

# RTL Design Sherpa

# RAPIDS Micro-Architecture Specification

**September 30, 2026**

# Table of Contents

1 Rapids Mas Index..... 37

2 Document Information..... 38

2.1 Revision History..... 39

2.2 References..... 44

2.2.1 Related Documents..... 44

2.3 Terminology..... 45

2.4 Document Conventions..... 47

3 Architecture Overview..... 48

3.1 Overview..... 49

3.1.1 Key Features..... 49

3.1.2 What Is Shared with RAPIDS Beats..... 49

3.2 Block Diagram..... 50

3.2.1 Figure 1.1.1: RAPIDS Architecture Block Diagram..... 50

3.3 Data Flow Overview..... 52

3.3.1 Sink Path (Network to Memory)..... 52

3.3.2 Figure 1.1.2: Sink Path Data Flow..... 52

3.3.3 Source Path (Memory to Network)..... 52

3.3.4 Figure 1.1.3: Source Path Data Flow..... 52

3.4 Byte-Granular Design Points..... 54

3.5 Chapter Map..... 55

4 Top-Level Port List..... 56

4.1 Parameters..... 57

4.2 Port List (324 ports)..... 59
4.2.1 Clock and Reset (shared - single clock/reset for the whole wrapper)..... 59
4.3 SOURCE HALF (u_src) - shared-infrastructure ports (src_ prefixed)..... 60
4.3.1 APB Programming Interface..... 60
4.3.2 Per-channel configuration..... 60
4.3.3 Scheduler Configuration (global)..... 60
4.3.4 Descriptor Engine Configuration (global)..... 61
4.3.5 Control Engine Configuration (Phase 2, global)..... 61
4.3.6 Descriptor AXI Monitor Configuration..... 61
4.3.7 Network-plane monitor input (rapids TASK-024)..... 62
4.3.8 Status..... 63
4.3.9 Descriptor AXI Monitor Status..... 63
4.3.10 Descriptor AXI Monitor perf window (feeds SRC_.MON.DAXMON_PERF_*)..... 64
4.3.11 Descriptor Fetch AXI Master..... 64
4.3.12 Control Read AXI Master (32-bit) [Phase 2]..... 65
4.3.13 Control Write AXI Master (32-bit) [Phase 2]..... 66
4.4 SOURCE HALF (u_src) - direction-unique ports (no prefix)..... 68
4.4.1 AXI Transfer Configuration (source-only)..... 68
4.4.2 Source Path - AXIS Master Interface (SRAM -> Network); tid = channel id ..... 68
4.4.3 AXI4 Master - Data Read (Memory -> Source SRAM)..... 68
4.5 SINK HALF (u_snk) - shared-infrastructure ports (snk_ prefixed)..... 70
4.5.1 APB Programming Interface..... 70

4.5.2 Per-channel configuration....70
4.5.3 Scheduler Configuration (global)....70
4.5.4 Descriptor Engine Configuration (global)....71
4.5.5 Control Engine Configuration (Phase 2, global)....71
4.5.6 Descriptor AXI Monitor Configuration....71
4.5.7 Network-plane monitor input (rapids TASK-024)....72
4.5.8 Status....73
4.5.9 Descriptor AXI Monitor Status....73
4.5.10 Descriptor AXI Monitor perf window (feeds SNK_.MON.DAXMON_PERF_*)....74
4.5.11 Descriptor Fetch AXI Master....74
4.5.12 Control Read AXI Master (32-bit) [Phase 2]....75
4.5.13 Control Write AXI Master (32-bit) [Phase 2]....76
4.6 Monitor Bus (SINGLE aggregated stream for the whole core)....78
4.6.1 Monitor Bus....78
4.7 SINK HALF (u_snk) - direction-unique ports (no prefix)....79
4.7.1 AXI Transfer Configuration (sink-only)....79
4.7.2 Sink Path - AXIS Slave Interface (Network -> SRAM); tid = channel id...79
4.7.3 AXI4 Master - Data Write (Sink SRAM -> Memory)....79
4.8 Debug Interface (per half, src_dbg_ / snk_dbg_ prefixed)....81
4.8.1 Source half debug....81
4.8.2 Sink half debug....81
4.8.3 Active-channel sideband for per-channel bus instrumentation....82
5 Clocks and Reset....83

5.1 Clock Domain....84
5.1.1 Clock Specifications....84
5.1.2 Clock Distribution....84
5.2 Reset Specification....85
5.2.1 Reset Signal....85
5.2.2 Reset Timing Diagram....85
5.2.3 Figure 1.3.1: Reset Timing....85
5.3 Reset Behavior....87
5.3.1 On Reset Assert....87
5.3.2 Reset Sequence....87
5.4 Configuration During Reset....88
5.5 Reset Macros....89
5.6 Power-On Reset Considerations....90
6 Scheduler....91
6.1 Overview....92
6.1.1 Structure: the FSM around scheduler_core....92
6.1.2 Key Features....92
6.1.3 Block Diagram....93
6.1.4 Figure 2.1.1: Scheduler Byte Math....93
6.2 Parameters....94
6.3 Port List....95
6.3.1 Clock, Reset and Configuration....95
6.3.2 Status....95
6.3.3 Descriptor Engine Interface....96

6.3.4 Descriptor Skid Buffer....96
6.3.5 Data Read Interface (to AXI Read Engine)....97
6.3.6 Data Write Interface (to AXI Write Engine)....97
6.3.7 Completion Interface....98
6.3.8 Control Engine Interface....98
6.3.9 Monitor Bus Interface....99
6.4 Descriptor Format....100
6.5 Byte-to-Beat Math....101
6.6 Address Advance....103
6.7 Packet Records....104
6.8 Finite State Machine....105
6.9 Operation....107
6.9.1 Data Request Outputs....107
6.9.2 Write Completion....107
6.9.3 Control Descriptors....107
6.10 Error Handling....108
6.11 Recoverable Write-Progress Timeout....109
6.12 MonBus Events....110
6.13 Run-Base Generator Flush....111
6.14 Waveforms....112
7 Descriptor Engine Specification....113
7.1 Overview....114
7.1.1 Key Features....114
7.1.2 Block Diagram....114

7.1.3 Figure 2.2.1: Descriptor Engine Block Diagram....114
7.2 Parameters....115
7.3 Port List....116
7.3.1 Clock and Reset....116
7.3.2 APB Programming Interface....116
7.3.3 Configuration Interface....116
7.3.4 AXI4 Read Master Interface....117
7.3.5 Scheduler Interface....118
7.4 FSM States....119
7.4.1 Figure 2.2.2: Descriptor Engine FSM....119
7.5 Operation....120
7.5.1 Descriptor Chain Processing....120
7.5.2 Figure 2.2.3: Descriptor Chain Timing....120
7.6 Address Range Checking....121
7.7 Prefetch and Autonomous Chaining....122
7.7.1 Chaining Decision....122
7.7.2 Deferred (Throttle-Blocked) Chain Fetch....122
8 AXI Read Engine....124
8.1 Overview....125
8.1.1 Key Features....125
8.1.2 Block Diagram....126
8.1.3 Figure 2.3.1: AXI Read Engine Block Diagram....126
8.2 Parameters....127
8.2.1 Derived Constants....127

8.3 Port List....129
8.3.1 Clock, Reset and Configuration....129
8.3.2 Scheduler Interface....129
8.3.3 SRAM Allocation and Fill Interface....130
8.3.4 AXI4 Read Master Interface....130
8.3.5 Debug....131
8.4 Operation....132
8.4.1 Burst Length Cap (rapids BUG-009)....132
8.4.2 Byte-Granular Burst Sizing....132
8.4.3 Streaming Pipeline....132
8.4.4 Figure 2.3.2: Burst Segmentation with a Mid-Beat Start....133
8.4.5 Done Strobe....133
8.5 Error Handling....134
8.5.1 Channel Reset....134
8.6 Differences from RAPIDS Beats....135
9 AXI Write Engine....136
9.1 Overview....137
9.1.1 Key Features....137
9.1.2 Block Diagram....138
9.1.3 Figure 2.4.1: AXI Write Engine Block Diagram....138
9.2 Parameters....139
9.2.1 Derived Constants....139
9.3 Port List....141
9.3.1 Clock, Reset and Configuration....141

9.3.2 Scheduler Interface....141
9.3.3 SRAM Reservation and Drain Interface....142
9.3.4 AXI4 Write Master Interface....143
9.3.5 Debug and Status....144
9.4 Operation....145
9.4.1 Burst Length Cap (rapids BUG-009)....145
9.4.2 Byte-Granular Burst Sizing....145
9.4.3 Write Strobes....145
9.4.4 Streaming Pipeline....145
9.4.5 Two Completion Strobes....146
9.5 Error Handling....147
9.5.1 Channel Reset....147
9.6 Differences from RAPIDS Beats....149
10 Beats Alloc Control Specification....150
10.1 Overview....151
10.1.1 Key Features....151
10.1.2 Block Diagram....151
10.1.3 Figure 2.5.1: Beats Alloc Control Block Diagram....151
10.2 Concept: Virtual FIFO....153
10.2.1 Use Case: Pre-Allocation for AXI Reads....153
10.3 Parameters....154
10.4 Port List....155
10.4.1 Clock and Reset....155
10.4.2 Write Interface (Allocation Requests)....155

10.4.3 Read Interface (Data Written)....155
10.4.4 Status Outputs....155
10.5 Operation....157
10.5.1 Pointer Arithmetic....157
10.5.2 Timing Diagram....157
10.5.3 Figure 2.5.2: Allocation and Release Timing....157
10.6 Integration Example....158
11 Beats Drain Control Specification....159
11.1 Overview....160
11.1.1 Key Features....160
11.1.2 Block Diagram....160
11.1.3 Figure 2.6.1: Beats Drain Control Block Diagram....160
11.2 Concept: Data Availability Tracking....162
11.2.1 Complementary Roles....162
11.3 Parameters....163
11.4 Port List....164
11.4.1 Clock and Reset....164
11.4.2 Write Interface (Data Written to SRAM)....164
11.4.3 Read Interface (Drain Requests)....164
11.4.4 Status Outputs....164
11.5 Operation....166
11.5.1 Pointer Arithmetic....166
11.5.2 Timing Diagram....166
11.5.3 Figure 2.6.2: Data Arrival and Drain Timing....166

11.6 Integration Example....167
11.7 Relationship: alloc_ctrl vs drain_ctrl....168
12 Latency Bridge (beats) Specification....169
12.1 Overview....170
12.1.1 Key Features....170
12.1.2 Block Diagram....171
12.1.3 Figure 2.7.1: Latency Bridge Block Diagram....171
12.2 Concept: bridging a registered read....172
12.3 Parameters....173
12.4 Port List....174
12.4.1 Clock and Reset....174
12.4.2 Upstream Interface (from the registered FIFO)....174
12.4.3 Downstream Interface (to the consumer)....174
12.4.4 Status and Debug....175
12.5 Operation....176
12.5.1 Figure 2.7.2: Latency Bridge Timing (consumer always ready)....176
12.6 Integration Context....177
12.7 Design Considerations....178
13 Control-Read Engine Specification....179
13.1 Overview....180
13.1.1 Key Features....180
13.1.2 Block Diagram....181
13.1.3 Figure 2.8.1: Control-Read Engine Block Diagram....181
13.2 Parameters....182

13.3 Port List....183
13.3.1 Clock and Reset....183
13.3.2 Scheduler Interface....183
13.3.3 Configuration Interface....184
13.3.4 AXI4 Read Master Interface....184
13.3.5 MonBus Interface....185
13.4 FSM States....186
13.4.1 Figure 2.8.2: Control-Read Engine FSM....186
13.5 Operation....188
13.5.1 Masked Compare....188
13.5.2 Retry Budget and Pacing....188
13.5.3 Null Address Fast-Path....188
13.5.4 AXI Read Transaction....188
13.5.5 Channel Reset....188
13.6 MonBus Reporting....190
13.7 Descriptor Field Mapping....191
14 Control-Write Engine Specification....192
14.1 Overview....193
14.1.1 Key Features....193
14.1.2 Block Diagram....194
14.1.3 Figure 2.9.1: Control-Write Engine Block Diagram....194
14.2 Parameters....195
14.3 Port List....196
14.3.1 Clock and Reset....196

14.3.2 Configuration Interface....196
14.3.3 Scheduler Interface....196
14.3.4 AXI4 Write Master Interface....197
14.3.5 MonBus Interface....198
14.4 FSM States....199
14.4.1 Figure 2.9.2: Control-Write Engine FSM....199
14.5 Operation....201
14.5.1 Doorbell Write Sequence....201
14.5.2 Figure 2.9.3: Control-Write Doorbell Timing....201
14.5.3 Request Handshake and Skid Buffer....202
14.5.4 Address Validation....202
14.5.5 Error Reporting....202
14.5.6 Channel Reset....202
14.6 Contrast with the Control-Read Engine....204
15 Shared Engines....205
15.1 Overview....206
15.1.1 Key Features....206
15.1.2 Cost....206
15.2 descriptor_engine_shared....207
15.2.1 Parameters....207
15.2.2 Port List....207
15.2.3 Per-Channel State....208
15.2.4 Operation....209
15.2.5 MonBus....210

15.3 ctrlrd_engine_shared and ctrlwr_engine_shared....211
15.3.1 Parameters....211
15.3.2 Port List....212
15.3.3 Trade-off....212
15.4 Related Chapters....213
16 Scheduler Group....214
16.1 Overview....215
16.1.1 Key Features....215
16.1.2 Block Diagram....216
16.1.3 Figure 3.1.1: Scheduler Group Block Diagram....216
16.2 Parameters....217
16.3 Port List....219
16.3.1 Clock, Reset and Kick-Off....219
16.3.2 Configuration Interface....219
16.3.3 Status....220
16.3.4 Descriptor AXI Master Interface....221
16.3.5 Control Read and Control Write AXI Interfaces....221
16.3.6 Scheduler Data Interface....222
16.3.7 Packet Record Interface....223
16.3.8 MonBus Interface....223
16.4 Internal Architecture....225
16.4.1 Descriptor Path....225
16.4.2 Control Engines....225
16.4.3 MonBus Aggregation....225

16.5 Differences from RAPIDS Beats....226
17 Scheduler Group Array....227
17.1 Overview....228
17.1.1 Key Features....228
17.1.2 Block Diagram....229
17.1.3 Figure 3.2.1: Scheduler Group Array Block Diagram....229
17.2 Parameters....230
17.3 Port List....232
17.3.1 Clock, Reset and Kick-Off....232
17.3.2 Global Scheduler and Descriptor Engine Configuration....232
17.3.3 Descriptor Monitor Configuration and Status....233
17.3.4 Descriptor-AXI Perf Window....234
17.3.5 Status....235
17.3.6 Shared AXI Masters....235
17.3.7 Per-Channel Scheduler Data Interface....236
17.3.8 Per-Channel Packet Record Interface....237
17.3.9 MonBus Interface....237
17.4 Shared Engines....238
17.5 Shared Master Arbitration (SHARED_ENGINES = 0)....240
17.5.1 Figure 3.2.2: Shared Master Arbitration....240
17.6 MonBus Aggregation....241
17.7 Differences from RAPIDS Beats....242
18 Sink Data Path....243
18.1 Overview....244

18.1.1 Key Features....244
18.1.2 Figure 3.3.1: Sink Data Path Block Diagram....244
18.2 Parameters....246
18.3 Interfaces....247
18.3.1 Fill Interface....247
18.3.2 Configuration....247
18.3.3 Scheduler Interface....248
18.3.4 AXI4 Write Master....248
18.4 Byte Enable Path....249
18.4.1 Channel Reset....249
18.5 Differences from RAPIDS Beats....250
19 Sink Data Path with AXIS (Ingress)....251
19.1 Overview....252
19.1.1 Key Features....252
19.1.2 Figure 3.4.1: Ingress Shifter....253
19.2 Parameters....254
19.3 Interfaces....255
19.3.1 AXI-Stream Slave....255
19.3.2 Scheduler Interface....255
19.3.3 Other Ports....256
19.4 Ingress Operation....257
19.4.1 Packet Records....257
19.4.2 Placement....257
19.4.3 Flush and Completion....257

19.4.4 Length Check....257
19.4.5 Channel Reset....257
19.4.6 Allocation....259
19.5 Differences from RAPIDS Beats....260
20 Sink SRAM Controller Specification....261
20.1 Overview....262
20.1.1 Key Features....262
20.1.2 Block Diagram....263
20.1.3 Figure 3.5.1: Sink SRAM Controller Block Diagram....263
20.2 Parameters....264
20.3 Port List....265
20.3.1 Clock and Reset....265
20.3.2 Fill Interface (ID-Selected)....265
20.3.3 Arbitrated Drain Interface (to AXI Write Engine)....265
20.3.4 Debug Interface....266
20.4 Drain Selection....267
20.4.1 Figure 3.5.2: Drain Selection by the Consumer....267
20.5 Integration with AXI Write Engine....268
21 Source Data Path Specification....269
21.1 Overview....270
21.1.1 Key Features....270
21.1.2 Block Diagram....270
21.1.3 Figure 3.6.1: Source Data Path Block Diagram....270
21.2 Parameters....272

21.3 Data Flow..... 273
21.3.1 Figure 3.6.2: Source Path Data Flow Sequence.....273
21.4 Port List.....274
21.4.1 Clock and Reset.....274
21.4.2 Configuration.....274
21.4.3 Scheduler Interface.....274
21.4.4 AXI Read Master Interface.....274
21.4.5 Drain Interface (Output to Network).....275
21.5 Timing Diagram.....277
21.5.1 Figure 3.6.3: Source Path Transfer Timing.....277
21.6 Integration Example.....278
22 Source Data Path with AXIS (Egress).....279
22.1 Overview.....280
22.1.1 Key Features.....280
22.1.2 Figure 3.7.1: Egress Shifter.....281
22.2 Parameters.....282
22.3 Interfaces.....283
22.3.1 Configuration.....283
22.3.2 Scheduler Interface.....283
22.3.3 AXI-Stream Master.....284
22.3.4 Other Ports.....284
22.4 Egress Operation.....285
22.4.1 Reservation and Drain.....285
22.4.2 Pop..... 285

22.4.3 Shift and Hold....285
22.4.4 Strobe and Last....286
22.4.5 Waveform 3.7: Egress Re-pack....286
22.5 Channel Reset....287
22.6 Differences from RAPIDS Beats....288
23 Source SRAM Controller Specification....289
23.1 Overview....290
23.1.1 Key Features....290
23.1.2 Block Diagram....291
23.1.3 Figure 3.8.1: Source SRAM Controller Block Diagram....291
23.2 Parameters....292
23.3 Port List....293
23.3.1 Clock and Reset....293
23.3.2 Fill Interface (ID-Selected, from AXI Read Engine)....293
23.3.3 Arbitrated Drain Interface (to Network)....293
23.3.4 Debug Interface....294
23.4 Drain Selection....295
23.4.1 Figure 3.8.2: Drain Selection by the Consumer....295
23.5 Comparison: Sink vs Source SRAM Controller....296
23.6 Integration Example....297
24 RAPIDS Core, Sink Half and Source Half....298
24.1 Overview....299
24.1.1 Figure 3.9.1: Core Block Diagram....299
24.2 Byte-Granular Content....300

24.2.1 Packet Record Nets..... 300
24.2.2 Port Renames..... 300
24.2.3 Data Ports..... 300
24.3 Parameters..... 302
24.3.1 Channel Reset Routing..... 303
24.4 Differences from RAPIDS Beats..... 305
25 RAPIDS Registers Specification..... 306
25.1 Overview..... 307
25.2 Base Register Map (0x100-0x3FF)..... 308
25.2.1 Key Register Fields..... 310
25.3 Monitor Register Map (rapids_mon_regs @ 0x1000)..... 311
26 RAPIDS Config Block Specification..... 313
26.1 Overview..... 314
26.2 Mapping Groups..... 315
26.2.1 Base Configuration..... 315
26.2.2 Monitor Configuration (hwif_out.MON.*)..... 315
26.2.3 Performance and Observation..... 316
27 RAPIDS Top..... 317
27.1 Overview..... 318
27.1.1 Channel-Count Variants..... 318
27.1.2 Figure 3.12.1: Top Hierarchy..... 319
27.2 APB Address Map..... 320
27.3 Parameters..... 321
27.4 Differences from RAPIDS Beats..... 323

28 Network Planes Specification....324
28.1 Overview....325
28.2 Generated Tops....326
28.3 CXS Planes....327
28.3.1 Figure 3.13.1: CXS Network Planes....327
28.3.2 Receive (sink) plane....327
28.3.3 Transmit (source) plane....328
28.3.4 Per-stream framing....328
28.3.5 Parameters....328
28.3.6 Monitoring....328
28.4 Proposed Changes....330
28.5 Verification....331
29 Tile Packet Header....332
29.1 Overview....333
29.2 Header Beat....334
29.2.1 Header Beat Chaining on Narrow Fabric Links (2026-10-19)....334
29.3 Source Half....335
29.4 Sink Half: rapids_tile_hdr_rx....336
29.5 Cell-Issued Descriptors: descriptor_engine_shared (INJ_EN)....337
29.6 End-to-End Credits: rapids_tile_credits (TILE_CREDITS)....338
29.7 Straps Toward RAPIDS (TILE_CREDITS)....340
29.8 Ordering Across the Fabric (open: delta DELTA-013)....343
29.9 Interleaved and Out-of-Order Intake (pencilled in, rapids TASK-033)....345
29.10 Status Registers....347

29.11 Verification....348
30 AXI4 Interface Specification....351
30.1 Overview....352
30.2 Byte Address to Beat Mapping....353
30.3 Burst Rules....354
30.4 Write Strobes....355
30.5 Limitations....356
31 AXI-Stream Interface Specification....357
31.1 Overview....358
31.2 Packet Contract....359
31.3 Source Master Signals....360
31.4 Sink Slave Signals....361
31.5 Length Check....362
32 MonBus Interface Specification....363
32.1 Overview....364
32.2 MonBus Signal Interface....365
32.2.1 Signal Table....365
32.2.2 Handshaking Protocol....365
32.2.3 Timing Diagram....365
32.2.4 Figure 4.3.1: MonBus Packet Transfer Timing....365
32.3 MonBus Packet Format....367
32.3.1 64-bit Packet Structure....367
32.3.2 Figure 4.3.2: MonBus Packet Format....367
32.4 Packet Types....368

32.4.1 Type Code Definitions....368
32.4.2 Payload Formats by Type....368
32.5 MonBus Source Aggregation....370
32.5.1 RAPIDS Core MonBus Sources....370
32.5.2 Aggregation Hierarchy....371
32.5.3 Figure 4.3.3: MonBus Aggregation Tree....371
32.6 Integration Guidelines....372
32.6.1 Consumer Requirements....372
32.6.2 Recommended Consumer Patterns....372
32.7 Timing Requirements....373
33 Packet Record Interface Specification....374
33.1 Overview....375
33.1.1 Figure 4.4.1: Scheduler Byte Math....375
33.2 Signals....376
33.3 Protocol....377
33.4 Unused Direction....378
33.5 Consumer Behavior....379
34 CXS Interface Specification....380
34.1 Overview....381
34.2 Link Profile....382
34.3 Signal Table....383
34.4 Credit Flow....385
34.5 Link Activation....386
34.6 Backpressure....387

34.7 Proposed Changes....388
35 rapids-g Overview....389
35.1 What rapids-g Is....390
35.2 Structure....391
35.3 Channels per gatling....392
35.4 What stays hardware, and why....393
36 scheduler_g....394
36.1 Overview....395
36.2 Sequencer Port....396
36.3 Register Window....397
36.4 Commands and the Edge Table....400
36.5 Pre-validation and the Automatic Edges....401
36.6 Firmware Packets....403
36.7 Attention....404
36.8 Run-Base FIFOs....405
37 rapids_seq_g and the Scheduler Firmware....406
37.1 rapids_seq_g....407
37.1.1 Data Memory Map....407
37.1.2 Register Mailbox....408
37.1.3 Faults....411
37.2 Firmware....412
37.2.1 The Executive....412
37.2.2 Figure 5.3.1: One Hart’s Poll....414
37.2.3 The Policy Table....415

37.2.4 Figure 5.3.2: step(), the Policy Table....417
37.2.5 EXT Run Bases....417
37.2.6 Next-Address Selector....417
37.2.7 Building the Image....418
37.2.8 Loading Firmware Live....418
38 The Generated rapids-g Tree....421
39 rapids-g Verification and Measurements....423
39.1 Verification....424
39.2 Performance on the Board....427
39.3 Scheduler Limit....431
39.3.1 Sixteen Channels....432
39.4 Sixteen Channels on the Genesys 2....434
39.4.1 Where rapids-g Catches Up....437
39.5 Area and Timing....441
40 RAPIDS Beats Architecture MAS....443
40.1 Document Organization....444
40.1.1 Front Matter....444
40.1.2 Chapter 1: Overview....444
40.1.3 Chapter 2: FUB (Functional Unit Blocks)....444
40.1.4 Chapter 3: Macro (Integration Blocks)....444
40.1.5 Chapter 4: Interfaces....444
40.2 Quick Reference....446
40.2.1 FUB Modules (fub_beats/)....446
40.2.2 Macro Modules (macro_beats/)....447

40.3 Architecture Comparison: RAPIDS vs STREAM....449
40.4 Clock and Reset Summary....450
40.4.1 Clock Domains....450
40.4.2 Reset Signals....450
40.5 Interface Summary....451
40.5.1 External Interfaces....451
40.5.2 Internal Buses....451
40.6 Related Documentation....452
40.7 Specification Conventions....453
40.7.1 Signal Naming....453
40.7.2 Parameter Naming....453
40.7.3 Figure and Table Numbering....453

# List of Figures

Figure 1.1.1: RAPIDS Architecture Block Diagram....50
Figure 1.1.2: Sink Path Data Flow....52
Figure 1.1.3: Source Path Data Flow....52
Figure 1.3.1: Reset Timing....85
Figure 2.1.1: Scheduler Byte Math....92
Figure 2.2.1: Descriptor Engine Block Diagram....112
Figure 2.2.2: Descriptor Engine FSM....117
Figure 2.2.3: Descriptor Chain Timing....118
Figure 2.3.1: AXI Read Engine Block Diagram....124
Figure 2.3.2: Burst Segmentation with a Mid-Beat Start....130
Figure 2.4.1: AXI Write Engine Block Diagram....136
Figure 2.5.1: Beats Alloc Control Block Diagram....149
Figure 2.5.2: Allocation and Release Timing....154
Figure 2.6.1: Beats Drain Control Block Diagram....157
Figure 2.6.2: Data Arrival and Drain Timing....162
Figure 2.7.1: Latency Bridge Block Diagram....167
Figure 2.7.2: Latency Bridge Timing (consumer always ready)....172
Figure 2.8.1: Control-Read Engine Block Diagram....177
Figure 2.8.2: Control-Read Engine FSM....182
Figure 2.9.1: Control-Write Engine Block Diagram....190
Figure 2.9.2: Control-Write Engine FSM....195
Figure 2.9.3: Control-Write Doorbell Timing....197
Figure 3.1.1: Scheduler Group Block Diagram....212
Figure 3.2.1: Scheduler Group Array Block Diagram....225
Figure 3.2.2: Shared Master Arbitration....236
Figure 3.3.1: Sink Data Path Block Diagram....240
Figure 3.4.1: Ingress Shifter....248
Figure 3.5.1: Sink SRAM Controller Block Diagram....258
Figure 3.5.2: Drain Selection by the Consumer....262
Figure 3.6.1: Source Data Path Block Diagram....265
Figure 3.6.2: Source Path Data Flow Sequence....267
Figure 3.6.3: Source Path Transfer Timing....271
Figure 3.7.1: Egress Shifter....275
Figure 3.8.1: Source SRAM Controller Block Diagram....285
Figure 3.8.2: Drain Selection by the Consumer....289
Figure 3.9.1: Core Block Diagram....293

Figure 3.12.1: Top Hierarchy....313
Figure 3.13.1: CXS Network Planes....321
Figure 4.3.1: MonBus Packet Transfer Timing....359
Figure 4.3.2: MonBus Packet Format....361
Figure 4.3.3: MonBus Aggregation Tree....365
Figure 4.4.1: Scheduler Byte Math....369
Figure 5.3.1: One Hart's Poll....408
Figure 5.3.2: step(), the Policy Table....411

# List of Tables

Table 1: Table 0.1: Revision History....39
Table 2: Table 0.2: Related Documents and Specifications....44
Table 3: Table 1.1.1: Block Roles....50
Table 4: Table 1.1.2: Design Points....54
Table 5: Table 1.2.1: Parameters (from rapids_core_beats.sv)....57
Table 6: Table 1.2.2: Clock and Reset (shared - single clock/reset for the whole wrapper)....59
Table 7: Table 1.2.3: APB Programming Interface....60
Table 8: Table 1.2.4: Per-channel configuration....60
Table 9: Table 1.2.5: Scheduler Configuration (global)....60
Table 10: Table 1.2.6: Descriptor Engine Configuration (global)....61
Table 11: Table 1.2.7: Control Engine Configuration (Phase 2, global)....61
Table 12: Table 1.2.8: Descriptor AXI Monitor Configuration....61
Table 13: Table 1.2.9: Network-plane monitor input (rapids TASK-024)....63
Table 14: Table 1.2.10: Status....63
Table 15: Table 1.2.11: Descriptor AXI Monitor Status....63
Table 16: Table 1.2.12: Descriptor AXI Monitor perf window (feeds SRC_.MON.DAXMON_PERF_*)....64
Table 17: Table 1.2.13: Descriptor Fetch AXI Master....64
Table 18: Table 1.2.14: Control Read AXI Master (32-bit) [Phase 2]....65
Table 19: Table 1.2.15: Control Write AXI Master (32-bit) [Phase 2]....66
Table 20: Table 1.2.16: AXI Transfer Configuration (source-only)....68
Table 21: Table 1.2.17: Source Path - AXIS Master Interface (SRAM -> Network); tid = channel id....68
Table 22: Table 1.2.18: AXI4 Master - Data Read (Memory -> Source SRAM)....68
Table 23: Table 1.2.19: APB Programming Interface....70
Table 24: Table 1.2.20: Per-channel configuration....70
Table 25: Table 1.2.21: Scheduler Configuration (global)....70
Table 26: Table 1.2.22: Descriptor Engine Configuration (global)....71
Table 27: Table 1.2.23: Control Engine Configuration (Phase 2, global)....71
Table 28: Table 1.2.24: Descriptor AXI Monitor Configuration....71
Table 29: Table 1.2.25: Network-plane monitor input (rapids TASK-024)....73
Table 30: Table 1.2.26: Status....73
Table 31: Table 1.2.27: Descriptor AXI Monitor Status....73
Table 32: Table 1.2.28: Descriptor AXI Monitor perf window (feeds SNK_.MON.DAXMON_PERF_*)....74

Table 33: Table 1.2.29: Descriptor Fetch AXI Master....74
Table 34: Table 1.2.30: Control Read AXI Master (32-bit) [Phase 2]....75
Table 35: Table 1.2.31: Control Write AXI Master (32-bit) [Phase 2]....76
Table 36: Table 1.2.32: Monitor Bus....78
Table 37: Table 1.2.33: AXI Transfer Configuration (sink-only)....79
Table 38: Table 1.2.34: Sink Path - AXIS Slave Interface (Network -> SRAM); tid = channel id....79
Table 39: Table 1.2.35: AXI4 Master - Data Write (Sink SRAM -> Memory)....79
Table 40: Table 1.2.36: Source half debug....81
Table 41: Table 1.2.37: Sink half debug....81
Table 42: Table 1.2.38: Active-channel sideband for per-channel bus instrumentation....82
Table 43: Table 1.3.1: Clock Specifications....84
Table 44: Table 1.3.2: Reset Specifications....85
Table 45: Table 1.3.3: Reset States....86
Table 46: Table 1.3.4: Configuration Timing Requirements....87
Table 47: Table 2.1.1: Scheduler Parameters....93
Table 48: Table 2.1.2: Clock, Reset and Configuration Ports....94
Table 49: Table 2.1.3: Status Ports....94
Table 50: Table 2.1.4: Descriptor Engine Interface....95
Table 51: Table 2.1.5: Data Read Interface....96
Table 52: Table 2.1.6: Data Write Interface....96
Table 53: Table 2.1.7: Completion and Error Interface....97
Table 54: Table 2.1.8: Control Engine Interface....97
Table 55: Table 2.1.9: Monitor Bus Interface....98
Table 56: Table 2.1.10: Descriptor Layout....99
Table 57: Table 2.1.11: Beat Totals for a 32-byte Beat....100
Table 58: Table 2.1.12: State Encoding....103
Table 59: Table 2.1.13: Transitions....103
Table 60: Table 2.1.14: MonBus Events....108
Table 61: Table 2.2.2: Clock and Reset....114
Table 62: Table 2.2.3: APB Programming Interface....114
Table 63: Table 2.2.4: Configuration Interface....114
Table 64: Table 2.2.5: AXI4 Read Master Interface....115
Table 65: Table 2.2.6: Scheduler Interface....116
Table 66: Table 2.3.1: AXI Read Engine Parameters....125
Table 67: Table 2.3.2: AXI Read Engine Derived Constants....125
Table 68: Table 2.3.3: Clock, Reset and Configuration....127

Table 69: Table 2.3.4: Scheduler Interface....127
Table 70: Table 2.3.5: SRAM Allocation and Fill Interface....128
Table 71: Table 2.3.6: AXI4 Read Master Interface....128
Table 72: Table 2.3.7: Debug Outputs....129
Table 73: Table 2.3.8: Burst Segmentation Example....131
Table 74: Table 2.3.9: Error Handling....132
Table 75: Table 2.3.10: Read Engine Delta....133
Table 76: Table 2.4.1: AXI Write Engine Parameters....137
Table 77: Table 2.4.2: AXI Write Engine Derived Constants....137
Table 78: Table 2.4.3: Clock, Reset and Configuration....139
Table 79: Table 2.4.4: Scheduler Interface....139
Table 80: Table 2.4.5: SRAM Drain Interface....140
Table 81: Table 2.4.6: AXI4 Write Master Interface....141
Table 82: Table 2.4.7: Debug and Status Outputs....142
Table 83: Table 2.4.8: Completion Strobes....144
Table 84: Table 2.4.9: Error Handling....145
Table 85: Table 2.4.10: Write Engine Delta....147
Table 86: Table 2.5.2: Clock and Reset....152
Table 87: Table 2.5.3: Write Interface....152
Table 88: Table 2.5.4: Read Interface....152
Table 89: Table 2.5.5: Status Outputs....152
Table 90: Table 2.6.2: Clock and Reset....160
Table 91: Table 2.6.3: Write Interface....160
Table 92: Table 2.6.4: Read Interface....160
Table 93: Table 2.6.5: Status Outputs....160
Table 94: Table 2.6.6: alloc_ctrl vs drain_ctrl Comparison....164
Table 95: Table 2.7.2: Clock and Reset....170
Table 96: Table 2.7.3: Upstream Interface....170
Table 97: Table 2.7.4: Downstream Interface....170
Table 98: Table 2.7.5: Status and Debug....170
Table 99: Table 2.7.6: Skid Depth Selection....174
Table 100: Table 2.8.2: Clock and Reset....179
Table 101: Table 2.8.3: Scheduler Interface....179
Table 102: Table 2.8.4: Configuration Interface....180
Table 103: Table 2.8.5: AXI4 Read Master Interface....180
Table 104: Table 2.8.6: MonBus Interface....181
Table 105: Table 2.8.7: MonBus Events....186
Table 106: Table 2.8.8: CTRL_READ Descriptor Field Mapping....187

Table 107: Table 2.9.2: Clock and Reset....192
Table 108: Table 2.9.3: Configuration Interface....192
Table 109: Table 2.9.4: Scheduler Interface....192
Table 110: Table 2.9.5: AXI4 Write Master Interface....193
Table 111: Table 2.9.6: MonBus Interface....194
Table 112: Table 2.10.1: descriptor_engine_shared Parameters....203
Table 113: Table 2.10.2: descriptor_engine_shared Ports....203
Table 114: Table 2.10.3: Per-Channel Fetch States....204
Table 115: Table 2.10.4: Shared Control Engine Wrapper....207
Table 116: Table 2.10.5: Shared Control Engine Parameters....207
Table 117: Table 2.10.6: Shared Control Engine Ports....208
Table 118: Table 3.1.1: Scheduler Group Parameters....213
Table 119: Table 3.1.2: Clock, Reset and Kick-Off....215
Table 120: Table 3.1.3: Configuration Interface....215
Table 121: Table 3.1.4: Status Interface....216
Table 122: Table 3.1.5: Descriptor AXI Master Interface....217
Table 123: Table 3.1.6: Scheduler Data Interface....218
Table 124: Table 3.1.7: Packet Record Interface....219
Table 125: Table 3.1.8: MonBus Interface....219
Table 126: Table 3.1.9: Scheduler Group Delta....222
Table 127: Table 3.2.1: Scheduler Group Array Parameters....226
Table 128: Table 3.2.2: Clock, Reset and Kick-Off....228
Table 129: Table 3.2.3: Global Configuration....228
Table 130: Table 3.2.4: Descriptor Monitor Configuration and Status....229
Table 131: Table 3.2.5: Descriptor-AXI Perf Window....231
Table 132: Table 3.2.6: Status....231
Table 133: Table 3.2.7: Per-Channel Scheduler Data Interface....232
Table 134: Table 3.2.8: Per-Channel Packet Record Interface....233
Table 135: Table 3.2.9: MonBus Interface....233
Table 136: Table 3.2.11: Shared Engines....234
Table 137: Table 3.2.10: MonBus Source Assignment....237
Table 138: Table 3.2.11: Array Delta....238
Table 139: Table 3.3.1: Sink Data Path Parameters....241
Table 140: Table 3.3.2: Fill Interface....242
Table 141: Table 3.3.3: Configuration....242
Table 142: Table 3.3.4: Scheduler Interface....243
Table 143: Table 3.3.5: Sink Data Path Delta....245
Table 144: Table 3.4.1: Additional Parameters....249

Table 145: Table 3.4.2: AXI-Stream Slave....250
Table 146: Table 3.4.3: Packet Record Ports....250
Table 147: Table 3.4.4: Channel Reset Actions....253
Table 148: Table 3.4.5: Ingress Delta....255
Table 149: Table 3.5.2: Clock and Reset....260
Table 150: Table 3.5.3: Fill Interface (ID-Selected)....260
Table 151: Table 3.5.4: Arbitrated Drain Interface....260
Table 152: Table 3.5.5: Debug Interface....261
Table 153: Table 3.6.2: Clock and Reset....268
Table 154: Table 3.6.3: Configuration....268
Table 155: Table 3.6.4: Scheduler Interface....268
Table 156: Table 3.6.5: AXI Read Master Interface....268
Table 157: Table 3.6.6: Drain Interface....269
Table 158: Table 3.7.1: Parameters....276
Table 159: Table 3.7.2: Configuration....277
Table 160: Table 3.7.3: Scheduler Interface....277
Table 161: Table 3.7.4: AXI-Stream Master....278
Table 162: Table 3.7.5: Egress Shift Rules....279
Table 163: Table 3.7.6: Channel Reset Actions....281
Table 164: Table 3.7.7: Egress Delta....282
Table 165: Table 3.8.2: Clock and Reset....287
Table 166: Table 3.8.3: Fill Interface (ID-Selected)....287
Table 167: Table 3.8.4: Arbitrated Drain Interface....287
Table 168: Table 3.8.5: Debug Interface....288
Table 169: Table 3.8.6: Sink vs Source Controller Comparison....290
Table 170: Table 3.9.1: Packet Record Nets per Half....294
Table 171: Table 3.9.2: Core Port Renames....294
Table 172: Table 3.9.3: Data Ports....294
Table 173: Table 3.9.4: Core Parameters....296
Table 174: Table 3.9.5: Channel Reset Routing....297
Table 175: Table 3.9.6: Core Delta....299
Table 176: Table 3.10.1: Base Register Map....302
Table 177: Table 3.10.2: Monitor Register Map (base 0x1000)....305
Table 178: Table 3.11.1: Monitor Register-to-Config Mapping....309
Table 179: Table 3.12.1: Top Variants....312
Table 180: Table 3.12.2: APB Address Map....314
Table 181: Table 3.12.3: Top Parameters....315
Table 182: Table 3.12.4: Top Delta....317

Table 183: Table 3.13.1: RAPIDS Network Planes....319
Table 184: Table 3.13.2: Generated Tops....320
Table 185: Table 3.13.3: CXS Plane Parameters....322
Table 186: Table 3.13.4: CXS Top Test Cells....325
Table 187: Table 3.14.1: Header Fields and Their Sources....328
Table 188: Table 3.14.2: Sink-Port Packet Handling....330
Table 189: Table 3.14.3: Credit Counter Index....332
Table 190: Table 3.14.4: Straps toward RAPIDS....334
Table 191: Table 3.14.5: Stalled-data checks....335
Table 192: Table 3.14.6: RAPIDS ordering hazards under multi-path routing....337
Table 193: Table 3.14.7: Tagged intake by class (pencilled in)....339
Table 194: Table 3.14.8: Model payload flits per cycle (link peak 1; headers take the rest)....340
Table 195: Table 3.14.7: Tile Header Test Cells....342
Table 196: Table 4.1.1: Address Split....347
Table 197: Table 4.1.2: Burst Rules....348
Table 198: Table 4.1.3: Byte-Granular Write Signal....349
Table 199: Table 4.2.1: Packet Contract....353
Table 200: Table 4.2.2: Source Master Signals....354
Table 201: Table 4.2.3: Sink Slave Signals....355
Table 202: Table 4.3.1: MonBus Interface Signals....359
Table 203: Table 4.3.2: MonBus Packet Fields....361
Table 204: Table 4.3.3: MonBus Packet Types....362
Table 205: Table 4.3.4: Descriptor Packet Payload....362
Table 206: Table 4.3.5: Scheduler Packet Payload....363
Table 207: Table 4.3.6: AXI Event Packet Payload....363
Table 208: Table 4.3.7: Error Packet Payload....363
Table 209: Table 4.3.8: MonBus Source Assignment....364
Table 210: Table 4.3.9: MonBus Timing Parameters....367
Table 211: Table 4.4.1: Packet Record Signals....370
Table 212: Table 4.4.2: Record Rules....371
Table 213: Table 4.4.1: CXS Link Profile....376
Table 214: Table 4.4.2: CXS Interface Signals....377
Table 215: Table 4.4.3: Link Activation Sequence....380
Table 216: Table 5.1.1: rapids and rapids-g, level by level....385
Table 217: Table 5.2.1: Sequencer port....390
Table 218: Table 5.2.2: The window, by word....391
Table 219: Table 5.2.3: STATUS....392

Table 220: Table 5.2.4: Legal edges....394
Table 221: Table 5.2.5: Firmware packets....397
Table 222: Table 5.3.1: rapids_seq_g....401
Table 223: Table 5.3.2: Data memory map....401
Table 224: Table 5.3.3: Register mailbox, per hart....402
Table 225: Table 5.3.4: Register-path masters....403
Table 226: Table 5.3.5: One gatling access, end to end....404
Table 227: Table 5.3.6: The policy table....409
Table 228: Table 5.3.7: Next-address algorithms in the image....412
Table 229: Table 5.3.8: Firmware load registers, per half....412
Table 230: Table 5.4.1: Generated files....415
Table 231: Table 5.5.1: rapids-g suites....418
Table 232: Table 5.5.2: Mutation, executive image....419
Table 233: Table 5.5.3: Genesys 2 gcompare, MB/s (peak 3,200; – = not measured: the source side has no interleaved variant)....421
Table 234: Table 5.5.4: Firmware cost per chained descriptor (the 16-channel rows' clocks are the point's measured cycles / 16)....423
Table 235: Table 5.5.5: Cycles to finish, schedulers alone....424
Table 236: Table 5.5.6: Cycles to finish at 16 channels....425
Table 237: Table 5.5.8: 16-channel bitstreams....427
Table 238: Table 5.5.9: Genesys 2 gcompare at 16 channels, MB/s (peak 3,200; – = no source variant)....427
Table 239: Table 5.5.10: rapids / rapids-g cycles, every channel busy....430
Table 240: Table 5.5.11: Bytes per descriptor for rapids-g to be within 10%, 5% and 1% of rapids....431
Table 241: Table 5.5.12: rapids / rapids-g cycles, one channel busy (-- = not in the sweep)....431
Table 242: Table 5.5.7: Out-of-context results....434
Table 243: FUB Module Summary....438
Table 244: Macro Module Summary....439
Table 245: RAPIDS Beats vs STREAM Comparison....441
Table 246: Clock Domains....442
Table 247: Reset Signals....442
Table 248: External Interfaces....443
Table 249: Internal Buses....443

# List of Waveforms

Waveform 2.1: Byte Descriptor Start.....110
Waveform 3.7: Egress Re-pack.....280

# 1 Rapids Mas Index

**Generated:** 2026-10-24

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 2 Document Information

This document describes RAPIDS, the byte-granular Rapid AXI Programmable In-band Descriptor System. RAPIDS provides network-to-memory and memory-to-network data transfer using descriptor-based DMA with 8 or 16 channels. A descriptor length is a byte count, source and destination addresses are byte addresses, and the sink write master drives byte enables. The earlier “Beats” architecture, which counted everything in beats, was the stepping stone to this design and is documented separately in the RAPIDS Beats MAS.

---

## 2.1 Revision History

*Table 0.1: Revision History*

| Version | Date | Author | Description |
|---|---|---|---|
| 0.6 | 2026-10-17 | RTL Design Sherpa | New Section 2.10 Shared Engines (`descriptor_engine_shared`, `ctrlrd_engine_shared`, `ctrlwr_engine_shared`: parameters, ports, per-channel state, operation, measured cost). `SHARED_ENGINES` and `TILE_HDR` added to the core and top parameter tables; 8- and 16-channel top variants; scheduler group marked per-channel-build only; scheduler group array overview corrected (one engine of each kind per array, not per channel) |
| 0.5 | 2026-10-17 | RTL Design Sherpa | New Section 3.14 Tile Packet Header (`TILE_HDR`: header build and decode, cell-issued descriptors, status registers). Scheduler group array: shared descriptor / control-read / control-write engines per half (`SHARED_ENGINES`, default 1), with block diagram |
| 0.4 | 2026-10-16 | RTL Design Sherpa | Shared Sections 3.13 and 4.4 point at the routing and payload-type proposal in the tile's TILE_SPEC Section |

| Version | Date | Author | Description |
|---|---|---|---|
| | | | 11 |
| 0.3 | 2026-10-16 | RTL Design Sherpa | Network planes (rapids TASK-024/025): `rapids_top` / `rapids_cxs_top` generated from `rapids_top_base`; links the shared Network Planes chapter and CXS Interface Specification; block diagrams updated for the planes; the CXS ID mapping is marked provisional. |
| 0.2 | 2026-09-30 | RTL Design Sherpa | Channel reset recovers every error class: `cfg_channel_reset` routed into the AXI read and write engines, both data paths and the SRAM wrappers; scheduler clears sticky flags on the reset. |
| 0.1 | 2026-09-30 | RTL Design Sherpa | First byte-granular MAS. Rewrites the chapters that changed from RAPIDS Beats and links the rest. |
| 0.7 | 2026-10-20 | RTL Design Sherpa | Block diagrams redrawn to the repo element key (v3): class fills, double-bordered registered blocks, green return paths and a generated key under each figure. No technical content changed. |
| 0.8 | 2026-10-20 | RTL Design Sherpa | Section (h2) headings now open a new page, the pipeline default. No |

| Version | Date | Author | Description |
|---|---|---|---|
| | | | technical content changed. |
| 0.81 | 2026-10-21 | RTL Design Sherpa | New Chapter 5, rapids-g: gatling-sequenced schedulers (`scheduler_g`, `rapids_seq_g`, firmware, the generated tree, verification, measured cost). Scheduler: split into the FSM and `scheduler_core`; MonBus events on state entry (rapids BUG-016); run-base generators flushed by the channel reset (rapids BUG-017). |
| 0.82 | 2026-10-22 | RTL Design Sherpa | rapids-g at one hart per channel (NUM_HARTS = NUM_CHANNELS): Section 5.3 describes the one-slot tight loop, with flow charts of the poll loop and `service` (Figures 5.3.1 and 5.3.2), and no longer says the firmware computes beat totals (the hardware does); Section 5.5 gains the Genesys 2 gcompare table for rapids-8ch and rapids-g-8ch (45 points, interleaved sink), sim/board match and the chunk warm-up, the firmware-cost history, and the 8-hart bench and out-of-context results. |
| 0.83 | 2026-10-23 | RTL Design | rapids-g firmware as an |

| Version | Date | Author | Description |
|---|---|---|---|
| | | Sherpa | executive over a policy table (rapids TASK-037): Section 5.3 rewritten (the poll and `step`, `ptab`, the policy table, the never-reached states and `fw_fault`), with new flow charts (Figures 5.3.1 and 5.3.2); the next-address selector and Z-order (TASK-035) in Sections 5.2 and 5.3; 16 channels (TASK-036), with 8 harts x 2 slots the measured default. Section 5.5: the mutation table re-run on the new image, the executive and 16-channel bench tables, out-of-context results at 8 and 16 channels, and the board run of the new image. |
| 0.84 | 2026-10-23 | RTL Design Sherpa | rapids-g firmware loadable live (rapids TASK-038): `FW_WRITABLE`, the `FW_*` registers, the load and its self-test, ROM once final (Section 5.3, Table 5.3.8). The register mailbox section rewritten as the serialized register path: one agent owns the path from command to response, with both arbitration levels and their signals (Tables 5.3.3-5.3.5). The two-slot poll's chain |

| Version | Date | Author | Description |
|---|---|---|---|
| | | | hook and why a chain is serial. Section 5.5: rapids-16ch and rapids-g-16ch on the Genesys 2 (fit, sim == board, MB/s at 16 channels) and the size sweep at 8 and 16 channels (Tables 5.5.8-5.5.10). Table 5.5.6 re-measured on the committed image. Table 5.5.10 extended to 128 KB per descriptor, with the source rows; new Table 5.5.11, the bytes per descriptor at which rapids-g is within 10%, 5% and 1% of rapids. One-channel sweep to 1 MB (Table 5.5.12) and its rows in Table 5.5.11. |

## 2.2 References

### 2.2.1 Related Documents

*Table 0.2: Related Documents and Specifications*

| Source | Title | Version |
|---|---|---|
| RTL Design Sherpa | RAPIDS Beats MAS (shared chapters) | 0.7 |
| RTL Design Sherpa | RAPIDS HAS | 0.93 |
| RTL Design Sherpa | RAPIDS Product Requirements Document | 1.0 |
| ARM | AMBA AXI and ACE Protocol Specification | IHI0022H |
| ARM | AMBA AXI-Stream Protocol Specification | IHI0051A |
| ARM | AMBA CXS Protocol Specification | IHI0079D |

## 2.3 Terminology

**Beat**

One data transfer of an AXI burst or an AXI-Stream transfer. One beat is `BYTE_LANES` bytes, where `BYTE_LANES = DATA_WIDTH/8`. The board design uses 256 bits, so a beat is 32 bytes.

**Byte offset**

The low `OFF_W` bits of a byte address, where `OFF_W = $clog2(BYTE_LANES)`. It is the lane at which a transfer begins inside its first memory beat.

**Memory beat**

A beat as it appears on the AXI side, aligned to `BYTE_LANES`. A packet that starts at a non-zero offset occupies one more memory beat than a stream of the same length would.

**Stream beat**

A beat as it appears on AXI-Stream, packed from lane 0 with no offset.

**Packet record**

The pair `{bytes, offset}` that the scheduler issues once per DATA descriptor, so the AXIS shifter knows how the memory-side and stream-side layouts relate.

**Spill**

The bytes of a stream beat that did not fit in the current memory beat during ingress shifting. They are held for the next memory beat.

**Channel**

One of 8 independent DMA channels. Each has its own descriptor chain, scheduler and SRAM allocation.

**Descriptor**

A 256-bit structure holding the source address, destination address, length in bytes, next pointer, and control flags.

**TYPE=EXT**

The descriptor type that describes a row/column transfer. It stays beat-aligned by design: addresses are aligned and lengths are multiples of the beat size.

**DMA**

Direct Memory Access. Data movement without processor involvement.

**MonBus**

The 64-bit monitor bus that carries event packets out of the design.

---

## 2.4 Document Conventions

- Shared chapters are linked from the RAPIDS Beats MAS and marked “(shared with RAPIDS Beats)” in the index. Module names in those chapters carry a `_beats` suffix that this design drops.
- Requirement priorities follow the RAPIDS PRD.
- No emojis appear in this document.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 3 Architecture Overview

**Module:** `rapids_core.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro/`
**Status:** Implemented **Last Updated:** 2026-09-30

---

## 3.1 Overview

RAPIDS moves data between AXI-Stream and system memory in units of bytes. A descriptor's length is a byte count in the existing 32-bit field, and its source and destination are byte addresses. A linear descriptor may start at any byte on either side. The design keeps the streaming, no-FSM data path of RAPIDS Beats and adds three things around it: byte-to-beat math in the scheduler, byte enables through the sink SRAM to the write master, and a shifter at each AXI-Stream edge that converts between packed stream beats and memory-aligned beats.

### 3.1.1 Key Features

- **8 or 16 Independent Channels:** Each channel has its own descriptor chain and SRAM allocation. The 16-channel tops use the 16-channel register map.
- **Shared Engines:** By default each half has one descriptor engine, one control-read engine and one control-write engine for all its channels (`SHARED_ENGINES = 1`).
- **Separate Data Paths:** Sink (network to memory) and source (memory to network).
- **Byte Granularity:** Length in bytes, byte addresses, any starting offset for linear descriptors.
- **Byte Enables:** The sink SRAM entry is `{strb, data}`; the write engine drives `WSTRB` from it.
- **Packet Records:** The scheduler issues one `{bytes, offset}` record per DATA descriptor to the AXIS shifter.
- **One AXIS Packet per Descriptor:** The source path emits exactly one packet, with `TLAST` on the final byte.
- **Safe Bursts:** AXI bursts never cross a 4 KB boundary; `AxADDR` is beat-aligned.
- **MonBus Integration:** Same monitor packets as RAPIDS Beats.

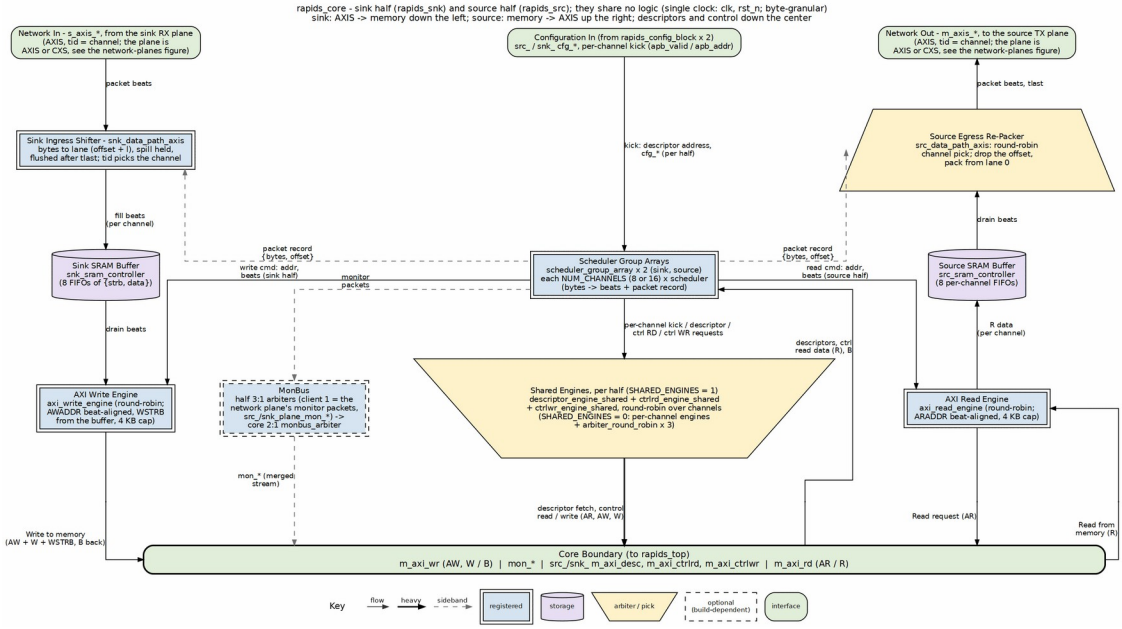
### 3.1.2 What Is Shared with RAPIDS Beats

The SRAM controllers, the register block and config block, and the MonBus specification are unchanged. Those chapters are linked from the RAPIDS Beats MAS. The per-channel descriptor, control-read and control-write engines are also unchanged and are built when `SHARED_ENGINES = 0`. The default build replaces them with the Shared Engines, which keep their per-channel behaviour.

---

## 3.2 Block Diagram

### 3.2.1 Figure 1.1.1: RAPIDS Architecture Block Diagram




*RAPIDS core block diagram*

**Source:** 04_rapids_core.dot

**Source:** `rapids_core.sv`, `rapids_snk.sv`, `rapids_src.sv`

*Table 1.1.1: Block Roles*

| Block | Role |
|---|---|
| scheduler | Loads a descriptor, computes beats from offset and length, issues the packet record, tracks completion |
| scheduler_group_array | Eight scheduler groups sharing one descriptor AXI master; per-channel packet-record ports |
| snk_data_path_axis | Converts packed AXIS beats into memory-aligned beats with byte enables; checks the packet length |
| snk_data_path | SRAM of `{strb, data}` entries plus the AXI write engine |
| src_data_path | SRAM of data entries plus the AXI |

| Block | Role |
| --- | --- |
| | read engine; unchanged from Beats |
| src_data_path_axis | Converts memory-aligned beats into packed stream beats with a partial last beat |

## 3.3 Data Flow Overview

### 3.3.1 Sink Path (Network to Memory)

### 3.3.2 Figure 1.1.2: Sink Path Data Flow

```
1. Scheduler loads a descriptor, computes beats_total, pulses
sched_wr_pkt_valid
        |   (record {bytes, offset} enters the per-channel record
queue)
        v
2. s_axis packet arrives; tready is held low until the channel's
record is known
        |
        v
3. Ingress shifter: stream beat << (offset*8), OR the spill of the
previous beat
        |   low DATA_WIDTH bits = this memory beat, high bits become
the new spill
        v
4. Fill logic writes {strb, data} into the sink SRAM
(snk_sram_controller)
        |
        v
5. axi_write_engine: aligned AWADDR, capped burst, WSTRB from the SRAM
entry
        |
        v
6. System memory; B response commits the beats and completes the
descriptor
```

### 3.3.3 Source Path (Memory to Network)

### 3.3.4 Figure 1.1.3: Source Path Data Flow

```
1. Scheduler loads a descriptor, computes beats_total, pulses
sched_rd_pkt_valid
        |
        v
2. axi_read_engine: aligned ARADDR, capped burst, beats into the
source SRAM
        |
        v
3. src_data_path_axis reserves drain blocks (round-robin,
cfg_drain_size beats)
        |
        v
4. Egress shifter: memory beat >> (offset*8), OR the next beat's low
lanes
```

```
        |   the first pop only primes the hold; the last beat is
partial
        v
5. m_axis: TDATA packed from lane 0, TSTRB contiguous, TLAST on the
final byte
```

---

## 3.4 Byte-Granular Design Points

*Table 1.1.2: Design Points*

| Point | Rule |
|---|---|
| Length unit | Bytes, in the existing 32-bit descriptor field |
| Beats moved | `(offset + length + BYTE_LANES - 1) >> OFF_W`, zero when length is zero |
| First burst | May begin mid-beat; the address it presents is beat-aligned and the offset is carried by the record |
| Later bursts | Beat-aligned |
| 4 KB rule | A burst is truncated at the next 4 KB boundary |
| Sink lanes not written | `WSTRB` bit is 0 |
| Source last beat | Partial; `TSTRB` marks the valid low lanes, `TLAST` set |
| TYPE=EXT | Beat-aligned by design (permanent): aligned addresses, beat-multiple lengths |

The 4 KB rule and the byte math are specified in Scheduler and AXI Read Engine.

---

## 3.5 Chapter Map

| Topic | Chapter |
|---|---|
| Scheduler byte math and record issue | Scheduler |
| Record queue interface | Packet Record Interface |
| Ingress shifter | Sink Data Path AXIS |
| Egress shifter | Source Data Path AXIS |
| Everything else | Linked from the RAPIDS Beats MAS via the index |

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

# 4 Top-Level Port List

**Module:** `rapids_core_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro_beats/` **Status:** Implemented **Last Updated:** 2026-10-16

---

## 4.1 Parameters

Every parameter of `rapids_core_beats`, as declared. The short aliases at the end are what the port widths below are written in (`NC`, `AW`, `DW`, `IW`, `SCW`, `CIW`, `SW`).

*Table 1.2.1: Parameters (from `rapids_core_beats.sv`)*

| Parameter | Type | Default | Notes |
|---|---|---|---|
| `NUM_CHANNELS` | int | `8` | - |
| `CHAN_WIDTH` | int | `$clog2(NUM_CHANNELS)` | - |
| `ADDR_WIDTH` | int | `64` | - |
| `DATA_WIDTH` | int | `512` | - |
| `AXI_ID_WIDTH` | int | `8` | - |
| `SRAM_DEPTH` | int | `512` | - |
| `SEG_COUNT_WIDTH` | int | `$clog2(SRAM_DEPTH) + 1` | - |
| `PIPELINE` | int | `1` | write-engine pipelining; 1 is the perf design point (see rapids_beats_top) |
| `AR_MAX_OUTSTANDING` | int | `8` | - |
| `AW_MAX_OUTSTANDING` | int | `8` | - |
| `W_PHASE_FIFO_DEPTH` | int | `64` | - |
| `B_PHASE_FIFO_DEPTH` | int | `16` | - |
| `AXIS_ID_WIDTH` | int | `8` | - |
| `AXIS_DEST_WIDTH` | int | `4` | - |
| `AXIS_USER_WIDTH` | int | `1` | - |
| `DESC_MON_BASE_AGENT_ID` | int | `16` | 0x10 - Descriptor Engines (16-23) |
| `SCHED_MON_BASE_AGENT_ID` | int | `48` | 0x30 - Schedulers (48-55) |
| `DESC_AXI_MON_AGE` | int | `8` | 0x08 - Descriptor |

| Parameter | Type | Default | Notes |
|---|---|---|---|
| NT_ID | | | AXI Master Monitor |
| MON_UNIT_ID | int | 1 | 0x1 |
| MON_MAX_TRANSACTIONS | int | 16 | - |
| USE_AXI_MONITORS | int | 1 | - |
| USE_ROW_COL_MAJOR_ADDRESSING | int | 1 | - |
| GEN_MON | bit | 1'b1 | - |
| NC | int | NUM_CHANNELS | - |
| AW | int | ADDR_WIDTH | - |
| DW | int | DATA_WIDTH | - |
| IW | int | AXI_ID_WIDTH | - |
| SD | int | SRAM_DEPTH | - |
| SCW | int | SEG_COUNT_WIDTH | - |
| CIW | int | (NC > 1) ? $clog2(NC) : 1 | - |
| SW | int | DW / 8 | - |

## 4.2 Port List (324 ports)

The module is two independent halves. Ports both halves carry are prefixed `src_` / `snk_`; the direction-unique ports (the source's AXI read master and AXIS egress, the sink's AXI write master and AXIS ingress) carry no prefix. Sections follow the declaration order in the RTL, so a reader can walk this page and the module side by side. Widths use the parameter aliases.

### 4.2.1 Clock and Reset (shared - single clock/reset for the whole wrapper)

*Table 1.2.2: Clock and Reset (shared - single clock/reset for the whole wrapper)*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `clk` | input | 1 | - |
| `rst_n` | input | 1 | - |

## 4.3 SOURCE HALF (u_src) - shared-infrastructure ports (src_ prefixed)

### 4.3.1 APB Programming Interface

*Table 1.2.3: APB Programming Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_apb_valid | input | NC | - |
| src_apb_ready | output | NC | - |
| src_apb_addr | input | NC x AW | - |

### 4.3.2 Per-channel configuration

*Table 1.2.4: Per-channel configuration*

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_cfg_channel_enable | input | NC | - |
| src_cfg_channel_reset | input | NC | - |

### 4.3.3 Scheduler Configuration (global)

*Table 1.2.5: Scheduler Configuration (global)*

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_cfg_sched_enable | input | 1 | - |
| src_cfg_sched_timeout_cycles | input | 32 | - |
| src_cfg_sched_timeout_limit | input | 8 | - |
| src_cfg_sched_timeout_enable | input | 1 | - |
| src_cfg_sched_err_enable | input | 1 | - |
| src_cfg_sched_compl_enable | input | 1 | - |

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_cfg_sched_perf_enable | input | 1 | - |

### 4.3.4 Descriptor Engine Configuration (global)

*Table 1.2.6: Descriptor Engine Configuration (global)*

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_cfg_descEng_enable | input | 1 | - |
| src_cfg_descEng_prefetch | input | 1 | - |
| src_cfg_descEng_fifo_thresh | input | 4 | - |
| src_cfg_descEng_addr0_base | input | AW | - |
| src_cfg_descEng_addr0_limit | input | AW | - |
| src_cfg_descEng_addr1_base | input | AW | - |
| src_cfg_descEng_addr1_limit | input | AW | - |

### 4.3.5 Control Engine Configuration (Phase 2, global)

*Table 1.2.7: Control Engine Configuration (Phase 2, global)*

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_cfg_ctrlrd_max_try | input | 9 | - |
| src_tick_1us | input | 1 | - |

### 4.3.6 Descriptor AXI Monitor Configuration

*Table 1.2.8: Descriptor AXI Monitor Configuration*

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_cfg_desc_mon_enable | input | 1 | - |
| src_cfg_desc_mon_err_enable | input | 1 | - |

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_cfg_desc_mon_perf_enable | input | 1 | - |
| src_cfg_desc_mon_perf_run | input | 1 | - |
| src_cfg_desc_mon_timeout_enable | input | 1 | - |
| src_cfg_desc_mon_timeout_cycles | input | 32 | - |
| src_cfg_desc_mon_latency_thresh | input | 32 | - |
| src_cfg_desc_mon_pkt_mask | input | 16 | - |
| src_cfg_desc_mon_err_select | input | 4 | - |
| src_cfg_desc_mon_err_mask | input | 8 | - |
| src_cfg_desc_mon_timeout_mask | input | 8 | - |
| src_cfg_desc_mon_compl_mask | input | 8 | - |
| src_cfg_desc_mon_thresh_mask | input | 8 | - |
| src_cfg_desc_mon_perf_mask | input | 8 | - |
| src_cfg_desc_mon_addr_mask | input | 8 | - |
| src_cfg_desc_mon_debug_mask | input | 8 | - |

### 4.3.7 Network-plane monitor input (rapids TASK-024)

The network endpoint and its monitor-lite live in the plane (`rtl/plane/`) in the top. Its monbus packets enter here as client 1 of this half's monbus arbiter; the plane's monitor configuration and status come from this half's register block in the top.

*Table 1.2.9: Network-plane monitor input (rapids TASK-024)*

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_plane_mon_valid | input | 1 | - |
| src_plane_mon_ready | output | 1 | - |
| src_plane_mon_packet | input | 128 | monitor_packet_t |
| src_plane_mon_timestamp | input | 64 | monbus_timestamp_t |

### 4.3.8 Status

*Table 1.2.10: Status*

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_system_idle | output | 1 | - |
| src_descriptor_engine_idle | output | NC | - |
| src_scheduler_idle | output | NC | - |
| src_scheduler_state | output | NC x 7 | - |
| src_sched_error | output | NC | - |

### 4.3.9 Descriptor AXI Monitor Status

*Table 1.2.11: Descriptor AXI Monitor Status*

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_cfg_sts_desc_mon_busy | output | 1 | - |
| src_cfg_sts_desc_mon_active_txns | output | 8 | - |
| src_cfg_sts_desc_mon_error_count | output | 16 | - |
| src_cfg_sts_desc_mon_txn_count | output | 32 | - |
| src_cfg_sts_d | output | 1 | - |

| Signal | Direction | Width | Description |
|---|---|---|---|
| esc_mon_conflict_error | | | |

### 4.3.10 Descriptor AXI Monitor perf window (feeds SRC_.MON.DAXMON_PERF_*).

*Table 1.2.12: Descriptor AXI Monitor perf window (feeds SRC_.MON.DAXMON_PERF_*).*

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_sts_desc_mon_win_active | output | 1 | - |
| src_sts_desc_mon_win_cycles | output | 32 | - |
| src_sts_desc_mon_prod_cycles | output | 32 | - |
| src_sts_desc_mon_bp_cycles | output | 32 | - |
| src_sts_desc_mon_starv_cycles | output | 32 | - |
| src_sts_desc_mon_idle_cycles | output | 32 | - |
| src_sts_desc_mon_beat_count | output | 32 | - |
| src_sts_desc_mon_byte_count | output | 64 | - |
| src_sts_desc_mon_burst_count | output | 32 | - |

### 4.3.11 Descriptor Fetch AXI Master

*Table 1.2.13: Descriptor Fetch AXI Master*

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_m_axi_desc_arvalid | output | 1 | - |

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_m_axi_desc_arready | input | 1 | - |
| src_m_axi_desc_araddr | output | AW | - |
| src_m_axi_desc_arlen | output | 8 | - |
| src_m_axi_desc_arsize | output | 3 | - |
| src_m_axi_desc_arburst | output | 2 | - |
| src_m_axi_desc_arid | output | IW | - |
| src_m_axi_desc_arlock | output | 1 | - |
| src_m_axi_desc_arcache | output | 4 | - |
| src_m_axi_desc_arprot | output | 3 | - |
| src_m_axi_desc_arqos | output | 4 | - |
| src_m_axi_desc_arregion | output | 4 | - |
| src_m_axi_desc_rvalid | input | 1 | - |
| src_m_axi_desc_rready | output | 1 | - |
| src_m_axi_desc_rdata | input | 256 | - |
| src_m_axi_desc_rresp | input | 2 | - |
| src_m_axi_desc_rlast | input | 1 | - |
| src_m_axi_desc_rid | input | IW | - |

### 4.3.12 Control Read AXI Master (32-bit) [Phase 2]

*Table 1.2.14: Control Read AXI Master (32-bit) [Phase 2]*

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_m_axi_ctrlrd_arvalid | output | 1 | - |

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_m_axi_ctrlrd_arready | input | 1 | - |
| src_m_axi_ctrlrd_araddr | output | AW | - |
| src_m_axi_ctrlrd_arlen | output | 8 | - |
| src_m_axi_ctrlrd_arsize | output | 3 | - |
| src_m_axi_ctrlrd_arburst | output | 2 | - |
| src_m_axi_ctrlrd_arid | output | IW | - |
| src_m_axi_ctrlrd_arlock | output | 1 | - |
| src_m_axi_ctrlrd_arcache | output | 4 | - |
| src_m_axi_ctrlrd_arprot | output | 3 | - |
| src_m_axi_ctrlrd_arqos | output | 4 | - |
| src_m_axi_ctrlrd_arregion | output | 4 | - |
| src_m_axi_ctrlrd_rvalid | input | 1 | - |
| src_m_axi_ctrlrd_rready | output | 1 | - |
| src_m_axi_ctrlrd_rdata | input | 32 | - |
| src_m_axi_ctrlrd_rresp | input | 2 | - |
| src_m_axi_ctrlrd_rlast | input | 1 | - |
| src_m_axi_ctrlrd_rid | input | IW | - |

### 4.3.13 Control Write AXI Master (32-bit) [Phase 2]

*Table 1.2.15: Control Write AXI Master (32-bit) [Phase 2]*

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_m_axi_ctrlwr_awvalid | output | 1 | - |

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_m_axi_ctrlwr_awready | input | 1 | - |
| src_m_axi_ctrlwr_awaddr | output | AW | - |
| src_m_axi_ctrlwr_awlen | output | 8 | - |
| src_m_axi_ctrlwr_awsize | output | 3 | - |
| src_m_axi_ctrlwr_awburst | output | 2 | - |
| src_m_axi_ctrlwr_awid | output | IW | - |
| src_m_axi_ctrlwr_awlock | output | 1 | - |
| src_m_axi_ctrlwr_awcache | output | 4 | - |
| src_m_axi_ctrlwr_awprot | output | 3 | - |
| src_m_axi_ctrlwr_awqos | output | 4 | - |
| src_m_axi_ctrlwr_awregion | output | 4 | - |
| src_m_axi_ctrlwr_wvalid | output | 1 | - |
| src_m_axi_ctrlwr_wready | input | 1 | - |
| src_m_axi_ctrlwr_wdata | output | 32 | - |
| src_m_axi_ctrlwr_wstrb | output | 4 | - |
| src_m_axi_ctrlwr_wlast | output | 1 | - |
| src_m_axi_ctrlwr_bvalid | input | 1 | - |
| src_m_axi_ctrlwr_bready | output | 1 | - |
| src_m_axi_ctrlwr_bid | input | IW | - |
| src_m_axi_ctrlwr_bresp | input | 2 | - |

## 4.4 SOURCE HALF (u_src) - direction-unique ports (no prefix)

### 4.4.1 AXI Transfer Configuration (source-only)

*Table 1.2.16: AXI Transfer Configuration (source-only)*

| Signal | Direction | Width | Description |
|---|---|---|---|
| cfg_axi_rd_xfer_beats | input | 8 | - |
| cfg_drain_size | input | 8 | source: beats drained per AXIS packet |

### 4.4.2 Source Path - AXIS Master Interface (SRAM -> Network); tid = channel id

*Table 1.2.17: Source Path - AXIS Master Interface (SRAM -> Network); tid = channel id*

| Signal | Direction | Width | Description |
|---|---|---|---|
| m_axis_tdata | output | DW | - |
| m_axis_tstrb | output | SW | - |
| m_axis_tlast | output | 1 | - |
| m_axis_tid | output | AXIS_ID_WIDTH | - |
| m_axis_tdest | output | AXIS_DEST_WIDTH | - |
| m_axis_tuser | output | AXIS_USER_WIDTH | - |
| m_axis_tvalid | output | 1 | - |
| m_axis_tready | input | 1 | - |

### 4.4.3 AXI4 Master - Data Read (Memory -> Source SRAM)

*Table 1.2.18: AXI4 Master - Data Read (Memory -> Source SRAM)*

| Signal | Direction | Width | Description |
|---|---|---|---|
| m_axi_rd_arid | output | IW | - |
| m_axi_rd_araddr | output | AW | - |
| m_axi_rd_arle | output | 8 | - |

| Signal | Direction | Width | Description |
|---|---|---|---|
| n | | | |
| m_axi_rd_arsize | output | 3 | - |
| m_axi_rd_arburst | output | 2 | - |
| m_axi_rd_arvalid | output | 1 | - |
| m_axi_rd_arready | input | 1 | - |
| m_axi_rd_rid | input | IW | - |
| m_axi_rd_rdata | input | DW | - |
| m_axi_rd_rresp | input | 2 | - |
| m_axi_rd_rlast | input | 1 | - |
| m_axi_rd_rvalid | input | 1 | - |
| m_axi_rd_rready | output | 1 | - |

## 4.5 SINK HALF (u_snk) - shared-infrastructure ports (snk_ prefixed)

### 4.5.1 APB Programming Interface

*Table 1.2.19: APB Programming Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_apb_valid | input | NC | - |
| snk_apb_ready | output | NC | - |
| snk_apb_addr | input | NC x AW | - |

### 4.5.2 Per-channel configuration

*Table 1.2.20: Per-channel configuration*

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_cfg_channel_enable | input | NC | - |
| snk_cfg_channel_reset | input | NC | - |

### 4.5.3 Scheduler Configuration (global)

*Table 1.2.21: Scheduler Configuration (global)*

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_cfg_sched_enable | input | 1 | - |
| snk_cfg_sched_timeout_cycles | input | 32 | - |
| snk_cfg_sched_timeout_limit | input | 8 | - |
| snk_cfg_sched_timeout_enable | input | 1 | - |
| snk_cfg_sched_err_enable | input | 1 | - |
| snk_cfg_sched_compl_enable | input | 1 | - |

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_cfg_sched_perf_enable | input | 1 | - |

### 4.5.4 Descriptor Engine Configuration (global)

*Table 1.2.22: Descriptor Engine Configuration (global)*

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_cfg_descneng_enable | input | 1 | - |
| snk_cfg_descneng_prefetch | input | 1 | - |
| snk_cfg_descneng_fifo_thresh | input | 4 | - |
| snk_cfg_descneng_addr0_base | input | AW | - |
| snk_cfg_descneng_addr0_limit | input | AW | - |
| snk_cfg_descneng_addr1_base | input | AW | - |
| snk_cfg_descneng_addr1_limit | input | AW | - |

### 4.5.5 Control Engine Configuration (Phase 2, global)

*Table 1.2.23: Control Engine Configuration (Phase 2, global)*

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_cfg_ctrlrd_max_try | input | 9 | - |
| snk_tick_1us | input | 1 | - |

### 4.5.6 Descriptor AXI Monitor Configuration

*Table 1.2.24: Descriptor AXI Monitor Configuration*

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_cfg_desc_mon_enable | input | 1 | - |
| snk_cfg_desc_mon_err_enable | input | 1 | - |

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_cfg_desc_mon_perf_enable | input | 1 | - |
| snk_cfg_desc_mon_perf_run | input | 1 | - |
| snk_cfg_desc_mon_timeout_enable | input | 1 | - |
| snk_cfg_desc_mon_timeout_cycles | input | 32 | - |
| snk_cfg_desc_mon_latency_thresh | input | 32 | - |
| snk_cfg_desc_mon_pkt_mask | input | 16 | - |
| snk_cfg_desc_mon_err_select | input | 4 | - |
| snk_cfg_desc_mon_err_mask | input | 8 | - |
| snk_cfg_desc_mon_timeout_mask | input | 8 | - |
| snk_cfg_desc_mon_compl_mask | input | 8 | - |
| snk_cfg_desc_mon_thresh_mask | input | 8 | - |
| snk_cfg_desc_mon_perf_mask | input | 8 | - |
| snk_cfg_desc_mon_addr_mask | input | 8 | - |
| snk_cfg_desc_mon_debug_mask | input | 8 | - |

### 4.5.7 Network-plane monitor input (rapids TASK-024)

The network endpoint and its monitor-lite live in the plane (`rtl/plane/`) in the top. Its monbus packets enter here as client 1 of this half's monbus arbiter; the plane's monitor configuration and status come from this half's register block in the top.

*Table 1.2.25: Network-plane monitor input (rapids TASK-024)*

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_plane_mon_valid | input | 1 | - |
| snk_plane_mon_ready | output | 1 | - |
| snk_plane_mon_packet | input | 128 | monitor_packet_t |
| snk_plane_mon_timestamp | input | 64 | monbus_timestamp_t |

### 4.5.8 Status

*Table 1.2.26: Status*

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_system_idle | output | 1 | - |
| snk_descriptor_engine_idle | output | NC | - |
| snk_scheduler_idle | output | NC | - |
| snk_scheduler_state | output | NC x 7 | - |
| snk_sched_error | output | NC | - |

### 4.5.9 Descriptor AXI Monitor Status

*Table 1.2.27: Descriptor AXI Monitor Status*

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_cfg_sts_desc_mon_busy | output | 1 | - |
| snk_cfg_sts_desc_mon_active_txns | output | 8 | - |
| snk_cfg_sts_desc_mon_error_count | output | 16 | - |
| snk_cfg_sts_desc_mon_txn_count | output | 32 | - |
| snk_cfg_sts_d | output | 1 | - |

| Signal | Direction | Width | Description |
|---|---|---|---|
| esc_mon_conflict_error | | | |

### 4.5.10 Descriptor AXI Monitor perf window (feeds SNK_.MON.DAXMON_PERF_*).

*Table 1.2.28: Descriptor AXI Monitor perf window (feeds SNK_.MON.DAXMON_PERF_*).*

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_sts_desc_mon_win_active | output | 1 | - |
| snk_sts_desc_mon_win_cycles | output | 32 | - |
| snk_sts_desc_mon_prod_cycles | output | 32 | - |
| snk_sts_desc_mon_bp_cycles | output | 32 | - |
| snk_sts_desc_mon_starv_cycles | output | 32 | - |
| snk_sts_desc_mon_idle_cycles | output | 32 | - |
| snk_sts_desc_mon_beat_count | output | 32 | - |
| snk_sts_desc_mon_byte_count | output | 64 | - |
| snk_sts_desc_mon_burst_count | output | 32 | - |

### 4.5.11 Descriptor Fetch AXI Master

*Table 1.2.29: Descriptor Fetch AXI Master*

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_m_axi_desc_arvalid | output | 1 | - |

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_m_axi_desc_arready | input | 1 | - |
| snk_m_axi_desc_araddr | output | AW | - |
| snk_m_axi_desc_arlen | output | 8 | - |
| snk_m_axi_desc_arsize | output | 3 | - |
| snk_m_axi_desc_arburst | output | 2 | - |
| snk_m_axi_desc_arid | output | IW | - |
| snk_m_axi_desc_arlock | output | 1 | - |
| snk_m_axi_desc_arcache | output | 4 | - |
| snk_m_axi_desc_arprot | output | 3 | - |
| snk_m_axi_desc_arqos | output | 4 | - |
| snk_m_axi_desc_arregion | output | 4 | - |
| snk_m_axi_desc_rvalid | input | 1 | - |
| snk_m_axi_desc_rready | output | 1 | - |
| snk_m_axi_desc_rdata | input | 256 | - |
| snk_m_axi_desc_rresp | input | 2 | - |
| snk_m_axi_desc_rlast | input | 1 | - |
| snk_m_axi_desc_rid | input | IW | - |

### 4.5.12 Control Read AXI Master (32-bit) [Phase 2]

*Table 1.2.30: Control Read AXI Master (32-bit) [Phase 2]*

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_m_axi_ctrlrd_arvalid | output | 1 | - |

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_m_axi_ctrlrd_arready | input | 1 | - |
| snk_m_axi_ctrlrd_araddr | output | AW | - |
| snk_m_axi_ctrlrd_arlen | output | 8 | - |
| snk_m_axi_ctrlrd_arsize | output | 3 | - |
| snk_m_axi_ctrlrd_arburst | output | 2 | - |
| snk_m_axi_ctrlrd_arid | output | IW | - |
| snk_m_axi_ctrlrd_arlock | output | 1 | - |
| snk_m_axi_ctrlrd_arcache | output | 4 | - |
| snk_m_axi_ctrlrd_arprot | output | 3 | - |
| snk_m_axi_ctrlrd_arqos | output | 4 | - |
| snk_m_axi_ctrlrd_arregion | output | 4 | - |
| snk_m_axi_ctrlrd_rvalid | input | 1 | - |
| snk_m_axi_ctrlrd_rready | output | 1 | - |
| snk_m_axi_ctrlrd_rdata | input | 32 | - |
| snk_m_axi_ctrlrd_rresp | input | 2 | - |
| snk_m_axi_ctrlrd_rlast | input | 1 | - |
| snk_m_axi_ctrlrd_rid | input | IW | - |

### 4.5.13 Control Write AXI Master (32-bit) [Phase 2]

*Table 1.2.31: Control Write AXI Master (32-bit) [Phase 2]*

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_m_axi_ctrlwr_awvalid | output | 1 | - |

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_m_axi_ctrlwr_awready | input | 1 | - |
| snk_m_axi_ctrlwr_awaddr | output | AW | - |
| snk_m_axi_ctrlwr_awlen | output | 8 | - |
| snk_m_axi_ctrlwr_awsize | output | 3 | - |
| snk_m_axi_ctrlwr_awburst | output | 2 | - |
| snk_m_axi_ctrlwr_awid | output | IW | - |
| snk_m_axi_ctrlwr_awlock | output | 1 | - |
| snk_m_axi_ctrlwr_awcache | output | 4 | - |
| snk_m_axi_ctrlwr_awprot | output | 3 | - |
| snk_m_axi_ctrlwr_awqos | output | 4 | - |
| snk_m_axi_ctrlwr_awregion | output | 4 | - |
| snk_m_axi_ctrlwr_wvalid | output | 1 | - |
| snk_m_axi_ctrlwr_wready | input | 1 | - |
| snk_m_axi_ctrlwr_wdata | output | 32 | - |
| snk_m_axi_ctrlwr_wstrb | output | 4 | - |
| snk_m_axi_ctrlwr_wlast | output | 1 | - |
| snk_m_axi_ctrlwr_bvalid | input | 1 | - |
| snk_m_axi_ctrlwr_bready | output | 1 | - |
| snk_m_axi_ctrlwr_bid | input | IW | - |
| snk_m_axi_ctrlwr_bresp | input | 2 | - |

## 4.6 Monitor Bus (SINGLE aggregated stream for the whole core)

The two halves' monitor outputs are merged through a top-level monbus_arbiter, so the core exposes exactly one monitor stream.

### 4.6.1 Monitor Bus

*Table 1.2.32: Monitor Bus*

| Signal | Direction | Width | Description |
|---|---|---|---|
| mon_valid | output | 1 | - |
| mon_ready | input | 1 | - |
| mon_packet | output | ocelus_common_pkg::monitor_packet_t | - |
| mon_timestamp | output | ocelus_common_pkg::monbus_timestamp_t | - |

## 4.7 SINK HALF (u_snk) - direction-unique ports (no prefix)

### 4.7.1 AXI Transfer Configuration (sink-only)

*Table 1.2.33: AXI Transfer Configuration (sink-only)*

| Signal | Direction | Width | Description |
|---|---|---|---|
| cfg_axi_wr_xfer_beats | input | 8 | - |
| cfg_alloc_size | input | 8 | sink: SRAM alloc size per AXIS fill |

### 4.7.2 Sink Path - AXIS Slave Interface (Network -> SRAM); tid = channel id

*Table 1.2.34: Sink Path - AXIS Slave Interface (Network -> SRAM); tid = channel id*

| Signal | Direction | Width | Description |
|---|---|---|---|
| s_axis_tdata | input | DW | - |
| s_axis_tstrb | input | SW | - |
| s_axis_tlast | input | 1 | - |
| s_axis_tid | input | AXIS_ID_WIDTH | - |
| s_axis_tdest | input | AXIS_DEST_WIDTH | - |
| s_axis_tuser | input | AXIS_USER_WIDTH | - |
| s_axis_tvalid | input | 1 | - |
| s_axis_tready | output | 1 | - |

### 4.7.3 AXI4 Master - Data Write (Sink SRAM -> Memory)

*Table 1.2.35: AXI4 Master - Data Write (Sink SRAM -> Memory)*

| Signal | Direction | Width | Description |
|---|---|---|---|
| m_axi_wr_awid | output | IW | - |
| m_axi_wr_awaddr | output | AW | - |
| m_axi_wr_awlen | output | 8 | - |

| Signal | Direction | Width | Description |
|---|---|---|---|
| m_axi_wr_awsize | output | 3 | - |
| m_axi_wr_awburst | output | 2 | - |
| m_axi_wr_awlock | output | 1 | - |
| m_axi_wr_awcache | output | 4 | - |
| m_axi_wr_awprot | output | 3 | - |
| m_axi_wr_awqos | output | 4 | - |
| m_axi_wr_awregion | output | 4 | - |
| m_axi_wr_awvalid | output | 1 | - |
| m_axi_wr_awready | input | 1 | - |
| m_axi_wr_wdata | output | DW | - |
| m_axi_wr_wstrb | output | (DW/8) | - |
| m_axi_wr_wlast | output | 1 | - |
| m_axi_wr_wvalid | output | 1 | - |
| m_axi_wr_wready | input | 1 | - |
| m_axi_wr_bid | input | IW | - |
| m_axi_wr_bresp | input | 2 | - |
| m_axi_wr_bvalid | input | 1 | - |
| m_axi_wr_bready | output | 1 | - |

## 4.8 Debug Interface (per half, src_dbg_ / snk_dbg_ prefixed)

### 4.8.1 Source half debug

*Table 1.2.36: Source half debug*

| Signal | Direction | Width | Description |
|---|---|---|---|
| src_dbg_rd_all_complete | output | NC | - |
| src_dbg_r_beats_rcvd | output | 32 | - |
| src_dbg_sram_writes | output | 32 | - |
| src_dbg_arb_request | output | NC | - |
| src_dbg_src_sram_bridge_pending | output | NC | - |
| src_dbg_src_sram_bridge_out_valid | output | NC | - |
| src_dbg_axis_beats_sent | output | 32 | - |
| src_dbg_axis_packets_sent | output | 32 | - |

### 4.8.2 Sink half debug

*Table 1.2.37: Sink half debug*

| Signal | Direction | Width | Description |
|---|---|---|---|
| snk_dbg_snk_sram_bridge_pending | output | NC | - |
| snk_dbg_snk_sram_bridge_out_valid | output | NC | - |
| snk_dbg_axis_beats_received | output | 32 | - |
| snk_dbg_axis_packets_received | output | 32 | - |

### 4.8.3 Active-channel sideband for per-channel bus instrumentation

(axi_bus_meter). The W bus carries no wid, so the write engine's channel index must travel out of band to reach the meter at the top.

*Table 1.2.38: Active-channel sideband for per-channel bus instrumentation*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `snk_active_channel_id` | output | CIW | - |
| `snk_active_channel_valid` | output | 1 | - |

**Verified:** every row above is a declared port of `rapids_core_beats` and every declared port has a row (324 of 324), generated from the module declaration on 2026-09-28. Regenerate rather than hand-edit when the interface changes.

# 5 Clocks and Reset

**Module:** `rapids_core_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

---

## 5.1 Clock Domain

RAPIDS Beats operates in a single clock domain for simplified design and maximum throughput.

### 5.1.1 Clock Specifications

*Table 1.3.1: Clock Specifications*

| Parameter | Specification |
|---|---|
| Clock Signal | `clk` (also `axi_aclk` in some submodules) |
| Frequency Range | 100 - 500 MHz |
| Duty Cycle | 45% - 55% |
| Clock Jitter | < 100 ps peak-to-peak |

### 5.1.2 Clock Distribution

```
clk (one clock domain; rst_n active-low, async assert)
├── scheduler_group_array_beats (one per half)
│   └── descriptor_engine_beats, scheduler_beats, ctrlrd_engine,
ctrlwr_engine (x8)
├── snk_data_path_axis_beats
│   └── snk_sram_controller_beats, axi_write_engine_beats
└── src_data_path_axis_beats
    └── axi_read_engine_beats, src_sram_controller_beats
```

---

## 5.2 Reset Specification

RAPIDS uses asynchronous assert, synchronous deassert reset methodology.

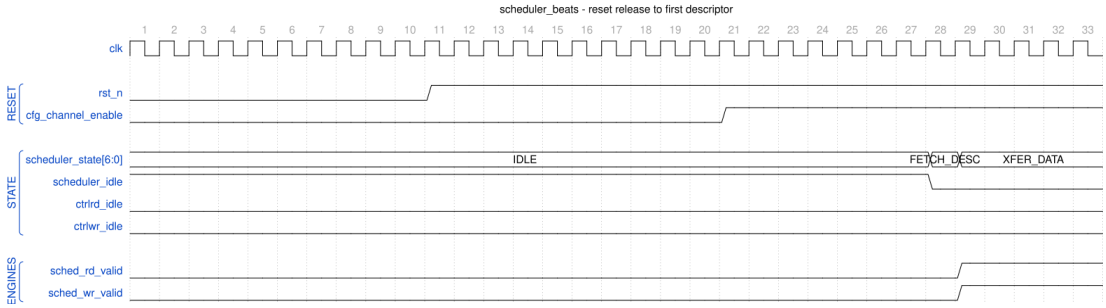
### 5.2.1 Reset Signal

*Table 1.3.2: Reset Specifications*

| Parameter | Specification |
|---|---|
| Reset Signal | `rst_n` (also `axi_aresetn` in some submodules) |
| Polarity | Active-low |
| Assert | Asynchronous (immediate) |
| Deassert | Synchronous (on rising clock edge) |
| Minimum Duration | 10 clock cycles |

### 5.2.2 Reset Timing Diagram

### 5.2.3 Figure 1.3.1: Reset Timing




*scheduler_beats - reset release to first descriptor*

**Source:** scheduler_reset.json, captured from `dv/tests/fub_beats/test_scheduler_beats.py` (basic transfer, the same run as Figure 2.1.3) with `WAVES=1`.

Reading it: `rst_n` is held low for ten cycles and released on a clock edge. Every state register is already at its reset value while reset is asserted – the FSM sits in `IDLE`, `scheduler_idle` is high, both control engines report idle and neither engine request is raised. Nothing changes at the release edge itself; the scheduler stays in `IDLE` until the testbench raises `cfg_channel_enable` ten cycles later and a descriptor becomes available, at which point it steps through `FETCH_DESC` to `XFER_DATA` and raises the read and write requests together.

---

## 5.3 Reset Behavior

### 5.3.1 On Reset Assert

All modules initialize to known states:

*Table 1.3.3: Reset States*

| Component | Reset State |
|---|---|
| Scheduler FSM | IDLE |
| Descriptor Engine | IDLE, FIFOs empty |
| AXI Read Engine | Idle, no outstanding transactions |
| AXI Write Engine | Idle, no outstanding transactions |
| SRAM Pointers | All zeros |
| Alloc/Drain Controllers | Empty, full space available |
| MonBus | No packets pending |

### 5.3.2 Reset Sequence

```
1. External reset asserted (rst_n = 0)
   - All flip-flops async reset
   - All outputs driven to safe states
   - No AXI transactions

2. System stabilization (minimum 10 cycles)
   - Clock running and stable
   - Configuration registers programmed (optional)

3. Reset deasserted (rst_n = 1, sync to clk rising edge)
   - FSMs begin operation
   - Ready to accept descriptors/data
```

---

## 5.4 Configuration During Reset

Some configuration signals MUST be stable before reset deassertion:

*Table 1.3.4: Configuration Timing Requirements*

| Signal | Requirement |
|---|---|
| `src_cfg_channel_enable` / `snk_cfg_channel_enable` | Stable before rst_n=1 (one per half) |
| `src_cfg_sched_timeout_cycles` / `snk_cfg_sched_timeout_cycles` | Stable before rst_n=1 (one per half) |
| `cfg_axi_rd_xfer_beats` | Stable before rst_n=1 |
| `cfg_axi_wr_xfer_beats` | Stable before rst_n=1 |

## 5.5 Reset Macros

RAPIDS uses standardized reset macros from `reset_defs.svh`:

```
`include "reset_defs.svh"

// Standard reset pattern
`ALWAYS_FF_RST(clk, rst_n,
    if (`RST_ASSERTED(rst_n)) begin
        r_state <= IDLE;
        r_counter <= '0;
    end else begin
        r_state <= w_next_state;
        r_counter <= w_next_counter;
    end
)
```

---

## 5.6 Power-On Reset Considerations

- External reset should be held for minimum 10 clock cycles after clock is stable
- Configuration registers should be programmed before or during reset
- No AXI transactions until reset is deasserted and channels are enabled

---

**Last Updated:** 2025-01-10

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 6 Scheduler

**Module:** `scheduler.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub/` **Status:** Implemented **Last Updated:** 2026-10-21

---

## 6.1 Overview

The scheduler coordinates one channel. It accepts a 256-bit descriptor from the descriptor engine, turns the descriptor's byte length and byte addresses into beat counts for the AXI engines, tells the data paths how the packet is laid out, and tracks progress until every source beat is read and every destination beat is committed.

Compared with the RAPIDS Beats scheduler, the state machine, the timeout logic, the control-descriptor path and the MonBus events are the same. Three things change: the beat math, the address advance, and the packet record.

### 6.1.1 Structure: the FSM around scheduler_core

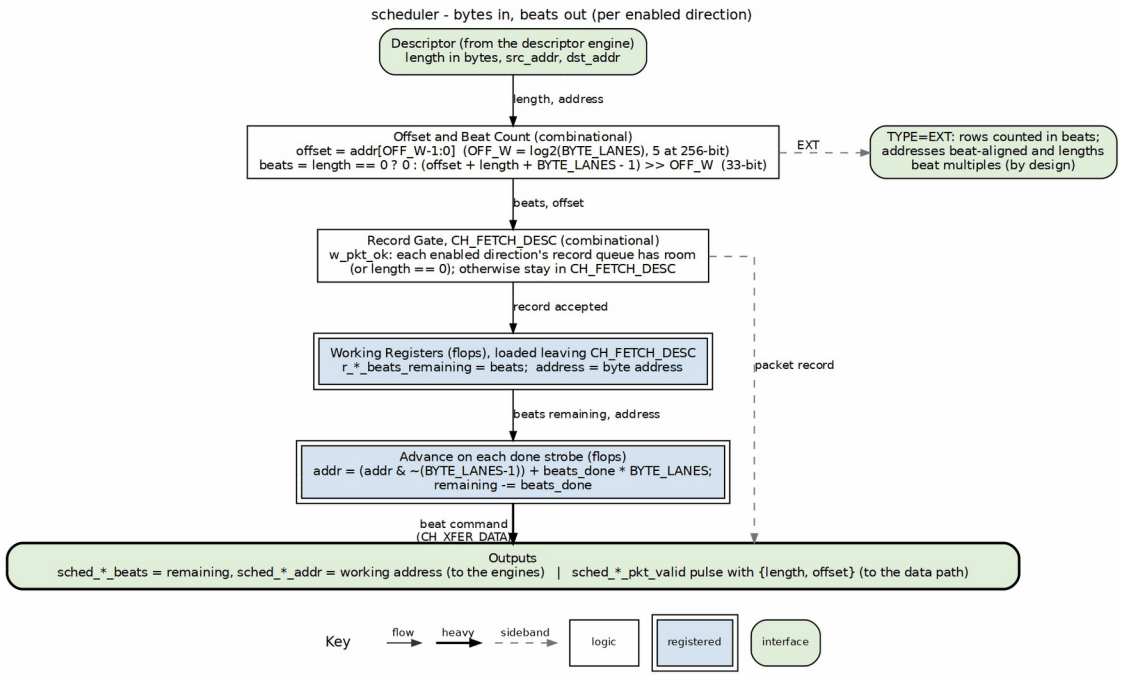
Since rapids TASK-034 the scheduler is two modules. `scheduler_core.sv` holds the datapath and nothing that decides: the descriptor skid and latch, the strobe-driven counters, the address advance, run accounting, timeouts, sticky errors, control-engine issue, tile checks, packet records and MonBus packets. Its state, its descriptor take, its beat totals and its EXT run bases are inputs. `scheduler.sv` keeps the port list it always had and supplies those inputs: the finite state machine below, the byte-to-beat ceiling and the two `stream_run_addr_gen` run-base generators. The gatling-sequenced `scheduler_g` (Chapter 5) wraps the same core and takes the four from firmware, so there is one copy of the datapath for both.

### 6.1.2 Key Features

- **Byte length:** the descriptor's 32-bit length is a byte count.
- **Byte addresses:** the offset of a transfer inside its first beat is `addr[OFF_W-1:0]`.
- **Beat math:** beats moved are derived from offset and length, in a 33-bit intermediate.
- **Packet record:** one `{bytes, offset}` pulse per DATA descriptor, per enabled direction, held back while the data path cannot take it.
- **Concurrent read and write:** both directions run inside one XFER_DATA state.
- **Recoverable write-progress timeout:** strikes escalate to a sticky error only when a limit is set.
- **Control descriptors:** CTRL_READ and CTRL_WRITE drive the control engines, unchanged.
- **Directional halves:** `EN_READ` and `EN_WRITE` select the source or sink half.

### 6.1.3 Block Diagram

### 6.1.4 Figure 2.1.1: Scheduler Byte Math




*Scheduler byte math*

**Source:** `assets/graphviz/03_scheduler_byte_math.dot`

---

## 6.2 Parameters

*Table 2.1.1: Scheduler Parameters*

| Parameter | Type | Default | Description |
|---|---|---|---|
| CHANNEL_ID | int | 0 | Channel index of this scheduler |
| NUM_CHANNELS | int | 8 | Channel count |
| CHAN_WIDTH | int | $clog2(NUM_CHANNELS) | Channel field width |
| ADDR_WIDTH | int | 64 | Address width |
| DATA_WIDTH | int | 512 | Data width in bits; the board design uses 256 |
| MON_AGENT_ID | logic [15:0] | 16'h0040 | MonBus agent ID |
| MON_UNIT_ID | logic [7:0] | 8'h01 | MonBus unit ID |
| MON_CHANNEL_ID | logic [8:0] | 9'h000 | MonBus base channel ID |
| DESC_WIDTH | int | 256 | Descriptor width; fixed at 256 (elaboration fails otherwise) |
| EN_READ | bit | 1 | Enable the read direction for DATA descriptors |
| EN_WRITE | bit | 1 | Enable the write direction for DATA descriptors |
| USE_ROW_COL_MAJOR_ADDRESSING | int | 1 | Enable TYPE=EXT run-base generators |
| BYTE_LANES | int | DATA_WIDTH/8 | Bytes per beat |
| OFF_W | int | (BYTE_LANES>1) ? $clog2(BYTE_LANES) : 1 | Offset field width |

BYTE_LANES and OFF_W are the two parameters that do not exist in the Beats scheduler. A directional half sets exactly one of EN_READ and EN_WRITE: the source half reads only, the sink half writes only.

## 6.3 Port List

### 6.3.1 Clock, Reset and Configuration

*Table 2.1.2: Clock, Reset and Configuration Ports*

| Signal | Dir | Width | Description |
|---|---|---|---|
| clk | in | 1 | Clock |
| rst_n | in | 1 | Active-low reset |
| cfg_channel_enable | in | 1 | Channel enable; IDLE leaves only when set |
| cfg_channel_reset | in | 1 | Channel reset; forces IDLE and clears sticky state |
| cfg_sched_timeout_cycles | in | 32 | Write-progress timeout window in cycles |
| cfg_sched_timeout_limit | in | 8 | Consecutive windows before escalation; 0 = never |
| cfg_sched_timeout_enable | in | 1 | Enable timeout detection |

### 6.3.2 Status

*Table 2.1.3: Status Ports*

| Signal | Dir | Width | Description |
|---|---|---|---|
| scheduler_idle | out | 1 | High in CH_IDLE with no channel reset pending and the descriptor skid empty |
| scheduler_state | out | 7 | One-hot state (see Table 2.1.12) |
| sched_error | out | 1 | Sticky, high in CH_ERROR |
| dbg_descriptor_error | out | 1 | Latched fatal descriptor-level error |
| dbg_read_error_sticky | out | 1 | Latched read-engine error |
| dbg_write_error_sticky | out | 1 | Latched write-engine error |
| dbg_timeout_expired | out | 1 | Live: the timeout window has elapsed |

### 6.3.3 Descriptor Engine Interface

*Table 2.1.4: Descriptor Engine Interface*

| Signal | Dir | Width | Description |
|---|---|---|---|
| descriptor_valid | in | 1 | A descriptor is available |
| descriptor_ready | out | 1 | The descriptor skid has room (see Descriptor Skid Buffer) |
| descriptor_packet | in | 256 | Descriptor (layout in Table 2.1.10) |
| descriptor_ext_packet | in | 256 | Extended chunk; used only when TYPE=EXT |
| descriptor_error | in | 1 | Descriptor fetch or parse error |

### 6.3.4 Descriptor Skid Buffer

Every scheduler takes its descriptors through a two-deep skid buffer (gaxi_skid_buffer, DEPTH 2) on {descriptor_packet, descriptor_ext_packet}, in front of the descriptor engine's per-channel FIFO, which stays (rapids TASK-028). The FSM reads the descriptor from the skid's flops rather than straight off the engine's 256-bit output, one engine for every channel under SHARED_ENGINES, and a running channel holds two more descriptors.

- descriptor_ready is the skid's wr_ready: high while it has room, whatever the FSM state. The FSM's own take (CH_IDLE while cfg_channel_enable is set, CH_NEXT_DESC) pops the skid.
- A disabled channel's descriptor waits in the skid (or behind it in the engine FIFO) until the channel is enabled; it is not consumed and dropped.
- scheduler_idle stays low while the skid holds a descriptor, so the engine and the scheduler never both read idle with work pending.
- Channel reset: the engine masks descriptor_valid during its reset, so nothing is written during one, and the skid drains while the reset is active. Each entry carries the reset epoch it was written in, and the epoch flips as a reset starts, so an entry a reset too short to drain it left behind is dropped at the head, never run.

### 6.3.5 Data Read Interface (to AXI Read Engine)

*Table 2.1.5: Data Read Interface*

| Signal | Dir | Width | Description |
|---|---|---|---|
| sched_rd_valid | out | 1 | Read request |
| sched_rd_addr | out | ADDR_WIDTH | Source byte address (working register) |
| sched_rd_beats | out | 32 | Beats remaining to read |
| sched_rd_pkt_valid | out | 1 | Packet record pulse to the source egress |
| sched_rd_pkt_ready | in | 1 | Source egress record queue has room |
| sched_rd_pkt_bytes | out | 32 | Descriptor length in bytes |
| sched_rd_pkt_offset | out | OFF_W | Source byte offset inside the first beat |

### 6.3.6 Data Write Interface (to AXI Write Engine)

*Table 2.1.6: Data Write Interface*

| Signal | Dir | Width | Description |
|---|---|---|---|
| sched_wr_valid | out | 1 | Write request |
| sched_wr_ready | in | 1 | Write engine can take the channel |
| sched_wr_addr | out | ADDR_WIDTH | Destination byte address (working register) |
| sched_wr_beats | out | 32 | Beats remaining to issue |
| sched_wr_pkt_valid | out | 1 | Packet record pulse to the sink ingress |
| sched_wr_pkt_ready | in | 1 | Sink ingress record queue has room |
| sched_wr_pkt_bytes | out | 32 | Descriptor length in bytes |
| sched_wr_pkt_offset | out | OFF_W | Destination byte offset inside the first beat |

### 6.3.7 Completion Interface

*Table 2.1.7: Completion and Error Interface*

| Signal | Dir | Width | Description |
|---|---|---|---|
| sched_rd_done_strobe | in | 1 | A read burst completed |
| sched_rd_beats_done | in | 32 | Beats in that burst |
| sched_wr_done_strobe | in | 1 | A write burst was issued (AW handshake) |
| sched_wr_beats_done | in | 32 | Beats in that burst |
| sched_wr_commit_strobe | in | 1 | A write burst was committed (B response) |
| sched_wr_commit_beats | in | 32 | Beats in that burst |
| sched_rd_error | in | 1 | Read engine error |
| sched_wr_error | in | 1 | Write engine error, including the sink packet length error |

### 6.3.8 Control Engine Interface

*Table 2.1.8: Control Engine Interface*

| Signal | Dir | Width | Description |
|---|---|---|---|
| ctrlrd_valid | out | 1 | CTRL_READ request |
| ctrlrd_ready | in | 1 | Engine accepted the request |
| ctrlrd_addr | out | ADDR_WIDTH | Poll address (descriptor source slot) |
| ctrlrd_data | out | 32 | Expected value (dst_addr[31:0]) |
| ctrlrd_mask | out | 32 | Compare mask (dst_addr[63:32]) |
| ctrlrd_error | in | 1 | Engine error (retries exhausted or AXI error) |
| ctrlrd_idle | in | 1 | Engine idle |
| ctrlwr_valid | out | 1 | CTRL_WRITE request |
| ctrlwr_ready | in | 1 | Engine accepted the request |

| Signal | Dir | Width | Description |
|---|---|---|---|
| ctrlwr_addr | out | ADDR_WIDTH | Doorbell address (descriptor source slot) |
| ctrlwr_data | out | 32 | Doorbell data (dst_addr[31:0]) |
| ctrlwr_error | in | 1 | Engine error |
| ctrlwr_idle | in | 1 | Engine idle |

### 6.3.9 Monitor Bus Interface

*Table 2.1.9: Monitor Bus Interface*

| Signal | Dir | Width | Description |
|---|---|---|---|
| i_mon_time | in | timestamp | Time base for packets |
| mon_valid | out | 1 | Packet valid |
| mon_ready | in | 1 | Consumer ready |
| mon_packet | out | 128 | Monitor packet |
| mon_timestamp | out | 64 | Side-band timestamp |

The control interface tables do not change from RAPIDS Beats. The packet-record ports in Tables 2.1.5 and 2.1.6 are the new ports.

---

## 6.4 Descriptor Format

*Table 2.1.10: Descriptor Layout*

| Bits | Field | Notes |
|---|---|---|
| [63:0] | `src_addr` | Byte address |
| [127:64] | `dst_addr` | Byte address |
| [159:128] | `length` | Bytes |
| [191:160] | `next_descriptor_ptr` | Chain pointer |
| [192] | `valid` | Must be 1 |
| [193] | `gen_irq` | Emit an IRQ event on completion |
| [194] | `last` | End of chain |
| [195] | `error` | Error flag |
| [199:196] | `channel_id` | |
| [207:200] | `priority` | |
| [209:208] | `opcode` | DATA = 0, CTRL_READ = 1, CTRL_WRITE = 2 |
| [212:210] | `type` | LEGACY = 0, EXT = 1 |
| [216:213] | tile `op`, `ptype` | `TILE_HDR` builds |
| [224:217] | `next_addr_sel` | must be 0 here; any other value is an invalid descriptor (rapids-g runs 1, HAS Section 5.1) |

For CTRL_READ and CTRL_WRITE the address and data slots are reinterpreted as in Tables 2.1.8; the length field does not describe a byte count and no packet record is issued.

---

## 6.5 Byte-to-Beat Math

For each enabled direction the scheduler forms the offset from the low `OFF_W` bits of that side's address and derives the number of memory beats the transfer touches.

```
w_rd_off        = src_addr[OFF_W-1:0]
w_wr_off        = dst_addr[OFF_W-1:0]
w_rd_span       = 33'(w_rd_off) + 33'(length) + 33'(BYTE_LANES-1)
w_wr_span       = 33'(w_wr_off) + 33'(length) + 33'(BYTE_LANES-1)
w_rd_beats_total = (length == 0) ? 0 : 32'(w_rd_span >> OFF_W)
w_wr_beats_total = (length == 0) ? 0 : 32'(w_wr_span >> OFF_W)
```

The span is 33 bits wide so a length near 4 GiB does not overflow. The read and write beat totals are computed separately because the two addresses can have different offsets; a 60-byte transfer read from offset 5 and written at offset 0 touches three source beats and two destination beats on a 32-byte design.

*Table 2.1.11: Beat Totals for a 32-byte Beat*

| Length (bytes) | Offset | Beats |
|---|---|---|
| 0 | any | 0 |
| 1 | 0 | 1 |
| 32 | 0 | 1 |
| 32 | 1 | 2 |
| 60 | 5 | 3 |
| 60 | 0 | 2 |
| 64 | 0 | 2 |

The values load in CH_FETCH_DESC:

- `r_beats_remaining` loads `EN_READ ? rd_total : wr_total`.
- `r_read_beats_remaining` loads `EN_READ ? rd_total : 0`.
- `r_write_beats_remaining` and `r_write_beats_to_commit` load `EN_WRITE ? wr_total : 0`.

A disabled direction therefore loads zero, so its valid never asserts and its completion term is true from the start. Completion collapses onto the enabled direction.

For TYPE=EXT the run counters load `min(run_size, beats_total)` instead of the whole total, and `stream_run_addr_gen.cfg_total_beats` takes the beats total. TYPE=EXT is beat-aligned by design: addresses are aligned and lengths are beat multiples, so the offset is zero and the math reduces to the Beats result.

## 6.6 Address Advance

The working address registers start at the descriptor's byte addresses. On each done strobe the address moves forward by the beats reported:

```
r_src_addr <= {r_src_addr[AW-1:OFF_W], {OFF_W{1'b0}}} +
(AW'(sched_rd_beats_done) << OFF_W)   // on sched_rd_done_strobe
r_dst_addr <= {r_dst_addr[AW-1:OFF_W], {OFF_W{1'b0}}} +
(AW'(sched_wr_beats_done) << OFF_W)   // on sched_wr_done_strobe
```

The low bits are cleared before the add. Only the first burst of a descriptor starts inside a beat; every later burst starts on a beat boundary, so aligning down first is correct for both. The write address advances on the AW handshake (issue), not on the B response. The B response decrements `r_write_beats_to_commit` instead.

The scheduler reports the working address to the engines unmodified. The engines align it (see AXI Read Engine); the offset reaches the data path through the packet record, not through the address bus.

---

## 6.7 Packet Records

A DATA descriptor with a non-zero length causes one record pulse per enabled direction, on the cycle the scheduler leaves CH_FETCH_DESC:

```
w_pkt_ok = !w_is_data || length == 0 ||
           ((!EN_READ  || sched_rd_pkt_ready) &&
            (!EN_WRITE || sched_wr_pkt_ready))

sched_{rd,wr}_pkt_valid = w_state_fetch_desc && r_descriptor.valid &&
w_is_data &&
                          EN_{READ,WRITE} && w_pkt_ok && (length != 0)
sched_{rd,wr}_pkt_bytes  = r_descriptor.length
sched_{rd,wr}_pkt_offset = w_{rd,wr}_off
```

Both directions share `w_pkt_ok`, so each enabled direction sees exactly one pulse. `w_pkt_ok` also gates the FETCH_DESC to XFER_DATA transition: while a record queue is full, the scheduler holds in CH_FETCH_DESC and the pulse does not fire. Control descriptors and zero-length transfers carry no record. See Packet Record Interface.

---

## 6.8 Finite State Machine

The state is one-hot and is `rapids_pkg::channel_state_t`.

*Table 2.1.12: State Encoding*

| State | Encoding | Description |
|---|---|---|
| CH_IDLE | 7'b0000001 | Waiting for a descriptor and `cfg_channel_enable` |
| CH_FETCH_DESC | 7'b0000010 | Descriptor latched; working registers and counters load; record pulses |
| CH_XFER_DATA | 7'b0000100 | Read and write engines run concurrently; control ops issue |
| CH_COMPLETE | 7'b0001000 | Descriptor done; chain decision |
| CH_NEXT_DESC | 7'b0010000 | Waiting for the chained descriptor |
| CH_ERROR | 7'b0100000 | Sticky; only channel reset leaves it |
| CH_RESERVED | 7'b1000000 | Unused |

Transition priority is: registered channel reset to IDLE, then a hard error or timeout escalation to ERROR, then the normal case.

*Table 2.1.13: Transitions*

| From | To | Condition |
|---|---|---|
| IDLE | FETCH_DESC | `descriptor_valid && cfg_channel_enable` |
| FETCH_DESC | XFER_DATA | `r_descriptor.valid && w_pkt_ok` |
| FETCH_DESC | FETCH_DESC | `r_descriptor.valid` and `w_pkt_ok` false (record queue full) |
| FETCH_DESC | ERROR | descriptor invalid: `valid` bit clear, or a next-address selector `[224:217]` other than 0 (rapids TASK-035) |
| XFER_DATA | COMPLETE | `w_exec_complete` |
| COMPLETE | NEXT_DESC | `next_descriptor_ptr != 0 && !last` |
| COMPLETE | IDLE | otherwise |

| From | To | Condition |
|---|---|---|
| NEXT_DESC | FETCH_DESC | `descriptor_valid` |
| ERROR | ERROR | always; channel reset is the only exit |
| any other | ERROR | default |

`w_exec_complete` is `w_transfer_complete` for a DATA descriptor and `w_ctrl_complete` for a control descriptor. `w_transfer_complete` is the read remaining count reaching zero together with the commit remaining count reaching zero. `w_ctrl_complete` is the control request having been accepted and the control engine having returned to idle.

`scheduler_idle` is `CH_IDLE && !r_channel_reset_active && !w_desc_skid_valid`. `sched_error` is the ERROR state.

---

## 6.9 Operation

### 6.9.1 Data Request Outputs

`sched_rd_valid` is high in XFER_DATA for a DATA descriptor while `!w_read_complete`, `!w_sched_rd_completing_this_cycle` and, for TYPE=EXT, `!w_rd_need_base`. `sched_rd_beats` is the read remaining count, or for EXT the current run's remainder. `sched_wr_valid` is high in XFER_DATA for a DATA descriptor while `r_write_beats_remaining != 0`, the commit count has not reached zero, `!w_sched_wr_completing_this_cycle` and `!w_wr_need_base`.

The look-ahead terms drop the valid on the same cycle the last done strobe arrives. Without them the engine would see a still-asserted valid for one more cycle and issue an extra burst.

### 6.9.2 Write Completion

The write direction has two counters. `r_write_beats_remaining` decrements on the AW done strobe and drives burst sizing and the destination address. `r_write_beats_to_commit` decrements on the B commit strobe and gates `w_write_complete`. The channel therefore completes only after every issued burst has been acknowledged.

### 6.9.3 Control Descriptors

CTRL_READ and CTRL_WRITE do not use the data engines. `ctrlrd_valid` and `ctrlwr_valid` assert in XFER_DATA until the engine accepts (`r_ctrl_issued`), and completion is the engine returning to idle after acceptance.

---

## 6.10 Error Handling

`w_hard_error` is the OR of:

- `descriptor_error`, `sched_rd_error`, `sched_wr_error`
- the sticky read and write error flags
- `ctrlrd_error` while the descriptor is CTRL_READ
- `ctrlwr_error` while the descriptor is CTRL_WRITE

A hard error, or a timeout escalation, moves the FSM to CH_ERROR. The sink packet length check reports through `sched_wr_error` (see Sink Data Path AXIS), so a stream packet whose byte count differs from the descriptor length is a hard error for that channel. Sticky read and write flags and the descriptor error clear in CH_IDLE and also on the registered channel reset, in every state. Clearing on the reset matters: the data engines and the data paths clear their own error levels on the same channel reset, so the scheduler no longer re-enters CH_ERROR from a stale level when it reaches CH_IDLE. A channel reset therefore recovers a data read or write response error and a sink packet length error, and the channel accepts a good descriptor afterwards. The reset also clears the descriptor-loaded flag, the remaining beat counts and the control issued flag, so nothing of the failed descriptor carries into the next one.

---

## 6.11 Recoverable Write-Progress Timeout

The timeout counter increments while `sched_wr_valid && !sched_wr_ready`. It resets on `sched_wr_done_strobe` or `sched_wr_commit_strobe`, and also when the condition is false. When the counter reaches `cfg_sched_timeout_cycles` with `cfg_sched_timeout_enable` set, `w_timeout_expired` pulses and the counter re-arms.

Each expiry adds one strike, in an 8-bit counter that saturates. Strikes clear on write progress, in CH_IDLE, and on channel reset. Escalation is:

```
w_timeout_escalate = (cfg_sched_timeout_limit != 0) && (strikes >=
cfg_sched_timeout_limit)
```

A limit of zero never escalates: the timeout is then purely informational.

---

## 6.12 MonBus Events

Packets are `PktTypeCompletion` from `PROTOCOL_CORE` unless noted.

*Table 2.1.14: MonBus Events*

| State | Event | Data |
|---|---|---|
| entry to CH_FETCH_DESC | `RAPIDS_EVENT_DESC_START` | descriptor length (bytes) |
| entry to CH_COMPLETE | `RAPIDS_EVENT_IRQ` when `gen_irq`, else `RAPIDS_EVENT_DESC_COMPLETE` | |
| CH_ERROR | `RAPIDS_EVENT_ERROR` (`PktTypeError`), once per error episode | `{29'h0, wr_sticky, rd_sticky, 33'h0}` |

There are no events during XFER_DATA. The only difference from RAPIDS Beats is that the DESC_START data field is a byte count.

DESC_START and the completion events fire on the first cycle of their state, from an entry pulse in `scheduler_core`. Until 2026-10-21 DESC_START fired on every cycle in CH_FETCH_DESC; the byte tree holds that state while the record queue is full or the tile advertisement window is open, and one held descriptor sent 62 packets in the record-backpressure test (rapids BUG-016).

## 6.13 Run-Base Generator Flush

A channel reset flushes both `stream_run_addr_gen` instances (`flush` = the registered channel reset): generation stops, results still in `dma_address_gen`'s pipeline are discarded, and the base FIFO (a `gaxi_drop_fifo_sync`) is dropped whole. Before 2026-10-21 nothing emptied it: a generator started for an EXT descriptor kept producing that descriptor's bases through the reset, and the next EXT descriptor began its second run at the reset descriptor's base (rapids BUG-017; the same in RAPIDS Beats and STREAM, fixed in all three).
`test_scheduler_ext_channel_reset` covers it.

---

## 6.14 Waveforms

#### *6.14.0.1 Waveform 2.1: Byte Descriptor Start*

The beat-count arithmetic and the record pulse are combinational functions of the latched descriptor, so the start of a descriptor is short:

```
cycle                    : 0     1        2        3
state                    : IDLE  FETCH    XFER     XFER
descriptor_valid         : 1     0        0        0
sched_wr_pkt_ready       : 1     1        1        1
sched_wr_pkt_valid       : 0     1        0        0     (one pulse,
the last FETCH cycle)
r_write_beats_remaining  : 0     0        N        N     (loads at the
end of FETCH)
sched_wr_valid           : 0     0        1        1
```

If `sched_wr_pkt_ready` is low in FETCH, the state holds and the pulse is delayed until it rises.

# 7 Descriptor Engine Specification

**Module:** `descriptor_engine_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

---

## 7.1 Overview

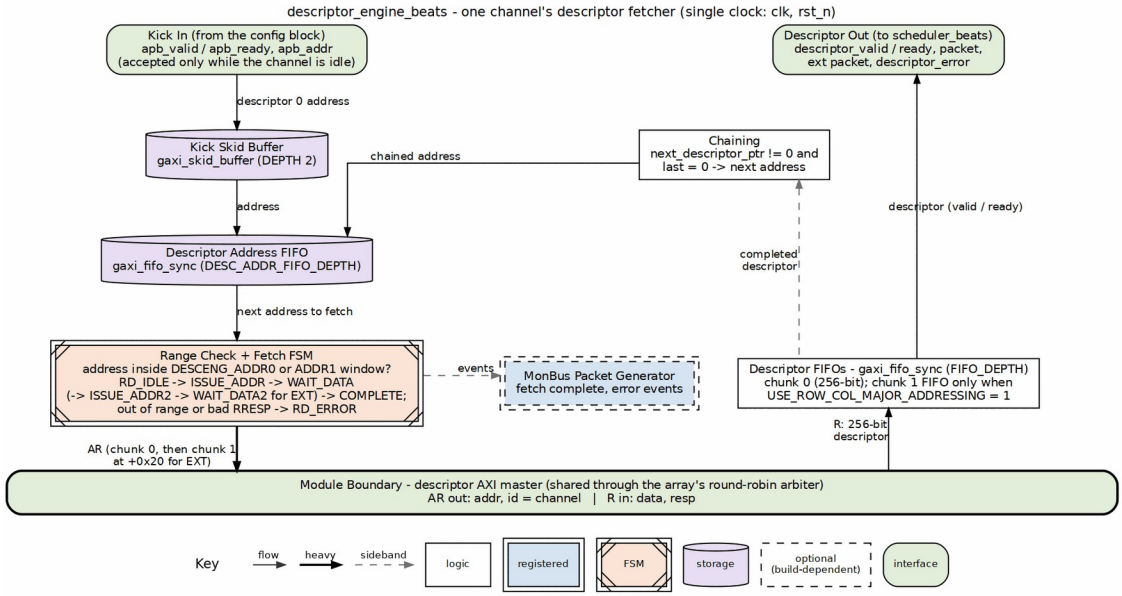
The Descriptor Engine fetches and manages 256-bit descriptors from memory via AXI4. It maintains a prefetch FIFO to hide memory latency and provides parsed descriptors to the scheduler.

### 7.1.1 Key Features

- **AXI4 Descriptor Fetch:** Single-beat 256-bit reads
- **Prefetch FIFO:** Configurable depth for latency hiding
- **Address Range Checking:** Validates descriptor addresses
- **Error Detection:** Address violations, AXI errors
- **MonBus Integration:** Fetch events and error reporting

### 7.1.2 Block Diagram

### 7.1.3 Figure 2.2.1: Descriptor Engine Block Diagram




*Descriptor engine block diagram*

**Source:** 02_descriptor_engine.dot

## 7.2 Parameters

```
parameter int CHANNEL_ID = 0;                    // Channel identifier
parameter int NUM_CHANNELS = 8;                  // Total channels
parameter int ADDR_WIDTH = 64;                   // Address bus width
parameter int FIFO_DEPTH = 4;                    // Prefetch FIFO
depth

// Monitor Bus Parameters
parameter logic [7:0] MON_AGENT_ID = 8'h10;      // Descriptor Engine
Agent ID
parameter logic [3:0] MON_UNIT_ID = 4'h1;        // Unit identifier
```

: Table 2.2.1: Descriptor Engine Parameters

## 7.3 Port List

### 7.3.1 Clock and Reset

*Table 2.2.2: Clock and Reset*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `clk` | input | 1 | System clock |
| `rst_n` | input | 1 | Active-low asynchronous reset |

### 7.3.2 APB Programming Interface

*Table 2.2.3: APB Programming Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `apb_valid` | input | 1 | Kick-off valid (start descriptor chain) |
| `apb_ready` | output | 1 | Ready to accept kick-off |
| `apb_addr` | input | AW | First descriptor address |

### 7.3.3 Configuration Interface

*Table 2.2.4: Configuration Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `cfg_prefetch_enable` | input | 1 | Enable prefetch chaining (0 = on-demand, ~1 ahead) |
| `cfg_fifo_threshold` | input | 4 | Prefetch depth (descriptors buffered ahead when enabled) |
| `cfg_addr0_base` | input | AW | Valid address range 0 base |
| `cfg_addr0_limit` | input | AW | Valid address range 0 limit |

| Signal | Direction | Width | Description |
|---|---|---|---|
| cfg_addr1_base | input | AW | Valid address range 1 base |
| cfg_addr1_limit | input | AW | Valid address range 1 limit |

### 7.3.4 AXI4 Read Master Interface

*Table 2.2.5: AXI4 Read Master Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| ar_valid | output | 1 | Address valid |
| ar_ready | input | 1 | Address ready |
| ar_addr | output | ADDR_WIDTH | Descriptor fetch address |
| ar_len | output | 8 | Burst length - 1 |
| ar_size | output | 3 | Burst size (log2 bytes) |
| ar_burst | output | 2 | Burst type (INCR) |
| ar_id | output | AXI_ID_WIDTH | Transaction ID |
| ar_lock | output | 1 | Lock type |
| ar_cache | output | 4 | Cache attributes |
| ar_prot | output | 3 | Protection attributes |
| ar_qos | output | 4 | Quality of service |
| ar_region | output | 4 | Region identifier |
| r_valid | input | 1 | Data valid |
| r_ready | output | 1 | Data ready |
| r_data | input | 256 | Descriptor payload (fixed 256-bit) |
| r_resp | input | 2 | Read response |

| Signal | Direction | Width | Description |
|---|---|---|---|
| r_last | input | 1 | Last beat |
| r_id | input | AXI_ID_WIDTH | Response ID |

### 7.3.5 Scheduler Interface

*Table 2.2.6: Scheduler Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| descriptor_valid | output | 1 | Descriptor valid |
| descriptor_ready | input | 1 | Scheduler ready |
| descriptor_packet | output | 256 | Parsed descriptor (fixed 256-bit) |
| descriptor_ext_packet | output | 256 | Extended chunk 1 (0 when unused) |
| descriptor_error | output | 1 | Error flag |
| descriptor_eos | output | 1 | End of Stream |
| descriptor_eol | output | 1 | End of Line |
| descriptor_eod | output | 1 | End of Data |
| descriptor_type | output | 2 | Packet type |
| channel_idle | input | 1 | Scheduler idle (enables APB kick-off) |
| descriptor_engine_idle | output | 1 | Engine idle |

## 7.4 FSM States

### 7.4.1 Figure 2.2.2: Descriptor Engine FSM

States and transitions of the `descriptor_engine_beats` fetch FSM.

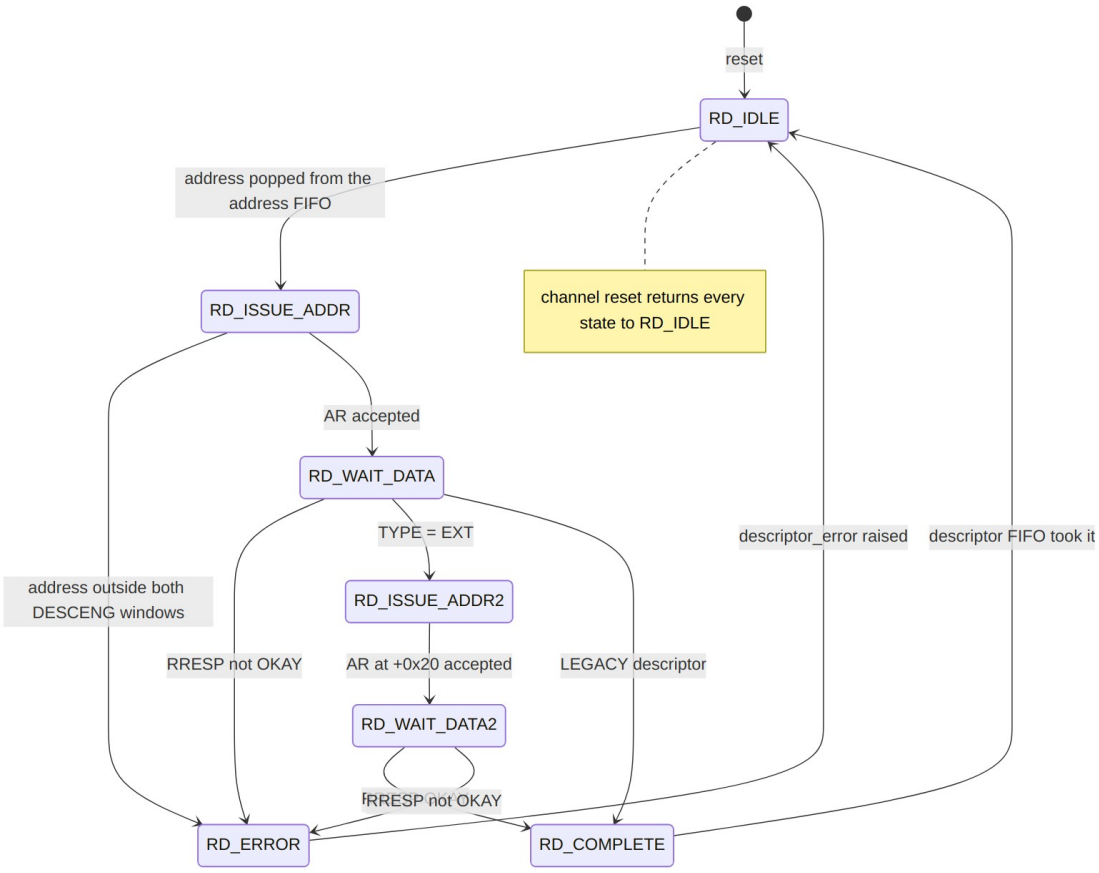
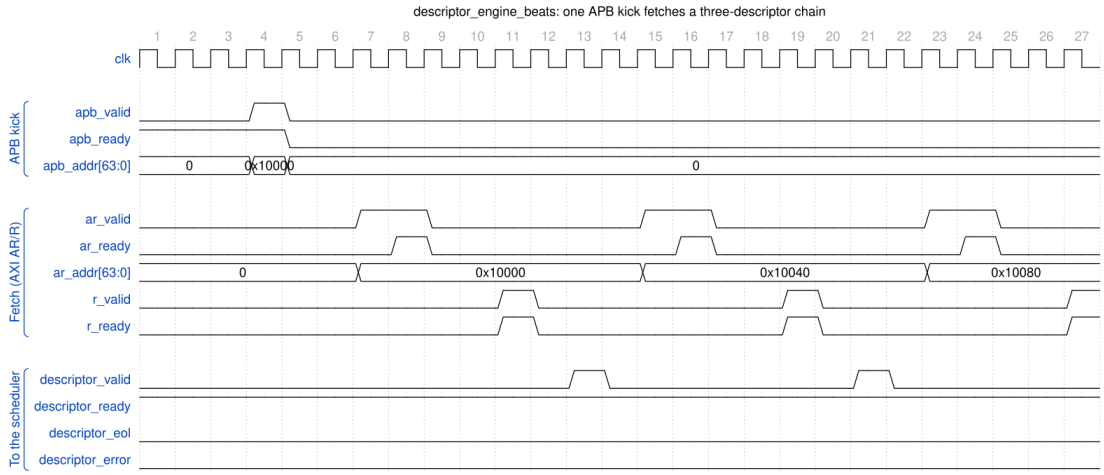



*Diagram 1*

## 7.5 Operation

### 7.5.1 Descriptor Chain Processing

### 7.5.2 Figure 2.2.3: Descriptor Chain Timing




*Descriptor Engine - three-descriptor chain*

**Source:** descriptor_engine_chain.json, captured from `dv/tests/fub_beats/test_descriptor_engine_beats.py` (`descriptor_chaining`, 32 channels, seed 7) with `WAVES=1`.

Reading it: one `apb_valid` kick carries the head address (0x10000). The engine issues an AXI read for it (`ar_valid`, `ar_addr`), the data returns on `r_valid`, and the parsed descriptor is presented on `descriptor_valid` to a ready scheduler. Because the descriptor's `next_ptr` points at 0x10040 and `last` is clear, the engine fetches the next one on its own – and the one after that at 0x10080 – with no further APB traffic; `descriptor_eol` marks the last of the chain. Each fetch costs the same AR -> R -> present latency, so a chain streams at one descriptor per fetch round trip.

---

## 7.6 Address Range Checking

Descriptors must fall within configured valid address ranges:

```
Valid if: (addr >= cfg_addr0_base && addr <= cfg_addr0_limit)
       || (addr >= cfg_addr1_base && addr <= cfg_addr1_limit)
```

Invalid addresses generate an error and halt the descriptor chain.

---

## 7.7 Prefetch and Autonomous Chaining

`cfg_prefetch_enable` and `cfg_fifo_threshold` are functional prefetch controls (previously declared-but-unused, which left chaining permanently greedy). They throttle how far ahead the engine fetches chained descriptors by comparing the output descriptor FIFO occupancy (`w_desc_fifo_count`) against a computed `w_prefetch_limit`:

```
if      (!cfg_prefetch_enable)      w_prefetch_limit = 1;   // on-
demand
else if (cfg_fifo_threshold == 0)   w_prefetch_limit = 1;   // guard
(0 would stall)
else                                w_prefetch_limit =
cfg_fifo_threshold;

w_prefetch_allows = (w_desc_fifo_count < w_prefetch_limit);
```

- **Prefetch OFF (`cfg_prefetch_enable = 0`):** limit forced to 1, so a chained fetch is only allowed when the output FIFO has drained – the engine stays roughly one descriptor ahead of the scheduler (on-demand).
- **Prefetch ON:** the engine buffers up to `cfg_fifo_threshold` descriptors ahead.

### 7.7.1 Chaining Decision

A chain fetch issues immediately when all of the following hold:

```
w_should_chain = w_chain_eligible     // valid non-last descriptor, in
range, no error
               && w_desc_committed      // current descriptor accepted
into output FIFO
               && w_prefetch_allows     // occupancy below prefetch
limit
               && w_desc_addr_fifo_wr_ready;  // address FIFO has a
slot
```

### 7.7.2 Deferred (Throttle-Blocked) Chain Fetch

If a descriptor commits and is chain-eligible but the fetch cannot issue this cycle (throttled by `w_prefetch_allows`, or the address FIFO is full), the owed fetch is recorded rather than dropped:

- `r_chain_pending` is set and the next-descriptor pointer is held in `r_pending_chain_addr`.
- The deferred fetch is reissued (`w_pending_push_fire`) once the output FIFO drains enough that `w_prefetch_allows` is true again and the address FIFO has a slot, clearing `r_chain_pending`.

**Last Updated:** 2026-07-02

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

# 8 AXI Read Engine

**Module:** `axi_read_engine.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub/`
**Status:** Implemented **Last Updated:** 2026-09-30

---

## 8.1 Overview

The AXI read engine turns per-channel read requests from the schedulers into AXI4 read bursts and routes the returning data to the SRAM controller. It is a streaming pipeline with no state machine. The channel identity travels in the AXI ID.

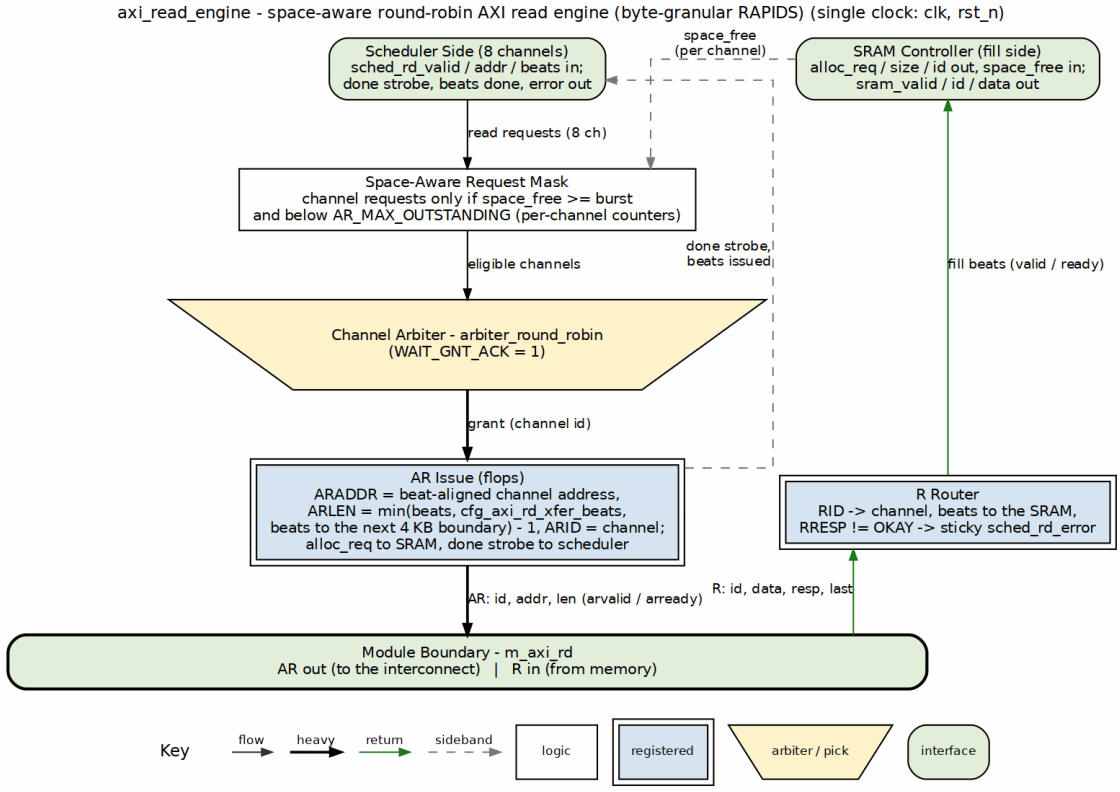
The engine is beat based, as in RAPIDS Beats. The scheduler has already converted the descriptor's byte length into a beat count. Two things change for byte-granular RAPIDS: the engine must cope with a channel address that is not beat aligned, and it must never let a burst cross a 4 KB boundary.

### 8.1.1 Key Features

- **Streaming pipeline:** no FSM, round-robin arbitration across channels.
- **Aligned AR address:** `ARADDR` is the channel address rounded down to a beat boundary. The lanes below the offset are read and ignored by the data path.
- **4 KB burst cap:** the burst length is limited so that no burst crosses a 4 KB boundary, whatever the start address.
- **Configurable burst length:** `cfg_axi_rd_xfer_beats`, clamped to what the buffer can hold (rapids BUG-009).
- **Space-checked:** a channel is only granted when the SRAM controller has room for the whole burst.
- **Sticky error:** SLVERR or DECERR on R sets `sched_rd_error` for the channel.

### 8.1.2 Block Diagram

### 8.1.3 Figure 2.3.1: AXI Read Engine Block Diagram




*AXI read engine block diagram*

**Source:** 05_axi_read_engine.dot

---

## 8.2 Parameters

*Table 2.3.1: AXI Read Engine Parameters*

| Parameter | Default | Description |
|---|---|---|
| NUM_CHANNELS | 8 | Number of channels. Channel id is carried in the AXI ID. |
| ADDR_WIDTH | 64 | AXI address width. |
| DATA_WIDTH | 512 | AXI data width in bits. The Genesys 2 build uses 256. |
| ID_WIDTH | 8 | AXI ID width. |
| SEG_COUNT_WIDTH | 8 | Width of the per-channel free-space count from the SRAM controller. Sets the burst clamp. |
| PIPELINE | 1 | 1 allows the next AR before the previous burst's data has returned. 0 waits for the burst to complete. |
| AR_MAX_OUTSTANDING | 8 | Maximum outstanding AR transactions. |
| STROBE_EVERY_BEAT | 0 | Declared for compatibility with STREAM. The completion strobe in this engine is generated on the AR handshake regardless of its value. |

The module also accepts the short aliases NC, AW, DW, IW, SCW and CIW (the channel id width, $clog2(NUM_CHANNELS)).

### 8.2.1 Derived Constants

*Table 2.3.2: AXI Read Engine Derived Constants*

| Constant | Definition | Meaning |
|---|---|---|
| BYTES_PER_BEAT | DW / 8 | Bytes per beat. |
| AXSIZE | $clog2(BYTES_PER_BEAT) | Value driven on ARSIZE. Full-width |

| Constant | Definition | Meaning |
| --- | --- | --- |
| | | beats. |
| SD_BEATS | 1 << (SCW - 1) | Buffer depth in beats implied by SEG_COUNT_WIDTH. |
| XFER_MAX | SD_BEATS < 256 ? SD_BEATS - 1 : 254 | Largest burst length value the engine will use. |

## 8.3 Port List

### 8.3.1 Clock, Reset and Configuration

*Table 2.3.3: Clock, Reset and Configuration*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `clk` | input | 1 | Clock. |
| `rst_n` | input | 1 | Active-low reset. |
| `cfg_axi_rd_xfer_beats` | input | 8 | Configured burst length (ARLEN value, 0 to 255). Clamped to `XFER_MAX`. |
| `cfg_channel_reset` | input | NC | Per-channel reset, level or pulse. Stops the channel's ARs, drains and discards its in-flight R data, and clears its error flag. |

### 8.3.2 Scheduler Interface

*Table 2.3.4: Scheduler Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `sched_rd_valid` | input | NC | Channel has a read request. |
| `sched_rd_addr` | input | NC x AW | Working source byte address. |
| `sched_rd_beats` | input | NC x 32 | Beats still to read for the channel. |
| `sched_rd_done_strobe` | output | NC | Registered pulse, one cycle after the AR handshake for the channel. |
| `sched_rd_beats_done` | output | NC x 32 | Beats issued by that AR (`ARLEN + 1`). |
| `sched_rd_error` | output | NC | Sticky read error for the channel, set by any non-OKAY R response. |

The address is a byte address and may carry an offset in its low bits. The scheduler advances it to the next beat boundary after each done strobe, so only the first burst of a descriptor can see a non-zero offset.

### 8.3.3 SRAM Allocation and Fill Interface

*Table 2.3.5: SRAM Allocation and Fill Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `axi_rd_alloc_req` | output | 1 | Registered pulse, one cycle after the AR handshake, reserving space for the burst. |
| `axi_rd_alloc_size` | output | 8 | Beats to reserve. |
| `axi_rd_alloc_id` | output | IW | Channel that owns the reservation. |
| `axi_rd_alloc_space_free` | input | NC x SCW | Free space per channel. A channel is granted only if it fits the burst. |
| `axi_rd_sram_valid` | output | 1 | Follows `m_axi_rvalid`. |
| `axi_rd_sram_ready` | input | 1 | SRAM controller accepts the beat. Drives `m_axi_rready`. |
| `axi_rd_sram_id` | output | IW | Follows `m_axi_rid`. |
| `axi_rd_sram_data` | output | DW | Follows `m_axi_rdata`. |

### 8.3.4 AXI4 Read Master Interface

*Table 2.3.6: AXI4 Read Master Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `m_axi_arid` | output | IW | Read address ID (the channel). |
| `m_axi_araddr` | output | AW | Beat-aligned read address. |
| `m_axi_arlen` | output | 8 | Burst length |

| Signal | Direction | Width | Description |
|---|---|---|---|
| | | | minus one. |
| m_axi_arsize | output | 3 | AXSIZE, full-width beats. |
| m_axi_arburst | output | 2 | INCR. |
| m_axi_arvalid | output | 1 | Address valid. |
| m_axi_arready | input | 1 | Address ready. |
| m_axi_rid | input | IW | Response ID. |
| m_axi_rdata | input | DW | Read data. |
| m_axi_rresp | input | 2 | Read response. |
| m_axi_rlast | input | 1 | Last beat of the burst. |
| m_axi_rvalid | input | 1 | Data valid. |
| m_axi_rready | output | 1 | Data ready. |

### 8.3.5 Debug

*Table 2.3.7: Debug Outputs*

| Signal | Direction | Width | Description |
|---|---|---|---|
| dbg_rd_all_complete | output | NC | Channel has no outstanding reads. |
| dbg_r_beats_rcvd | output | 32 | Total R beats received. |
| dbg_sram_writes | output | 32 | Total beats written to the SRAM controller. |
| dbg_arb_request | output | NC | Arbiter request vector. |

## 8.4 Operation

### 8.4.1 Burst Length Cap (rapids BUG-009)

`cfg_axi_rd_xfer_beats` is an ARLEN value, but a burst must fit in the buffer that stages it. The engine clamps the configured value to `XFER_MAX`, giving the working limit `w_xfer_cfg`. Without the clamp an ARLEN of 255 wraps the 8-bit `ARLEN + 1` and a channel is granted with no space reserved. This is the same fix as in RAPIDS Beats.

### 8.4.2 Byte-Granular Burst Sizing

The engine sizes each burst from the channel's current byte address and remaining beats. Three steps produce `ARLEN`:

1. Distance to the next 4 KB boundary, in beats: `w_beats_to_4k = (4096 - {sched_rd_addr[11:AXSIZE], AXSIZE'b0}) >> AXSIZE`. The offset bits are dropped first, so the distance is measured from the aligned beat.
2. Burst cap: `w_cap_beats = min(w_xfer_cfg + 1, w_beats_to_4k)`.
3. Burst length: `w_transfer_size = (sched_rd_beats <= w_cap_beats) ? sched_rd_beats - 1 : w_cap_beats - 1`.

`m_axi_araddr` is `{sched_rd_addr[grant][AW-1:AXSIZE], AXSIZE'b0}`. The offset never reaches the bus. The first beat of the burst carries the bytes before the offset, and the data path discards them.

Because the cap always rounds toward the boundary, a burst can be shorter than the configured length, and a later burst restarts on the boundary. The scheduler needs no knowledge of this. It only sees `beats_done` per burst.

### 8.4.3 Streaming Pipeline

An arbiter picks a channel that has `sched_rd_valid` set and enough `axi_rd_alloc_space_free` for its burst. When the AR handshakes, the engine pulses `axi_rd_alloc_req` one cycle later with the burst size so the SRAM controller reserves the space, and pulses `sched_rd_done_strobe` for the channel. R beats pass straight through: `axi_rd_sram_valid`, `axi_rd_sram_id` and `axi_rd_sram_data` follow the R channel, and `m_axi_rready` is `axi_rd_sram_ready`.

With `PIPELINE = 1` the next AR for a channel can issue while earlier data is still in flight. With `PIPELINE = 0` the channel waits for the burst to finish. The top and core default to 1.

### 8.4.4 Figure 2.3.2: Burst Segmentation with a Mid-Beat Start

The example uses 32-byte beats (OFF_W 5) and a configured burst of 16 beats. A descriptor reads 8,200 bytes from address 0xFF8. The scheduler computes offset 0x18 (24) and `beats_total = (24 + 8200 + 31) >> 5 = 257`.

*Table 2.3.8: Burst Segmentation Example*

| Burst | Channel address | ARADDR | Beats to 4 KB | ARLEN | Beats left after |
|---|---|---|---|---|---|
| 1 | 0x0FF8 | 0x0FE0 | 1 | 0 | 256 |
| 2 | 0x1000 | 0x1000 | 128 | 15 | 240 |
| 3 | 0x1200 | 0x1200 | 112 | 15 | 224 |
| … | | | | 15 | continues |

The first burst is one beat because the aligned start is 32 bytes below the 4 KB boundary. From then on each burst is the configured 16 beats until the remainder runs out. The scheduler advances its address to the next beat boundary after each done strobe, which is why burst 2 starts at 0x1000, not 0x0FF8 plus a beat.

### 8.4.5 Done Strobe

The engine pulses `sched_rd_done_strobe[ch]` once per AR handshake, with `sched_rd_beats_done[ch]` equal to `ARLEN + 1`. The scheduler subtracts this from its remaining count and advances the address. The strobe therefore reports beats issued, not beats returned.

---

## 8.5 Error Handling

*Table 2.3.9: Error Handling*

| Error | Detection | Response |
|---|---|---|
| AXI SLVERR | `m_axi_rresp == 2'b10` on an accepted beat | Set sticky `sched_rd_error` for the channel |
| AXI DECERR | `m_axi_rresp == 2'b11` on an accepted beat | Set sticky `sched_rd_error` for the channel |
| Any other non-zero response | `m_axi_rresp != 2'b00` on an accepted beat | Same: the test is not-OKAY, and the channel is taken from `RID` |

The scheduler turns the sticky error into its ERROR state and a MonBus event. The engine does not abort the burst: the remaining R beats still pass through, so the interconnect is not left mid-burst.

### 8.5.1 Channel Reset

`cfg_channel_reset[ch]` clears the sticky `sched_rd_error[ch]`, so a channel reset recovers a read error without `aresetn`. The reset also has to settle AR bursts that are already on the bus, because an AXI master cannot recall them:

- A channel in reset, or flushing, issues no new AR and does not enter the allocation pipeline.
- `r_rd_flush[ch]` is set by the reset and holds until the channel has no burst outstanding. While it holds, each R beat whose `RID` names the channel is accepted (`m_axi_rready` is forced high) and dropped: `axi_rd_sram_valid` stays low, so nothing reaches the SRAM, and its `RRESP` does not set the error flag.
- The completion strobe and pending allocation of the channel are cleared, so the scheduler does not see a stale done pulse after the reset.
- Other channels are not affected: every term is indexed by channel, and R beats of other channels pass as usual.

The channel's SRAM partition is cleared by the SRAM controller wrapper (see Sink Data Path and Source Data Path AXIS). The discard is what guarantees that no beat of the reset channel reaches the partition after it has been cleared.

---

## 8.6 Differences from RAPIDS Beats

*Table 2.3.10: Read Engine Delta*

| Item | RAPIDS Beats | RAPIDS |
|---|---|---|
| ARADDR | Channel address as given (always aligned) | Aligned down to the beat |
| Burst cap | Configured length and remaining beats | Also capped at the 4 KB boundary |
| Everything else | | Unchanged |

The beat-count chapter for the beats engine is AXI Read Engine (RAPIDS Beats). The scheduler that feeds this engine is described in Scheduler.

---

**Last Updated:** 2026-09-30

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 9 AXI Write Engine

**Module:** `axi_write_engine.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub/`
**Status:** Implemented **Last Updated:** 2026-09-30

---

## 9.1 Overview

The AXI write engine drains data from the SRAM controller and issues AXI4 write bursts to system memory. It is a streaming pipeline with no state machine. It also tracks the write responses, which is how the scheduler learns that data has been committed.

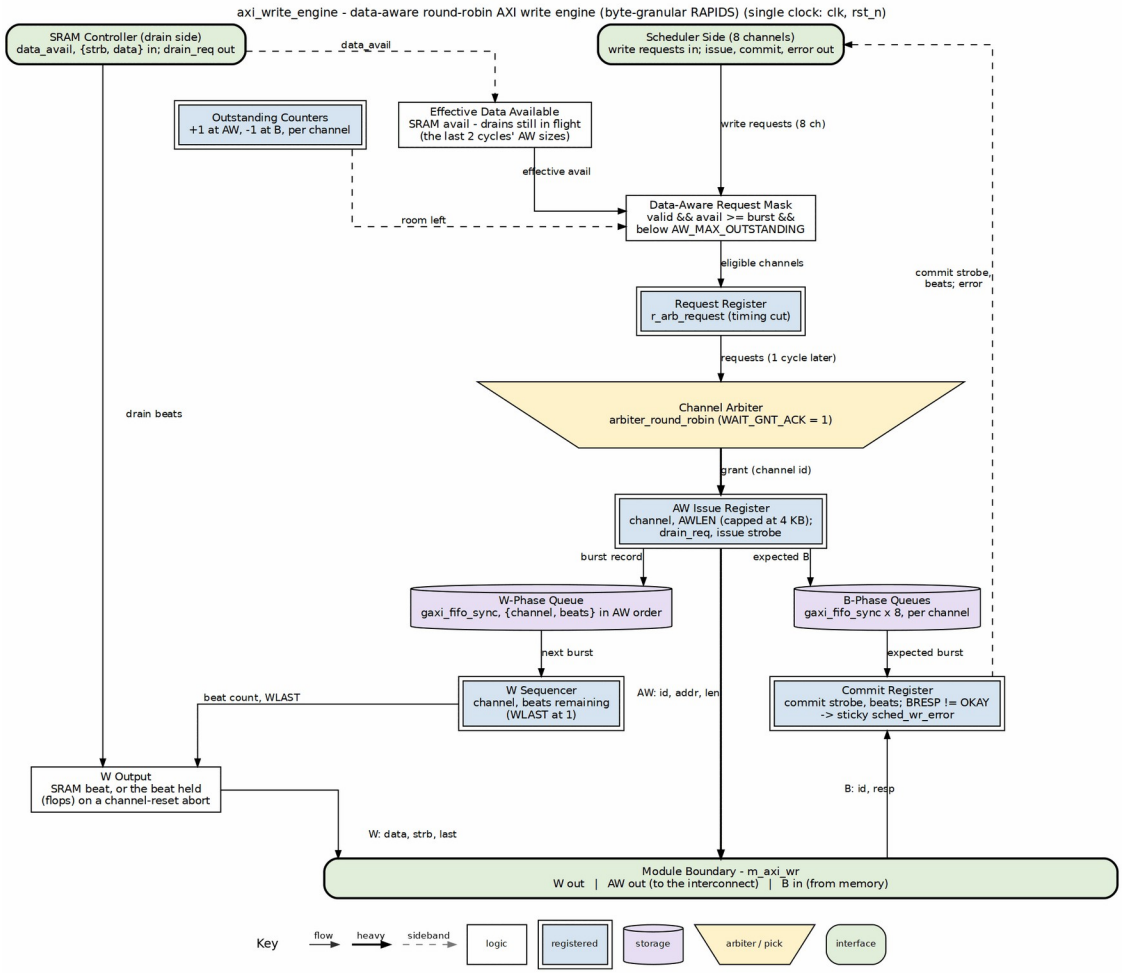
Byte-granular RAPIDS changes the engine in three places. Write strobes now come from the SRAM instead of being all ones, so a partial first or last beat writes only the bytes that belong to the packet. Burst lengths are capped at the 4 KB boundary. `AWADDR` is beat aligned.

### 9.1.1 Key Features

- **Streaming pipeline:** no FSM, round-robin arbitration across channels.
- **Real write strobes:** `m_axi_wstrb` is the strobe word stored beside each data beat (`axi_wr_sram_strb`). Lanes outside the packet are not written.
- **Aligned AW address:** `AWADDR` is the channel address rounded down to a beat boundary.
- **4 KB burst cap:** the same cap as the read engine, applied to both the burst length and the final-burst test.
- **Configurable burst length:** `cfg_axi_wr_xfer_beats`, clamped to what the buffer can hold (rapids BUG-009).
- **Two completion strobes:** done on the AW handshake, commit on the B response.
- **Sticky error:** a non-OKAY B response sets `sched_wr_error`.

### 9.1.2 Block Diagram

### 9.1.3 Figure 2.4.1: AXI Write Engine Block Diagram




*AXI write engine block diagram*

**Source:** 06_axi_write_engine.dot

---

## 9.2 Parameters

*Table 2.4.1: AXI Write Engine Parameters*

| Parameter | Default | Description |
|---|---|---|
| NUM_CHANNELS | 8 | Number of channels. Channel id is carried in the AXI ID. |
| ADDR_WIDTH | 64 | AXI address width. |
| DATA_WIDTH | 512 | AXI data width in bits. The Genesys 2 build uses 256. |
| ID_WIDTH | 8 | AXI ID width. |
| USER_WIDTH | 8 | Width of `m_axi_wuser`, which carries the active channel. |
| SEG_COUNT_WIDTH | 8 | Width of the per-channel available-data count from the SRAM controller. Sets the burst clamp. |
| PIPELINE | 1 | 1 allows further AWs before earlier B responses return. 0 waits for the B. |
| AW_MAX_OUTSTANDING | 8 | Maximum outstanding AW transactions per channel. |
| W_PHASE_FIFO_DEPTH | 64 | Depth of the W-phase transaction FIFO, kept in order with the AW issue. |
| B_PHASE_FIFO_DEPTH | 16 | Depth of the B-phase transaction FIFO used to match responses. |

The module also accepts the short aliases `NC`, `AW`, `DW`, `IW`, `UW`, `SCW` and `CIW` (the channel id width).

### 9.2.1 Derived Constants

*Table 2.4.2: AXI Write Engine Derived Constants*

| Constant | Definition | Meaning |
|---|---|---|
| BYTES_PER_BEAT | `DW / 8` | Bytes per beat. |
| AXSIZE | `$clog2(BYTES_PER_BEAT)` | Value driven on |

| Constant | Definition | Meaning |
| --- | --- | --- |
| XFER_MAX | min(2^(SCW-1) - 1, 254) | AWSIZE.<br>Largest burst length value the engine will use. |

## 9.3 Port List

### 9.3.1 Clock, Reset and Configuration

*Table 2.4.3: Clock, Reset and Configuration*

| Signal | Direction | Width | Description |
|---|---|---|---|
| clk | input | 1 | Clock. |
| rst_n | input | 1 | Active-low reset. |
| cfg_axi_wr_xfer_beats | input | 8 | Configured burst length (AWLEN value, 0 to 255). Clamped to XFER_MAX. |
| cfg_channel_reset | input | NC | Per-channel reset, level or pulse. Stops new bursts, completes open bursts with null beats, and clears the channel's error flag. |

### 9.3.2 Scheduler Interface

*Table 2.4.4: Scheduler Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| sched_wr_valid | input | NC | Channel has a write request. |
| sched_wr_ready | output | NC | Pulses when the B response of the last burst of a descriptor arrives. |
| sched_wr_addr | input | NC x AW | Working destination byte address. |
| sched_wr_beats | input | NC x 32 | Beats still to write for the channel. |
| sched_wr_burst_len | input | NC x 8 | Requested burst length. Present on the port; the engine sizes bursts from cfg_axi_wr_xfer_beats and the 4 KB cap. |
| sched_wr_don | output | NC | Pulses on the AW |

| Signal | Direction | Width | Description |
|---|---|---|---|
| e_strobe | | | handshake for the channel. |
| sched_wr_beats_done | output | NC x 32 | Beats issued by that AW (AWLEN + 1). |
| sched_wr_commit_strobe | output | NC | Pulses when the B response for the channel arrives. |
| sched_wr_commit_beats | output | NC x 32 | Beats confirmed by that B response, recovered from the B-phase FIFO. |
| sched_wr_error | output | NC | Sticky write error for the channel. |

The done strobe advances the scheduler's destination address, which lets the next AW issue without waiting for a response. The commit strobe is what the scheduler counts when it decides the descriptor is complete. Both strobes are per channel and report beat counts, not bytes.

### 9.3.3 SRAM Reservation and Drain Interface

*Table 2.4.5: SRAM Drain Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| axi_wr_drain_req | output | NC | Pulses for the granted channel on the AW handshake. |
| axi_wr_drain_size | output | NC x 8 | Beats to reserve (AWLEN + 1). |
| axi_wr_drain_data_avail | input | NC x SCW | Data available per channel after reservations. |
| axi_wr_sram_valid | input | NC | Per-channel valid, registered. Used for arbitration. |
| axi_wr_sram_valid_comb | input | NC | Per-channel valid, combinational. Gates m_axi_wvalid. |
| axi_wr_sram_ | output | 1 | Drain request. Equal to |

| Signal | Direction | Width | Description |
|---|---|---|---|
| drain | | | wvalid && wready. |
| axi_wr_sram_id | output | CIW | Channel selected for the drain. |
| axi_wr_sram_data | input | DW | Data from the selected channel. |
| axi_wr_sram_strb | input | DW/8 | Byte strobes stored with that data beat. New in RAPIDS. |

### 9.3.4 AXI4 Write Master Interface

*Table 2.4.6: AXI4 Write Master Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| m_axi_awid | output | IW | Write address ID (the channel). |
| m_axi_awaddr | output | AW | Beat-aligned write address. |
| m_axi_awlen | output | 8 | Burst length minus one. |
| m_axi_awsize | output | 3 | AXSIZE, full-width beats. |
| m_axi_awburst | output | 2 | INCR. |
| m_axi_awvalid | output | 1 | Address valid. |
| m_axi_awready | input | 1 | Address ready. |
| m_axi_wdata | output | DW | Write data. |
| m_axi_wstrb | output | DW/8 | Write strobes, equal to axi_wr_sram_strb. |
| m_axi_wlast | output | 1 | Last beat of the burst. |
| m_axi_wuser | output | UW | Channel of the beat being written. |
| m_axi_wvalid | output | 1 | Data valid. |
| m_axi_wready | input | 1 | Data ready. |
| m_axi_bid | input | IW | Response ID. |
| m_axi_bresp | input | 2 | Write response. |

| Signal | Direction | Width | Description |
|---|---|---|---|
| m_axi_bvalid | input | 1 | Response valid. |
| m_axi_bready | output | 1 | Response ready. |

### 9.3.5 Debug and Status

*Table 2.4.7: Debug and Status Outputs*

| Signal | Direction | Width | Description |
|---|---|---|---|
| dbg_wr_all_complete | output | NC | Channel has no outstanding writes. |
| dbg_aw_transactions | output | 32 | Total AW handshakes. |
| dbg_w_beats | output | 32 | Total W beats. |
| o_active_channel_id | output | CIW | Channel whose burst is driving, or about to drive, the W bus. |
| o_active_channel_valid | output | 1 | o_active_channel_id is meaningful. |

## 9.4 Operation

### 9.4.1 Burst Length Cap (rapids BUG-009)

`cfg_axi_wr_xfer_beats` is an AWLEN value, but a burst cannot be larger than the SRAM that stages it: the engine waits for the whole burst to be present before issuing the AW, and a burst the buffer cannot hold would wait forever. The engine clamps the configured value to `XFER_MAX`. Without the clamp an AWLEN of 255 wraps the 8-bit `AWLEN + 1` and the engine issues an AW on an empty buffer. This is the same fix as in RAPIDS Beats.

### 9.4.2 Byte-Granular Burst Sizing

The sizing follows the read engine:

1. `w_beats_to_4k` is the distance from the aligned channel address to the next 4 KB boundary, in beats.
2. `w_cap_beats = min(w_xfer_cfg + 1, w_beats_to_4k)`.
3. `w_transfer_size` (the AWLEN) is `sched_wr_beats - 1` if `sched_wr_beats <= w_cap_beats`, otherwise `w_cap_beats - 1`.

A channel may issue when the buffer holds at least `w_transfer_size + 1` beats (`w_has_data`), or when this is the final burst. The final-burst test is `sched_wr_beats > 0`, `sched_wr_beats <= w_cap_beats`, and the available data covers all remaining beats. The 4 KB cap therefore applies to both tests, so a descriptor that ends just past a boundary still finishes on a short final burst.

`m_axi_awaddr` is `{sched_wr_addr[ch][AW-1:AXSIZE], AXSIZE'b0}`. The offset of a descriptor that starts mid-beat never reaches the bus. The first W beat of that descriptor carries `WSTRB` bits only for the bytes at and above the offset, and the last carries only the bytes up to the end of the packet.

### 9.4.3 Write Strobes

Each SRAM word holds a data beat and its strobe word side by side (see Sink Data Path). The write engine takes both from the drain port and presents them together, so the strobe always matches the data. In RAPIDS Beats `WSTRB` was tied high.

### 9.4.4 Streaming Pipeline

Each channel with `sched_wr_valid` and a burst that passes the data test requests arbitration. On grant the AW is issued, and on the AW handshake the engine pulses `axi_wr_drain_req` with the burst size so the drain controller reserves the beats. W beats stream from the drain port that `axi_wr_sram_id` selects. `axi_wr_sram_drain` equals `m_axi_wvalid && m_axi_wready`. `m_axi_wlast` closes each burst, and `m_axi_wuser` carries the channel.

With `PIPELINE = 1` a channel may issue further AWs before earlier B responses return, up to `AW_MAX_OUTSTANDING`. With `PIPELINE = 0` the channel waits for each B. That costs throughput, so the top and core default to 1.

### 9.4.5 Two Completion Strobes

*Table 2.4.8: Completion Strobes*

| Strobe | Raised when | Scheduler use |
|---|---|---|
| `sched_wr_done_strobe` | AW handshake for the channel | Advance the destination address and the issued-beat count. |
| `sched_wr_commit_strobe` | B response for the channel | Count committed beats. The descriptor is complete when all beats are committed. |

`sched_wr_ready` is a third, coarser pulse: it fires when the B response belongs to the last burst of a descriptor.

---

## 9.5 Error Handling

*Table 2.4.9: Error Handling*

| Error | Detection | Response |
|---|---|---|
| AXI SLVERR | `m_axi_bresp == 2'b10` | Set sticky `sched_wr_error` for the channel |
| AXI DECERR | `m_axi_bresp == 2'b11` | Set sticky `sched_wr_error` for the channel |
| Any other non-zero response | `m_axi_bresp != 2'b00` | Same: the test is not-OKAY, and the channel is taken from `BID` |

The sink data path ORs `sched_wr_error` with its own packet-length error before passing it to the scheduler.

### 9.5.1 Channel Reset

`cfg_channel_reset[ch]` clears the sticky `sched_wr_error[ch]`, so a channel reset recovers a B-response error without `aresetn`. A burst whose AW has been accepted must still receive all of its W beats, so the engine settles in-flight work rather than dropping it:

- `w_kill[ch]` is the reset level ORed with `r_wr_flush[ch]`. The flush bit is set by the reset and holds until the channel has no open burst and no B response due.
- No new AW is arbitrated for a channel with `w_kill` set. `AWADDR` comes from `sched_wr_addr`, which the scheduler does not clear, so the value on the bus stays stable across the reset.
- An open burst is completed with null W beats: `WSTRB` is zero, `WDATA` repeats the last captured value and the SRAM drain is not popped, so the null beats modify no byte at the destination. A beat that was presented on W and not accepted when the reset hit is the one exception: it is replayed unchanged, with its own strobes, from a captured copy, so the W channel does not change its payload while `WVALID` is high. At most one beat per reset is replayed this way.
- B responses of a channel with `w_kill` set are consumed. The commit strobe, `sched_wr_ready`, the done strobe and the error flag are suppressed for them.
- Other channels are not affected.

The reset therefore leaves no partial state in the engine: the B-phase FIFO drains through the flush, and the channel restarts with clean counters when the scheduler next raises `sched_wr_valid`.

---

## 9.6 Differences from RAPIDS Beats

*Table 2.4.10: Write Engine Delta*

| Item | RAPIDS Beats | RAPIDS |
|---|---|---|
| `WSTRB` | All ones | `axi_wr_sram_strb` |
| SRAM drain port | Data only | Data plus strobe word |
| `AWADDR` | Channel address as given (always aligned) | Aligned down to the beat |
| Burst cap and final-burst test | Configured length and remaining beats | Also capped at the 4 KB boundary |
| Everything else | | Unchanged |

The beats chapter is AXI Write Engine (RAPIDS Beats). The read-side counterpart is AXI Read Engine.

---

**Last Updated:** 2026-09-30

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 10 Beats Alloc Control Specification

**Module:** `alloc_ctrl_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

> **No longer in the SRAM datapath (2026-09-26).** RAPIDS' SRAM path now uses STREAM's `stream_alloc_ctrl` / `stream_drain_ctrl` / `stream_latency_bridge`, reached through the `snk_/src_sram_controller_beats` naming wrappers (`bdf4e0dff`). This FUB is still built and still verified – it keeps its own test, its own filelist and its `rapids_all.f` entry – it simply is not instantiated by the SRAM controllers any more. Treat this page as the FUB's own specification, not as a description of the current SRAM path.

---

## 10.1 Overview

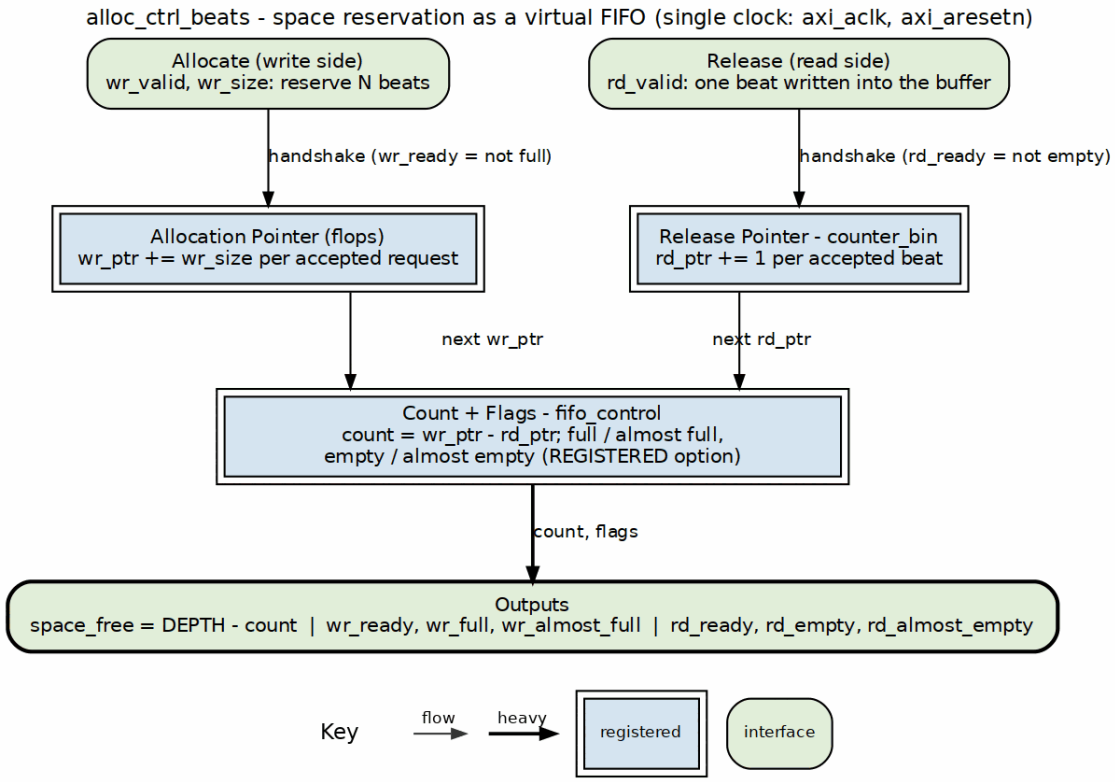
The Beats Alloc Control is a “virtual FIFO” that tracks space allocation without storing actual data. It uses FIFO pointer logic to manage pre-allocation for AXI engines, preventing race conditions between allocation and actual data arrival.

### 10.1.1 Key Features

- **Virtual FIFO:** Pointer tracking without data storage
- **Variable-Size Allocation:** Supports multi-beat allocation requests
- **Single-Beat Release:** Data writes release one beat at a time
- **Status Flags:** Full, almost full, empty, almost empty
- **Space Tracking:** Reports available space for flow control

### 10.1.2 Block Diagram

### 10.1.3 Figure 2.5.1: Beats Alloc Control Block Diagram




*Alloc control block diagram*

**Source:** 05_alloc_ctrl.dot

## 10.2 Concept: Virtual FIFO

Unlike a traditional FIFO that stores data, alloc_ctrl_beats only tracks **space reservations**:

| | Traditional FIFO | Virtual FIFO (alloc_ctrl_beats) |
|---|---|---|
| Write side | `wr_data` into storage | `wr_size` advances a pointer |
| Read side | `rd_data` out of storage | one beat advances a pointer |
| Output | data | `space_free` |

### 10.2.1 Use Case: Pre-Allocation for AXI Reads

```
1. AXI engine requests N beats of space
   - alloc_ctrl.wr_valid = 1
   - alloc_ctrl.wr_size = N
   - wr_ptr advances by N

2. As AXI read data arrives (1 beat at a time)
   - alloc_ctrl.rd_valid = 1 (each beat)
   - rd_ptr advances by 1

3. Space becomes available again
   - space_free reflects available capacity
```

## 10.3 Parameters

```
parameter int DEPTH = 512;                    // Virtual FIFO depth
parameter int ALMOST_WR_MARGIN = 1;           // Almost full margin
parameter int ALMOST_RD_MARGIN = 1;           // Almost empty margin
parameter int REGISTERED = 1;                 // Registered outputs

// Derived
parameter int AW = $clog2(DEPTH);             // Address width
```

: Table 2.5.1: Beats Alloc Control Parameters

## 10.4 Port List

### 10.4.1 Clock and Reset

*Table 2.5.2: Clock and Reset*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `axi_aclk` | input | 1 | System clock |
| `axi_aresetn` | input | 1 | Active-low reset |

### 10.4.2 Write Interface (Allocation Requests)

*Table 2.5.3: Write Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `wr_valid` | input | 1 | Allocate space |
| `wr_size` | input | 8 | Number of beats to allocate |
| `wr_ready` | output | 1 | Space available for allocation |

### 10.4.3 Read Interface (Data Written)

*Table 2.5.4: Read Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `rd_valid` | input | 1 | One beat of data written |
| `rd_ready` | output | 1 | Not full (can accept) |

### 10.4.4 Status Outputs

*Table 2.5.5: Status Outputs*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `space_free` | output | AW+1 | Available space in beats |
| `wr_full` | output | 1 | No space |

| Signal | Direction | Width | Description |
|---|---|---|---|
| | | | available |
| wr_almost_full | output | 1 | Near full |
| rd_empty | output | 1 | No allocations pending |
| rd_almost_empty | output | 1 | Near empty |

## 10.5 Operation

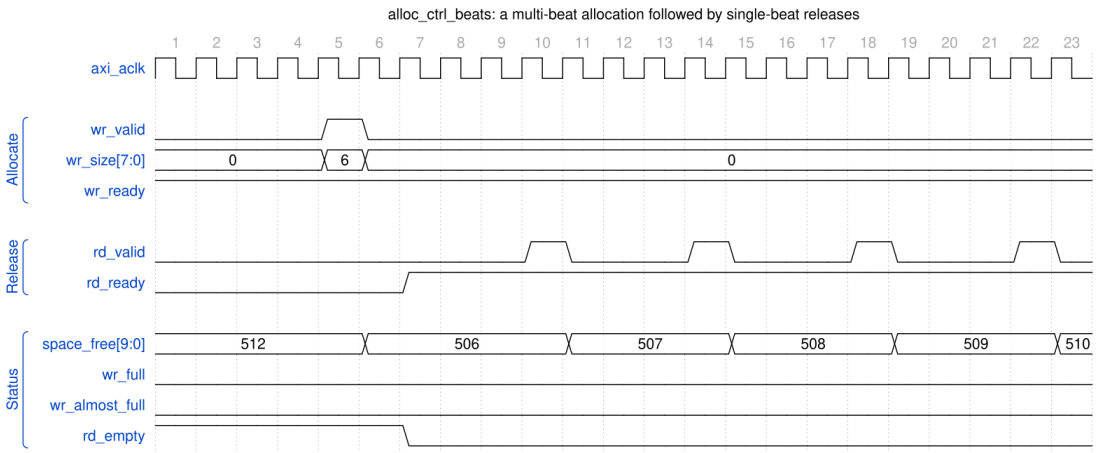
### 10.5.1 Pointer Arithmetic

```
space_free = DEPTH - (wr_ptr - rd_ptr)

Full condition:    wr_ptr - rd_ptr >= DEPTH
Empty condition:   wr_ptr == rd_ptr
```

### 10.5.2 Timing Diagram

### 10.5.3 Figure 2.5.2: Allocation and Release Timing




*Allocation control - allocate then release*

**Source:** alloc_ctrl_beats_alloc_release.json, captured from `dv/tests/fub_beats/test_alloc_ctrl_beats.py` (`basic_alloc_drain`, depth 512, seed 7) with `WAVES=1`.

Reading it: one `wr_valid` handshake with `wr_size = 6` reserves six beats and `space_free` drops 512 to 506 the next cycle; `rd_empty` falls because reserved space is now outstanding. Each later `rd_valid` handshake releases one beat and `space_free` climbs back one at a time (507, 508, ...). `wr_full` and `wr_almost_full` stay low: the virtual FIFO is nowhere near its depth.

---

## 10.6 Integration Example

```
// Source path: Pre-allocate SRAM space before AXI read
alloc_ctrl_beats #(
    .DEPTH(SRAM_DEPTH),
    .ALMOST_WR_MARGIN(8)
) u_src_alloc (
    .axi_aclk       (clk),
    .axi_aresetn    (rst_n),

    // Scheduler allocates space before issuing AXI read
    .wr_valid       (sched_rd_valid),
    .wr_size        (sched_rd_beats[7:0]),
    .wr_ready       (alloc_ready),

    // AXI read data arrival releases allocation
    .rd_valid       (m_axi_rvalid && m_axi_rready),
    .rd_ready       (),  // Not used for single-beat

    // Status
    .space_free     (sram_space_available),
    .wr_full        (sram_full),
    .wr_almost_full (sram_almost_full)
);
```

---

**Last Updated:** 2025-01-10

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 11 Beats Drain Control Specification

**Module:** `drain_ctrl_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

> **No longer in the SRAM datapath (2026-09-26).** RAPIDS' SRAM path now uses STREAM's `stream_alloc_ctrl` / `stream_drain_ctrl` / `stream_latency_bridge`, reached through the `snk_/src_sram_controller_beats` naming wrappers (`bdf4e0dff`). This FUB is still built and still verified – it keeps its own test, its own filelist and its `rapids_all.f` entry – it simply is not instantiated by the SRAM controllers any more. Treat this page as the FUB's own specification, not as a description of the current SRAM path.

---

## 11.1 Overview

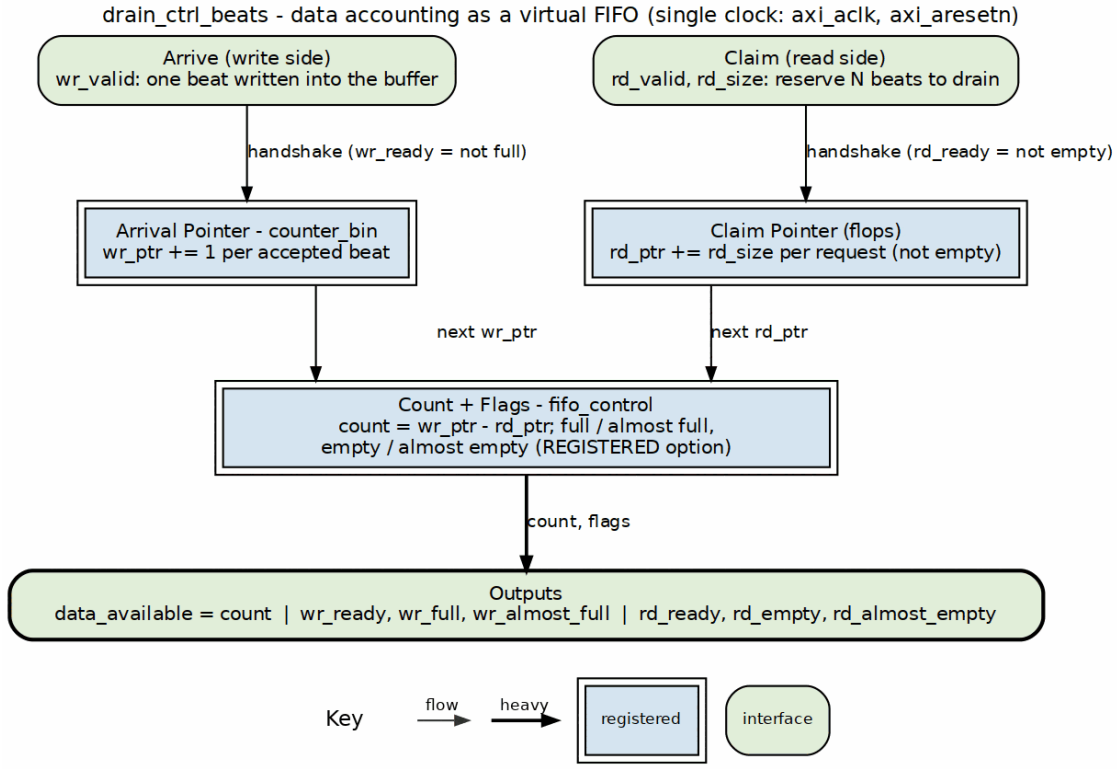
The Beats Drain Control is a “virtual FIFO” that tracks data availability without storing actual data. It complements alloc_ctrl_beats by tracking when data has been written to SRAM and is available for draining.

### 11.1.1 Key Features

- **Virtual FIFO:** Pointer tracking without data storage
- **Single-Beat Write:** Data arrivals tracked one beat at a time
- **Variable-Size Drain:** Supports multi-beat drain requests
- **Status Flags:** Full, almost full, empty, almost empty
- **Data Availability:** Reports beats available for drain

### 11.1.2 Block Diagram

### 11.1.3 Figure 2.6.1: Beats Drain Control Block Diagram




*Drain control block diagram*

**Source:** 06_drain_ctrl.dot

## 11.2 Concept: Data Availability Tracking

The drain_ctrl tracks when data has been written to SRAM:

```
alloc_ctrl:  Tracks SPACE reservations (allocation -> actual write)
drain_ctrl:  Tracks DATA availability (data write -> drain request)
```

### 11.2.1 Complementary Roles

```
Source Path:
  1. Scheduler allocates space (alloc_ctrl.wr)
  2. AXI reads data into SRAM
  3. Data written to SRAM (drain_ctrl.wr)
  4. Drain consumer reads (drain_ctrl.rd)

Sink Path:
  1. Fill allocates space (alloc_ctrl.wr)
  2. Fill writes to SRAM (alloc_ctrl.rd)
  3. Data available in SRAM (drain_ctrl.wr)
  4. AXI write drains SRAM (drain_ctrl.rd)
```

---

## 11.3 Parameters

```
parameter int DEPTH = 512;                    // Virtual FIFO depth
parameter int ALMOST_WR_MARGIN = 1;           // Almost full margin
parameter int ALMOST_RD_MARGIN = 1;           // Almost empty margin
parameter int REGISTERED = 1;                 // Registered outputs

// Derived
parameter int AW = $clog2(DEPTH);             // Address width
```

: Table 2.6.1: Beats Drain Control Parameters

## 11.4 Port List

### 11.4.1 Clock and Reset

*Table 2.6.2: Clock and Reset*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `axi_aclk` | input | 1 | System clock |
| `axi_aresetn` | input | 1 | Active-low reset |

### 11.4.2 Write Interface (Data Written to SRAM)

*Table 2.6.3: Write Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `wr_valid` | input | 1 | One beat written to SRAM |
| `wr_ready` | output | 1 | Can track more data |

### 11.4.3 Read Interface (Drain Requests)

*Table 2.6.4: Read Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `rd_valid` | input | 1 | Drain request |
| `rd_size` | input | 8 | Number of beats to drain |
| `rd_ready` | output | 1 | Data available for drain |

### 11.4.4 Status Outputs

*Table 2.6.5: Status Outputs*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `data_available` | output | AW+1 | Beats available to drain |
| `wr_full` | output | 1 | Tracking full |

| Signal | Direction | Width | Description |
|---|---|---|---|
| wr_almost_full | output | 1 | Near full |
| rd_empty | output | 1 | No data to drain |
| rd_almost_empty | output | 1 | Near empty |

## 11.5 Operation

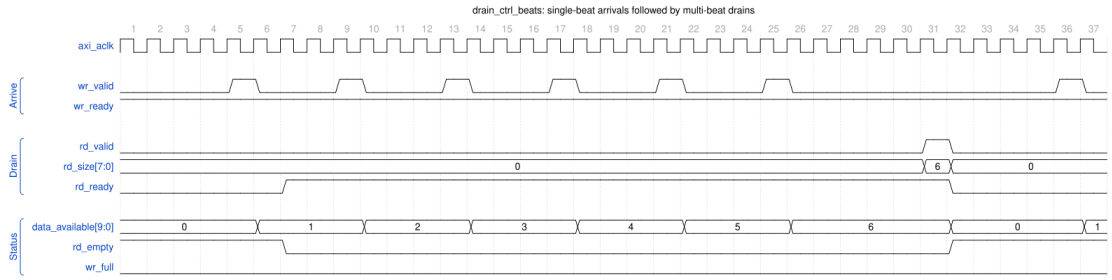
### 11.5.1 Pointer Arithmetic

```
data_avail = wr_ptr - rd_ptr

Full condition:    wr_ptr - rd_ptr >= DEPTH
Empty condition:   wr_ptr == rd_ptr
```

### 11.5.2 Timing Diagram

### 11.5.3 Figure 2.6.2: Data Arrival and Drain Timing




*Drain control - single-beat arrivals then a multi-beat drain*

**Source:** drain_ctrl_beats_arrive_drain.json, captured from `dv/tests/fub_beats/test_drain_ctrl_beats.py` (`basic_write_drain`, depth 512, seed 7) with `WAVES=1`.

Reading it: six `wr_valid` handshakes, one beat each, raise `data_available` 1, 2, … 6 and drop `rd_empty` on the first arrival. A single `rd_valid` handshake with `rd_size = 6` then reserves all six at once: `data_available` returns to 0 and `rd_empty` rises the next cycle. The counter tracks beats, not bytes, and never touches the data itself.

---

## 11.6 Integration Example

```
// Sink path: Track data availability for AXI write engine
drain_ctrl_beats #(
    .DEPTH(SRAM_DEPTH),
    .ALMOST_RD_MARGIN(8)
) u_snk_drain (
    .axi_aclk       (clk),
    .axi_aresetn    (rst_n),

    // Fill writes data to SRAM
    .wr_valid       (fill_valid && fill_ready),
    .wr_ready       (),  // Single-beat, always ready

    // AXI write engine drains data
    .rd_valid       (axi_wr_drain_req),
    .rd_size        (axi_wr_burst_len),
    .rd_ready       (data_ready_for_drain),

    // Status
    .data_available (beats_to_write),
    .rd_empty       (no_data_to_write)
);
```

---

## 11.7 Relationship: alloc_ctrl vs drain_ctrl

*Table 2.6.6: alloc_ctrl vs drain_ctrl Comparison*

| Aspect | alloc_ctrl_beats | drain_ctrl_beats |
|---|---|---|
| **Tracks** | Space reservations | Data availability |
| **Write port** | Multi-beat allocation | Single-beat arrival |
| **Read port** | Single-beat release | Multi-beat drain |
| **Output** | `space_free` | `data_available` |
| **Use case** | Pre-allocate before fill | Know when to drain |

**Last Updated:** 2025-01-10

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

# 12 Latency Bridge (beats) Specification

**Module:** `latency_bridge_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub_beats/` **Status:** Implemented, tested, not in the SRAM path **Last Updated:** 2026-09-27

> **Not in the SRAM datapath since 2026-09-26.** RAPIDS' SRAM path is STREAM's `sram_controller`, which carries its own `stream_latency_bridge` (the same design), reached through the `snk_/src_sram_controller_beats` naming wrappers (`bdf4e0dff`). This FUB is still built and verified – it keeps its test, its filelist and its `rapids_all.f` entry – but nothing in `rapids_core_beats` instantiates it. This page is the FUB's own specification.

---

## 12.1 Overview

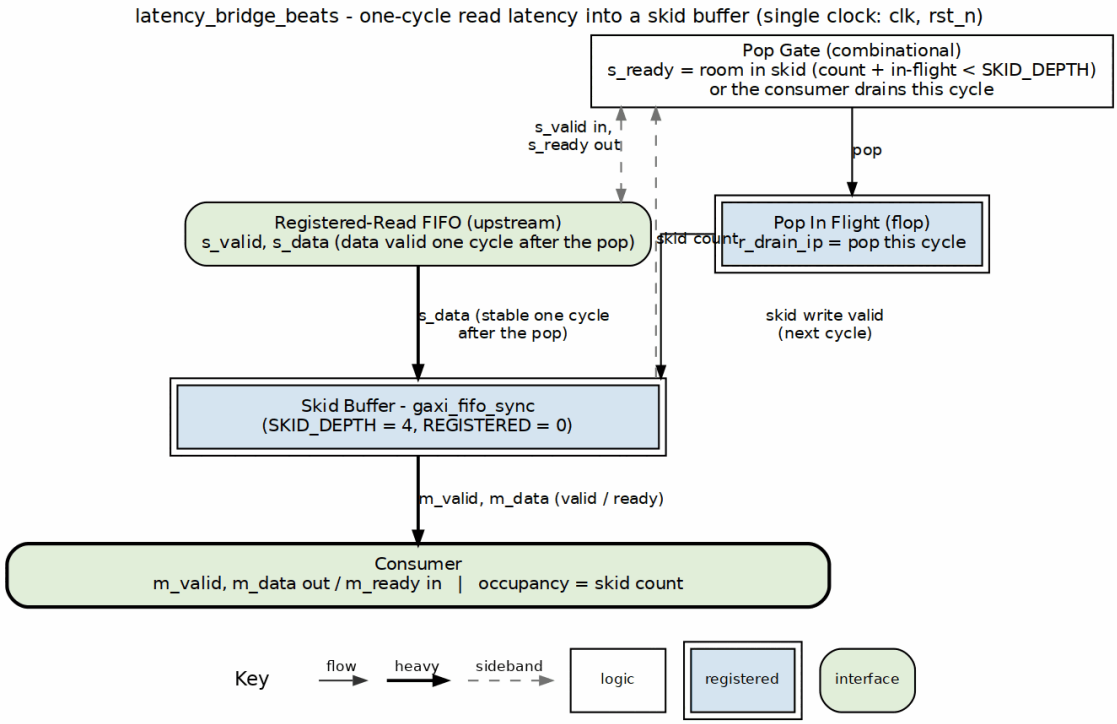
`latency_bridge_beats` turns a **registered-read FIFO** – one whose data appears the cycle *after* the read handshake – into a plain **valid/ready stream** the consumer can stall at will. It is a data-width bridge: it moves `DATA_WIDTH` bits per beat and carries nothing else. There is no beat count, no channel ID and no programmable delay in it; an earlier version of this page described such an interface, and it never existed in this module.

### 12.1.1 Key Features

- **One-cycle glue, one skid buffer:** a single flop (`r_drain_ip`) remembers that a FIFO read was accepted, so the data that lands the next cycle is pushed into a `gaxi_fifo_sync` skid of depth `SKID_DEPTH`
- **Full throughput when the consumer is ready:** back-to-back reads are accepted as long as the skid keeps draining
- **Backpressure absorbed in the skid:** `s_ready` drops only when a skid write is actually stalled, so the FIFO is never read into a slot that does not exist
- **Occupancy reported:** `occupancy` counts the beats the bridge holds (the one in flight plus the skid contents), which is what an occupancy-based data-available counter needs
- **Debug taps:** `dbg_r_pending` and `dbg_r_out_valid` expose the in-flight flop and the skid output valid, for catching stuck data from a test

### 12.1.2 Block Diagram

### 12.1.3 Figure 2.7.1: Latency Bridge Block Diagram




*Latency bridge block diagram*

**Source:** 07_latency_bridge.dot

---

## 12.2 Concept: bridging a registered read

A FIFO with `REGISTERED=1` read (the SRAM style this repo uses so BRAM infers cleanly) answers a read handshake with data **one cycle later**. A consumer speaking valid/ready expects data *with* valid, and may deassert ready at any time. Without a bridge the FIFO has already popped a beat the consumer did not take.

```
cycle 0   s_valid && s_ready         FIFO read accepted
r_drain_ip <= 1
cycle 1   data arrives on s_data     skid_wr_valid = r_drain_ip, data
pushed
cycle N   m_valid && m_ready         consumer drains the skid at its
own pace
```

The bridge never guesses about downstream readiness: `s_ready` is derived from the skid's room, counting only a write that is *stalled* this cycle as pending (a write that completes this cycle frees its slot at the same edge). That is why full throughput holds while writes complete immediately, and why the FIFO is never over-read when they do not.

---

## 12.3 Parameters

```
parameter int DATA_WIDTH = 64;   // Beat width
parameter int SKID_DEPTH = 4;    // Skid buffer depth (2-4
recommended)
parameter int DW = DATA_WIDTH;   // Short alias
```

: Table 2.7.1: Latency Bridge Parameters

`occupancy` is 3 bits wide and counts 0..5 at the default depth (one beat in flight plus four in the skid); a larger `SKID_DEPTH` needs the port widened.

---

## 12.4 Port List

### 12.4.1 Clock and Reset

*Table 2.7.2: Clock and Reset*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `clk` | input | 1 | System clock |
| `rst_n` | input | 1 | Active-low reset |

### 12.4.2 Upstream Interface (from the registered FIFO)

*Table 2.7.3: Upstream Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `s_valid` | input | 1 | FIFO has a beat to read (not empty) |
| `s_ready` | output | 1 | Bridge accepts a read this cycle; the data lands NEXT cycle |
| `s_data` | input | DW | Read data, valid one cycle after the `s_valid && s_ready` handshake |

### 12.4.3 Downstream Interface (to the consumer)

*Table 2.7.4: Downstream Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `m_valid` | output | 1 | Beat available |
| `m_ready` | input | 1 | Consumer takes the beat |
| `m_data` | output | DW | Beat data, valid with `m_valid` |

### 12.4.4 Status and Debug

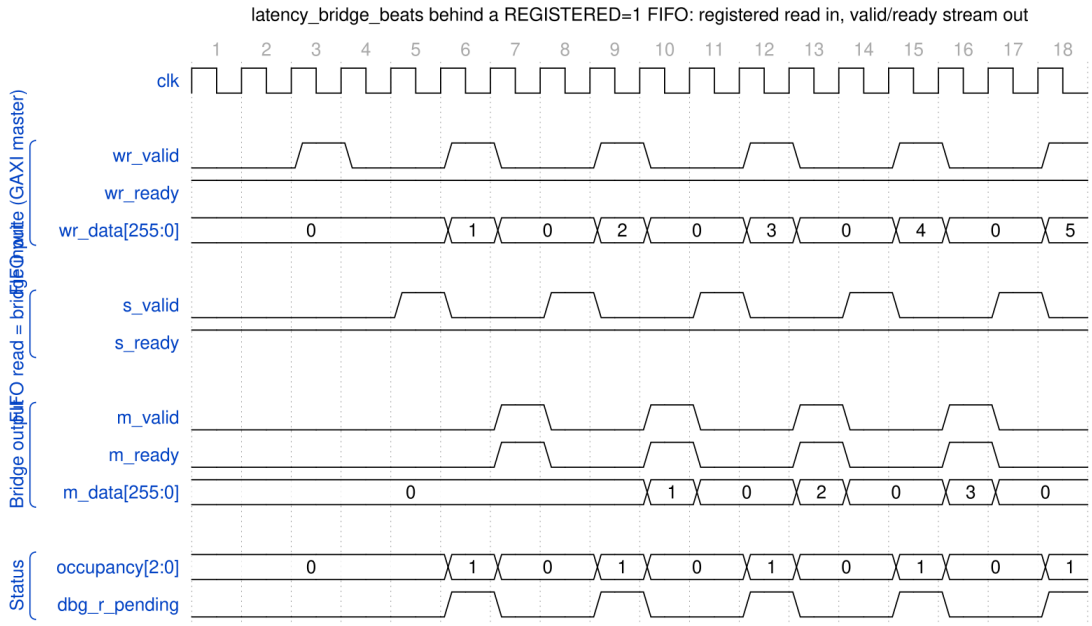
*Table 2.7.5: Status and Debug*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `occupancy` | output | 3 | Beats held by the bridge: |

| Signal | Direction | Width | Description |
|---|---|---|---|
| | | | the in-flight read plus the skid contents (0..5) |
| `dbg_r_pending` | output | 1 | `r_drain_ip`: a read was accepted and its data is due next cycle |
| `dbg_r_out_valid` | output | 1 | Skid output valid (mirrors `m_valid`) |

## 12.5 Operation

### 12.5.1 Figure 2.7.2: Latency Bridge Timing (consumer always ready)




*Latency bridge behind a registered-read FIFO*

**Source:** latency_bridge_beats_streaming.json, captured from `dv/tests/fub_beats/test_latency_bridge_beats.py` (`streaming`, 256-bit, seed 7) with `WAVES=1`. The test drives a `REGISTERED = 1` `gaxi_fifo_sync` in front of the bridge (`dv/tb/latency_bridge_beats_tb_top.sv`), because that is the only way to produce the bridge's upstream contract: a valid/ready master presents data *with* valid, a registered FIFO presents it one cycle *after* the read handshake.

Reading it: each `wr_valid` beat lands in the FIFO and `s_valid` (FIFO not empty) rises a cycle later; the bridge reads it at once (`s_ready` stays high) and `dbg_r_pending` marks the cycle the data is in flight. Two cycles after the write the beat is on `m_valid`/`m_data` with its payload intact (1, 2, 3, ...), and `occupancy` shows exactly one beat inside the bridge per transfer. With a consumer that is always ready the bridge adds the FIFO's read latency and no more.

An earlier revision of this test drove `s_*` directly from the BFM and never compared `m_data`; every beat came out 0 and the test passed (rapids TASK-003). (`dv/tests/fub_beats/test_latency_bridge_beats.py`).

## 12.6 Integration Context

The bridge sits between a `gaxi_fifo_sync` with `REGISTERED=1` and whatever consumes the stream. The same arrangement, with STREAM's twin `stream_latency_bridge`, is what `sram_controller_unit` uses inside the shared SRAM controller today; `occupancy` there is what the drain accounting must allow for (stream BUG-011).

```
latency_bridge_beats #(
    .DATA_WIDTH (512),
    .SKID_DEPTH (4)
) u_bridge (
    .clk             (clk),
    .rst_n           (rst_n),
    // registered-read FIFO side
    .s_valid         (fifo_rd_valid),
    .s_ready         (fifo_rd_ready),
    .s_data          (fifo_rd_data),
    // consumer side
    .m_valid         (drain_valid),
    .m_ready         (drain_read),
    .m_data          (drain_data),
    // status / debug
    .occupancy       (bridge_occupancy),
    .dbg_r_pending   (dbg_bridge_pending),
    .dbg_r_out_valid (dbg_bridge_out_valid)
);
```

---

## 12.7 Design Considerations

*Table 2.7.6: Skid Depth Selection*

| SKID_DEPTH | Behaviour |
|---|---|
| 2 | Minimum: one stall of the consumer costs a bubble on the next read |
| 4 | Default: absorbs a short consumer stall with no upstream bubble |
| 8 | Deeper stall absorption; `occupancy` must be widened past 3 bits |

The skid does not need to cover the FIFO's read latency (that is the one `r_drain_ip` flop's job); it needs to cover how long the *consumer* stalls without the bridge stopping upstream reads.

---

**Last Updated:** 2026-09-27

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 13 Control-Read Engine Specification

**Module:** `ctrlrd_engine.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub/`
**Status:** Implemented **Last Updated:** 2026-07-12

---

## 13.1 Overview

The Control-Read Engine implements the CTRL_READ descriptor opcode (DESC_OP_CTRL_READ = 2'b01) as a **consumer gate**. When the scheduler decodes a CTRL_READ descriptor, it hands the engine a poll address, an expected value, and a compare mask. The engine issues an AXI4 read to the poll address, applies the mask to the returned data, and compares (read_data & mask) against the expected value. If they match, the gate is satisfied and the descriptor chain proceeds; if not, the engine retries. Retries are bounded so a gate that never matches cannot hang the channel.

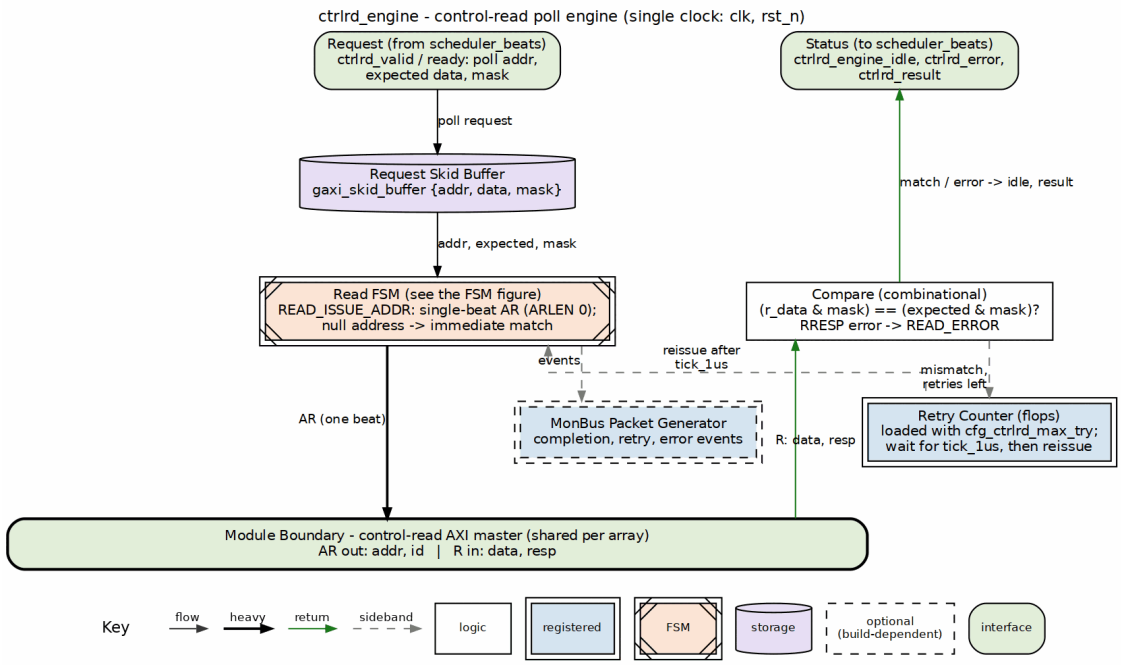
One Control-Read Engine is instantiated per channel in scheduler_group_beats.sv (u_ctrlrd_engine), configured for 32-bit AXI reads.

### 13.1.1 Key Features

- **Consumer Gate Semantics:** Polls memory until (read_data & mask) == expected
- **Masked Comparison:** Only the masked bits of the polled word participate in the match
- **Bounded Retry:** cfg_ctrlrd_max_try (0-511) caps the number of poll attempts
- **1 microsecond Retry Pacing:** Retries wait for tick_1us from the scheduler group
- **Error on Exhaustion:** Asserts ctrlrd_error if the gate never matches within budget
- **Null Address Fast-Path:** A poll address of zero completes immediately as a match
- **Channel Reset Support:** cfg_channel_reset returns the FSM to idle and drains any read still on the fabric
- **MonBus Integration:** Completion, retry, and error events

### 13.1.2 Block Diagram

### 13.1.3 Figure 2.8.1: Control-Read Engine Block Diagram




*Control-read engine block diagram*

**Source:** 08_ctrlrd_engine.dot

---

## 13.2 Parameters

```
parameter int CHANNEL_ID = 0;                    // Channel identifier
parameter int NUM_CHANNELS = 32;                 // Total channels
parameter int CHAN_WIDTH = $clog2(NUM_CHANNELS); // Channel index
width (derived)
parameter int ADDR_WIDTH = 64;                   // Address bus width
parameter int AXI_DATA_WIDTH = 64;               // AXI read data
width (>= 32)
parameter int AXI_ID_WIDTH = 8;                  // AXI ID width (>=
CHAN_WIDTH)

// Monitor Bus Parameters
parameter logic [15:0] MON_AGENT_ID  = 16'h0030; // Control-Read
Engine Agent ID
parameter logic [7:0]  MON_UNIT_ID   = 8'h02;    // Unit identifier
parameter logic [8:0]  MON_CHANNEL_ID = 9'h000;  // Base channel ID
```

: Table 2.8.1: Control-Read Engine Parameters

Parameter validation enforces `AXI_ID_WIDTH >= CHAN_WIDTH` and `AXI_DATA_WIDTH >= 32` (the engine consumes only the lower 32 bits of the read data). The per-channel instance in `scheduler_group_beats.sv` overrides `AXI_DATA_WIDTH` to 32.

---

## 13.3 Port List

### 13.3.1 Clock and Reset

*Table 2.8.2: Clock and Reset*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `clk` | input | 1 | System clock |
| `rst_n` | input | 1 | Active-low asynchronous reset |

### 13.3.2 Scheduler Interface

*Table 2.8.3: Scheduler Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `ctrlrd_valid` | input | 1 | Gate request valid from scheduler |
| `ctrlrd_ready` | output | 1 | Engine ready to accept a request |
| `ctrlrd_pkt_addr` | input | AW | Poll address (descriptor `poll_addr[63:0]`) |
| `ctrlrd_pkt_data` | input | 32 | Expected value (descriptor `expected[95:64]`) |
| `ctrlrd_pkt_mask` | input | 32 | Compare mask (descriptor `mask[127:96]`) |
| `ctrlrd_error` | output | 1 | Gate failed (retries exhausted or AXI error) |
| `ctrlrd_result` | output | 32 | Last read value (lower 32 bits) |
| `ctrlrd_engine_idle` | output | 1 | Engine idle (post-issue completion indicator) |

### 13.3.3 Configuration Interface

*Table 2.8.4: Configuration Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `cfg_ctrlrd_max_try` | input | 9 | Poll retry budget, 0-511 (from `CTRL_CONFIG.CTRLRD_MAX_TRY`) |
| `cfg_channel_reset` | input | 1 | Per-channel reset request |
| `tick_1us` | input | 1 | 1 microsecond tick that paces retries |

### 13.3.4 AXI4 Read Master Interface

*Table 2.8.5: AXI4 Read Master Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `ar_valid` | output | 1 | Read address valid |
| `ar_ready` | input | 1 | Read address ready |
| `ar_addr` | output | AW | Read address (poll address) |
| `ar_len` | output | 8 | Burst length (`8'h00`, single beat) |
| `ar_size` | output | 3 | Burst size (`3'b010`, 4 bytes) |
| `ar_burst` | output | 2 | Burst type (`2'b01`, INCR) |
| `ar_id` | output | ID | Transaction ID (channel-derived) |
| `ar_lock` | output | 1 | Lock (`1'b0`, normal access) |
| `ar_cache` | output | 4 | Cache (`4'b0010`, non-cacheable bufferable) |
| `ar_prot` | output | 3 | Protection (`3'b000`) |
| `ar_qos` | output | 4 | Quality of service (`4'h0`) |
| `ar_region` | output | 4 | Region (`4'h0`) |
| `r_valid` | input | 1 | Read data valid (shared |

| Signal | Direction | Width | Description |
|---|---|---|---|
| | | | channel) |
| r_ready | output | 1 | Read data ready |
| r_data | input | ADW | Read data (lower 32 bits used) |
| r_id | input | ID | Read response ID (matched to our request) |
| r_resp | input | 2 | Read response |
| r_last | input | 1 | Read last beat |

### 13.3.5 MonBus Interface

*Table 2.8.6: MonBus Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| i_mon_time | input | 64 | Side-band monitor timestamp |
| mon_valid | output | 1 | Monitor packet valid |
| mon_ready | input | 1 | Monitor consumer ready |
| mon_packet | output | 128 | Monitor packet (monitor_packet_t) |
| mon_timestamp | output | 64 | Monitor packet timestamp |

## 13.4 FSM States

### 13.4.1 Figure 2.8.2: Control-Read Engine FSM

States and transitions of the `ctrlrd_engine` poll FSM.

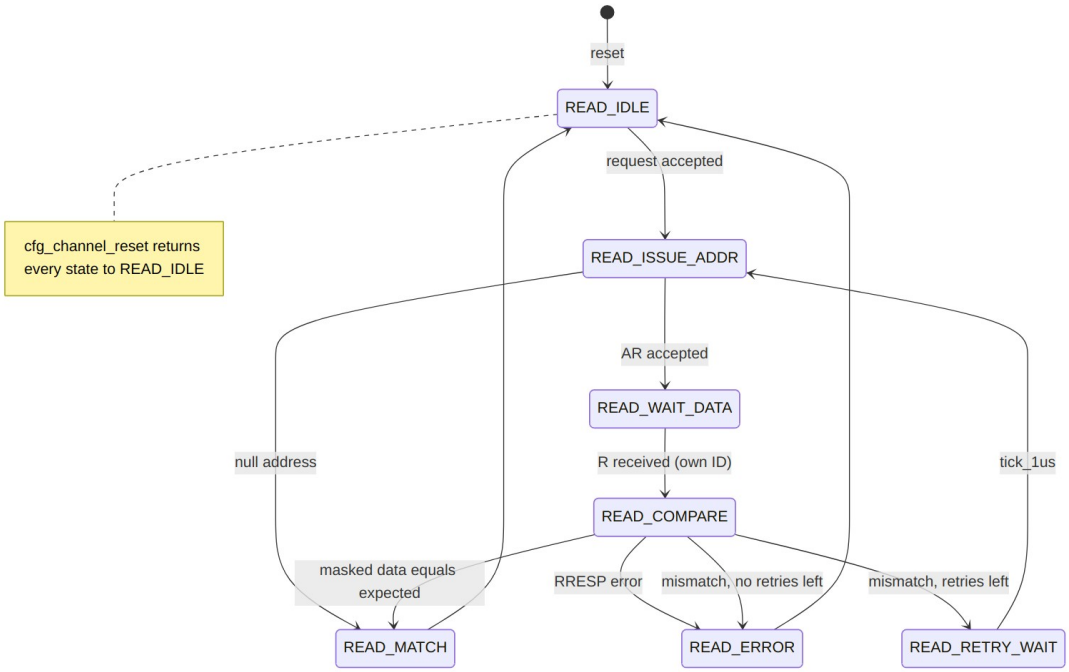



*Diagram 2*

State summary:

- **READ_IDLE:** Waits for a request from the scheduler (via a two-deep skid buffer). Latches poll address, expected value, mask, and the AXI ID; initializes the retry counter from `cfg_ctrlrd_max_try`.
- **READ_ISSUE_ADDR:** Drives `ar_valid`. A null address (`64'h0`) shortcuts to `READ_MATCH`; otherwise, once `ar_ready` handshakes, advances to `READ_WAIT_DATA`.
- **READ_WAIT_DATA:** Accepts the read response whose `r_id` matches the channel ID, capturing the lower 32 bits and the response code.
- **READ_COMPARE:** On an AXI error response, goes to `READ_ERROR`. On a masked match, goes to `READ_MATCH`. Otherwise, if retries remain, decrements the counter and goes to `READ_RETRY_WAIT`; if the budget is exhausted, goes to `READ_ERROR`.
- **READ_RETRY_WAIT:** Waits for `tick_1us`, then reissues the read.

- **READ_MATCH:** Gate satisfied. Clears the error flag and returns to `READ_IDLE`.
- **READ_ERROR:** Sets `ctrlrd_error` and returns to `READ_IDLE`.

---

## 13.5 Operation

### 13.5.1 Masked Compare

The engine gates on a masked equality check:

```
w_masked_expected = ctrlrd_pkt_data & ctrlrd_pkt_mask
w_masked_actual   = r_data[31:0]    & ctrlrd_pkt_mask
w_data_match      = (w_masked_expected == w_masked_actual)
```

Only the bits set in `mask` participate. A mask of `32'hFFFFFFFF` compares the full word; a mask of a single bit turns the gate into a flag-poll.

### 13.5.2 Retry Budget and Pacing

The retry counter is loaded from `cfg_ctrlrd_max_try` when the request is accepted. On each non-matching, non-error compare with retries remaining, the counter decrements and the engine parks in `READ_RETRY_WAIT` until the next `tick_1us`, which paces successive polls at roughly 1 microsecond intervals. When the counter reaches zero without a match, the engine transitions to `READ_ERROR` and asserts `ctrlrd_error`, so a never-matching gate cannot stall the channel indefinitely.

`cfg_ctrlrd_max_try` is driven by the `CTRL_CONFIG` register at offset `0x240`, field `CTRLRD_MAX_TRY[8:0]` (0-511, reset 16).

### 13.5.3 Null Address Fast-Path

If the latched poll address is `64'h0`, the engine treats the gate as already satisfied: `READ_ISSUE_ADDR` transitions directly to `READ_MATCH` without issuing an AXI read. This lets software emit an unconditional (no-op) gate.

### 13.5.4 AXI Read Transaction

Polls are single-beat, 4-byte INCR reads (`ar_len = 0`, `ar_size = 3'b010`, `ar_burst = 2'b01`). The `ar_id` is derived from `CHANNEL_ID`, and only responses whose `r_id` matches are consumed on the shared read data channel. `ctrlrd_result` exposes the last captured 32-bit value; in the per-channel instantiation the scheduler gates on match/error rather than the raw value, so `ctrlrd_result` is left unconnected there.

### 13.5.5 Channel Reset

`cfg_channel_reset` is registered into `r_channel_reset_active`. While asserted, the engine refuses new requests (`ctrlrd_ready` is forced low), the retry and error state is cleared, and the FSM is driven back to `READ_IDLE`.

What the reset cannot do is cancel a read already on the fabric, because AXI has no cancel. The engine therefore drains it (rapids TASK-014, 2026-09-27):

- an AR that was raised and not yet accepted stays raised until the slave takes it (`r_drain_ar`), then
- the R beat that the accepted AR is owed is taken and discarded (`r_drain_pending`, which alone lifts `r_ready` outside `READ_WAIT_DATA`).

Until the beat is gone `ctrlrd_engine_idle` stays low and the request path stays closed, so the next read on this ID cannot be answered by the stale beat. Before this the reset dropped the FSM to idle with `r_ready` low, the beat sat on the bus, and a re-kicked channel took it as its own answer.

`ctrlrd_engine_idle` asserts only in `READ_IDLE` with no pending request, no active channel reset and nothing left to drain.

---

## 13.6 MonBus Reporting

The engine emits monitor packets on notable events:

*Table 2.8.7: MonBus Events*

| Event | Packet Type | Code | Payload |
|---|---|---|---|
| Gate satisfied (non-null) | `PktTypeCompletion` | `CORE_COMPL_CTRLRD_COMPLETED` | Poll address |
| Retry attempt | `PktTypePerf` | `CORE_PERF_CTRLRD_RETRY` | Remaining retry count |
| Retries exhausted | `PktTypeError` | `CORE_ERR_CTRLRD_MAX_RETRIES` | Poll address |
| AXI error response | `PktTypeError` | `AXI_ERR_RESP_SLVERR` / `AXI_ERR_RESP_DECERR` | Response code |

All packets carry `MON_AGENT_ID`, `MON_UNIT_ID`, and `MON_CHANNEL_ID`, and are timestamped from `i_mon_time`.

---

## 13.7 Descriptor Field Mapping

The CTRL_READ opcode reinterprets the shared 256-bit descriptor (the descriptor engine's existing field extraction is reused):

*Table 2.8.8: CTRL_READ Descriptor Field Mapping*

| Descriptor Field | Bits | Engine Port |
|---|---|---|
| poll_addr | [63:0] | ctrlrd_pkt_addr |
| expected | [95:64] | ctrlrd_pkt_data |
| mask | [127:96] | ctrlrd_pkt_mask |
| max_try | [143:128] | Per-descriptor budget (0 = use cfg_ctrlrd_max_try default) |

**Last Updated:** 2026-07-12

# 14 Control-Write Engine Specification

**Module:** `ctrlwr_engine.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub/`
**Status:** Implemented **Last Updated:** 2025-01-10

---

## 14.1 Overview

The Control-Write Engine executes a CTRL_WRITE control descriptor (DESC_OP_CTRL_WRITE = 2'b10). A CTRL_WRITE is a producer doorbell: the engine performs a single-beat AXI4 write of the descriptor's 32-bit wr_data value to its 64-bit wr_addr target, then completes so the descriptor chain continues. One instance is dedicated per channel.

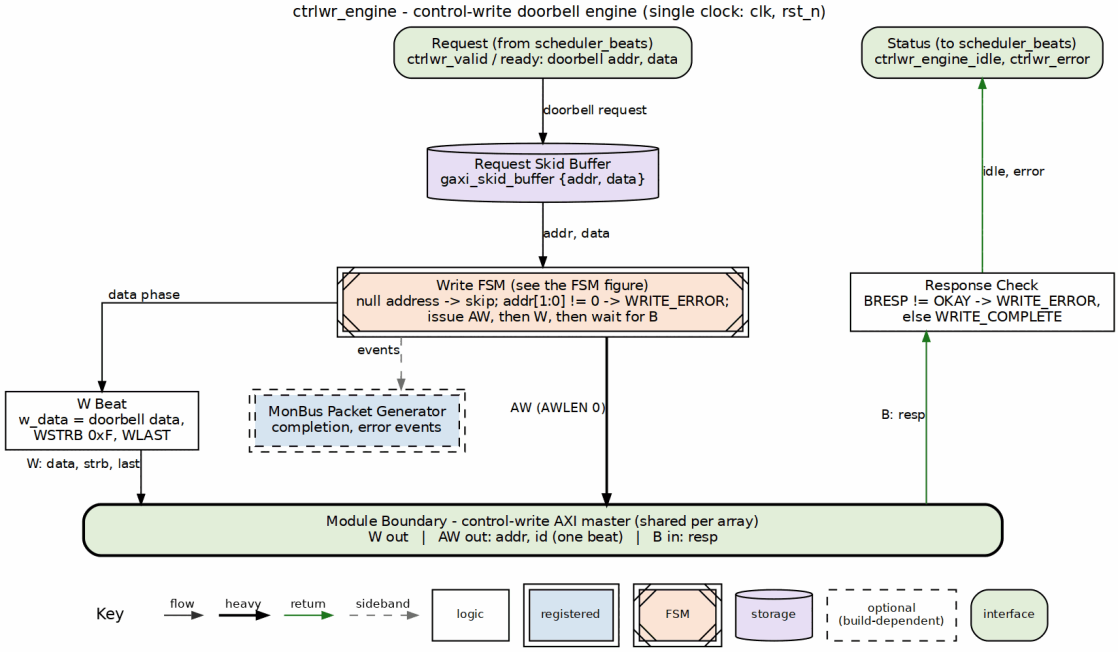
The write target address must be 4-byte aligned because the transfer is a 32-bit (aw_size = 3'b010) single-beat write. A null address (wr_addr == 0) is treated as a no-op and the request completes without issuing an AXI transaction.

### 14.1.1 Key Features

- **Single-Beat AXI4 Write:** One 32-bit doorbell write per request (aw_len = 0, w_last = 1)
- **Producer Doorbell Semantics:** Write value to target, then release the chain (no poll, no retry)
- **Address Alignment Check:** Requires 4-byte-aligned wr_addr; misalignment raises an error
- **Null-Address Skip:** wr_addr == 0 completes as a no-op
- **Channel Reset Support:** cfg_channel_reset returns the FSM to idle and drains any write still on the fabric
- **MonBus Integration:** Completion and error event reporting

### 14.1.2 Block Diagram

### 14.1.3 Figure 2.9.1: Control-Write Engine Block Diagram




*Control-write engine block diagram*

**Source:** 09_ctrlwr_engine.dot

---

## 14.2 Parameters

```
parameter int CHANNEL_ID = 0;                    // Channel identifier
parameter int NUM_CHANNELS = 32;                 // Total channels
parameter int CHAN_WIDTH = $clog2(NUM_CHANNELS); // Derived channel
index width
parameter int ADDR_WIDTH = 64;                   // Address bus width
parameter int AXI_ID_WIDTH = 8;                  // AXI transaction ID
width


// Monitor Bus Parameters
parameter logic [15:0] MON_AGENT_ID  = 16'h0020; // Ctrlwr Engine
Agent ID
parameter logic [7:0]  MON_UNIT_ID   = 8'h01;    // Unit identifier
parameter logic [8:0]  MON_CHANNEL_ID = 9'h000;  // Base channel ID
```

: Table 2.9.1: Control-Write Engine Parameters

Note: `AXI_ID_WIDTH` must be `>= CHAN_WIDTH` (checked by an elaboration-time `$fatal`). When instantiated in `scheduler_group_beats.sv` as `u_ctrlwr_engine`, `MON_AGENT_ID` is overridden to `16'h0021` (`CTRLWR_MON_AGENT_ID`) to distinguish it from the Control-Read Engine's `16'h0020`.

---

## 14.3 Port List

### 14.3.1 Clock and Reset

*Table 2.9.2: Clock and Reset*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `clk` | input | 1 | System clock |
| `rst_n` | input | 1 | Active-low asynchronous reset |

### 14.3.2 Configuration Interface

*Table 2.9.3: Configuration Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `cfg_channel_reset` | input | 1 | Per-channel reset: blocks new requests, drains transaction, forces idle |

### 14.3.3 Scheduler Interface

*Table 2.9.4: Scheduler Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `ctrlwr_valid` | input | 1 | Request valid (doorbell write requested) |
| `ctrlwr_ready` | output | 1 | Ready to accept request |
| `ctrlwr_pkt_addr` | input | AW | Doorbell target address (`wr_addr`, 4-byte aligned) |
| `ctrlwr_pkt_data` | input | 32 | Doorbell write value (`wr_data`) |
| `ctrlwr_error` | output | 1 | Error flag (alignment or AXI response error) |
| `ctrlwr_engine_idle` | output | 1 | Engine idle (in `WRITE_IDLE`, no active transaction, not in channel reset) |

### 14.3.4 AXI4 Write Master Interface

*Table 2.9.5: AXI4 Write Master Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `aw_valid` | output | 1 | Write address valid |
| `aw_ready` | input | 1 | Write address ready |
| `aw_addr` | output | AW | Write address (= `wr_addr`) |
| `aw_len` | output | 8 | Burst length (`0` = single beat) |
| `aw_size` | output | 3 | Transfer size (`3'b010` = 4 bytes / 32-bit) |
| `aw_burst` | output | 2 | Burst type (`2'b01` = INCR) |
| `aw_id` | output | AXI_ID_WIDTH | Transaction ID (derived from `CHANNEL_ID`) |
| `aw_lock` | output | 1 | Lock type (`0` = normal) |
| `aw_cache` | output | 4 | Cache type (`4'b0010` = non-cacheable bufferable) |
| `aw_prot` | output | 3 | Protection type (`3'b000`) |
| `aw_qos` | output | 4 | Quality of service (`0`) |
| `aw_region` | output | 4 | Region identifier (`0`) |
| `w_valid` | output | 1 | Write data valid |
| `w_ready` | input | 1 | Write data ready |
| `w_data` | output | 32 | Write data (= `wr_data`) |
| `w_strb` | output | 4 | Write strobes (`4'b1111` = all bytes) |
| `w_last` | output | 1 | Last beat (`1` = single beat) |
| `b_valid` | input | 1 | Write response valid |
| `b_ready` | output | 1 | Write response ready |
| `b_id` | input | AXI_ID_WIDTH | Response ID (matched against expected AXI ID) |
| `b_resp` | input | 2 | Write response code |

### 14.3.5 MonBus Interface

*Table 2.9.6: MonBus Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `i_mon_time` | input | 64 | Monitor timestamp side-band input |
| `mon_valid` | output | 1 | Monitor packet valid |
| `mon_ready` | input | 1 | Monitor packet ready |
| `mon_packet` | output | 128 | Monitor packet (`monitor_packet_t`) |
| `mon_timestamp` | output | 64 | Monitor packet timestamp |

## 14.4 FSM States

### 14.4.1 Figure 2.9.2: Control-Write Engine FSM

States and transitions of the `ctrlwr_engine` doorbell FSM.

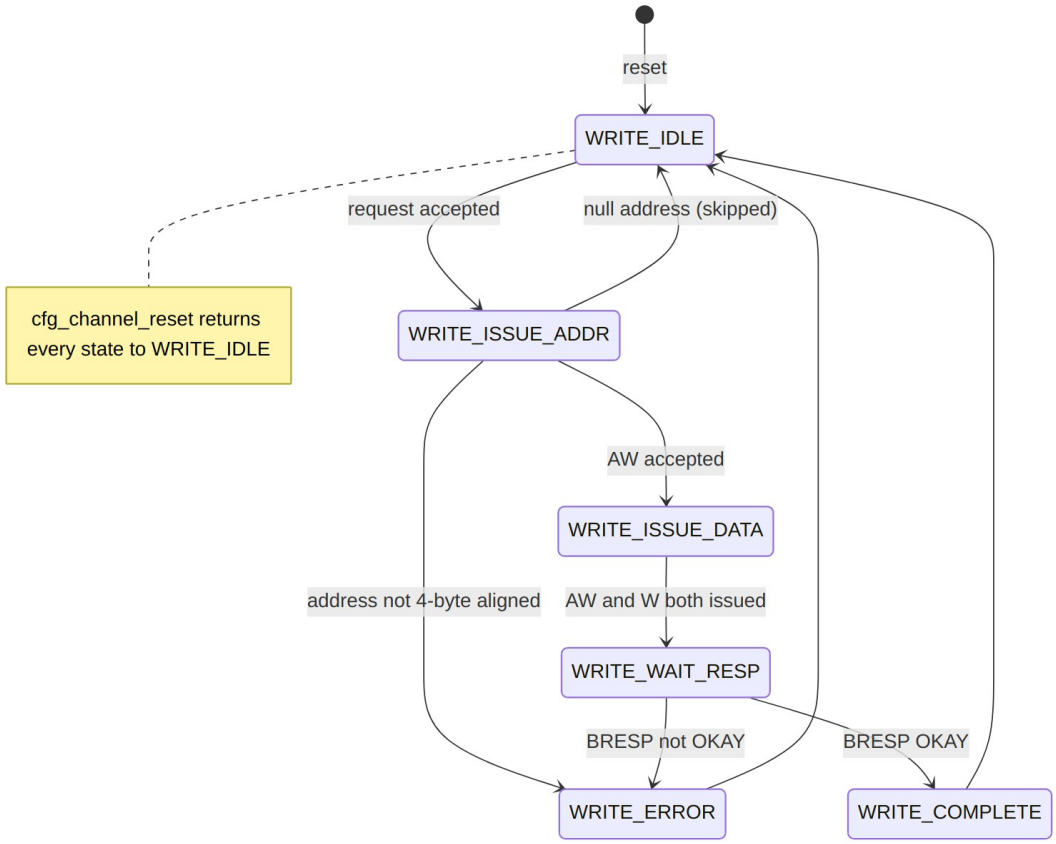



*Diagram 3*

State summary:

- **WRITE_IDLE:** Wait for a request from the request skid buffer; latch `wr_addr`/`wr_data` and the expected AXI ID.
- **WRITE_ISSUE_ADDR:** Drive `aw_valid` until `aw_ready`. A misaligned address routes to `WRITE_ERROR`; a null address returns to `WRITE_IDLE` (no-op skip).
- **WRITE_ISSUE_DATA:** Drive `w_valid` until `w_ready`; advance once both address and data phases have been issued.
- **WRITE_WAIT_RESP:** Accept the matching `b`-channel response; a non-OKAY `b_resp` routes to `WRITE_ERROR`, otherwise to `WRITE_COMPLETE`.

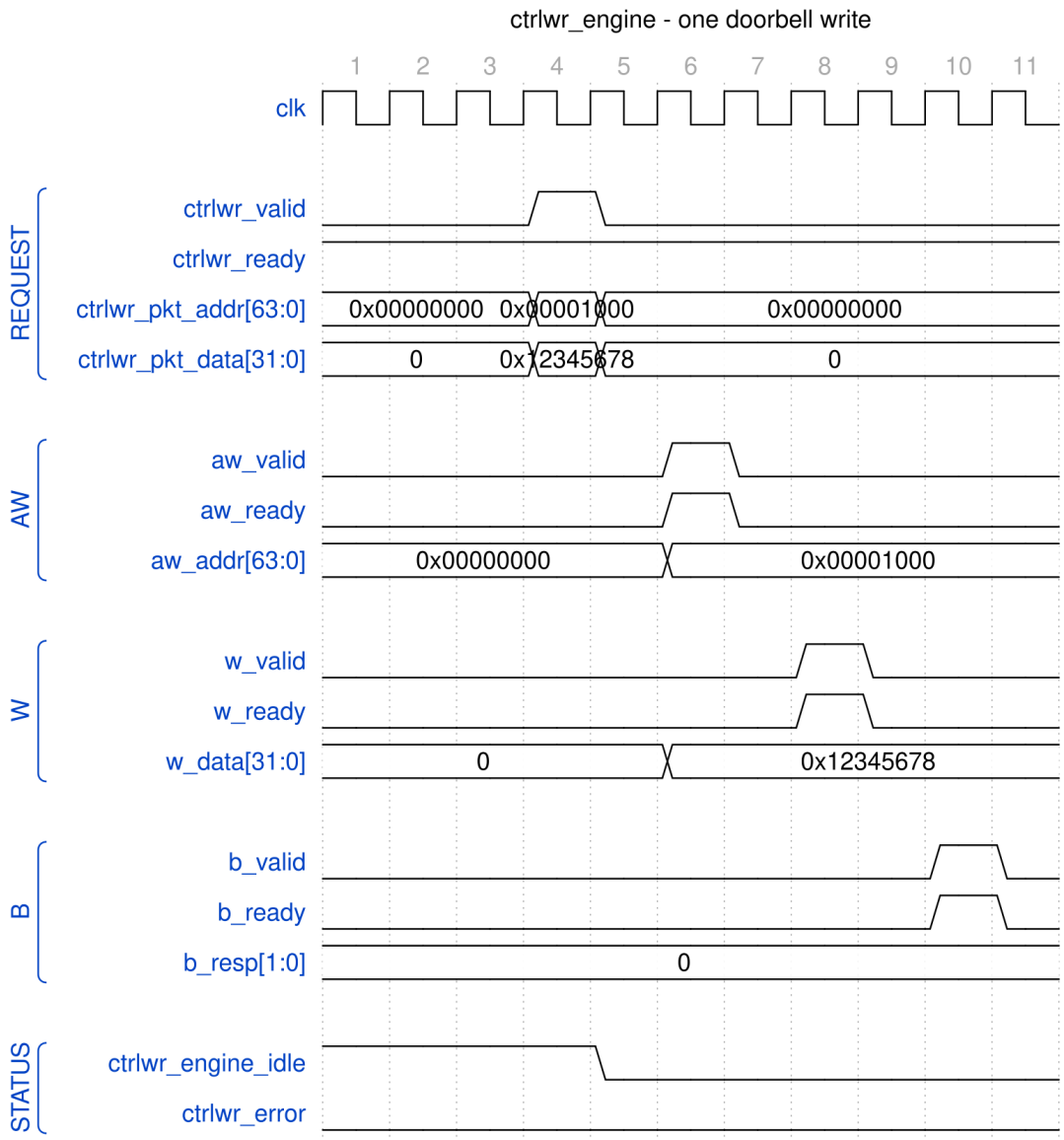
- **WRITE_COMPLETE:** Emit a completion MonBus packet (unless null address) and return to `WRITE_IDLE`.
- **WRITE_ERROR:** Latch `ctrlwr_error`, emit an error MonBus packet, and return to `WRITE_IDLE`. The error flag persists until reset or `cfg_channel_reset`.

---

## 14.5 Operation

### 14.5.1 Doorbell Write Sequence

### 14.5.2 Figure 2.9.3: Control-Write Doorbell Timing




*ctrlwr_engine - one doorbell write*

**Source:** ctrlwr_engine_doorbell.json, captured from `dv/tests/fub/test_ctrlwr_engine.py` (`basic_write`, channel 0, 64-bit address, `REG_LEVEL=GATE`) with `WAVES=1`; the AXI side is the framework write slave.

Reading it: the request handshakes in one cycle (ctrlwr_ready is held high by the skid buffer) with the doorbell address 0x1000 and data 0x12345678, and the engine leaves idle the next cycle. AW goes out two cycles after the request, W two cycles after AW, and the slave answers B (OKAY) two cycles after W; ctrlwr_engine_idle returns the cycle after the B handshake, eight cycles after the request. Nothing is pipelined across doorbells: the engine takes one request at a time.

### 14.5.3 Request Handshake and Skid Buffer

Incoming requests are captured through a depth-2 gaxi_skid_buffer. The engine accepts a request (ctrlwr_ready) only when not in an active channel reset. The FSM pops the buffered request in WRITE_IDLE, latching {wr_addr, wr_data} and computing the expected AXI ID from CHANNEL_ID:

```
aw_id = {{(AXI_ID_WIDTH-CHAN_WIDTH){1'b0}}, CHANNEL_ID[CHAN_WIDTH-
1:0]};
```

The b-channel response is matched against this expected ID (b_id == r_expected_axi_id) so a shared write master can be demultiplexed per channel.

### 14.5.4 Address Validation

```
Null (skip)  : wr_addr == 64'h0
Align error  : wr_addr[1:0] != 2'b00  (and not null)
```

A null address completes immediately as a no-op with no AXI activity and no completion packet. A misaligned address never issues on AXI; it transitions to WRITE_ERROR, asserts ctrlwr_error, and emits an error MonBus packet.

### 14.5.5 Error Reporting

ctrlwr_error is a registered, sticky flag. It is set on an alignment error or on a non-OKAY AXI write response (b_resp != 2'b00), and it persists until rst_n or cfg_channel_reset. The MonBus error packet carries either the offending address (alignment error) or the captured b_resp code (AXI response error).

### 14.5.6 Channel Reset

Asserting cfg_channel_reset registers r_channel_reset_active, which blocks new requests (ctrlwr_ready low), clears the error flag and the phase-issued state, and drives the FSM back to WRITE_IDLE.

A write already on the fabric cannot be cancelled, so the engine drains it (rapids TASK-014, 2026-09-27): a raised AW stays up until accepted (r_drain_aw), an accepted AW is followed by its W beat with the latched data (r_drain_w), and the B response is taken and discarded (r_drain_b, which alone lifts b_ready outside WRITE_WAIT_RESP). The write that was already issued therefore lands. Until B is taken ctrlwr_engine_idle stays low and the request path stays closed. Before this the reset abandoned whatever phase was in flight and left a dangling response on the channel's ID.

`ctrlwr_engine_idle` asserts only in `WRITE_IDLE` with no pending request, no active channel reset and nothing left to drain.

---

## 14.6 Contrast with the Control-Read Engine

The Control-Write Engine is the producer counterpart to the Control-Read Engine (Section 2.8). Both are per-channel control-descriptor executors, but they implement opposite half of the producer/consumer handshake:

| Aspect | Control-Read Engine (2.8) | Control-Write Engine (2.9) |
|---|---|---|
| Opcode | `DESC_OP_CTRL_READ` (`2'b01`) | `DESC_OP_CTRL_WRITE` (`2'b10`) |
| Role | Consumer gate | Producer doorbell |
| AXI direction | Read (poll) | Single-beat write |
| Semantics | Poll until `(rd & mask) == expected` | Write `wr_data` to `wr_addr`, then continue |
| Retry budget | `max_try` retry budget (config default) | None (single unconditional write) |
| Chain effect | Holds off the chain until the gate passes | Releases the chain after the write completes |
| Descriptor fields | `poll_addr` / `expected` / `mask` / `max_try` | `wr_addr` / `wr_data` |

A `CTRL_WRITE` therefore has no maximum-try configuration: it is one unconditional 32-bit write, whereas `CTRL_READ` polls with a bounded retry budget before either passing the gate or timing out.

---

**Last Updated:** 2026-07-12

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 15 Shared Engines

**Modules:** `descriptor_engine_shared.sv`, `ctrlrd_engine_shared.sv`, `ctrlwr_engine_shared.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub/`
**Status:** Implemented **Last Updated:** 2026-10-17

---

## 15.1 Overview

With `SHARED_ENGINES = 1` (the default) each scheduler group array, one per half, builds one descriptor engine, one control-read engine and one control-write engine for all of its channels. These replace the `NUM_CHANNELS` copies of each that `SHARED_ENGINES = 0` builds. Each shared engine picks the next channel to serve round-robin on the scheduler side and owns its AXI master outright, so the array's AXI-side arbiters are gone. ARID and AWID carry the channel.

Each scheduler sees the same per-channel interface as with a dedicated engine. The schedulers, the array's ports and everything above the array are the same in both builds. The Scheduler Group Array chapter describes how the array instantiates the engines and the MonBus sources they add. This chapter describes the engines themselves.

### 15.1.1 Key Features

- **One fetch path per half:** a single AR issuer, one set of address-range comparators and one R path for every channel.
- **Per-channel ordering:** each channel keeps one descriptor fetch in flight, so its descriptors arrive in order. Fetches for different channels overlap on the bus under their own ARIDs.
- **No R-channel blocking:** a fetch issues only when its channel's FIFO has room, and an R beat for a channel reset mid-fetch is accepted and dropped.
- **Owner-register control engines:** one `ctrlrd_engine` and one `ctrlwr_engine`, each wrapped with a round-robin pick and an owner register.
- **Descriptor injection (`INJ_EN`):** a second writer into the descriptor FIFOs for the tile's cell-issued descriptors. See Tile Packet Header.

### 15.1.2 Cost

On the Genesys 2 board build (8 channels per half, 256 bits) the three per-channel engine sets took 15,283 LUTs and 19,308 flip-flops across both halves:

- descriptor engines: 11,009 LUTs and 7,360 flip-flops for 16 engines;
- control-read engines: 2,388 LUTs and 7,038 flip-flops;
- control-write engines: 1,886 LUTs and 4,910 flip-flops.

Sharing removed 11,024 LUTs (20%) and 16,483 flip-flops (37%) from RAPIDS. The HAS Resource Estimates chapter has the full before and after.

---

## 15.2 descriptor_engine_shared

### 15.2.1 Parameters

*Table 2.10.1: descriptor_engine_shared Parameters*

| Parameter | Default | Description |
|---|---|---|
| NUM_CHANNELS | 8 | Channels served. |
| CHAN_WIDTH | $clog2(NUM_CHANNELS) (1 minimum) | Channel ID width. AXI_ID_WIDTH must be at least this; elaboration fails otherwise. |
| ADDR_WIDTH | 64 | Address width. |
| AXI_ID_WIDTH | 8 | ARID and RID width. ARID = channel. |
| FIFO_DEPTH | 8 | Descriptor FIFO depth per channel. |
| USE_ROW_COL_MAJOR_ADDRESSING | 1 | Enables the EXTENDED descriptor's second fetch and its chunk-1 FIFO. |
| MON_AGENT_BASE | 0x10 | MonBus agent is MON_AGENT_BASE + channel. |
| MON_UNIT_ID | 0x01 | MonBus unit ID. |
| MON_FIFO_DEPTH | 4 | Depth of the MonBus packet FIFO. |
| INJ_EN | 0 | 1 builds the injection slot (set by the array when TILE_HDR = 1). |

### 15.2.2 Port List

Every per-channel port is a packed [NUM_CHANNELS-1:0] vector of the signal a dedicated descriptor_engine has, so channel c sees bit or element c.

*Table 2.10.2: descriptor_engine_shared Ports*

| Group | Signals | Description |
|---|---|---|
| Clock and reset | clk, rst_n | Single clock, active-low asynchronous reset. |

| Group | Signals | Description |
|---|---|---|
| Kick (per channel) | apb_valid, apb_ready, apb_addr | Staged descriptor address from the configuration block. |
| Scheduler (per channel) | channel_idle, descriptor_valid, descriptor_ready, descriptor_packet, descriptor_ext_packet, descriptor_error, descriptor_eos, descriptor_eol, descriptor_eod, descriptor_type | The same handshake and flags as descriptor_engine. |
| AXI4 read master | ar_*, r_* (256-bit r_data) | Owned outright; one master per half. |
| Configuration | cfg_prefetch_enable, cfg_fifo_threshold, cfg_addr0_base/limit, cfg_addr1_base/limit, cfg_channel_reset | Shared by every channel, as before; the reset is per channel. |
| Injection | inj_valid, inj_ch, inj_data, inj_dropped, inj_dropped_clr | Unused (tied off) unless INJ_EN = 1. |
| Status | descriptor_engine_idle (per channel) | Channel has no fetch pending or in flight. |
| MonBus | i_mon_time, mon_valid, mon_ready, mon_packet, mon_timestamp | One stream for every channel. |

### 15.2.3 Per-Channel State

Each channel keeps only what a fetch needs between the kick and the FIFO write:

*Table 2.10.3: Per-Channel Fetch States*

| State | Meaning |
|---|---|
| S_IDLE | Nothing to fetch. |

| State | Meaning |
|---|---|
| S_PEND | Chunk-0 address pending (from a kick or from chaining). |
| S_ISSUE | Chunk-0 request is in the shared issue slot. |
| S_WAIT | Chunk-0 AR accepted, waiting for R. |
| S_PEND2, S_ISSUE2, S_WAIT2 | The same three steps for an EXTENDED descriptor's chunk 1. |

Beside the state code each channel has one pending fetch address, a kick-in-progress flag, an owed-beat flag (r_owe), its descriptor FIFO and, when EXTENDED descriptors are built, a chunk-1 FIFO kept in lockstep with it. An injection slot is added when INJ_EN = 1.

### 15.2.4 Operation

**Issue.** A round-robin pick among channels in S_PEND or S_PEND2 loads one request into the issue slot. A channel is eligible only while its FIFO has a free entry, so a channel that has stopped consuming never takes an R beat it cannot store. The range check runs at the slot: an address outside both configured ranges is refused with no AR and raises the channel's descriptor error. Once ARVALID is high, the address and ID hold until accepted.

**Return.** Each R beat is routed by RID to its channel and written straight into that channel's FIFO, so there is no 256-bit holding register per channel. The exception is an EXTENDED descriptor's chunk 0, which waits for chunk 1. A non-OKAY response, or a beat that does not parse as a valid descriptor, raises the channel's descriptor error.

**Chaining and prefetch.** The next-pointer decision and the prefetch throttle are the per-channel engine's: autonomous chaining while the descriptor allows it, up to the FIFO threshold plus one buffered.

**Channel reset.** A reset in the middle of a fetch sets r_owe. The owed beat is accepted and dropped when it arrives, and the channel takes no new kick until then. Reset also drains the channel's FIFO, so descriptors prefetched for an aborted chain are never delivered.

**Injection.** An injected descriptor is a pulse with no backpressure. It waits in its channel's one-entry slot and is written into the FIFO, with no fetch, once the channel's fetch side is idle and the FIFO has room. An injection therefore never takes an entry an R beat needs. A descriptor that arrives while the slot is full, while the channel is held in reset, or that is EXTENDED is dropped and sets the channel's sticky inj_dropped bit. A drop is not a descriptor error.

### 15.2.5 MonBus

Completion and error packets go through a MON_FIFO_DEPTH-entry FIFO. Two behaviours differ from the per-channel engine:

- A refused kick address reports 0 in the response field. The per-channel engine reported its last latched read response, which was stale.
- An issue-side refusal and an R-side event can occur in the same cycle. The issue-side packet waits one cycle. If a second collision arrives while it waits, the waiting packet is dropped. Tests that count descriptor-engine packets must allow for this.

---

## 15.3 ctrlrd_engine_shared and ctrlwr_engine_shared

The two control engines share one structure: an unmodified `ctrlrd_engine` or `ctrlwr_engine` with three things around it. A control engine runs one request at a time, so nothing inside it has to change.

*Table 2.10.4: Shared Control Engine Wrapper*

| Element | Behaviour |
|---|---|
| Round-robin pick | Offered to the engine only while it is idle, its last MonBus packet has drained, and no error remains from the previous owner. A stale error is cleared with a one-cycle engine reset before the next channel is taken. |
| Owner register | Set when the engine accepts a request. ARID or AWID carries the owner, and every R or B response belongs to the engine. |
| Per-channel view | The owner sees the engine's `*_idle` and `*_error` as a dedicated engine would show them. Every other channel sees idle 1 and error 0. |
| Channel reset | The owner's `cfg_channel_reset` resets the engine only while the owner's operation is in flight, so a channel held in reset cannot hold the engine for the others. |
| MonBus | Packets are re-tagged with the owner's channel and agent ID: 0x20 + channel for control reads, 0x28 + channel for control writes. |

### 15.3.1 Parameters

*Table 2.10.5: Shared Control Engine Parameters*

| Parameter | Default | Description |
|---|---|---|
| `NUM_CHANNELS` | 8 | Channels served. |
| `CHAN_WIDTH` | `$clog2(NUM_CHANNELS)` (1 minimum) | Channel ID width. |

| Parameter | Default | Description |
|---|---|---|
| ADDR_WIDTH | 64 | Address width. |
| AXI_ID_WIDTH | 8 | ARID or AWID width. |
| MON_AGENT_BASE | 0x20 (read), 0x28 (write) | MonBus agent is MON_AGENT_BASE + channel. |
| MON_UNIT_ID | 0x01 | MonBus unit ID. |

### 15.3.2 Port List

*Table 2.10.6: Shared Control Engine Ports*

| Group | ctrlrd_engine_shared | ctrlwr_engine_shared |
|---|---|---|
| Requests (per channel) | ctrlrd_valid, ctrlrd_ready, ctrlrd_addr, ctrlrd_data, ctrlrd_mask | ctrlwr_valid, ctrlwr_ready, ctrlwr_addr, ctrlwr_data |
| Status (per channel) | ctrlrd_error, ctrlrd_idle | ctrlwr_error, ctrlwr_idle |
| Configuration | cfg_ctrlrd_max_try, cfg_channel_reset, tick_1us | cfg_channel_reset |
| AXI4 master | ar_*, r_* (32-bit data) | aw_*, w_* (32-bit data, 4-bit strobe), b_* |
| MonBus | i_mon_time, mon_valid, mon_ready, mon_packet, mon_timestamp | the same |

### 15.3.3 Trade-off

A CTRL_READ that keeps polling holds the control-read engine, so other channels' control reads wait behind it, up to cfg_ctrlrd_max_try retries. Data transfers are not affected. The control-write engine keeps one write in flight, which the array-level serializer of the per-channel build also enforced.

---

## 15.4 Related Chapters

- Scheduler Group Array: instantiation, MonBus sources, and the `SHARED_ENGINES = 0` AXI arbiters.
- Descriptor Engine, Control-Read Engine, Control-Write Engine: the per-channel engines whose behaviour the shared engines keep.
- Tile Packet Header: the descriptor injection path.



---

# 16 Scheduler Group

**Module:** `scheduler_group.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro/` **Status:** Implemented **Last Updated:** 2026-09-30

---

## 16.1 Overview

The scheduler group is used only when `SHARED_ENGINES = 0`. The default build instantiates the schedulers directly and uses the Shared Engines instead. The scheduler group wraps one channel: a descriptor engine, the scheduler, the two control engines (control read and control write) and a MonBus arbiter that merges their monitor packets. It is the unit the scheduler group array instantiates once per channel.

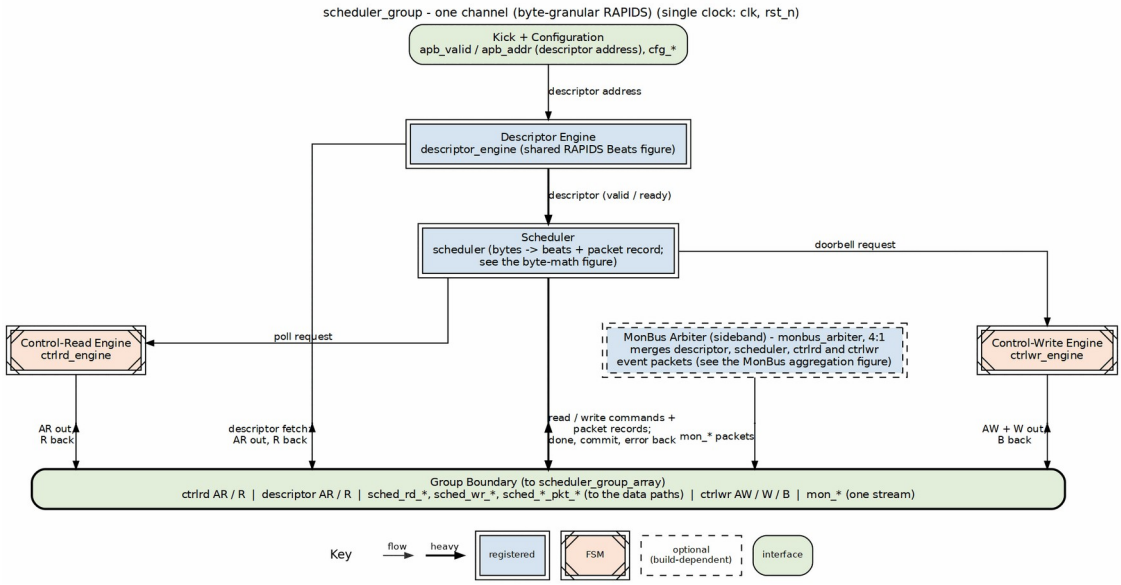
The structure is identical to the RAPIDS Beats group. The one change is the packet-record port set: the scheduler now publishes a per-direction record of the packet's byte length and start offset, and the group passes it through unchanged.

### 16.1.1 Key Features

- **One channel, four engines:** descriptor engine, scheduler, control read engine, control write engine.
- **Packet-record pass-through:** `sched_rd_pkt_*` and `sched_wr_pkt_*` connect the scheduler straight to the group boundary.
- **MonBus aggregation:** a four-client arbiter merges the four engines' packets.
- **Completion gating:** `cfg_sched_compl_enable` drops CORE Completion packets from the descriptor engine and the scheduler at the arbiter input and acknowledges the emitters, so nothing stalls (rapids ISSUE-005). Errors are never gated.
- **Status aggregation:** idle, FSM state and sticky error are exported.

### 16.1.2 Block Diagram

### 16.1.3 Figure 3.1.1: Scheduler Group Block Diagram




*Scheduler group block diagram*

**Source:** 07_scheduler_group.dot

---

## 16.2 Parameters

*Table 3.1.1: Scheduler Group Parameters*

| Parameter | Default | Description |
|---|---|---|
| CHANNEL_ID | 0 | Channel identifier. |
| NUM_CHANNELS | 8 | Total channels. |
| CHAN_WIDTH | $clog2(NUM_CHANNELS) | Channel id width. |
| ADDR_WIDTH | 64 | Address width. |
| DATA_WIDTH | 512 | Data width in bits. Sets the beat size seen by the scheduler. |
| AXI_ID_WIDTH | 8 | AXI ID width for the descriptor and control masters. |
| DESC_MON_AGENT_ID | 16 | MonBus agent id of the descriptor engine. |
| SCHED_MON_AGENT_ID | 48 | MonBus agent id of the scheduler. |
| CTRLRD_MON_AGENT_ID | 32 | MonBus agent id of the control read engine. |
| CTRLWR_MON_AGENT_ID | 33 | MonBus agent id of the control write engine. |
| MON_UNIT_ID | 1 | MonBus unit id. |
| MON_CHANNEL_ID | 0 | Base MonBus channel id. |
| EN_READ | 1 | Enable the read direction of the scheduler. |
| EN_WRITE | 1 | Enable the write direction of the scheduler. |
| USE_ROW_COL_MAJOR_ADDRESSING | 1 | Descriptor engine addressing mode. |
| GEN_MON | 1 | Enable monitor packet generation. |
| OFF_W | $clog2(DATA_WIDTH/8) | Width of the packet-record byte offset. New in RAPIDS. |

## 16.3 Port List

### 16.3.1 Clock, Reset and Kick-Off

*Table 3.1.2: Clock, Reset and Kick-Off*

| Signal | Direction | Width | Description |
|---|---|---|---|
| clk | input | 1 | System clock. |
| rst_n | input | 1 | Active-low reset. |
| apb_valid | input | 1 | Channel kick-off. |
| apb_ready | output | 1 | Descriptor engine ready for kick-off. |
| apb_addr | input | AW | First descriptor address. |

### 16.3.2 Configuration Interface

*Table 3.1.3: Configuration Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| cfg_channel_enable | input | 1 | Enable this channel. |
| cfg_channel_reset | input | 1 | Per-channel soft reset. |
| cfg_sched_timeout_cycles | input | 32 | Write-progress timeout window in cycles. |
| cfg_sched_timeout_limit | input | 8 | Consecutive timeout windows before a fatal error. 0 means never. |
| cfg_sched_timeout_enable | input | 1 | Enable timeout detection. |
| cfg_sched_err_enable | input | 1 | Enable error reporting. |
| cfg_sched_compl_enable | input | 1 | 1 passes CORE Completion packets to the monitor bus; 0 drops them at the arbiter |

| Signal | Direction | Width | Description |
|---|---|---|---|
| | | | inputs. |
| cfg_sched_perf_enable | input | 1 | Enable performance monitoring. |
| cfg_ctrlrd_max_try | input | 9 | Control-read poll retry budget, 0 to 511. |
| tick_1us | input | 1 | 1 us tick for control-read retry spacing. |
| cfg_desceng_prefetch | input | 1 | Enable descriptor prefetch. |
| cfg_desceng_fifo_thresh | input | 4 | Prefetch threshold. |
| cfg_desceng_addr0_base | input | AW | Valid descriptor address range 0 base. |
| cfg_desceng_addr0_limit | input | AW | Valid descriptor address range 0 limit. |
| cfg_desceng_addr1_base | input | AW | Valid descriptor address range 1 base. |
| cfg_desceng_addr1_limit | input | AW | Valid descriptor address range 1 limit. |

### 16.3.3 Status

*Table 3.1.4: Status Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| descriptor_engine_idle | output | 1 | Descriptor engine idle. |
| scheduler_idle | output | 1 | Scheduler idle. |
| scheduler_state | output | 7 | Scheduler FSM state, one-hot. |
| sched_error | output | 1 | Scheduler error, sticky. |

### 16.3.4 Descriptor AXI Master Interface

*Table 3.1.5: Descriptor AXI Master Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `desc_ar_valid` / `desc_ar_ready` | output / input | 1 | AR handshake. |
| `desc_ar_addr` | output | AW | Descriptor fetch address. |
| `desc_ar_len` | output | 8 | Burst length minus one. |
| `desc_ar_size` | output | 3 | Burst size. |
| `desc_ar_burst` | output | 2 | Burst type. |
| `desc_ar_id` | output | IW | Transaction ID. |
| `desc_ar_lock`, `_cache`, `_prot`, `_qos`, `_region` | output | 1, 4, 3, 4, 4 | Attribute fields. |
| `desc_r_valid` / `desc_r_ready` | input / output | 1 | R handshake. |
| `desc_r_data` | input | 256 | Descriptor payload, fixed 256 bits. |
| `desc_r_resp` | input | 2 | Read response. |
| `desc_r_last` | input | 1 | Last beat. |
| `desc_r_id` | input | IW | Response ID. |

### 16.3.5 Control Read and Control Write AXI Interfaces

The control read engine fetches a 32-bit semaphore word: `ctrlrd_ar_*` carries the AR channel (address, length, size, burst, id and the same attribute fields as above) and `ctrlrd_r_valid`, `ctrlrd_r_ready`, `ctrlrd_r_data[31:0]`, `ctrlrd_r_id`, `ctrlrd_r_resp` and `ctrlrd_r_last` the R channel. The control write engine writes a 32-bit word: `ctrlwr_aw_*` carries the AW channel, `ctrlwr_w_valid`, `ctrlwr_w_ready`, `ctrlwr_w_data[31:0]`, `ctrlwr_w_strb[3:0]` and `ctrlwr_w_last` the W channel, and `ctrlwr_b_valid`, `ctrlwr_b_ready`, `ctrlwr_b_id` and `ctrlwr_b_resp` the B channel. These are unchanged from RAPIDS Beats. See Control Read Engine and Control Write Engine.

### 16.3.6 Scheduler Data Interface

*Table 3.1.6: Scheduler Data Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| sched_rd_valid | output | 1 | Read request. |
| sched_rd_addr | output | AW | Working source byte address. |
| sched_rd_beats | output | 32 | Beats still to read. |
| sched_wr_valid | output | 1 | Write request. |
| sched_wr_ready | input | 1 | Write engine reports the last burst of the descriptor committed. |
| sched_wr_addr | output | AW | Working destination byte address. |
| sched_wr_beats | output | 32 | Beats still to write. |
| sched_rd_done_strobe | input | 1 | Read burst issued. |
| sched_rd_beats_done | input | 32 | Beats in that burst. |
| sched_wr_done_strobe | input | 1 | Write AW issued. |
| sched_wr_beats_done | input | 32 | Beats in that AW. |
| sched_wr_commit_strobe | input | 1 | Write B response received. |
| sched_wr_commit_beats | input | 32 | Beats committed by that B response. |
| sched_rd_error | input | 1 | Sticky read error from the read engine. |
| sched_wr_error | input | 1 | Sticky write error, including a packet-length error from the sink path. |

### 16.3.7 Packet Record Interface

*Table 3.1.7: Packet Record Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `sched_rd_pkt_valid` | output | 1 | One-cycle pulse: a read-side packet record is available. |
| `sched_rd_pkt_ready` | input | 1 | Consumer has room in its record queue. |
| `sched_rd_pkt_bytes` | output | 32 | Packet length in bytes. |
| `sched_rd_pkt_offset` | output | OFF_W | Byte offset of the packet start within its first memory beat. |
| `sched_wr_pkt_valid` | output | 1 | One-cycle pulse: a write-side packet record is available. |
| `sched_wr_pkt_ready` | input | 1 | Consumer has room in its record queue. |
| `sched_wr_pkt_bytes` | output | 32 | Packet length in bytes. |
| `sched_wr_pkt_offset` | output | OFF_W | Byte offset of the packet start within its first memory beat. |

The group wires these signals directly to the scheduler. The read-side record goes to the source data path and the write-side record to the sink data path. The scheduler holds a descriptor in its fetch state while a required record queue is full, using the ready inputs. See Packet Record Interface.

### 16.3.8 MonBus Interface

*Table 3.1.8: MonBus Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `i_mon_time` | input | `monbus_timestamp_t` | Shared monitor timebase. |
| `mon_valid` | output | 1 | Monitor packet valid. |
| `mon_ready` | input | 1 | Consumer ready. |

| Signal | Direction | Width | Description |
|---|---|---|---|
| mon_packet | output | monitor_packet_t | Monitor packet. |
| mon_timestamp | output | monbus_timestamp_t | Packet timestamp. |

## 16.4 Internal Architecture

### 16.4.1 Descriptor Path

The descriptor engine fetches 256-bit descriptors from the address range the configuration allows and hands each one to the scheduler with its type, end-of-stream, end-of-list and error flags. The scheduler reports `scheduler_idle` back to the descriptor engine as `channel_idle`. No byte or beat arithmetic happens in the descriptor engine.

### 16.4.2 Control Engines

When a descriptor asks the scheduler to wait on or update a semaphore, the scheduler talks to the control engines through valid and ready handshakes with an address, data and mask. Both engines share the group's `AXI_ID_WIDTH` and each raises its own MonBus packets.

### 16.4.3 MonBus Aggregation

A four-client arbiter merges the descriptor engine, scheduler, control read and control write packets. `cfg_sched_compl_enable` gates only the CORE Completion packets of the descriptor engine and the scheduler: with it low, those packets are consumed at the arbiter input and never reach `mon_packet`. Error packets are never gated. `GEN_MON` masks every arbiter input when clear.

---

## 16.5 Differences from RAPIDS Beats

*Table 3.1.9: Scheduler Group Delta*

| Item | RAPIDS Beats | RAPIDS |
|---|---|---|
| Parameter `OFF_W` | absent | present, `$clog2(DATA_WIDTH/8)` |
| `sched_rd_pkt_*`, `sched_wr_pkt_*` ports | absent | present, wired to the scheduler |
| Everything else | | Unchanged |

The beats chapter is Beats Scheduler Group. The scheduler is described in Scheduler.

---

**Last Updated:** 2026-09-30

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 17 Scheduler Group Array

**Module:** `scheduler_group_array.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro/` **Status:** Implemented **Last Updated:** 2026-10-16

---

## 17.1 Overview

The scheduler group array runs one scheduler per channel and three engines that issue the array's AXI traffic: the descriptor fetch engine, the control-read engine and the control-write engine. `SHARED_ENGINES` (default 1) selects how the engines are built. With 1, the array has one of each engine for all of its channels, and each engine picks among the schedulers round-robin (see Shared Engines below and the Shared Engines chapter). With 0, every channel has its own three engines inside a scheduler group, and three AXI-side arbiters merge their masters (the original structure). It also owns the descriptor-AXI monitor and merges every channel's MonBus output with the monitor's into one stream.

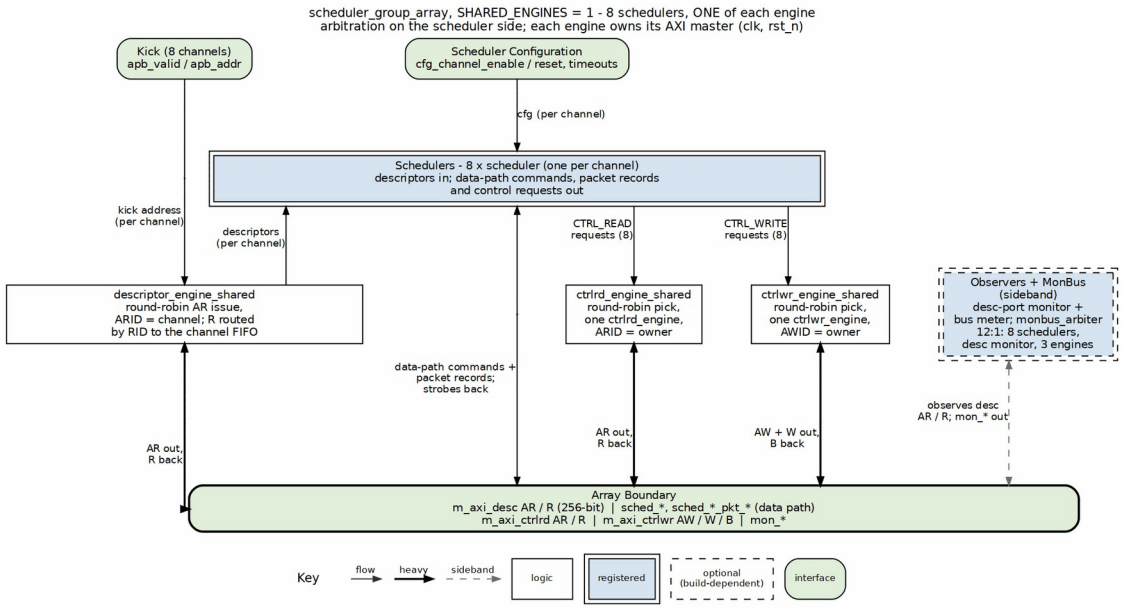
Compared with the RAPIDS Beats array, the sharing, arbitration and monitoring are unchanged. The array adds one parameter, `OFF_W`, and the per-channel packet-record ports, which it connects to each group by channel index.

### 17.1.1 Key Features

- **One group per channel:** `NUM_CHANNELS` scheduler groups.
- **Shared engines (`SHARED_ENGINES = 1`, default):** one `descriptor_engine_shared`, one `ctrlrd_engine_shared` and one `ctrlwr_engine_shared` per array, arbitrating on the scheduler side and owning their AXI masters.
- **Per-channel engines (`SHARED_ENGINES = 0`):** one scheduler group per channel, and three round-robin arbiters with grant acknowledgment on the AXI side (`u_desc_ar_arbiter`, `u_ctrlrd_ar_arbiter`, `u_ctrlwr_aw_arbiter`).
- **Descriptor-AXI monitor:** a lite AXI read monitor on the shared descriptor master, plus an always-on bus meter for the perf window.
- **Unified MonBus:** one round-robin arbiter over `NUM_CHANNELS + 4` sources (shared engines) or `NUM_CHANNELS + 1` (per-channel engines).
- **Direction gating:** `EN_READ` and `EN_WRITE` let the source array be read-only and the sink array write-only.
- **Per-channel packet records:** vector ports for the byte length and offset of each channel's packet.

### 17.1.2 Block Diagram

### 17.1.3 Figure 3.2.1: Scheduler Group Array Block Diagram




*Scheduler group array block diagram*

**Source:** 08_scheduler_group_array.dot

---

## 17.2 Parameters

*Table 3.2.1: Scheduler Group Array Parameters*

| Parameter | Default | Description |
|---|---|---|
| NUM_CHANNELS | 8 | Number of channels. |
| CHAN_WIDTH | $clog2(NUM_CHANNELS) | Channel id width. |
| ADDR_WIDTH | 64 | Address width. |
| DATA_WIDTH | 512 | Data width in bits. |
| AXI_ID_WIDTH | 8 | AXI ID width. |
| DESC_MON_BASE_AGENT_ID | 16 | Base MonBus agent id of the descriptor engines. |
| SCHED_MON_BASE_AGENT_ID | 48 | Base MonBus agent id of the schedulers. |
| DESC_AXI_MON_AGENT_ID | 8 | MonBus agent id of the descriptor AXI monitor. |
| MON_UNIT_ID | 1 | MonBus unit id. |
| MON_MAX_TRANSACTIONS | 16 | Transaction table depth of the descriptor monitor. |
| EN_READ | 1 | Enable the read direction in every scheduler. Clear for a sink array. |
| EN_WRITE | 1 | Enable the write direction in every scheduler. Clear for a source array. |
| USE_AXI_MONITORS | 1 | 0 omits the descriptor-AXI monitor hardware and ties its MonBus source to zero. |
| USE_ROW_COL_MAJOR_ADDRESSING | 1 | 0 compiles out the extended addressing path. |
| GEN_MON | 1 | 0 omits the per-channel completion and error emitters. |
| SHARED_ENGINES | 1 | 1: one descriptor, control-read and control-write engine for |

| Parameter | Default | Description |
|---|---|---|
| | | the array. 0: one of each per channel. |
| OFF_W | $clog2(DATA_WIDTH/8) | Width of the packet-record byte offset. New in RAPIDS. |

## 17.3 Port List

Vector ports are NUM_CHANNELS wide (NC), indexed by channel.

### 17.3.1 Clock, Reset and Kick-Off

*Table 3.2.2: Clock, Reset and Kick-Off*

| Signal | Direction | Width | Description |
|---|---|---|---|
| clk | input | 1 | System clock. |
| rst_n | input | 1 | Active-low reset. |
| apb_valid | input | NC | Per-channel kick-off. |
| apb_ready | output | NC | Per-channel ready. |
| apb_addr | input | NC x AW | First descriptor address per channel. |
| cfg_channel_enable | input | NC | Per-channel enable. |
| cfg_channel_reset | input | NC | Per-channel soft reset. |

### 17.3.2 Global Scheduler and Descriptor Engine Configuration

These inputs are applied to every channel.

*Table 3.2.3: Global Configuration*

| Signal | Direction | Width | Description |
|---|---|---|---|
| cfg_sched_enable | input | 1 | Master scheduler enable. |
| cfg_sched_timeout_cycles | input | 32 | Timeout window in cycles. |
| cfg_sched_timeout_limit | input | 8 | Consecutive timeouts before a fatal error. 0 means never. |
| cfg_sched_timeout_enable | input | 1 | Enable timeout detection. |

| Signal | Direction | Width | Description |
|---|---|---|---|
| cfg_sched_err_enable | input | 1 | Enable error reporting. |
| cfg_sched_compl_enable | input | 1 | Enable CORE Completion packets. |
| cfg_sched_perf_enable | input | 1 | Enable performance monitoring. |
| cfg_desceng_enable | input | 1 | Master descriptor engine enable. |
| cfg_desceng_prefetch | input | 1 | Enable descriptor prefetch. |
| cfg_desceng_fifo_thresh | input | 4 | Prefetch threshold. |
| cfg_desceng_addr0_base, _limit | input | AW | Valid descriptor address range 0. |
| cfg_desceng_addr1_base, _limit | input | AW | Valid descriptor address range 1. |
| cfg_ctrlrd_max_try | input | 9 | Control-read poll retry budget. |
| tick_1us | input | 1 | 1 us tick for control-read retry spacing. |

### 17.3.3 Descriptor Monitor Configuration and Status

*Table 3.2.4: Descriptor Monitor Configuration and Status*

| Signal | Direction | Width | Description |
|---|---|---|---|
| cfg_desc_mon_enable | input | 1 | Enable the descriptor AXI monitor. |
| cfg_desc_mon_err_enable | input | 1 | Enable error packets. |
| cfg_desc_mon_perf_enable | input | 1 | Enable performance packets. |
| cfg_desc_mon_perf_run | input | 1 | Perf window control. |
| cfg_desc_mon_timeout_ena | input | 1 | Enable timeout packets. |

| Signal | Direction | Width | Description |
|---|---|---|---|
| ble | | | |
| cfg_desc_mon_timeout_cycles | input | 32 | Timeout threshold. |
| cfg_desc_mon_latency_thresh | input | 32 | Latency threshold. |
| cfg_desc_mon_pkt_mask | input | 16 | Packet type mask. |
| cfg_desc_mon_err_select | input | 4 | Error select. |
| cfg_desc_mon_err_mask, _timeout_mask, _compl_mask, _thresh_mask, _perf_mask, _addr_mask, _debug_mask | input | 8 each | Per-class event masks. |
| cfg_sts_desc_mon_busy | output | 1 | Monitor busy. |
| cfg_sts_desc_mon_active_txns | output | 8 | Active transactions. |
| cfg_sts_desc_mon_error_count | output | 16 | Error count. |
| cfg_sts_desc_mon_txn_count | output | 32 | Transaction count. |
| cfg_sts_desc_mon_conflict_error | output | 1 | Conflict error. |

### 17.3.4 Descriptor-AXI Perf Window

The perf counters come from an always-on bus meter on the descriptor R channel, not from the monitor. `cfg_desc_mon_perf_run` opens the window: counters clear on its rising edge, count while it is high and hold while it is low. `USE_AXI_MONITORS` does not gate the meter.

*Table 3.2.5: Descriptor-AXI Perf Window*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `sts_desc_mon_win_active` | output | 1 | Window open. |
| `sts_desc_mon_win_cycles` | output | 32 | Cycles in the window. |
| `sts_desc_mon_prod_cycles` | output | 32 | R valid and ready. |
| `sts_desc_mon_bp_cycles` | output | 32 | R valid, not ready. |
| `sts_desc_mon_starv_cycles` | output | 32 | R ready, not valid. |
| `sts_desc_mon_idle_cycles` | output | 32 | Neither valid nor ready. |
| `sts_desc_mon_beat_count` | output | 32 | R handshakes. |
| `sts_desc_mon_byte_count` | output | 64 | Bytes moved on R (32 per descriptor beat). |
| `sts_desc_mon_burst_count` | output | 32 | AR handshakes. |

### 17.3.5 Status

*Table 3.2.6: Status*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `descriptor_engine_idle` | output | NC | Per-channel descriptor engine idle. |
| `scheduler_idle` | output | NC | Per-channel scheduler idle. |
| `scheduler_state` | output | NC x 7 | Per-channel FSM state, one-hot. |
| `sched_error` | output | NC | Per-channel scheduler error, sticky. |

### 17.3.6 Shared AXI Masters

The three shared masters carry the same signals as the per-group ports in Scheduler Group, prefixed `desc_axi_`, `ctrlrd_axi_` and `ctrlwr_axi_`. The descriptor master returns 256-bit data. The control read master returns 32-bit data. The control write master is 32 bits wide with a 4-bit

strobe. Each is a single interface: the arbiter selects one channel's request onto it and steers the response back by ID.

### 17.3.7 Per-Channel Scheduler Data Interface

*Table 3.2.7: Per-Channel Scheduler Data Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| sched_rd_valid | output | NC | Read request per channel. |
| sched_rd_addr | output | NC x AW | Working source byte address. |
| sched_rd_beats | output | NC x 32 | Beats still to read. |
| sched_wr_valid | output | NC | Write request per channel. |
| sched_wr_ready | input | NC | Last burst of the descriptor committed. |
| sched_wr_addr | output | NC x AW | Working destination byte address. |
| sched_wr_beats | output | NC x 32 | Beats still to write. |
| sched_rd_done_strobe | input | NC | Read burst issued. |
| sched_rd_beats_done | input | NC x 32 | Beats in that burst. |
| sched_wr_done_strobe | input | NC | Write AW issued. |
| sched_wr_beats_done | input | NC x 32 | Beats in that AW. |
| sched_wr_commit_strobe | input | NC | Write B response received. |
| sched_wr_commit_beats | input | NC x 32 | Beats committed. |
| sched_rd_error | input | NC | Sticky read error. |
| sched_wr_error | input | NC | Sticky write error. |

### 17.3.8 Per-Channel Packet Record Interface

*Table 3.2.8: Per-Channel Packet Record Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `sched_rd_pkt_valid` | output | NC | Read-side record pulse per channel. |
| `sched_rd_pkt_ready` | input | NC | Read-side consumer has room. |
| `sched_rd_pkt_bytes` | output | NC x 32 | Packet length in bytes. |
| `sched_rd_pkt_offset` | output | NC x OFF_W | Start offset in the first memory beat. |
| `sched_wr_pkt_valid` | output | NC | Write-side record pulse per channel. |
| `sched_wr_pkt_ready` | input | NC | Write-side consumer has room. |
| `sched_wr_pkt_bytes` | output | NC x 32 | Packet length in bytes. |
| `sched_wr_pkt_offset` | output | NC x OFF_W | Start offset in the first memory beat. |

Element `[ch]` of each vector connects to the matching port of `scheduler_group[ch]`. The array adds no logic between them. A source array uses only the read set and a sink array only the write set.

### 17.3.9 MonBus Interface

*Table 3.2.9: MonBus Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `i_mon_time` | input | `monbus_timestamp_t` | Shared monitor timebase. |
| `mon_valid` | output | 1 | Packet valid. |
| `mon_ready` | input | 1 | Consumer ready. |
| `mon_packet` | output | `monitor_packet_t` | Packet. |
| `mon_timestamp` | output | `monbus_timestamp_t` | Timestamp. |

## 17.4 Shared Engines

With `SHARED_ENGINES = 1` the per-channel descriptor and control engines are replaced by one of each for the whole array. The arbitration moves from the AXI side to the scheduler side: each engine picks the next channel to serve round-robin, and it owns its AXI master outright, with no arbiter behind it. ARID and AWID carry the channel. The schedulers, the array's ports and every interface above the array are unchanged.

*Table 3.2.11: Shared Engines*

| Engine | What is shared | What stays per channel |
|---|---|---|
| `descriptor_engine_shared` | AR issuer, address-range comparators, the R path (each beat routed by RID straight into its channel's FIFO), MonBus reporter | State code, one pending fetch address, kick-in-progress flag, owed-beat flag, descriptor FIFO (and EXT chunk-1 FIFO) |
| `ctrlrd_engine_shared` | One `ctrlrd_engine` | Nothing: an owner register says which channel the engine serves |
| `ctrlwr_engine_shared` | One `ctrlwr_engine` | Nothing: an owner register says which channel the engine serves |

**Descriptor engine.** Each channel keeps one fetch in flight and stays in order. Different channels' fetches overlap on the bus because each carries its own ARID. Each scheduler sees exactly the per-channel engine's behaviour: - the same kick policy, and refusal of a kicked address outside both ranges; - the extended-descriptor second fetch; - the prefetch throttle, up to the threshold plus one buffered; - autonomous chaining, the error pulse, and completion and error MonBus packets.

A fetch issues only when its channel's FIFO has a free entry, so a channel that stops consuming can never hold the shared R channel. A channel reset in the middle of a fetch records the owed beat. The beat is drained when it arrives, and the channel takes no new kick until then. Reset also drains the channel's FIFO, so descriptors prefetched for an aborted chain are not delivered afterwards. Once ARVALID is raised, the AR address and ID hold until accepted.

The shared descriptor engine's MonBus output differs from the per-channel engine's in two ways: - **Range refusal:** the error packet for a refused kicked address carries 0 in its response field. The per-channel engine carried whatever read response it had last latched, which was stale. - **Colliding events:** packets go through a small FIFO, and two events can occur in one cycle: a refusal on the issue side, and a completion or error on the R side. The issue-side packet waits one cycle. If another collision arrives while it waits, the waiting packet is dropped. The per-channel engine also dropped packets, whenever MonBus backpressured it. Tests that count descriptor-engine packets should allow for this.

**Control engines.** The engine runs one request at a time. - A request is offered only while the engine is idle, its last MonBus packet has drained, and no error remains from the previous owner.

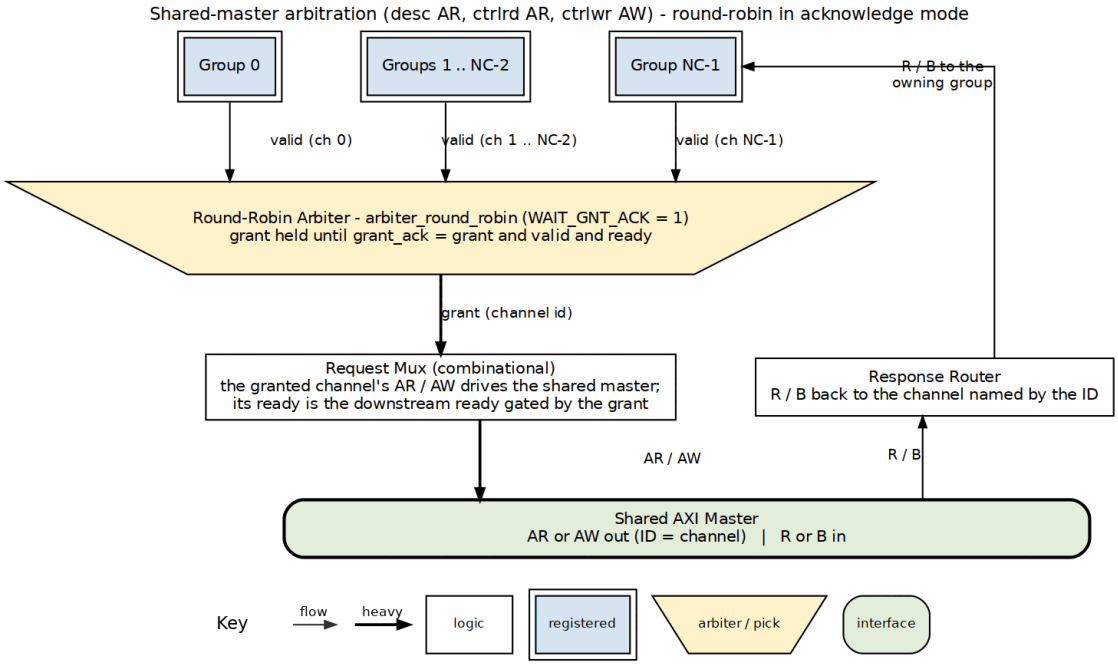
A stale error is cleared with a one-cycle engine reset before the next channel's request is taken. - The owner sees the engine's idle and error exactly as a dedicated engine would show them; every other channel sees idle 1 and error 0. - The owner's channel reset resets the engine only while the owner's operation is in flight, so a channel parked in reset cannot hold the engine for everyone else. - MonBus packets are re-tagged with the owner's channel and agent ID (0x20 + channel for control reads, 0x28 + channel for control writes).

Trade-off: a `CTRL_READ` that keeps polling holds the control-read engine, so other channels' control reads wait behind it. Data transfers are unaffected.

## 17.5 Shared Master Arbitration (SHARED_ENGINES = 0)

Each shared master has a round-robin arbiter over `NUM_CHANNELS` requesters in acknowledge mode. The grant is held until the granted channel's request is accepted (`grant_ack`), so a granted channel always sees its own valid and ready. The AR or AW ready seen by a channel is the downstream ready gated by that channel's grant.

### 17.5.1 Figure 3.2.2: Shared Master Arbitration




*Shared master arbitration*

**Source:** 11_shared_master_arbitration.dot

---

## 17.6 MonBus Aggregation

*Table 3.2.10: MonBus Source Assignment*

| Source | Origin | Content |
|---|---|---|
| 0 to NC-1 | `scheduler_group[ch]` (per-channel engines) or `scheduler[ch]` (shared engines) | Per-channel engines: that channel's descriptor engine, scheduler, control read and control write packets. Shared engines: that channel's scheduler packets |
| NC | Descriptor AXI monitor | Descriptor fetch monitor packets |
| NC+1 | Shared descriptor engine (shared engines only) | Every channel's descriptor completion and error packets |
| NC+2 | Shared control-read engine (shared engines only) | Every channel's control-read packets |
| NC+3 | Shared control-write engine (shared engines only) | Every channel's control-write packets |

One round-robin arbiter with acknowledge mode merges the sources into `mon_*`: `NUM_CHANNELS + 4` with shared engines, `NUM_CHANNELS + 1` with per-channel engines. Each source carries its own timestamp alongside the packet.

---

## 17.7 Differences from RAPIDS Beats

*Table 3.2.11: Array Delta*

| Item | RAPIDS Beats | RAPIDS |
|---|---|---|
| Parameter `OFF_W` | absent | present |
| `sched_rd_pkt_*`, `sched_wr_pkt_*` vectors | absent | present, wired by channel |
| Everything else | | Unchanged |

The beats chapter is Beats Scheduler Group Array.

---

**Last Updated:** 2026-09-30

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 18 Sink Data Path

**Module:** `snk_data_path.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro/`
**Status:** Implemented **Last Updated:** 2026-09-30

---

## 18.1 Overview

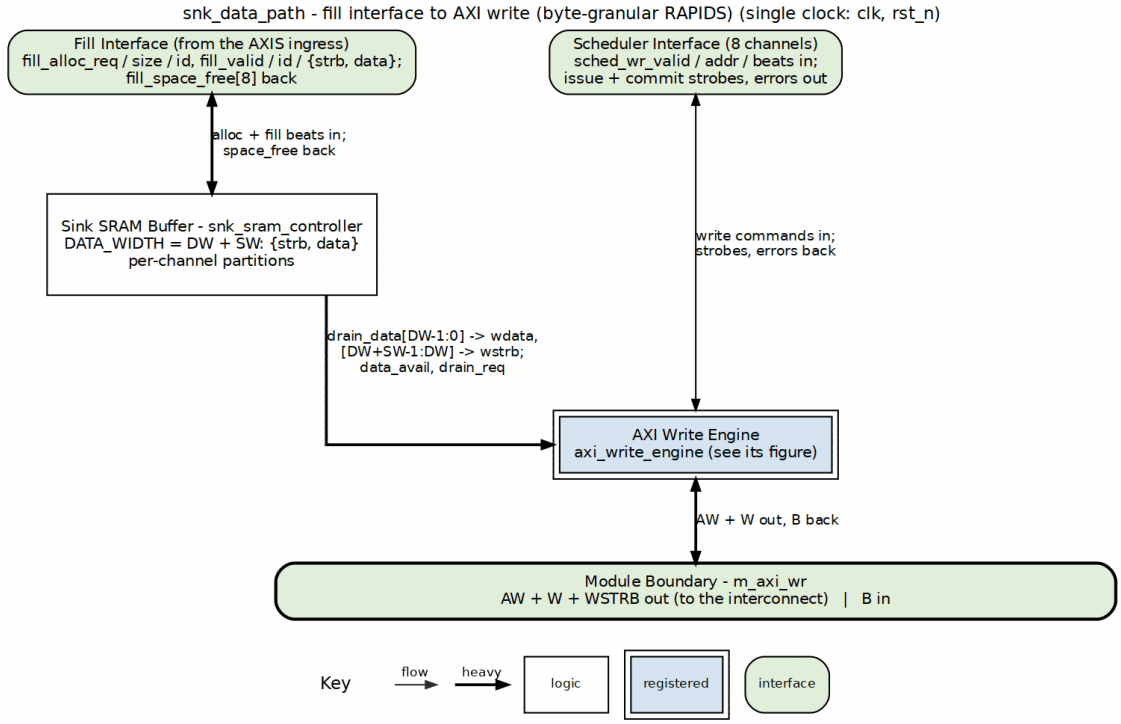
The sink data path joins the per-channel SRAM controller to the AXI write engine. Data enters through the fill interface, waits in the channel's SRAM partition and leaves as AXI4 write bursts.

In RAPIDS every SRAM word carries the byte enables of its data. The fill interface widens from `DW` to `DW + SW` bits, where `SW = DW/8`, and the write engine drives `m_axi_wstrb` from the stored enables. Everything else, including the allocation protocol, the scheduler interface and the burst rules, is as in RAPIDS Beats.

### 18.1.1 Key Features

- **Per-channel SRAM partitions:** flow control by allocation and space-free counts.
- **Byte enables stored with the data:** the SRAM is `DW + SW` bits wide.
- **AXI write engine:** 4 KB-safe, beat-aligned bursts. See AXI Write Engine.
- **Scheduler integration:** beat counts in, issue and commit strobes out.

### 18.1.2 Figure 3.3.1: Sink Data Path Block Diagram




*Sink data path block diagram*

**Source:** 09_snk_data_path.dot

## 18.2 Parameters

*Table 3.3.1: Sink Data Path Parameters*

| Parameter | Default | Description |
|---|---|---|
| NUM_CHANNELS | 8 | Number of channels (alias NC). |
| ADDR_WIDTH | 64 | Address width (alias AW). |
| DATA_WIDTH | 512 | Data width in bits (alias DW). The board uses 256. |
| AXI_ID_WIDTH | 8 | AXI ID width (alias IW). |
| SRAM_DEPTH | 512 | Words per channel partition (alias SD). |
| SEG_COUNT_WIDTH | $clog2(SRAM_DEPTH) + 1 | Space counter width (alias SCW). |
| PIPELINE | 1 | SRAM read pipeline setting. |
| AW_MAX_OUTSTANDING | 8 | AW commands the write engine may have in flight. |
| W_PHASE_FIFO_DEPTH | 64 | Depth of the write engine W-phase queue. |
| B_PHASE_FIFO_DEPTH | 16 | Depth of the write engine B-phase queue. |
| CIW | $clog2(NC) | Channel id width. |
| SW | DW / 8 | Byte enables per beat. New in RAPIDS. |

## 18.3 Interfaces

### 18.3.1 Fill Interface

*Table 3.3.2: Fill Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `fill_alloc_req` | input | 1 | Space allocation request. |
| `fill_alloc_size` | input | 8 | Beats to allocate. |
| `fill_alloc_id` | input | CIW | Channel to allocate in. |
| `fill_space_free` | output | NC x SCW | Free space per channel. |
| `fill_valid` | input | 1 | Fill beat valid. |
| `fill_ready` | output | 1 | Ready for a fill beat. |
| `fill_id` | input | CIW | Channel of the fill beat. |
| `fill_data` | input | DW + SW | `{byte enables, data}`. Bits `[DW-1:0]` are data. Bits `[DW+SW-1:DW]` are enables. |

### 18.3.2 Configuration

*Table 3.3.3: Configuration*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `cfg_axi_wr_xfer_beats` | input | 8 | Write burst size cap in beats, minus one. Renamed from `cfg_axi_wr_xfer`. |
| `cfg_channel_reset` | input | NC | Per-channel reset, level or pulse. Forwarded to the write engine and the SRAM controller wrapper. |

### 18.3.3 Scheduler Interface

*Table 3.3.4: Scheduler Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| sched_wr_valid | input | NC | Channel requests writes. |
| sched_wr_ready | output | NC | Last burst of a descriptor committed. |
| sched_wr_addr | input | NC x AW | Working destination byte address. |
| sched_wr_beats | input | NC x 32 | Beats still to write. |
| sched_wr_burst_len | input | NC x 8 | Requested burst length. Not used for sizing. |
| sched_wr_done_strobe | output | NC | AW issued. |
| sched_wr_beats_done | output | NC x 32 | Beats in that AW. |
| sched_wr_commit_strobe | output | NC | B response received. |
| sched_wr_commit_beats | output | NC x 32 | Beats committed by that B. |
| sched_wr_error | output | NC | Sticky write error (bad B response). |

### 18.3.4 AXI4 Write Master

The write master is `m_axi_aw*`, `m_axi_w*` and `m_axi_b*` with `IW`-bit IDs and `DW`-bit data. See AXI4 Interface Specification. `m_axi_wstrb` is `DW/8` bits and carries the stored byte enables.

---

## 18.4 Byte Enable Path

1. The upstream stage presents `{strb, data}` on `fill_data` with `fill_valid`.
2. The SRAM controller stores the whole `DW + SW` word. It is instantiated with `DATA_WIDTH = DW + SW`.
3. The drain side returns the same word as `drain_data`.
4. `drain_data[DW-1:0]` goes to `axi_wr_sram_data` and `drain_data[DW+SW-1:DW]` goes to `axi_wr_sram_strb` on the write engine.
5. The write engine drives `m_axi_wdata` and `m_axi_wstrb` from those two.

The SRAM depth counts words, so capacity in beats is unchanged. Only the width grows, by one bit per data byte.

### 18.4.1 Channel Reset

`cfg_channel_reset[ch]` reaches two blocks. The AXI Write Engine stops issuing for the channel, completes an open burst with null beats and clears its error flag. The SRAM controller wrapper (`snk_sram_controller`) holds one single-channel `sram_controller` per channel. Each instance's reset is a registered flop, `r_ch_rst_n[ch]`, which goes low one cycle after `cfg_channel_reset[ch]` rises, so the partition's pointers and counters return to empty. The shared STREAM `sram_controller` has no channel reset of its own and is not modified; giving each channel its own instance and reset is what makes the reset per channel. A reset of one channel leaves the instances of the other channels untouched.

---

## 18.5 Differences from RAPIDS Beats

*Table 3.3.5: Sink Data Path Delta*

| Item | RAPIDS Beats | RAPIDS |
|---|---|---|
| `fill_data` width | `DW` | `DW + SW` |
| SRAM word width | `DW` | `DW + SW` |
| `m_axi_wstrb` | all ones | stored per-beat enables |
| Config port name | `cfg_axi_wr_xfer` | `cfg_axi_wr_xfer_beats` |
| Scheduler port names | `sched_wr`, `sched_wr_done`, `sched_wr_commit` | `sched_wr_beats`, `sched_wr_beats_done`, `sched_wr_commit_beats` |
| Fill allocation, drain, error export | | Unchanged |

The beats chapter is Sink Data Path. Its timing diagrams apply to the AW, W and B sequencing unchanged.

---

**Last Updated:** 2026-09-30

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 19 Sink Data Path with AXIS (Ingress)

**Module:** `snk_data_path_axis.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro/` **Status:** Implemented **Last Updated:** 2026-09-30

---

## 19.1 Overview

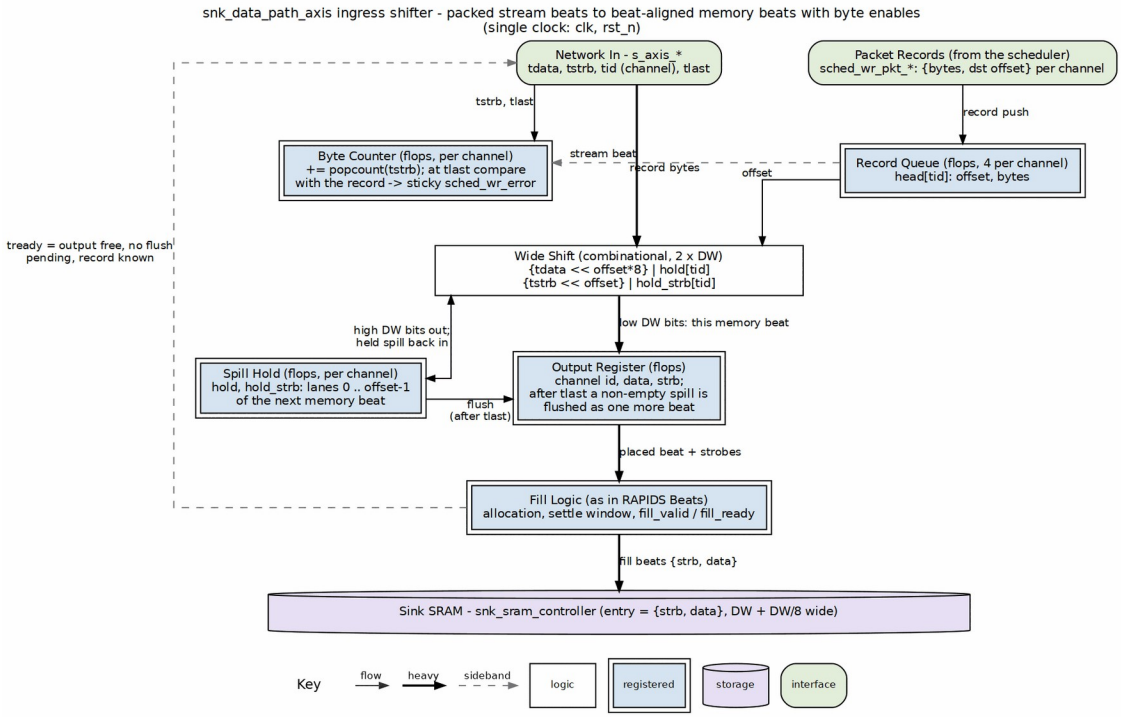
This module puts an AXI-Stream slave in front of the Sink Data Path and converts a packed byte stream into beat-aligned memory beats with byte enables. It is the block that makes the sink side byte-granular.

A packet arrives with its bytes packed from lane 0. Its destination in memory starts at a byte offset inside the first memory beat. The scheduler supplies that offset in a packet record. The module shifts each stream beat up by the offset, holds the bytes that spill past the beat boundary, and merges them into the next memory beat. After the last stream beat, a non-empty spill becomes one more, partial, memory beat.

### 19.1.1 Key Features

- **Barrel shift by the destination offset:** stream lane `l` lands in memory lane `offset + l`.
- **Byte enables follow the data:** the shifted strobe is stored beside the data and becomes WSTRB.
- **Per-channel state:** spill hold, received-byte count and length-error flag per channel, so packets of different channels interleave by beat.
- **Packet record queue:** four records per channel, fed by the scheduler.
- **Descriptor gating:** the stream is accepted only when the channel has a packet record.
- **Length check:** a packet whose byte count differs from the descriptor sets a sticky channel error.
- **Allocation logic unchanged from RAPIDS Beats**, including the settle wait.

### 19.1.2 Figure 3.4.1: Ingress Shifter




*Ingress shifter*

**Source:** 01_ingress_shifter.dot

---

## 19.2 Parameters

Parameters of Sink Data Path apply unchanged. The additions are:

*Table 3.4.1: Additional Parameters*

| Parameter | Default | Description |
|---|---|---|
| `AXIS_ID_WIDTH` | 8 | `tid` width. The low `CIW` bits select the channel. |
| `AXIS_DEST_WIDTH` | 4 | `tdest` width. Not used to select a channel. |
| `AXIS_USER_WIDTH` | 1 | `tuser` width. Not used. |
| `SW` | `DW / 8` | Byte lanes per beat. |
| `OFF_W` | `$clog2(SW)` | Width of the packet-record offset. |

## 19.3 Interfaces

### 19.3.1 AXI-Stream Slave

*Table 3.4.2: AXI-Stream Slave*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `s_axis_tdata` | input | DW | Packed data. |
| `s_axis_tstrb` | input | SW | Byte strobes. Contiguous from lane 0. Partial only on the `tlast` beat. |
| `s_axis_tlast` | input | 1 | Last beat of the packet. |
| `s_axis_tid` | input | AXIS_ID_WIDTH | Channel select. |
| `s_axis_tdest` | input | AXIS_DEST_WIDTH | Unused. |
| `s_axis_tuser` | input | AXIS_USER_WIDTH | Unused. |
| `s_axis_tvalid` | input | 1 | Beat valid. |
| `s_axis_tready` | output | 1 | Beat accepted. |

`s_axis_tready` is high when the output register is free, no flush beat is pending and the addressed channel has a packet record queued. A channel's stream therefore stalls until its scheduler has fetched the descriptor. `s_axis_tready` is a single signal qualified by the TID of the beat on the bus, so a beat for a channel that has no record blocks the stream, and any channel's beats behind it, until that record arrives. Channels on one stream are not isolated from each other; a system that needs isolation gives each channel its own stream or guarantees the descriptor is queued before the packet is sent. `s_axis_tready` is also low for a beat whose TID names a channel in reset, and high without a record for a channel that is discarding the tail of a cut packet (see Channel Reset).

### 19.3.2 Scheduler Interface

The write request and completion ports are those of Sink Data Path. The packet record ports are new:

*Table 3.4.3: Packet Record Ports*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `sched_wr_pkt` | input | NC | Record pulse from the |

| Signal | Direction | Width | Description |
|---|---|---|---|
| _valid | | | scheduler. |
| sched_wr_pkt_ready | output | NC | Record queue not full. |
| sched_wr_pkt_bytes | input | NC x 32 | Packet length in bytes. |
| sched_wr_pkt_offset | input | NC x OFF_W | Destination offset in the first memory beat. |
| cfg_channel_reset | input | NC | Per-channel reset, level or pulse. Clears the channel's queue, hold, byte count and length error, and discards the tail of a cut packet. |

`sched_wr_error` is the OR of the write engine's B-response error and the packet length error of the channel. Both clear on `cfg_channel_reset` (see Channel Reset).

### 19.3.3 Other Ports

The AXI4 write master is that of Sink Data Path. `cfg_alloc_size` (8 bits) is the allocation segment per request, with 0 treated as 1. The debug outputs are `dbg_sram_bridge_pending`, `dbg_sram_bridge_out_valid`, `dbg_axis_beats_received` and `dbg_axis_packets_received`. `o_active_channel_id` and `o_active_channel_valid` carry the write engine's active channel to the per-channel bus meter.

---

## 19.4 Ingress Operation

### 19.4.1 Packet Records

Each channel has a four-entry queue of `{offset, bytes}`. A record is pushed on `sched_wr_pkt_valid` when the queue is not full and popped when the packet completes. The head record supplies the offset for every beat of the packet and the expected length at `tlast`.

### 19.4.2 Placement

For an accepted stream beat, with `off` the head record's offset:

- The wide word is `(tdata_m << off*8) | hold_data`, of width `2*DW`, where `tdata_m` is `tdata` with every lane whose `tstrb` bit is clear forced to zero.
- The wide strobe is `(tstrb << off) | hold_strb`, of width `2*SW`.
- The low half goes to the output register as the memory beat and its enables.
- The high half becomes the new hold for the channel.

The strobe mask matters on a partial last beat: the non-strobed lanes carry whatever the sender left there, and unmasked they would shift into the spill hold. A packet that ends without a spill never flushes the hold, so that junk would be ORed into the first memory beat of the channel's next packet. Masking by `tstrb` keeps the hold to real payload bytes.

The first memory beat has its low `off` strobe bits clear, so the bytes below the start address are not written.

### 19.4.3 Flush and Completion

On `tlast` the received-byte total is compared with the head record. If the wide strobe's high half is non-zero, a flush is scheduled and `s_axis_tready` drops until the hold has been emitted as the last memory beat. The packet then completes and its record pops. If the high half is empty the packet completes on the `tlast` beat itself.

### 19.4.4 Length Check

`r_pkt_rx_bytes` counts the popcount of `tstrb` over the packet. At `tlast`, a total that differs from the record's byte count sets the channel's sticky error. Nothing is padded or dropped. The error is visible on `sched_wr_error` and the scheduler treats it as fatal. It clears on `cfg_channel_reset`.

### 19.4.5 Channel Reset

`cfg_channel_reset[ch]` is a register level in the system (`SNK.CHANNEL_RESET.CH_RST` ORed with `GLOBAL_CTRL.GLOBAL_RST`), so a pulse and a held level must both work. The ingress registers it once into `r_rst_d1` and uses `w_rst[ch] = cfg_channel_reset[ch] | r_rst_d1[ch]`, which stretches a one-cycle pulse to two cycles and lets every clear see the reset on a cycle where it is asserted. The stretch is what settles an output-register beat and an allocation

that were launched on the cycle of the reset. The reset is written for one channel; every term is indexed by channel, so the other channels are not affected.

*Table 3.4.4: Channel Reset Actions*

| State | Action on `w_rst[ch]` |
|---|---|
| Packet record queue | Write and read pointers cleared. A record arriving on the reset cycle is not pushed. |
| Spill hold, `r_pkt_rx_bytes` | Cleared. |
| `r_pkt_len_error` | Cleared. The channel's `sched_wr_error` contribution drops. |
| `r_pending_alloc`, `r_alloc_settle` | Cleared, so no allocation request of the old packet is issued. |
| Output register (`r_out_valid`), flush beat (`r_flush_valid`) | Dropped when the beat belongs to the channel, so no beat of the old packet enters the SRAM after the reset. |
| `s_axis_tready` | Low for a beat whose TID names a channel in reset. |
| `r_discard[ch]` | Set when a packet was part-way through the stream (received bytes non-zero and no flush of its own pending). |

With `r_discard[ch]` set the ingress accepts the channel's beats without a record and drops them: `s_axis_tready` does not wait for a packet record, and no state is updated. `r_discard[ch]` clears on the dropped beat that carries `tlast`. The next beat is then an ordinary first beat of a new packet. The sender is not asked to do anything special: it finishes the packet it was sending and the tail is discarded. A packet whose beats had all been accepted (the flush beat aside) is not discarded.

Head-of-line blocking is unchanged. A beat for a channel with no record, and not discarding, still holds `s_axis_tready` low for every channel. After a reset the next packet for the channel waits for its descriptor as before.

The channel's write engine and SRAM partition are cleared by the blocks that own them; see AXI Write Engine and Sink Data Path. All of them settle within a few cycles of the reset, and the scheduler's own reset returns it to CH_IDLE, so a channel reset recovers the channel without `aresetn`.

### 19.4.6 Allocation

The output register holds one memory beat for the fill interface. Allocation follows the beats design:

- A channel requests `cfg_alloc_size` beats, or the remainder when less is free.
- The next allocation of a channel waits three cycles after its previous one so `fill_space_free` is current.
- A beat leaves the output register only when the fill interface accepts it.

---

## 19.5 Differences from RAPIDS Beats

*Table 3.4.5: Ingress Delta*

| Item | RAPIDS Beats | RAPIDS |
|---|---|---|
| Stream beat to memory beat | one to one, full lanes | shifted by the offset, spill held |
| `s_axis_tstrb` | full only | partial on the `tlast` beat |
| Byte enables into SRAM | none | `{strb, data}` word |
| Packet record queue | absent | four per channel |
| Stream gating | buffers before the descriptor | waits for the packet record |
| Length mismatch | not checked | sticky channel error, cleared by channel reset |
| Channel reset | not applicable | per-channel clear, cut packet tail discarded |

The beats chapter is Sink Data Path with AXIS.

---

**Last Updated:** 2026-09-30

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 20 Sink SRAM Controller Specification

**Module:** `snk_sram_controller_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

---

## 20.1 Overview

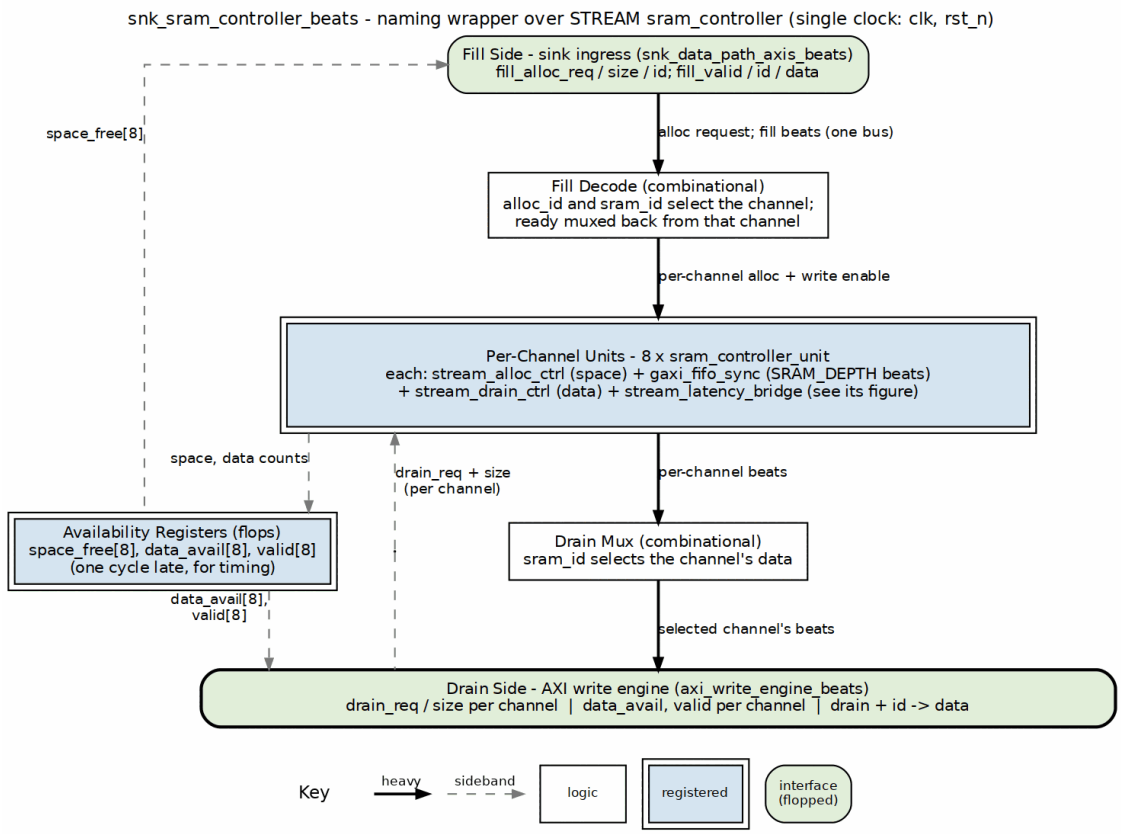
The Sink SRAM Controller is a naming wrapper, with no logic of its own, around STREAM's `sram_controller` – the one per-channel SRAM implementation both DMAs share since `bdf4e0dff`. It maps the RAPIDS `fill_*`/`drain_*` names onto STREAM's ports; the eight per-channel units (allocation counter, FIFO, drain counter, latency bridge) live inside STREAM's block. It manages 8 channel units, providing per-channel buffering with channel arbitration for the shared AXI write engine.

### 20.1.1 Key Features

- **One shared implementation:** instantiates STREAM's `sram_controller` once; it holds 8 `sram_controller_units`, each a `gaxi_fifo_sync` FIFO with `stream_alloc_ctrl`, `stream_drain_ctrl` and `stream_latency_bridge`
- **Channel Arbitration:** none inside the controller – the AXI write engine picks a channel and presents it on `drain_id`
- **Flow Control Integration:** per-channel `stream_alloc_ctrl` / `stream_drain_ctrl` virtual FIFOs; the drain counter's virtual depth is 2 x SRAM_DEPTH (stream BUG-011)
- **Space Tracking:** Reports available space per channel

### 20.1.2 Block Diagram

### 20.1.3 Figure 3.5.1: Sink SRAM Controller Block Diagram




*Sink SRAM controller block diagram*

**Source:** 15_snk_sram_controller.dot

---

## 20.2 Parameters

```
parameter int NUM_CHANNELS = 8;
parameter int DATA_WIDTH = 512;
parameter int SRAM_DEPTH = 512;                    // Per-channel depth
parameter int ADDR_WIDTH = $clog2(SRAM_DEPTH);
```

: Table 3.5.1: Sink SRAM Controller Parameters

## 20.3 Port List

### 20.3.1 Clock and Reset

*Table 3.5.2: Clock and Reset*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `clk` | input | 1 | System clock |
| `rst_n` | input | 1 | Active-low reset |

### 20.3.2 Fill Interface (ID-Selected)

*Table 3.5.3: Fill Interface (ID-Selected)*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `fill_alloc_req` | input | 1 | Allocation request (single, ID-selected) |
| `fill_alloc_size` | input | 8 | Beats to allocate |
| `fill_alloc_id` | input | CIW | Transaction ID selects the channel |
| `fill_space_free` | output | NC x SCW | Available space per channel |
| `fill_valid` | input | 1 | Fill data valid |
| `fill_ready` | output | 1 | Ready for fill data |
| `fill_id` | input | CIW | Transaction ID selects the channel |
| `fill_data` | input | DW | Fill data |

### 20.3.3 Arbitrated Drain Interface (to AXI Write Engine)

*Table 3.5.4: Arbitrated Drain Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `drain_data_avail` | output | NC x SCW | Data available per channel |
| `drain_req` | input | NC | Drain reservation request per channel |
| `drain_size` | input | NC x 8 | Beats to reserve |

| Signal | Direction | Width | Description |
|---|---|---|---|
| drain_valid | output | NC | Drain valid, registered (for arbitration) |
| drain_valid_comb | output | NC | Drain valid, combinational (gates m_axi_wvalid) |
| drain_read | input | 1 | Consumer read strobe |
| drain_id | input | CIW | Channel ID select for drain |
| drain_data | output | DW | Drain data (muxed from selected channel) |

### 20.3.4 Debug Interface

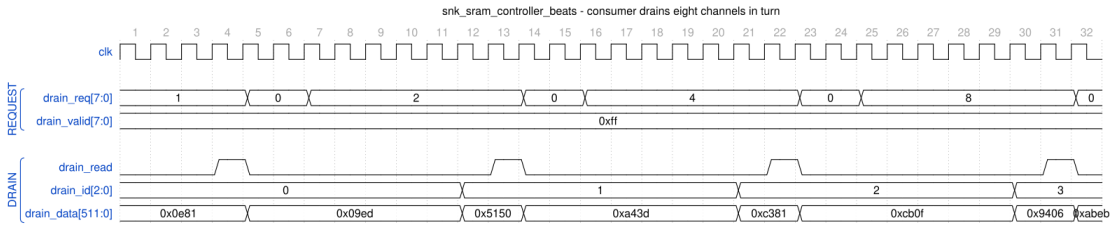
*Table 3.5.5: Debug Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| dbg_bridge_pending | output | NC | Bridge has a pending beat per channel |
| dbg_bridge_out_valid | output | NC | Bridge output valid per channel |

## 20.4 Drain Selection

There is no arbiter inside the controller. Every channel unit raises its own `drain_valid[ch]` / `drain_size[ch]` when it has data; the AXI write engine picks a channel, drives it on `drain_id`, and the controller steers that channel's beats to the single `drain_*` port. Switching channels costs nothing beyond the consumer's own decision, and a channel keeps presenting `drain_valid` until it is empty.

### 20.4.1 Figure 3.5.2: Drain Selection by the Consumer




*snk_sram_controller_beats - consumer drains eight channels in turn*

**Source:** snk_sram_controller_drain_select.json, captured from `dv/tests/macro_beats/test_snk_sram_controller_beats.py` (`multi_channel`, 8 channels, 512-bit data, 512-entry SRAM, `REG_LEVEL=GATE`) with `WAVES=1`.

Reading it: all eight channels have data (`drain_valid` = 0xff) and the consumer walks them in turn. It raises one bit of `drain_req` at a time; the controller answers with `drain_id` naming that channel and `drain_data` presenting its head word on the same cycle, and the consumer's `drain_read` pulse pops it. Each channel's turn lasts nine cycles here because the test releases the request and re-arms the next one; the controller itself needs no gap between channels.

## 20.5 Integration with AXI Write Engine

```
snk_sram_controller_beats #(
    .NUM_CHANNELS(8),
    .DATA_WIDTH(512),
    .SRAM_DEPTH(512)
) u_snk_sram_ctrl (
    .clk                    (clk),
    .rst_n                  (rst_n),

    // Per-channel fill interface
    .fill_alloc_req         (fill_alloc_req),
    .fill_alloc_size        (fill_alloc_size),
    .fill_space_free        (fill_space_free),
    .fill_valid             (fill_valid),
    .fill_ready             (fill_ready),
    .fill_data              (fill_data),

    // Arbitrated drain interface
    .drain_req              (sram_drain_req),
    .drain_id               (sram_drain_id),
    .drain_size             (sram_drain_beats),

    // Status
    .drain_data_avail       (channel_data_avail),
    .dbg_bridge_pending     (channel_empty),
    .dbg_bridge_out_valid   (channel_full)
);
```

---

**Last Updated:** 2025-01-10

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 21 Source Data Path Specification

**Module:** `src_data_path_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

---

## 21.1 Overview

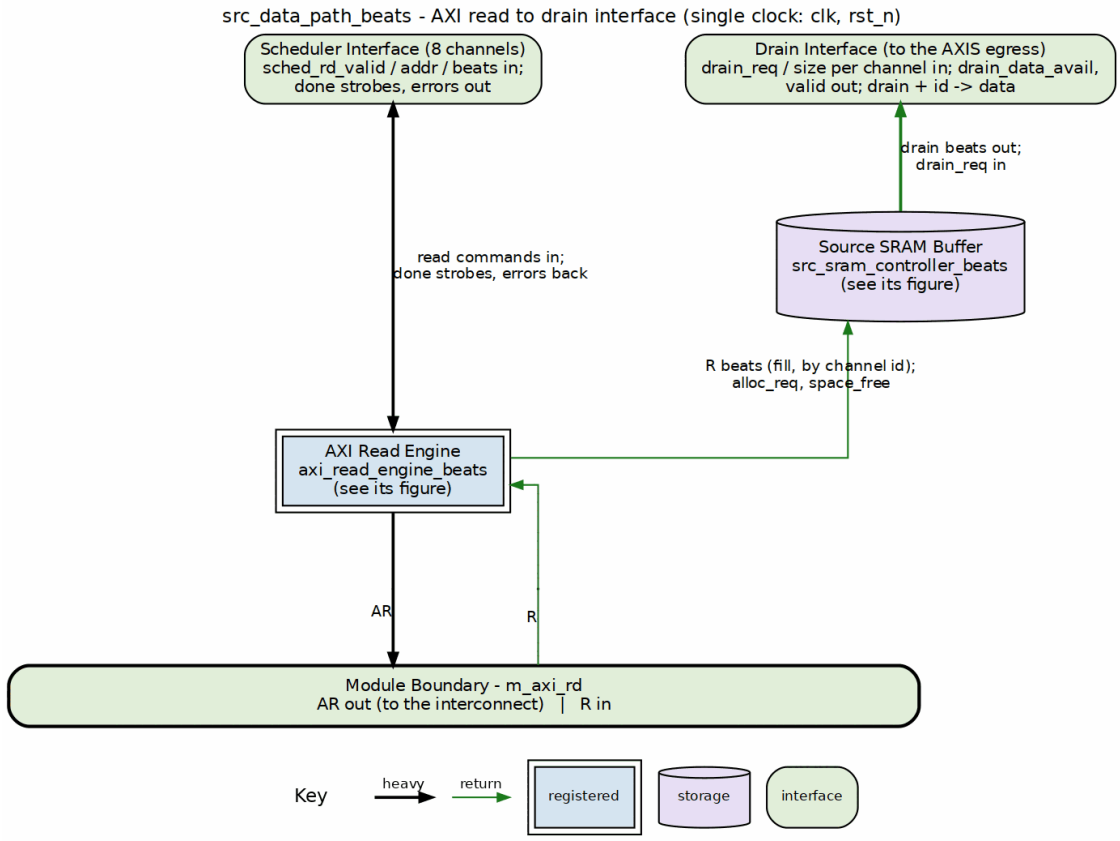
The Source Data Path integrates the AXI read engine and SRAM controller for memory-to-network transfers. Data is read from system memory via AXI4, buffered in SRAM, and delivered through the drain interface to the network.

### 21.1.1 Key Features

- **AXI Read Engine:** Streaming burst reads from system memory
- **8-Channel SRAM Controller:** Per-channel buffering with flow control
- **Beat-Level Tracking:** Precise flow control at beat granularity
- **Scheduler Integration:** Receives transfer requests from scheduler

### 21.1.2 Block Diagram

### 21.1.3 Figure 3.6.1: Source Data Path Block Diagram




*Source data path block diagram*

**Source:** 16_source_data_path.dot

## 21.2 Parameters

```
parameter int NUM_CHANNELS = 8;
parameter int ADDR_WIDTH = 64;
parameter int DATA_WIDTH = 512;
parameter int AXI_ID_WIDTH = 8;
parameter int SRAM_DEPTH = 512;
parameter int AR_MAX_OUTSTANDING = 8;
parameter int R_PHASE_FIFO_DEPTH = 64;
```

: Table 3.6.1: Source Data Path Parameters

## 21.3 Data Flow

### 21.3.1 Figure 3.6.2: Source Path Data Flow Sequence

```
1. Scheduler Read Request
   - sched_rd_valid = 1
   - sched_rd_addr = source address
   - sched_rd_beats = N
   - sched_rd_id = channel
        |
        v
2. Space Check (stream_alloc_ctrl inside STREAM's sram_controller)
   - Check src_drain_space_free[channel] >= N
   - Allocate N beats of space
        |
        v
3. AXI Read Transaction
   - axi_read_engine issues AR
   - m_axi_araddr = sched_rd_addr
   - m_axi_arlen = calculated from beats
        |
        v
4. R Channel Data
   - Data arrives on m_axi_rdata
   - Written to channel's SRAM partition
        |
        v
5. SRAM Write Complete
   - stream_drain_ctrl notified via the unit's latency bridge
   - Data available for drain
        |
        v
6. Drain Interface
   - src_drain_valid = 1
   - Data delivered to network master
        |
        v
7. Completion
   - sched_rd_done_strobe = 1
   - Scheduler notified of completion
```

---

## 21.4 Port List

### 21.4.1 Clock and Reset

*Table 3.6.2: Clock and Reset*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `clk` | input | 1 | System clock |
| `rst_n` | input | 1 | Active-low reset |

### 21.4.2 Configuration

*Table 3.6.3: Configuration*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `cfg_axi_rd_xfer_beats` | input | 8 | AXI read transfer size in beats (all channels) |

### 21.4.3 Scheduler Interface

*Table 3.6.4: Scheduler Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `sched_rd_valid` | input | NC | Channel requests read |
| `sched_rd_addr` | input | NC x AW | Source addresses |
| `sched_rd_beats` | input | NC x 32 | Beats remaining to read |
| `sched_rd_done_strobe` | output | NC | Burst completed (pulsed 1 cycle) |
| `sched_rd_beats_done` | output | NC x 32 | Beats completed in burst |
| `sched_rd_error` | output | NC | Sticky error flag per channel |

### 21.4.4 AXI Read Master Interface

*Table 3.6.5: AXI Read Master Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `m_axi_arvalid` | output | 1 | AR channel valid |

| Signal | Direction | Width | Description |
|---|---|---|---|
| m_axi_arready | input | 1 | AR channel ready |
| m_axi_araddr | output | AW | Read address |
| m_axi_arlen | output | 8 | Burst length |
| m_axi_arsize | output | 3 | Burst size |
| m_axi_arburst | output | 2 | Burst type |
| m_axi_arid | output | ID_W | Transaction ID |
| m_axi_rvalid | input | 1 | R channel valid |
| m_axi_rready | output | 1 | R channel ready |
| m_axi_rdata | input | DW | Read data |
| m_axi_rresp | input | 2 | Read response |
| m_axi_rid | input | ID_W | Response ID |
| m_axi_rlast | input | 1 | Last beat |

### 21.4.5 Drain Interface (Output to Network)

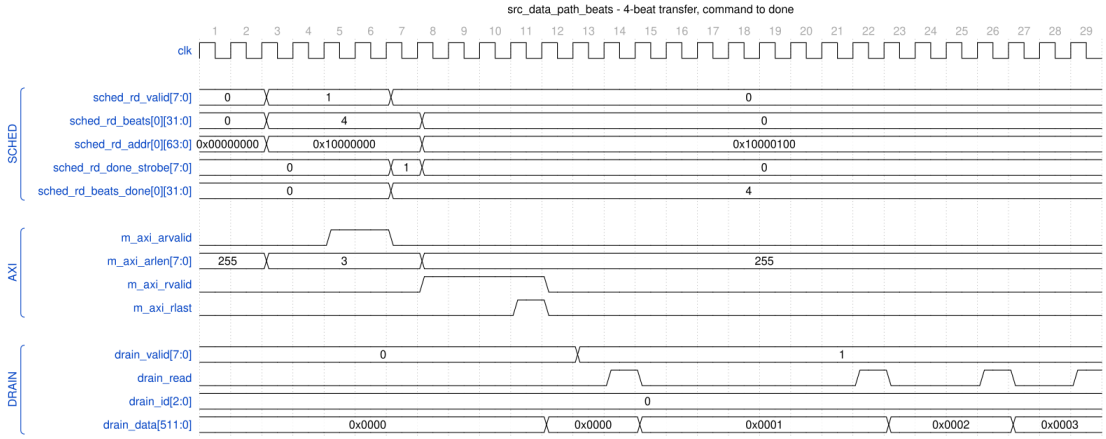
*Table 3.6.6: Drain Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| drain_data_avail | output | NC x SCW | Data available per channel |
| drain_req | input | NC | Drain reservation request per channel |
| drain_size | input | NC x 8 | Beats to reserve |
| drain_valid | output | NC | Drain data valid per channel |
| drain_valid_comb | output | NC | Combinational per-channel valid (beat gate) |
| drain_read | input | 1 | Consumer read strobe |
| drain_id | input | CIW | Channel ID select for drain |
| drain_data | output | DW | Drain data (muxed from |

| Signal | Direction | Width | Description |
|---|---|---|---|
| | | | selected channel) |

## 21.5 Timing Diagram

### 21.5.1 Figure 3.6.3: Source Path Transfer Timing




*src_data_path_beats - 4-beat transfer, command to done*

**Source:** src_data_path_transfer.json, captured inside `rapids_core_beats.u_src.u_source_data_path`, captured from `dv/tests/top_beats/test_rapids_core_beats.py` (source path, channel 0, 4 beats, 512-bit data, `REG_LEVEL=GATE`) with `WAVES=1`.

Reading it: the scheduler holds `sched_rd_valid[0]` with 4 beats at 0x10000000. The read engine issues the AR (`arlen = 3`) two cycles later; the four R beats land from cycle 8, `rlast` on the fourth, and the cycle after it the engine pulses `sched_rd_done_strobe[0]` with `beats_done = 4` – the scheduler advances its address to 0x10000100 on that strobe. The data reaches the source SRAM two cycles after the last R beat (`drain_valid[0]` rises) and the AXIS side drains it one word per `drain_read` pulse at the pace the egress consumer allows.

---

## 21.6 Integration Example

```
src_data_path_beats #(
    .NUM_CHANNELS(8),
    .ADDR_WIDTH(64),
    .DATA_WIDTH(512),
    .SRAM_DEPTH(512)
) u_source_path (
    .clk                    (clk),
    .rst_n                  (rst_n),

    // Scheduler interface
    .sched_rd_valid         (sched_rd_valid),
    .sched_rd_addr          (sched_rd_addr),
    .sched_rd_beats         (sched_rd_beats),
    .sched_rd_done_strobe   (sched_rd_done_strobe),
    .sched_rd_beats_done    (sched_rd_beats_done),
    .sched_rd_error         (sched_rd_error),

    // AXI read master
    .m_axi_arvalid          (src_axi_arvalid),
    .m_axi_arready          (src_axi_arready),
    .m_axi_araddr           (src_axi_araddr),
    .m_axi_arlen            (src_axi_arlen),
    .m_axi_rvalid           (src_axi_rvalid),
    .m_axi_rready           (src_axi_rready),
    .m_axi_rdata            (src_axi_rdata),
    .m_axi_rlast            (src_axi_rlast),

    // Drain interface
    .drain_valid            (src_drain_valid),
    .drain_data             (src_drain_data),
    .drain_id               (src_drain_id),
    .drain_data_avail       (src_data_avail)
);
```

---

**Last Updated:** 2025-01-10

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 22 Source Data Path with AXIS (Egress)

**Module:** `src_data_path_axis.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro/` **Status:** Implemented **Last Updated:** 2026-09-30

---

## 22.1 Overview

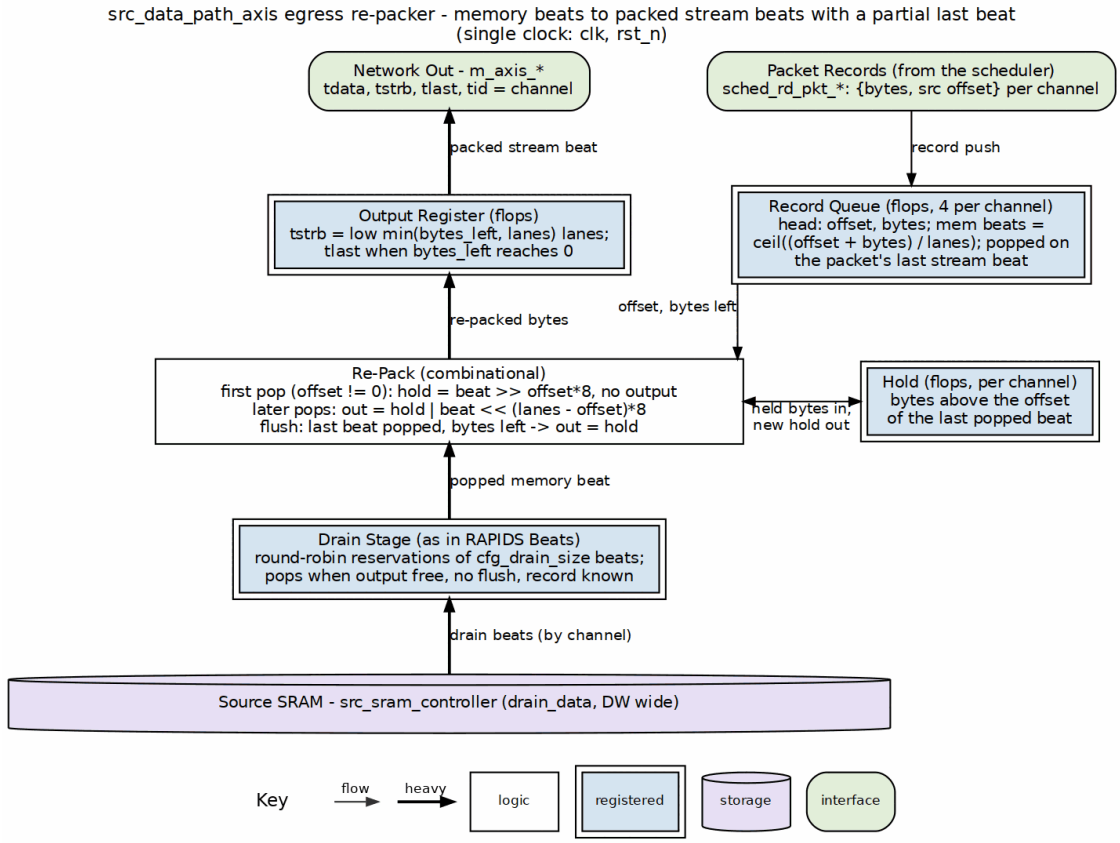
This module puts an AXI-Stream master behind the source data path. It pops beat-aligned memory beats from the SRAM and re-packs them into stream beats that start at lane 0, so a packet leaves as exactly the bytes its descriptor named.

A descriptor's source address may start in the middle of a beat. The scheduler supplies that offset and the byte length in a packet record. The module shifts each memory beat down by the offset, holds the bytes that belong to the next stream beat, and marks the last stream beat with a contiguous strobe and `tlast`. One descriptor produces one AXI-Stream packet.

### 22.1.1 Key Features

- **Barrel shift by the source offset:** memory lane `offset + l` goes out on stream lane `l`.
- **Partial last beat:** `tstrb` is the low `min(bytes_left, SW)` lanes and `tlast` marks the final beat.
- **Packets follow descriptors,** not drain reservations.
- **Two-stage reservation and drain,** unchanged from RAPIDS Beats. The drain stage advances on memory-beat pops, not on stream handshakes.
- **Per-channel state:** hold, bytes-left and beats-left per channel, so reservations of different channels interleave.
- **Packet record queue:** four records per channel, fed by the scheduler.

### 22.1.2 Figure 3.7.1: Egress Shifter




*Egress shifter*

**Source:** 02_egress_shifter.dot

---

## 22.2 Parameters

*Table 3.7.1: Parameters*

| Parameter | Default | Description |
|---|---|---|
| NUM_CHANNELS | 8 | Channels (alias NC). |
| ADDR_WIDTH | 64 | Address width (alias AW). |
| DATA_WIDTH | 512 | Data width in bits (alias DW). The board uses 256. |
| AXI_ID_WIDTH | 8 | AXI ID width (alias IW). |
| SRAM_DEPTH | 512 | Words per channel partition. |
| SEG_COUNT_WIDTH | $clog2(SRAM_DEPTH) + 1 | Space counter width. |
| PIPELINE | 1 | SRAM read pipeline setting. |
| AR_MAX_OUTSTANDING | 8 | ARs the read engine may have in flight. |
| AXIS_ID_WIDTH | 8 | tid width. |
| AXIS_DEST_WIDTH | 4 | tdest width. |
| AXIS_USER_WIDTH | 1 | tuser width. |
| SW | DW / 8 | Byte lanes per beat. |
| OFF_W | $clog2(SW) | Width of the packet-record offset. |

## 22.3 Interfaces

### 22.3.1 Configuration

*Table 3.7.2: Configuration*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `cfg_axi_rd_xfer_beats` | input | 8 | Read burst size cap in beats, minus one. |
| `cfg_drain_size` | input | 8 | Beats per drain reservation. 0 is treated as 1. |
| `cfg_channel_reset` | input | NC | Per-channel reset, level or pulse. Clears the channel's queue, hold, egress counters, drain reservations and error flag. |

### 22.3.2 Scheduler Interface

*Table 3.7.3: Scheduler Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `sched_rd_valid` | input | NC | Channel requests reads. |
| `sched_rd_addr` | input | NC x AW | Working source byte address. |
| `sched_rd_beats` | input | NC x 32 | Beats still to read. |
| `sched_rd_pkt_valid` | input | NC | Record pulse from the scheduler. |
| `sched_rd_pkt_ready` | output | NC | Record queue not full. |
| `sched_rd_pkt_bytes` | input | NC x 32 | Packet length in bytes. |
| `sched_rd_pkt_offset` | input | NC x OFF_W | Source offset in the first memory beat. |
| `sched_rd_done_strobe` | output | NC | AR issued. |
| `sched_rd_bea` | output | NC x 32 | Beats in that AR. |

| Signal | Direction | Width | Description |
|---|---|---|---|
| ts_done | | | |
| sched_rd_error | output | NC | Sticky read error. |

### 22.3.3 AXI-Stream Master

*Table 3.7.4: AXI-Stream Master*

| Signal | Direction | Width | Description |
|---|---|---|---|
| m_axis_tdata | output | DW | Packed data. |
| m_axis_tstrb | output | SW | Contiguous from lane 0. Partial only on the tlast beat. |
| m_axis_tlast | output | 1 | Last beat of the packet. |
| m_axis_tid | output | AXIS_ID_WIDTH | Channel. |
| m_axis_tdest | output | AXIS_DEST_WIDTH | Channel. |
| m_axis_tuser | output | AXIS_USER_WIDTH | Zero. |
| m_axis_tvalid | output | 1 | Beat valid. |
| m_axis_tready | input | 1 | Consumer ready. |

### 22.3.4 Other Ports

The AXI4 read master is that of the AXI Read Engine, with IW-bit IDs and DW-bit data. The debug outputs are dbg_rd_all_complete, dbg_r_beats_rcvd, dbg_sram_writes, dbg_arb_request, dbg_sram_bridge_pending, dbg_sram_bridge_out_valid, dbg_axis_beats_sent and dbg_axis_packets_sent.

---

## 22.4 Egress Operation

### 22.4.1 Reservation and Drain

Unchanged from RAPIDS Beats. A round-robin decision per cycle selects a channel whose available data, less reservations still in flight, reaches `cfg_drain_size` beats. It may also select one whose fill is finished and has any data left. Each decision is a one-cycle `drain_req` of up to `cfg_drain_size` beats. Up to four reservations queue ahead of the drain stage, which loads the next entry on the last beat of the current one.

### 22.4.2 Pop

A memory beat is popped when all of these hold:

- The drain stage is active on channel `c` and has a beat.
- The output register is free or being taken.
- No flush is pending.
- The channel's packet record is known.

Each pop counts against the reservation block, so the drain stage advances on pops.

### 22.4.3 Shift and Hold

With `off` the head record's offset:

*Table 3.7.5: Egress Shift Rules*

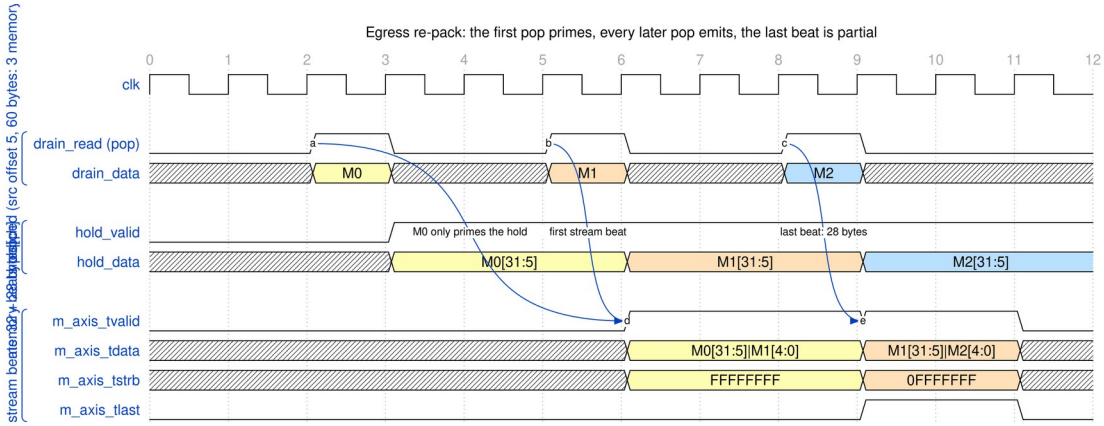
| Situation | Action |
|---|---|
| `off = 0` | Every pop emits a stream beat. |
| `off > 0`, first pop | The pop only primes the hold with `beat >> off*8`. It emits only when the packet fits in that single memory beat. |
| `off > 0`, later pops | Emit `hold \| (beat << (SW - off)*8)`. The new hold is `beat >> off*8`. |
| Last memory beat popped, bytes left | The flush emits the hold alone. |

The memory beat count of a packet is `(off + bytes + SW - 1) >> OFF_W`, computed in 33 bits.

### 22.4.4 Strobe and Last

Each emitted beat carries `n = min(bytes_left, SW)` bytes. `tstrb` is the low `n` lanes. `tlast` is set on the beat where `bytes_left` equals `n`. That beat also retires the packet record and clears the channel's hold, started flag and counters.

### 22.4.5 Waveform 3.7: Egress Re-pack




*Egress re-pack*

**Source:** 01_egress_repack.json

Offset 5, 60 bytes and a 256-bit beat give three memory beats. The first pop, M0, only primes the hold. The second pop emits the first stream beat with 32 bytes. The third pop emits the last beat with 28 bytes, `tstrb` `0FFFFFFF` and `tlast`.

---

## 22.5 Channel Reset

`cfg_channel_reset[ch]` is a register level, so a pulse and a held level must both work. As in the sink ingress the data path forms `w_rst[ch] = cfg_channel_reset[ch] | r_rst_d1[ch]`, a two-cycle stretch of a pulse. The read engine clears its error flag and settles the reads in flight (see AXI Read Engine); the data path clears everything that would otherwise let the old descriptor keep streaming:

*Table 3.7.6: Channel Reset Actions*

| State | Action on `w_rst[ch]` |
|---|---|
| Packet record queue | Pointers cleared. A record arriving on the reset cycle is not pushed. |
| Egress state | `r_eg_started`, `r_hold_valid`, `r_hold_data`, `r_eg_bytes_left` and `r_eg_mem_left` cleared. |
| Drain reservation | Grants for the channel are masked. `r_drain_tminus1[ch]` is cleared. |
| Reservation queue | Entries that belong to the channel are zeroed. An entry of size 0 loads the drain stage idle, so the drain stage skips it. A reservation being drained for the channel is dropped. |
| Output register | A beat already in `r_out` completes, so `m_axis` stays stable. |

The channel's SRAM partition is cleared by `src_sram_controller`, which holds one single-channel `sram_controller` per channel with a registered per-channel reset `r_ch_rst_n[ch]`. The shared STREAM controller has no channel reset of its own and is not modified. Reads of the old descriptor that are already on the bus are discarded by the read engine, so no stale data reaches the cleared partition.

**A packet cut by the reset is left unterminated.** If the reset lands after the first beat of a packet has gone out and before its `tlast` beat, the data path does not send a `tlast` and does not send a null terminator. The downstream receiver sees the packet stop. A packet that had not started, or whose `tlast` beat was already in `r_out` when the reset hit, is unaffected. A receiver that needs to resynchronize after a channel reset treats the reset as the end of the packet.

Other channels are unaffected: every term above is indexed by channel, the drain and reservation stages skip only the channel in reset, and their beats keep flowing through the shared `m_axis`.

---

## 22.6 Differences from RAPIDS Beats

*Table 3.7.7: Egress Delta*

| Item | RAPIDS Beats | RAPIDS |
|---|---|---|
| Stream beat from memory beat | one to one, full lanes | shifted by the offset, hold merged |
| `m_axis_tstrb` | all ones | contiguous, partial on the last beat |
| `m_axis_tlast` | on the drain block boundary | on the last byte of the descriptor |
| Drain advance | stream handshakes | memory-beat pops |
| Packet framing | one per drain block | one per descriptor |
| Packet record queue | absent | four per channel |
| Channel reset | not applicable | per-channel clear of every stage, cut packet left unterminated |
| Reservation stage, allocation of drain | | Unchanged |

The beats chapter is Source Data Path with AXIS.

---

**Last Updated:** 2026-09-30

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 23 Source SRAM Controller Specification

**Module:** `src_sram_controller_beats.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro_beats/` **Status:** Implemented **Last Updated:** 2025-01-10

---

## 23.1 Overview

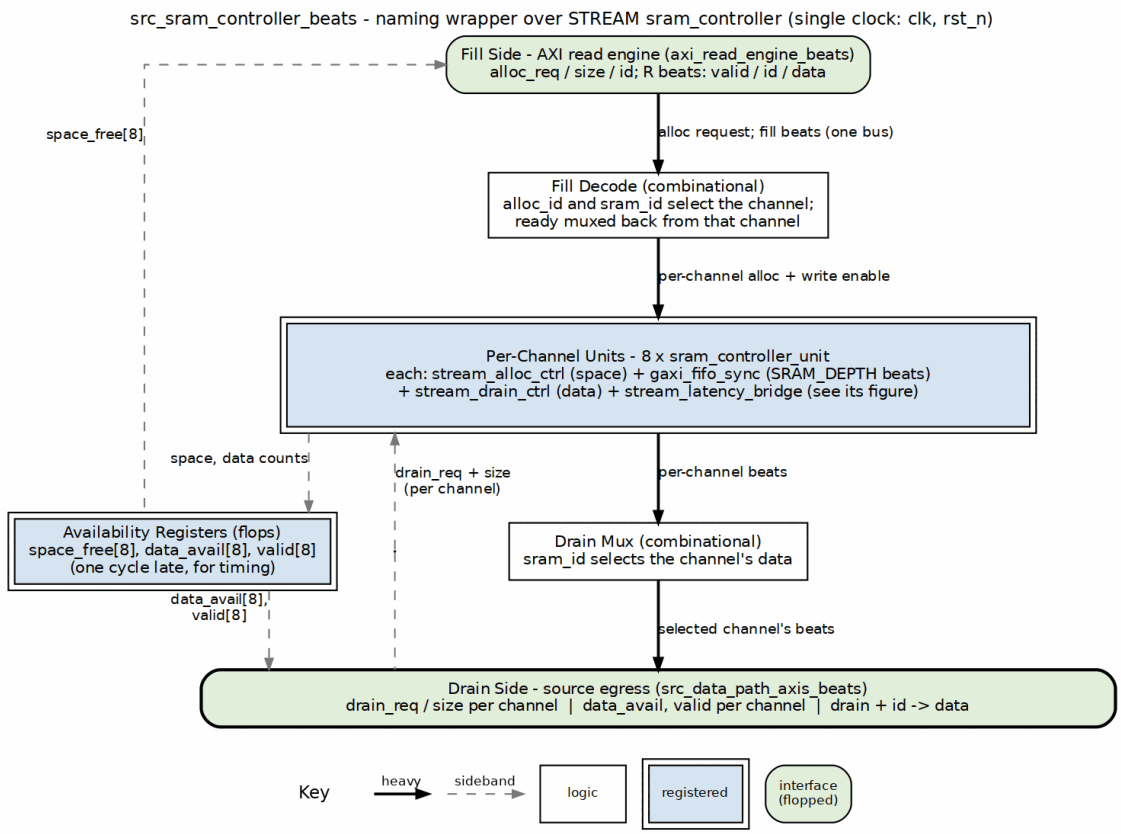
The Source SRAM Controller is a naming wrapper, with no logic of its own, around STREAM's `sram_controller` – the one per-channel SRAM implementation both DMAs share since `bdf4e0dff`. It maps the RAPIDS `fill_*`/`drain_*` names onto STREAM's ports; the eight per-channel units (allocation counter, FIFO, drain counter, latency bridge) live inside STREAM's block. It manages 8 channel units for the source data path, providing per-channel buffering with channel arbitration for data delivery to the network.

### 23.1.1 Key Features

- **One shared implementation:** instantiates STREAM's `sram_controller` once; it holds 8 `sram_controller_units`, each a `gaxi_fifo_sync` FIFO with `stream_alloc_ctrl`, `stream_drain_ctrl` and `stream_latency_bridge`
- **Channel Arbitration:** none inside the controller – the AXIS egress arbiter picks a channel and presents it on `drain_id`
- **Flow Control Integration:** per-channel `stream_alloc_ctrl` / `stream_drain_ctrl` virtual FIFOs; the drain counter's virtual depth is 2 x SRAM_DEPTH (stream BUG-011)
- **Data Availability Tracking:** Reports available data per channel

### 23.1.2 Block Diagram

### 23.1.3 Figure 3.8.1: Source SRAM Controller Block Diagram




*Source SRAM controller block diagram*

**Source:** 18_src_sram_controller.dot

---

## 23.2 Parameters

```
parameter int NUM_CHANNELS = 8;
parameter int DATA_WIDTH = 512;
parameter int SRAM_DEPTH = 512;                    // Per-channel depth
parameter int ADDR_WIDTH = $clog2(SRAM_DEPTH);
```

: Table 3.8.1: Source SRAM Controller Parameters

## 23.3 Port List

### 23.3.1 Clock and Reset

*Table 3.8.2: Clock and Reset*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `clk` | input | 1 | System clock |
| `rst_n` | input | 1 | Active-low reset |

### 23.3.2 Fill Interface (ID-Selected, from AXI Read Engine)

*Table 3.8.3: Fill Interface (ID-Selected)*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `fill_alloc_req` | input | 1 | Allocation request (single, ID-selected) |
| `fill_alloc_size` | input | 8 | Beats to allocate |
| `fill_alloc_id` | input | CIW | Transaction ID selects the channel |
| `fill_space_free` | output | NC x SCW | Available space per channel |
| `fill_valid` | input | 1 | Fill data valid |
| `fill_ready` | output | 1 | Ready for fill data |
| `fill_id` | input | CIW | Transaction ID selects the channel |
| `fill_data` | input | DW | Fill data |

### 23.3.3 Arbitrated Drain Interface (to Network)

*Table 3.8.4: Arbitrated Drain Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| `drain_data_avail` | output | NC x SCW | Data available per channel |
| `drain_req` | input | NC | Drain reservation request per channel |
| `drain_size` | input | NC x 8 | Beats to reserve |

| Signal | Direction | Width | Description |
|---|---|---|---|
| drain_valid | output | NC | Drain data valid per channel |
| drain_valid_comb | output | NC | Combinational per-channel valid (beat gate) |
| drain_read | input | 1 | Consumer read strobe |
| drain_id | input | CIW | Channel ID select for drain |
| drain_data | output | DW | Drain data (muxed from selected channel) |

### 23.3.4 Debug Interface

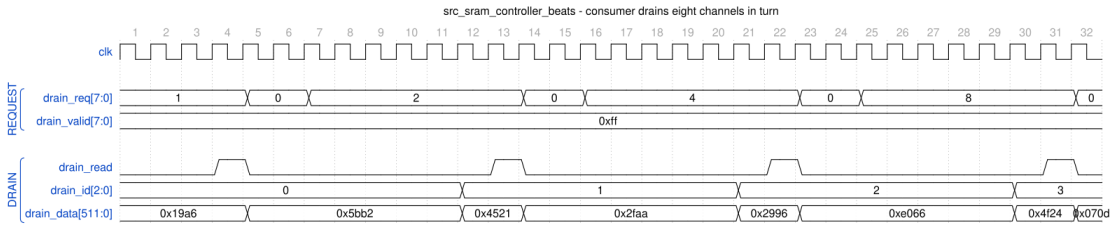
*Table 3.8.5: Debug Interface*

| Signal | Direction | Width | Description |
|---|---|---|---|
| dbg_bridge_pending | output | NC | Bridge has a pending beat per channel |
| dbg_bridge_out_valid | output | NC | Bridge output valid per channel |

## 23.4 Drain Selection

There is no arbiter inside the controller. Every channel unit raises its own `drain_valid[ch]` / `drain_size[ch]` when it has data; the AXIS egress arbiter picks a channel, drives it on `drain_id`, and the controller steers that channel's beats to the single `drain_*` port. Switching channels costs nothing beyond the consumer's own decision, and a channel keeps presenting `drain_valid` until it is empty.

### 23.4.1 Figure 3.8.2: Drain Selection by the Consumer




*src_sram_controller_beats - consumer drains eight channels in turn*

**Source:** src_sram_controller_drain_select.json, captured from `dv/tests/macro_beats/test_src_sram_controller_beats.py` (`multi_channel`, 8 channels, 512-bit data, 512-entry SRAM, `REG_LEVEL=GATE`) with `WAVES=1`.

Reading it: the same selection contract as the sink controller (Figure 3.5.2). With `drain_valid` = 0xff the consumer asks for one channel at a time through `drain_req`; `drain_id` and `drain_data` follow the request on the same cycle and `drain_read` pops the word. The nine-cycle turn per channel is the test's pacing, not the controller's.

---

## 23.5 Comparison: Sink vs Source SRAM Controller

*Table 3.8.6: Sink vs Source Controller Comparison*

| Aspect | Sink Controller | Source Controller |
|---|---|---|
| Fill Source | Network ingress | AXI read data |
| Drain Destination | AXI write engine | Network egress |
| Fill Interface | Per-channel from network | Per-channel from AXI R |
| Drain Interface | Arbitrated SRAM read | Arbitrated drain to network |
| Primary Use | Network -> Memory | Memory -> Network |

## 23.6 Integration Example

```
src_sram_controller_beats #(
    .NUM_CHANNELS(8),
    .DATA_WIDTH(512),
    .SRAM_DEPTH(512)
) u_src_sram_ctrl (
    .clk                    (clk),
    .rst_n                  (rst_n),

    // Per-channel fill interface (from AXI read engine)
    .fill_alloc_req         (fill_alloc_req),
    .fill_alloc_size        (fill_alloc_size),
    .fill_space_free        (fill_space_free),
    .fill_valid             (fill_valid),
    .fill_ready             (fill_ready),
    .fill_data              (fill_data),

    // Arbitrated drain interface (to network)
    .drain_valid            (src_drain_valid),
    .drain_data             (src_drain_data),
    .drain_id               (src_drain_id),

    // Status
    .drain_data_avail       (channel_data_avail),
    .dbg_bridge_pending     (channel_empty),
    .dbg_bridge_out_valid   (channel_full)
);
```

---

**Last Updated:** 2025-01-10

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 24 RAPIDS Core, Sink Half and Source Half

**Module:** `rapids_core.sv`, `rapids_snk.sv`, `rapids_src.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro/` **Status:** Implemented **Last Updated:** 2026-09-30

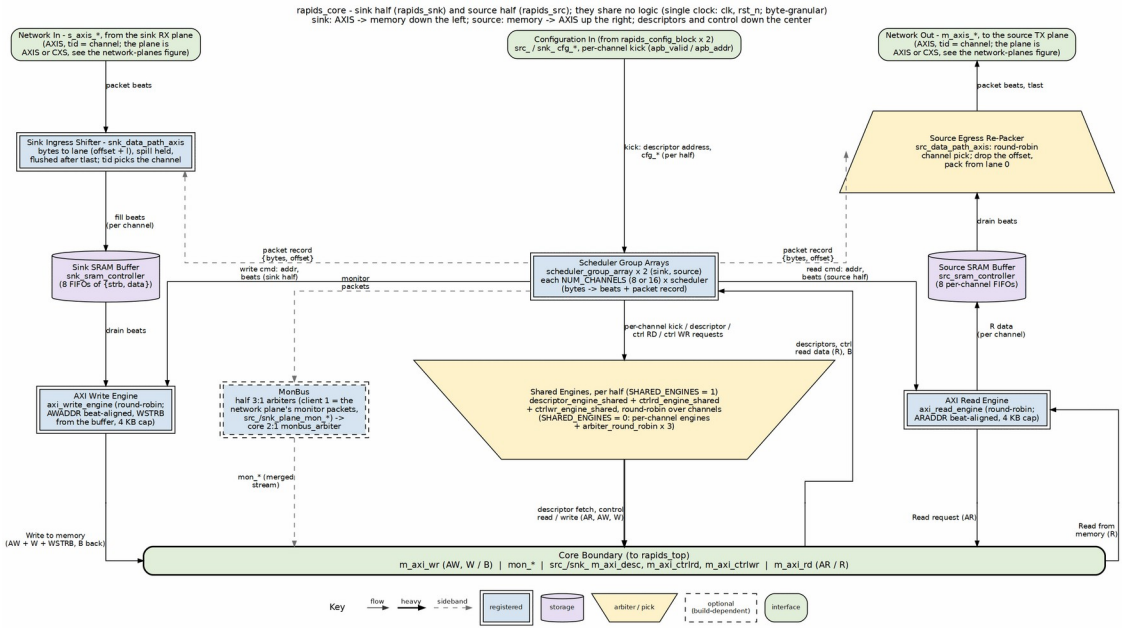
---

## 24.1 Overview

`rapids_core` is a thin structural wrapper over two independent halves. `rapids_src` moves memory to the network. `rapids_snk` moves the network to memory. The halves share no logic and no signals. The core only presents one boundary and merges the two MonBus streams.

The structure, the port naming convention and the register and monitor plumbing are those of RAPIDS Beats. This chapter records what the byte-granular RTL contains and how it differs. The full port tables are in RAPIDS Core Beats, with the port renames below applied.

### 24.1.1 Figure 3.9.1: Core Block Diagram




*Core block diagram*

**Source:** 04_rapids_core.dot

## 24.2 Byte-Granular Content

### 24.2.1 Packet Record Nets

Each half declares the packet record nets of its direction and connects them between its scheduler group array and its AXIS data path:

*Table 3.9.1: Packet Record Nets per Half*

| Half | Nets | Producer | Consumer |
|---|---|---|---|
| rapids_src | sched_rd_pkt_valid, _ready, _bytes, _offset | array read set | src_data_path_axis |
| rapids_snk | sched_wr_pkt_valid, _ready, _bytes, _offset | array write set | snk_data_path_axis |

The offset nets are $clog2(DW/8) bits wide. The unused direction of each array is tied off: valid, bytes and offset are left open and ready is held at all ones so nothing stalls. The record nets are internal, so the core port list has no packet record ports.

### 24.2.2 Port Renames

The core ports that carry the beat-count unit are renamed. Everything else keeps its Beats name.

*Table 3.9.2: Core Port Renames*

| RAPIDS Beats | RAPIDS |
|---|---|
| cfg_axi_rd_xfer | cfg_axi_rd_xfer_beats |
| cfg_axi_wr_xfer | cfg_axi_wr_xfer_beats |
| src_dbg_r_rcvd | src_dbg_r_beats_rcvd |
| src_dbg_axis_sent | src_dbg_axis_beats_sent |
| snk_dbg_axis_received | snk_dbg_axis_beats_received |

### 24.2.3 Data Ports

*Table 3.9.3: Data Ports*

| Interface | Signals | Notes |
|---|---|---|
| Source AXIS | m_axis_* | tstrb is contiguous, partial |

| Interface | Signals | Notes |
|---|---|---|
| | | on the `tlast` beat. |
| Sink AXIS | `s_axis_*` | `tid` selects the channel. Partial `tstrb` only on `tlast`. |
| Source AXI read | `m_axi_rd_*` | ARADDR is beat-aligned, bursts stop at 4 KB. |
| Sink AXI write | `m_axi_wr_*` | `wstrb` carries the stored byte enables. |

## 24.3 Parameters

*Table 3.9.4: Core Parameters*

| Parameter | Default | Description |
|---|---|---|
| NUM_CHANNELS | 8 | Channels. |
| ADDR_WIDTH | 64 | Address width. |
| DATA_WIDTH | 512 | Data width in bits. The board uses 256. |
| AXI_ID_WIDTH | 8 | AXI ID width. |
| SRAM_DEPTH | 512 | Words per channel. |
| PIPELINE | 1 | Write engine pipelining. |
| AR_MAX_OUTSTANDING, AW_MAX_OUTSTANDING | 8 | Outstanding bursts. |
| W_PHASE_FIFO_DEPTH, B_PHASE_FIFO_DEPTH | 64, 16 | Write engine queues. |
| AXIS_ID_WIDTH, AXIS_DEST_WIDTH, AXIS_USER_WIDTH | 8, 4, 1 | AXIS side widths. |
| DESC_MON_BASE_AGENT_ID | 16 | Descriptor engine agents. |
| SCHED_MON_BASE_AGENT_ID | 48 | Scheduler agents. |
| DESC_AXI_MON_AGENT_ID | 8 | Descriptor AXI monitor. |
| MON_UNIT_ID, MON_MAX_TRANSACTIONS | 1, 16 | Monitor unit id and table depth. |
| USE_AXI_MONITORS | 1 | 0 omits the monitor hardware. |
| USE_ROW_COL_MAJOR_ADDRESSING | 1 | 0 compiles out extended addressing. |
| GEN_MON | 1 | 0 omits the completion and error emitters. |
| SHARED_ENGINES | 1 | 1: one descriptor, control-read and control-write engine per half (Shared Engines). 0: one of each per channel. |

| Parameter | Default | Description |
|---|---|---|
| TILE_HDR, TILE_SRC_COORD | 0, 14'h3000 | Tile packet header on both network ports (Tile Packet Header). Needs SHARED_ENGINES = 1. |
| TILE_CREDITS | 0 | End-to-end tile credits: 64 destination counters (each node's internal and external ID) in the source half, the advertisement window and the legal-destination mask (Tile Packet Header). Needs TILE_HDR = 1. |

The network endpoint and its monitor-lite moved to the top's network planes (rapids TASK-024), so SNK_AXIS_MON_AGENT_ID, SRC_AXIS_MON_AGENT_ID and ACLK_MHZ are top parameters now. The core takes each plane's monbus packets on src_plane_mon_* / snk_plane_mon_*. See docs/rapids_planes.md.

### 24.3.1 Channel Reset Routing

Each half has its own configuration block, and that block drives the half's per-channel reset from its CHANNEL_RESET.CH_RST field ORed with GLOBAL_CTRL.GLOBAL_RST. The core exposes it as src_cfg_channel_reset and snk_cfg_channel_reset, and each half forwards it twice: to its scheduler group array (scheduler, descriptor engine, control engines) and to its byte data path (src_data_path_axis or snk_data_path_axis, through which it reaches the AXI engine and the SRAM wrapper). A reset therefore reaches every block that holds a sticky flag or an in-flight beat of the channel, and a channel reset recovers every error class without aresetn. Table 3.9.5 lists the blocks reached.

*Table 3.9.5: Channel Reset Routing*

| Half | Blocks reached | Chapter |
|---|---|---|
| rapids_snk | scheduler, axi_write_engine, snk_sram_controller, snk_data_path_axis | Sink Data Path AXIS |
| rapids_src | scheduler, axi_read_engine, src_sram_controller, src_data_path_axis | Source Data Path AXIS |

The two halves are independent: a reset of a sink channel does not touch the source channel with the same index, so software resets both directions of a channel by writing both halves' reset fields. A reset of one channel never disturbs another channel of the same half.

---

## 24.4 Differences from RAPIDS Beats

*Table 3.9.6: Core Delta*

| Item | RAPIDS Beats | RAPIDS |
|---|---|---|
| Half structure, monitor merge | | Unchanged |
| Packet record nets inside each half | absent | present |
| Data path instances | `*_axis_beats` | byte-granular `*_axis` |
| Beat-unit port names | no suffix | `_beats` suffix, see Table 3.9.2 |

The Beats chapters for the halves are folded into the core chapter there.

---

**Last Updated:** 2026-09-30

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 25 RAPIDS Registers Specification

**Module:** `rapids_regs.sv` (generated) **Source:** `projects/components/dma-ip/rapids/rtl/macro_beats/rapids_regs.rdl`, `rapids_mon_regs.rdl` **Generated:** `projects/components/dma-ip/rapids/regs/generated/rtl/rapids_regs.sv` **Status:** Implemented (PeakRDL-generated)

---

## 25.1 Overview

`rapids_regs` is the PeakRDL-generated register block for the RAPIDS Beats top-level. It exposes configuration and status through a single command/response CPU interface and drives the hardware via a `hwif_out` structure that `rapids_config_block` maps onto the core/monitor `cfg_*` signals.

A single addrmap (one APB slave) contains two regfiles:

- **Base config/status** registers at `0x100`-`0x3FF`.
- A nested **`rapids_mon_regs`** monitor regfile at base `0x1000`, decoded under `hwif_out.MON.*` (all DAXMON / RDMON / WRMON / MON_FIFO and per-monitor performance registers).

Because the monitor regfile lives at `0x1000`, the register-block address decode requires at least **13 bits** (`cpuif_addr` is a 13-bit compare in the generated RTL). The APB address bus feeding the block (`APB_ADDR_WIDTH`) must therefore be >= 13 to reach the monitor regfile.

---

## 25.2 Base Register Map (0x100-0x3FF)

*Table 3.10.1: Base Register Map*

| Offset | Name | Access | Description |
|---|---|---|---|
| 0x100 | GLOBAL_CTRL | RW | GLOBAL_EN[0], GLOBAL_RST[1] (level; write 0 to release) |
| 0x104 | GLOBAL_STATUS | RO | SYSTEM_IDLE[0] (all channels idle) |
| 0x108 | VERSION | RO | MINOR[7:0]=0x5A, MAJOR[15:8], NUM_CHANNELS[23:16]=0x08 |
| 0x120 | CHANNEL_ENABLE | RW | CH_EN[7:0] per-channel enable |
| 0x124 | CHANNEL_RESET | RW | CH_RST[7:0] per-channel reset (level; write 0 to release) |
| 0x140 | CHANNEL_IDLE | RO | CH_IDLE[7:0] – not driven at the top level, reads 0 |
| 0x144 | DESC_ENGINE_IDLE | RO | Per-channel descriptor-engine idle |
| 0x148 | SCHEDULER_IDLE | RO | Per-channel scheduler idle (excludes CH_ERROR) |
| 0x150-0x16C | CH_STATE[0..7].STATE | RO | Per-channel FSM state (stride 0x4) |
| 0x170 | SCHED_ERROR | RO | Per-channel sticky scheduler error |
| 0x174 | AXI_RD_COMPLETE | RO | Per-channel read-complete flags (SRC half only; reads 0 in SNK) |
| 0x178 | AXI_WR_COMPLETE | RO | Per-channel write-complete flags – not driven, reads 0 |
| 0x200 | SCHED_TIMEOUT_CYCLES | RW | TIMEOUT_CYCLES[31:0] write-progress window (reset 1000) |

| Offset | Name | Access | Description |
|---|---|---|---|
| 0x204 | SCHED_CONFIG | RW | SCHED_EN, TIMEOUT_EN, ERR_EN, COMPL_EN, PERF_EN |
| 0x208 | SCHED_TIMEOUT_LIMIT | RW | LIMIT[7:0] consecutive-timeout escalation limit (reset 4, 0 = never) |
| 0x220 | DESCENG_CONFIG | RW | DESCENG_EN[0], PREFETCH_EN[1], FIFO_THRESH[5:2] (reset 8) |
| 0x224 | DESCENG_ADDR0_BASE | RW | Descriptor address range 0 base [31:0] |
| 0x228 | DESCENG_ADDR0_LIMIT | RW | Descriptor address range 0 limit [31:0] |
| 0x22C | DESCENG_ADDR1_BASE | RW | Descriptor address range 1 base [31:0] |
| 0x230 | DESCENG_ADDR1_LIMIT | RW | Descriptor address range 1 limit [31:0] |
| 0x240 | CTRL_CONFIG | RW | CTRLRD_MAX_TRY[8:0] control-read poll retry budget (0-511, reset 16) |
| 0x2A0 | AXI_XFER_CONFIG | RW | AXI transfer sizing configuration |
| 0x2B0 | PERF_CONFIG | RW | Performance profiler configuration |
| 0x2C0 | OBS_CTRL | RW | Observation mux control (channel/category select) |
| 0x2C4 | OBS_FLAGS | RO | Observation flags |
| 0x2C8 | OBS_DATA0 | RO | Observation data word 0 |
| 0x2CC | OBS_DATA1 | RO | Observation data word 1 |
| 0x300-0x31C | CH_SCRATCH[n].SCRATCH | RW | Eight software bits per channel, SCRATCH[7:0] (reset 0); storage only, for flags or semaphores between the host and, in rapids-g, the half's gatling. |

| Offset | Name | Access | Description |
|---|---|---|---|
| | | | Not atomic across masters: give each bit one writer |
| 0x35C | PERF_CH_SEL | RW | Performance channel select |
| 0x378 | HIST_SEL | RW | Histogram bucket select |
| 0x37C | HIST_DATA | RO | Histogram bucket data |
| 0x380 | HIST_TOTAL | RO | Histogram total count |

### 25.2.1 Key Register Fields

**SCHED_TIMEOUT_LIMIT (0x208)** – `LIMIT[7:0]` sets the number of consecutive write-progress timeout windows a channel tolerates before the recoverable timeout escalates to a fatal, sticky `CH_ERROR`. `0` = never escalate (pure soft timeout). Total time to escalate is approximately `LIMIT x TIMEOUT_CYCLES`. Reset value is 4.

**DESCENG_CONFIG (0x220)** – `PREFETCH_EN[1]` and `FIFO_THRESH[5:2]` control the now-functional descriptor prefetch: prefetch off = on-demand chaining (~1 ahead), prefetch on = buffer up to `FIFO_THRESH` descriptors ahead.

**CTRL_CONFIG (0x240)** – `CTRLRD_MAX_TRY[8:0]` bounds the control-read poll retry budget (0-511, reset 16) fed to every channel's `ctrlrd_engine`. A `CTRL_READ` descriptor polls its gate address once per retry; if the masked value never matches within `CTRLRD_MAX_TRY` attempts the engine raises `ctrlrd_error` so a never-satisfied gate cannot hang the channel. See the Control-Read / Control-Write Engine specs (Sections 2.8 / 2.9) and the Scheduler control interface (Section 2.1).

---

## 25.3 Monitor Register Map (rapids_mon_regs @ 0x1000)

The monitor regfile is instantiated at base `0x1000` under `hwif_out.MON.*`. It holds the MonBus FIFO status, the three AXI-monitor configuration groups (DAXMON = descriptor monitor, RDMON = read monitor, WRMON = write monitor), and per-monitor performance counters.

*Table 3.10.2: Monitor Register Map (base 0x1000)*

| Offset | Name | Access | Description |
|---|---|---|---|
| 0x1000 | `MON.MON_FIFO_STATUS` | RO | MonBus capture/error FIFO status |
| 0x1004 | `MON.MON_FIFO_COUNT` | RO | MonBus FIFO occupancy |
| 0x10C0 | `MON.DAXMON_ENABLE` | RW | Descriptor-monitor enables (incl. `COMPRESS_EN`) |
| 0x10C4 | `MON.DAXMON_TIMEOUT` | RW | Descriptor-monitor timeout cycles |
| 0x10C8 | `MON.DAXMON_LATENCY_THRESH` | RW | Descriptor-monitor latency threshold |
| 0x10CC | `MON.DAXMON_PKT_MASK` | RW | Descriptor-monitor packet mask |
| 0x10D0 | `MON.DAXMON_ERR_CFG` | RW | Descriptor-monitor error select/mask |
| 0x10D4-0x10DC | `MON.DAXMON_MASK1/2/3` | RW | Descriptor-monitor category masks |
| 0x10E0-0x10FC | `MON.RDMON_*` | RW | Source-egress AXIS monitor-lite config (same layout as DAXMON; SRC instance, rapids TASK-015) |
| 0x1100-0x111C | `MON.WRMON_*` | RW | Sink-ingress AXIS monitor-lite config (same layout as DAXMON; SNK instance, rapids TASK-015) |
| 0x1150-0x1178 | `MON.DAXMON_PERF_*` | RO/RW | Descriptor-monitor performance counters |
| 0x1180-0x11A8 | `MON.RDMON_PERF_*` | RO/RW | Read-monitor performance counters |

| Offset | Name | Access | Description |
|---|---|---|---|
| 0x11B0-0x11D8 | MON.WRMON_PERF_* | RO/RW | Write-monitor performance counters |
| 0x11E0-0x11EC | MON.{RD,WR}MON_PERF_CH_* | RO | Per-channel producer/backpressure/starve/idle |
| 0x11F0 | MON.RDMON_PERF_CH_OVERFLOW | RO | Read-monitor per-channel overflow |
| 0x11F4 | MON.WRMON_PERF_CH_OVERFLOW | RO | Write-monitor per-channel overflow |

Each *_ENABLE register carries MON_EN, ERR_EN, COMPL_EN, TIMEOUT_EN (and COMPRESS_EN on the write monitor); each *_PERF_* group provides window, producer, backpressure, starve, idle, beat, byte (lo/hi), and burst counters.

---

**Last Updated:** 2026-07-02

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 26 RAPIDS Config Block Specification

**Module:** `rapids_config_block.sv` **Location:** `projects/components/dma-ip/rapids/rtl/macro_beats/` **Status:** Implemented

---

## 26.1 Overview

`rapids_config_block` is a combinational adapter that maps the register block's `hwif_out` structure onto the flat `cfg_*` signals consumed by `rapids_core_beats` and the AXI monitors. It contains no state; every output is a direct assignment (with a small number of width expansions and global-enable gates).

The monitor registers arrive under `hwif_out.MON.*`; base configuration and status arrive at the top level of `hwif_out`.

---

## 26.2 Mapping Groups

### 26.2.1 Base Configuration

Routes scheduler, channel, and descriptor-engine configuration to the core:

- `cfg_channel_enable = CHANNEL_ENABLE.CH_EN & {8{GLOBAL_CTRL.GLOBAL_EN}}` (per-channel enable gated by the global master enable).
- `cfg_sched_timeout_cycles = SCHED_TIMEOUT_CYCLES.TIMEOUT_CYCLES` (32-bit).
- `cfg_sched_timeout_limit  = SCHED_TIMEOUT_LIMIT.LIMIT` (8-bit, direct).
- `cfg_sched_{timeout,err,compl,perf}_enable = SCHED_CONFIG.{TIMEOUT,ERR,COMPL,PERF}_EN`. `COMPL_EN` is the one the scheduler group acts on for its monitor packets: 0 drops the scheduler's and descriptor engine's CORE Completion packets at the group (emitters still acknowledged); `ERR_EN` and `PERF_EN` reach the group but gate nothing today (errors always reported, perf not implemented).
- `cfg_desceng_*` from `DESCENG_CONFIG` (enable / prefetch / FIFO threshold).
- Descriptor address ranges (`cfg_desceng_addr{0,1}_{base,limit}`) are zero-extended from the 32-bit register fields to the 64-bit `ADDR_WIDTH`.

### 26.2.2 Monitor Configuration (hwif_out.MON.*)

Three parallel groups map to the descriptor-read monitor and, since rapids TASK-015, the two AXIS data-path monitor-lites (one per half):

*Table 3.11.1: Monitor Register-to-Config Mapping*

| Register group | cfg_* target |
|---|---|
| `MON.DAXMON_*` | `cfg_desc_mon_*` (descriptor monitor) |
| `MON.RDMON_*` | `cfg_rdeng_mon_*` (source-egress AXIS monitor-lite; wired in the SRC half only) |
| `MON.WRMON_*` | `cfg_wreng_mon_*` (sink-ingress AXIS monitor-lite; wired in the SNK half only) |

The `*_ENABLE` bits map onto the AXIS monitor-lite's class enables: `MON_EN` (gated by `GLOBAL_EN`), `ERR_EN` (error, channel-change and TSTRB checks), `COMPL_EN` (one Completion/STREAM_END per

packet), PERF_EN (stream start/pause/resume and Credit/BACKPRESSURE stall reports), TIMEOUT_EN. *_TIMEOUT is MICROSECONDS in its low 16 bits (0 = never), *_LATENCY_THRESH the stall CYCLES that raise a backpressure report, and *_PKT_MASK is a MASK: bit[type] = 1 masks (drops) that packet type at the monitor-lite and again at the monbus group. It resets to all-masked (0xFFFF); the host clears the bits of the classes it wants to see (rapids BUG-008 corrected the description, which used to read "1 = enable"; the polarity and the reset are as intended).

Within each group the enable/timeout/latency/mask fields map 1:1, e.g. `cfg_desc_mon_enable = MON.DAXMON_ENABLE.MON_EN & GLOBAL_CTRL.GLOBAL_EN`, `cfg_desc_mon_timeout_cycles = MON.DAXMON_TIMEOUT.TIMEOUT_CYCLES`, and the category masks (`timeout`/`compl`/`thresh`/`perf`/`addr`/`debug`) pass through directly. Key enables are gated by `GLOBAL_CTRL.GLOBAL_EN`; masks pass through ungated.

### 26.2.3 Performance and Observation

PERF_CONFIG, PERF_CH_SEL, and OBS_CTRL map to the profiler and channel-observation-mux `cfg_*` signals.

---

**Last Updated:** 2026-07-02

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 27 RAPIDS Top

**Module:** `rapids_top.sv`, `rapids_cxs_top.sv`, `rapids_8ch_top.sv`, `rapids_8ch_cxs_top.sv`, `rapids_16ch_top.sv`, `rapids_16ch_cxs_top.sv` (all generated), `rapids_top_base.sv` **Location:** `projects/components/dma-ip/rapids/rtl/top/` **Status:** Implemented **Last Updated:** 2026-10-17

---

## 27.1 Overview

`rapids_top` is the synthesizable top level. It wraps RAPIDS Core with an APB4 register interface, two config blocks, the register file, per-channel bus meters and the MonBus AXI-Lite group. It has the same hierarchy as RAPIDS Beats. Only the beat-count unit names differ.

The byte-granular behavior is entirely below the top: the packet record nets are internal to the core halves, so the top port list has no new signals. The full port list is in Top-Level Port List and the top integration description is in RAPIDS Beats Top.

Both tops are generated by `bin/gen_plane_tops.py` from `rapids_top_base`: `rapids_top` adds the AXIS network planes and `rapids_cxs_top` the CXS planes, one CXS link per direction with `CXSSRCID` carrying the channel. The CXS planes carry `tstrb` in `CXSUSER`, so the byte-granular partial last beats and unaligned starts cross the link intact; that caps `DATA_WIDTH` at 1024. The planes are shared with RAPIDS Beats: see Network Planes and the CXS Interface Specification.

### 27.1.1 Channel-Count Variants

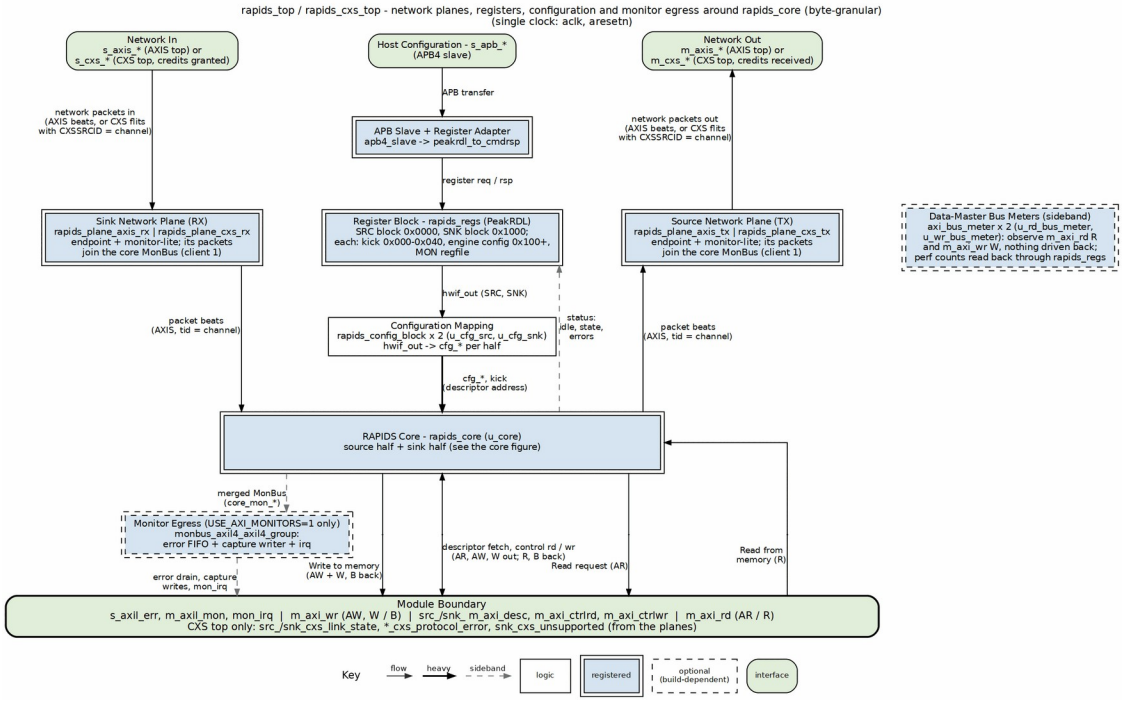
The generator also emits fixed-width variants that pair `NUM_CHANNELS` with the register map built for it:

*Table 3.12.1: Top Variants*

| Top | NUM_CHANNELS | Register map | Filelist |
|---|---|---|---|
| `rapids_top`, `rapids_cxs_top` | 8 (parameter) | 8-channel | `rtl/filelists/top/rapids_top.f`, `rapids_cxs_top.f` |
| `rapids_8ch_top`, `rapids_8ch_cxs_top` | 8 | 8-channel | `rapids_8ch_top.f`, `rapids_8ch_cxs_top.f` |
| `rapids_16ch_top`, `rapids_16ch_cxs_top` | 16 | 16-channel (`regs/generated_16ch/`) | `rapids_16ch_top.f`, `rapids_16ch_cxs_top.f` |

`rapids_top_base` stops elaboration if `NUM_CHANNELS` is larger than the register map compiled with it. The 16-channel map is described in the HAS 16-Channel Register Map.

### 27.1.2 Figure 3.12.1: Top Hierarchy




*RAPIDS top integration*

**Source:** 10_rapids_top.dot

---

## 27.2 APB Address Map

The APB address is 13 bits. Bit 12 selects the half.

*Table 3.12.2: APB Address Map*

| Range | Content |
|---|---|
| `0x0000` to `0x003F` | Source staged descriptor addresses. |
| `0x0040` | Source kick-off, one single-pulse bit per channel. |
| `0x0040` to `0x0FFF` | Source configuration registers. |
| `0x1000` to `0x103F` | Sink staged descriptor addresses. |
| `0x1040` | Sink kick-off, one single-pulse bit per channel. |
| `0x1040` to `0x1FFF` | Sink configuration registers. |

The monitor register windows sit at `+0x800` of each half's 4 KB space. With `USE_MON_REGS` clear the windows answer with an APB error, so a host can tell a build without monitors from a zero setting.

---

## 27.3 Parameters

*Table 3.12.3: Top Parameters*

| Parameter | Default | Description |
|---|---|---|
| NUM_CHANNELS | 8 | Channels. |
| DATA_WIDTH | 512 | Data width in bits. The board uses 256. |
| ADDR_WIDTH | 64 | Address width. |
| AXI_ID_WIDTH | 8 | AXI ID width. |
| SRAM_DEPTH | 4096 | Words per channel at the top. |
| APB_ADDR_WIDTH, APB_DATA_WIDTH | 13, 32 | APB widths. |
| MON_MAX_TRANSACTIONS | 16 | Monitor table depth. |
| SNK_AXIS_MON_AGENT_ID, SRC_AXIS_MON_AGENT_ID | 9, 10 | Network-plane monitor agent ids (AXIS or CXS plane). |
| ACLK_MHZ | 100 | Clock in MHz for the network-plane monitor tick. |
| USE_AXI_MONITORS | 1 | 0 omits the monitors and the MonBus AXI-Lite group. |
| USE_ROW_COL_MAJOR_ADDRESSING | 1 | 0 compiles out extended addressing. |
| GEN_MON | 1 | 0 omits the completion and error emitters. |
| SHARED_ENGINES | 1 | 1: one descriptor, control-read and control-write engine per half. 0: one of each per channel. See Shared Engines. |
| TILE_HDR, TILE_SRC_COORD | 0, 14'h3000 | Tile packet header; needs SHARED_ENGINES = 1 and DATA_WIDTH of at least 256. See Tile Packet Header. |
| TILE_CREDITS | 0 | End-to-end tile credits and the legal-destination mask (SRC.TILE_ADV_WINDOW, SRC.TILE_LEGAL_DEST_LO / |

| Parameter | Default | Description |
|---|---|---|
| | | _HI, SRC.TILE_CREDIT_STATUS); needs TILE_HDR = 1. See Tile Packet Header. |
| USE_MON_REGS | USE_AXI_MONITORS != 0 | Wire the monitor register windows. |
| AR_MAX_OUTSTANDING, AW_MAX_OUTSTANDING | 8 | Outstanding bursts. |
| PIPELINE | 1 | Write engine pipelining. |
| AXIS_ID_WIDTH, AXIS_DEST_WIDTH, AXIS_USER_WIDTH | 8, 4, 1 | AXIS side widths. |

Clock and reset are aclk and aresetn. cam_clear synchronously clears the MonBus group transaction tables.

Each half's configuration block drives its per-channel reset vector (src_cfg_channel_reset, snk_cfg_channel_reset) from SRC/SNK.CHANNEL_RESET.CH_RST and GLOBAL_CTRL.GLOBAL_RST. The top wires each vector to its half of the core (see RAPIDS Core), and it reaches every block that can hold a channel error: a channel reset recovers every error class, including the AXI response and sink packet length errors, and aresetn is needed only at power-up. The top has no port change.

---

## 27.4 Differences from RAPIDS Beats

*Table 3.12.4: Top Delta*

| Item | RAPIDS Beats | RAPIDS |
|---|---|---|
| Module | `rapids_beats_top` | `rapids_top` |
| Core instance | `rapids_core_beats` | `rapids_core` |
| Register file, APB map, config blocks | | Unchanged |
| Bus meter beat counters | `r_rd`, `r_wr` | `r_rd_beats`, `r_wr_beats` |
| Config nets | `*_xfer` | `*_xfer_beats` |
| Port list | | Unchanged |

The register-file field names `RD_XFER_BEATS` and `WR_XFER_BEATS` are the same in both designs. Their meaning is beats in both, since bursts are beat-based.

---

**Last Updated:** 2026-09-30

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 28 Network Planes Specification

**Modules:** `rapids_plane_axis_rx.sv`, `rapids_plane_axis_tx.sv`, `rapids_plane_cxs_rx.sv`, `rapids_plane_cxs_tx.sv` **Location:** `projects/components/dma-ip/rapids/rtl/plane/` **Generator:** `projects/components/dma-ip/rapids/bin/gen_plane_tops.py` **Status:** Implemented (rapids TASK-024, TASK-025)

---

## 28.1 Overview

A network plane is the endpoint between the core's network-side AXIS stream and the pins: the RX plane feeds the sink half's ingress, the TX plane takes the source half's egress. Each plane carries the monitor-lite for its port. Both RAPIDS trees (byte-granular `rtl/top`, beat-granular `rtl/top_beats`) share the same four planes.

Every plane has the same parameters and the same inner interface, so a top selects AXIS or CXS by module name alone:

- `fub_axis_*`: the core-facing AXIS stream (`tdata`, `tstrb`, `tlast`, `tid` = channel, `tdest`, `tuser`)
- `cfg_mon_*` in, `sts_mon_*` out: the `SNK.WRMON_*` / `SRC.RDMON_*` monitor CSRs, from `rapids_config_block`
- `monbus_*` out: client 1 of the half's 3:1 `monbus_arbiter`, entering the core on `snk_plane_mon_*` / `src_plane_mon_*`

*Table 3.13.1: RAPIDS Network Planes*

| Plane | Contents | Outer pins |
|---|---|---|
| `rapids_plane_axis_rx` | `axis4_slave_monlite` (skid stage, `SKID_DEPTH` = 4, + `axis_monitor_lite` tap) | `s_axis_*` |
| `rapids_plane_axis_tx` | `axis4_master_monlite` (skid stage + tap) | `m_axis_*` |
| `rapids_plane_cxs_rx` | `cxs_slave_monlite` (`u_endpoint`) + `cxs_to_axis` (`u_cxs_to_axis`) | `s_cxs_*`, `snk_cxs_*` status |
| `rapids_plane_cxs_tx` | `axis_to_cxs` (`u_axis_to_cxs`) + `cxs_master_monlite` (`u_endpoint`) | `m_cxs_*`, `src_cxs_*` status |

The monitor counters of every plane are at `u_snk_plane.u_endpoint` and `u_src_plane.u_endpoint` in the top.

## 28.2 Generated Tops

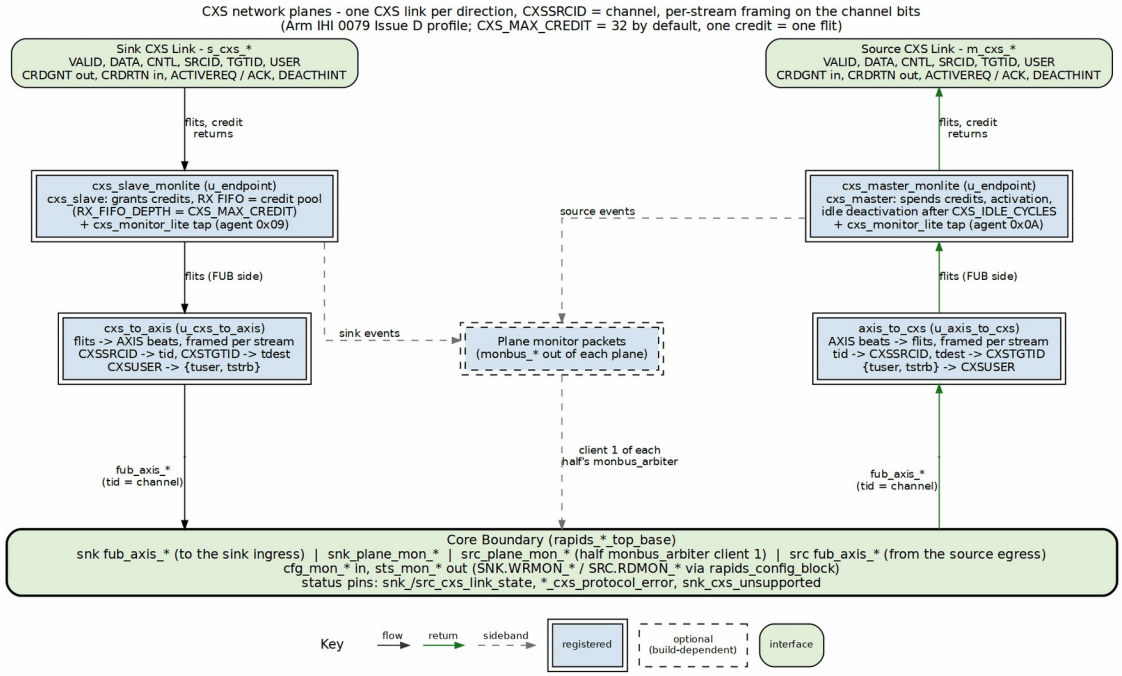
*Table 3.13.2: Generated Tops*

| Top | Tree | Base | Planes |
|---|---|---|---|
| rapids_top | byte | rapids_top_base | AXIS |
| rapids_cxs_top | byte | rapids_top_base | CXS |
| rapids_beats_top | beats | rapids_beats_top_base | AXIS |
| rapids_beats_cxs_top | beats | rapids_beats_top_base | CXS |

gen_plane_tops.py copies the base's parameter and port text verbatim, then instantiates the base (u_base) and the two planes (u_snk_plane, u_src_plane). The AXIS tops therefore keep exactly the interface they had before the split, and existing instantiations are unchanged. Run the generator with --check to fail on a stale top.

## 28.3 CXS Planes

### 28.3.1 Figure 3.13.1: CXS Network Planes




*CXS network planes*

**Source:** 23_cxs_planes.dot

### 28.3.2 Receive (sink) plane

`cxs_slave_monlite` is `cxs_slave` with a `cxs_monitor_lite` tap. `cxs_slave` grants credits up to `CXS_MAX_CREDIT`, runs the activation handshake, and holds received flits in a receive FIFO of `RX_FIFO_DEPTH` (default `CXS_MAX_CREDIT`) entries. The FIFO is the credit pool: a credit is only granted for a free entry, so a flit always has space. `cxs_to_axis` turns flits back into AXIS beats: `CXSSRCID` becomes `tid`, `CXSTGTID` becomes `tdest`, and `CXSUSER` is split into `{tuser, tstrb}`. When the core stops taking beats the FIFO fills, and the slave stops granting credits; that is the CXS form of the sink's `tready` backpressure.

`cxs_to_axis` is combinational and stateless, so it can only represent the flits `axis_to_cxs` makes: at most one START and one END per flit, with a packet starting at byte 0. Any other flit (a transmitter that packs two packets into one flit) is still passed on, with `tlast` from END[0], and sets the sticky `snk_cxs_unsupported`.

### 28.3.3 Transmit (source) plane

axis_to_cxs packs AXIS beats into flits (tid -> CXSSRCID, tdest -> CXSTGTID, {tuser, tstrb} -> CXSUSER) and cxs_master_monlite sends them, one credit per flit. The master requests activation when it has a flit to send and deactivates the link after CXS_IDLE_CYCLES idle cycles.

### 28.3.4 Per-stream framing

axis_to_cxs and both endpoints' control checkers track an open packet per stream, keyed on the low CXS_STREAM_SRCID_BITS = CIW bits of CXSSRCID (the channel); cxs_to_axis needs no state because each beat carries its own tid. The byte-granular source arbitrates per drain block while a packet spans the whole descriptor, so with two or more channels active the egress switches channel in the middle of a packet. Per-link framing (one open packet per link) rejects that traffic; per-stream framing accepts it, as IHI 0079D 2.5.2 allows.

### 28.3.5 Parameters

*Table 3.13.3: CXS Plane Parameters*

| Parameter | Default | Description |
|---|---|---|
| CXS_MAX_CREDIT | 32 | Credits granted by the receiver. One credit is one flit; the grant -> flit -> free round trip is about 3 cycles |
| CXS_RX_FIFO_DEPTH | CXS_MAX_CREDIT | Receive FIFO depth (the credit pool); need not be a power of two |
| CXS_CREDIT_BEYOND_SPEC | CXS_MAX_CREDIT > 63 | Permits more credits than IHI 0079D allows; the far end must match |
| CXS_IDLE_CYCLES | 32 | Idle cycles before the transmitter deactivates the link |
| CXS_IW, CXS_DESTW, CXS_UW, CXS_CNTL_PW | derived | Port widths; do not override |

DATA_WIDTH must be at most 1024: CXSUSER is at most 128 bits and carries tstrb (DATA_WIDTH/8 bits). tuser is dropped from CXSUSER when DATA_WIDTH/8 + AXIS_USER_WIDTH > 128.

### 28.3.6 Monitoring

The CXS planes drive cxs_monitor_lite from the same cfg_mon_* CSRs as the AXIS planes and use the same agent IDs (sink 0x09, source 0x0A). Packets are tagged PROTOCOL_CXS; the top's

monitor group filters them through its AXIS mask slot. A PACKET_END completion carries the channel in `event_data[55:48]`; `channel_id` holds `{PRCLTYPE[2:0], TGTID[5:0]}`. The lite's output queue is 4 entries deep; when events outrun it the lite drops and counts them (an Error / EVENT_DROPPED packet), so a check on completions counts delivered plus dropped.

## 28.4 Proposed Changes

A proposal (2026-10-16, not implemented) puts a tile packet header at the front of every packet: destination and source coordinates, payload type (`cfg` / `prog` / `mem`), length and target address. It is described once in the compute-engine tile's `TILE_SPEC.md` Section 11; CXS carries it as the first flit. The link signals, credits and per-stream framing on this page do not change, because `CXSSRCID` keeps the channel.

## 28.5 Verification

*Table 3.13.4: CXS Top Test Cells*

| Suite | Cells |
|---|---|
| `dv/tests/top/test_rapids_cxs_top.py` | source, sink, source_bytes, sink_bytes, source_concurrent, sink_concurrent, cxs_mon, ext_addressing |
| `dv/tests/top_beats/test_rapids_beats_cxs_top.py` | source, sink, source_concurrent, sink_concurrent, cxs_mon, ext_addressing |

The concurrent cells start four channels with one `KICK_ENABLE` write, so channels interleave on the link. Every cell also requires a clean link: no BFM or wire-monitor violation and the DUT's protocol-error flags low. The core testbenches drive packets on `plane_mon_*` and require them on the MonBus output.

---

**Last Updated:** 2026-10-16

# 29 Tile Packet Header

**Modules:** `rapids_tile_hdr_rx.sv` and `rapids_tile_credits.sv` (new), with header and credit logic in `scheduler.sv`, `src_data_path_axis.sv` and `descriptor_engine_shared.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub/`, `rtl/macro/` **Status:** Header behind `TILE_HDR` = 1, end-to-end credits behind `TILE_CREDITS` = 1 (both default 0) **Last Updated:** 2026-10-18

---

## 29.1 Overview

With TILE_HDR = 1, RAPIDS is the I/O of a compute-engine tile. Every packet on the source port and the sink port starts with one header beat. The header and its packing are defined in the tile's TILE_SPEC Section 11.5; this section describes the RAPIDS hardware that builds and reads it. The network planes do not change: on a CXS link the header is the data of the first flit of the packet, which is what TILE_SPEC 11.6 asks of CXS, and an AXIS plane carries it as the first beat in the same way.

TILE_HDR needs SHARED_ENGINES = 1, because cell-issued descriptors are written into the shared descriptor engine's queues, and DATA_WIDTH >= 256, so that a descriptor arrives in one beat. Both are elaboration-time checks.

## 29.2 Header Beat

The header occupies bits [127:0] of its beat (`rapids_pkg::tile_hdr_pack`) and its strobe is the low 16 bytes. Bits [127:119] are zero. A coordinate is 15 bits, `{ext, level, X, Y, Z}`: `ext` picks the node's internal ID (0, the node itself) or its external ID (1, what is attached to it, such as a cell).

*Table 3.14.1: Header Fields and Their Sources*

| Bits | Field | Source half fills it from |
|---|---|---|
| [14:0] | Destination coordinate `{ext, level, X, Y, Z}` | descriptor `dst_addr[63:49]` |
| [29:15] | Source coordinate | `TILE_SRC_COORD` (level 3, the tile I/O) |
| [32:30] | `ptype` | descriptor [215:213] |
| [33] | `op` (0 write, 1 read) | descriptor [216] |
| [65:34] | Length in bytes | descriptor `length` |
| [114:66] | Target address | descriptor `dst_addr[48:0]` |
| [118:115] | Tag | the issuing channel |

### 29.2.1 Header Beat Chaining on Narrow Fabric Links (2026-10-19)

RAPIDS ports are 256 bits or wider, so the 128-bit tile header is one payload beat and the existing `DATA_WIDTH` >= 256 elaboration check stays in force. On narrower fabric links the header occupies the first `ceil(128 / DATA_WIDTH)` payload beats, LSB-first, per TILE_SPEC §11.6. That chaining rule is the contract toward narrow delta links (delta `DATA_WIDTH` = 64 today → 2 beats); consumers on those links parse the header per TILE_SPEC §11.6, and RAPIDS does not insert a width converter for the header.

## 29.3 Source Half

**Scheduler.** It latches `ptype` and `op` with the descriptor and passes `{op, ptype, dst_addr}` (`sched_rd_pkt_tile`) with the packet record it already sends the source data path. A read request (`op` = 1) loads zero read beats, so no memory is read, and completes like any data descriptor once its record is queued. A source descriptor whose `ptype` is not `cfg`, `prog` or `mem` is refused as an invalid descriptor (CH_ERROR): `credit` and `desc` packets are built by the tile, never by a descriptor.

**Source data path.** The packet-record queue stores the tile fields beside the byte length and offset. Before a packet's first data beat, the egress emits its header from the head record:

- The output register takes, in priority order, a flush beat, a header beat, then a data pop.
- A data packet's header goes only when its first memory beat is at the drain stage, so header and payload leave back to back. Only the channel the drain stage is parked on can need a header, so it is served first.
- A header-only packet (read request) can go at any time. When it competes with a data pop, the two alternate, and header-only channels are scanned round-robin, so neither a run of read requests nor a long data packet starves the other.
- The header-only record is popped when its header leaves.

## 29.4 Sink Half: rapids_tile_hdr_rx

The module sits between the sink plane and snk_data_path_axis. Packets are framed per stream, and the stream is the TID (CXSSRCID, the channel), so the “next beat is a header” state is kept per stream.

*Table 3.14.2: Sink-Port Packet Handling*

| Header | Action |
|---|---|
| cfg / prog / mem, op = 0 | Header dropped; payload beats to the sink data path unchanged, delivered by stream ID |
| desc, op = 0 | First payload beat [255:0] goes out on desc_inj_* with the tag as the source channel; never stored in the sink |
| credit | Consumed and counted (TILE_COUNTS.CREDIT_COUNT); {source coordinate, op, count, class} goes to the source half’s counters on credit_* (used when TILE_CREDITS = 1) |
| Anything else | Consumed; sets UNSUPPORTED: a read request to RAPIDS, a reserved ptype, a desc with no payload or a tag past NUM_CHANNELS, extra desc payload, or DATA_WIDTH < 256 |

Only data payload waits on the sink data path’s tready. Header, desc, credit and dropped beats are taken at once, so a full sink channel never holds a descriptor or a credit behind it. A data packet whose tag differs from its stream ID is still delivered by stream ID and sets TAG_MISMATCH.

## 29.5 Cell-Issued Descriptors: `descriptor_engine_shared` (`INJ_EN`)

`desc_inj_*` is a single-cycle pulse with no backpressure. Each channel has a one-descriptor slot:

- **Capture.** A descriptor for a channel whose slot is empty is captured. The chain decision (non-zero next pointer, not last, valid, in range) is taken once at capture, so one set of comparators serves every channel.
- **Write.** The slot is written into the channel's descriptor FIFO when the channel's fetch side is idle (`S_IDLE`), no R beat is owed from a fetch a channel reset aborted, and the FIFO has room. Idle means no fetch is in flight, so the R path never writes that FIFO in the same cycle and the injection never takes the entry an outstanding fetch needs.
- **Start and chain.** An idle channel starts on the descriptor as if kicked; a busy one runs it after what is already queued. A chaining descriptor moves the channel to `S_PEND` with its next pointer, and the chain continues from memory.
- **Kicks** wait while the slot is full.
- **Drops.** A descriptor that arrives while the slot is full, while the channel is held in reset, or of type EXT (its second chunk has no way in) is dropped and sets the channel's sticky drop bit. A drop is not a descriptor error: the channel and the work it holds carry on.
- **Hold (`INJ_HOLD`, `TILE_CREDITS` = 1).** `inj_*` becomes a valid/ready handshake from the descriptor intake queue (below). A descriptor for a channel whose slot is full waits (`inj_ready` low) instead of being dropped; one for a channel in reset, or an EXT one, is still taken and dropped.

## 29.6 End-to-End Credits: rapids_tile_credits (TILE_CREDITS)

With TILE_CREDITS = 1 (it needs TILE_HDR = 1), a packet leaves the source port only when its destination has room for it (TILE_SPEC 11.9, 11.10). The credits are end to end, between RAPIDS and the cells and control cores. A delta node only transports: it never originates a packet, so it never advertises or returns credits, and it consumes what is addressed to it on arrival.

**Counters.** The source half keeps 64 counters in flits, one per destination index {ext, n} (rapids_pkg::tile_dest_index): the internal and the external ID of every node. Only the external IDs carry credits: a node's internal ID is legal by topology, and a packet to it is never held for credits or charged. An external ID with nothing behind it never advertises and so is never legal.

*Table 3.14.3: Credit Counter Index*

| Index | Destination |
|---|---|
| 0-15 | Cell-level node (internal), {Y[1:0], X[1:0]} |
| 16-19 | Tetrad node (internal), {Y[0], X[0]} of its group |
| 20-21 | Root node 0, 1 (internal) |
| 32-47 | Cell: the cell-level node's external ID |
| 48-51 | Tetrad control core: the tetrad node's external ID |
| others | Spare: never advertised, always illegal |

A coordinate outside this map (the tile I/O itself, a cell past 4x4, a tetrad past 2x2, a root past 1) has no index. Every external counter starts at 0. A class-0 credit packet from an external ID fills it: op = 1 (set) replaces the count and op = 0 (add) adds to it, saturating. Class-1 credits (descriptor entries), credits carrying a node's internal ID and credits from a coordinate with no index are ignored.

**Advertisement window.** rapids_tile_credits counts cycles from reset up to SRC.TILE_ADV_WINDOW (default 4096). When the window closes, every node index is legal and every external counter still at 0 marks an illegal destination, and the 64-bit result is saved in SRC.TILE_LEGAL_DEST_LO (internal IDs) and _HI (external IDs) until the next reset. SRC.TILE_CREDIT_STATUS.ADV_DONE reports the close. A later advertisement updates the count but never makes a destination legal.

**Scheduler.** A tile data descriptor with a non-zero length waits in CH_FETCH_DESC until the window closes. Its destination must then be legal (an index whose bit is set in the mask), or the

descriptor is refused as invalid (CH_ERROR) and nothing is sent. A read request to a node's internal ID is refused the same way, with or without `TILE_CREDITS`: a node never replies.

**Source data path.** Each channel asks for its head packet's flits, the header and `ceil(bytes / beat)` data beats or the header alone for a read request, against its destination (`tile_acq_*`). `rapids_tile_credits` grants one request per cycle, round-robin, and only when the counter covers the whole packet. It charges the counter in the same cycle and pulses `tile_acq_gnt` the next. A request to a node is granted without a count check and charges nothing. Until the grant, neither the packet's header nor any further SRAM reservation of the channel goes ahead, so a channel waiting for credits does not hold the shared drain stage. A reservation made under the previous packet can still run into this one's first beat, and the drain stage then waits there for the grant. Counters are per destination, so concurrent channels to one cell share its buffer.

**Limits.** A channel reset that discards a packet whose credits are already charged does not refund them. With `TILE_CREDITS` a channel reset acts only while the tile is quiescent (below), when no packet is in flight against a charge, so the case does not arise under the quiescent rule.

## 29.7 Straps Toward RAPIDS (TILE_CREDITS)

What a cell or control core sends into the sink port is charged against a strap RAPIDS reserves, and RAPIDS returns the credit as it consumes (TILE_SPEC 11.9, 11.11). The straps are fixed by construction, not granted: a sender starts from its strap after reset. They are Hive-level parameters that the cells and cores must be built with too.

*Table 3.14.4: Straps toward RAPIDS*

| Sender (external slot = index - 32) | Strap | Reserved in RAPIDS | Returned when |
|---|---|---|---|
| Cell, slots 0-15 | TILE_LAND_FLITS flits (default 16), header included | A landing space of TILE_LAND_FLITS beats | Each beat leaves the landing space; the header when it is decoded |
| Control core, slots 16-19 | TILE_DESC_PER_CORE entries (default 2), credit class 1 | A share of the intake queue (4 x TILE_DESC_PER_CORE deep) | The descriptor moves into the descriptor queue |

**Header decode (`rapids_tile_hdr_rx`, `STRAPS`).** The sender is the header's source coordinate as an external slot (`rapids_pkg::tile_ext_slot`). Data payload goes on with the sender's cell slot to the landing space; the header flit returns at once. A packet the decode drops (a read request into RAPIDS, a reserved `ptype`) returns every beat. A `desc` packet refused at its header returns its entry. A data packet from a sender with no landing space (not a cell) is dropped and flagged `UNSUPPORTED`, and returns nothing: it was charged to no strap. Credit packets are never charged.

**Landing space (`rapids_tile_landing`, in `rapids_snk`).** One FIFO of `TILE_LAND_FLITS` beats per cell, holding payload by sender. A cell under its strap can never fill its FIFO, so the sink port never waits on a landing space. The drain moves one whole packet at a time into `snk_data_path_axis`, round-robin over the cells. A packet is a candidate once its `tlast` beat has landed, so the drain never waits on the network, and only when its channel can take all of it now: `snk_data_path_axis` `chan_can_take` says the channel is out of reset, has the packet's record (its descriptor is running), no spill is waiting, and `TILE_LAND_FLITS + 4` beats are free (the packet, a spill and what the previous packet may still hold in the shifter). A packet for a disabled channel is drained and thrown away. A cell whose channel has no descriptor yet holds only its own landing space; every other cell's data keeps flowing. Each beat that leaves, delivered or thrown away, returns one flit. A beat arriving for a full landing space (a cell past its strap) waits and sets `TILE_CREDIT_STATUS.LAND_OVERRUN`. The per-channel sink SRAM must be at least `TILE_LAND_FLITS + 4` beats (an elaboration check).

**Descriptor intake (`rapids_tile_desc_intake`, in `rapids_core`).** `desc` packets go into a FIFO `4 x TILE_DESC_PER_CORE` deep with their channel and sender, and leave in order through the `INJ_HOLD` handshake into `descriptor_engine_shared`. A descriptor for a busy channel at the head waits there; the schedulers' two-deep descriptor skid keeps that short. When an entry leaves,

its class-1 credit returns. A `desc` arriving at a full queue (a core past its strap) is dropped, sets `TILE_CREDIT_STATUS.INTAKE_OVERRUN`, and still returns its entry: no path consumes a descriptor silently.

**Credit returns (`rapids_tile_credit_tx`, in `rapids_core`).** Pending returns are kept per external slot and class and taken round-robin into one output register as soon as it is free; whatever arrives while a return waits is coalesced into that slot's next one, and nothing is held back to build a batch. A return is a header-only credit packet (`ptype` = `credit`, `op` = `add`, count in the length, class in target bits [1:0]) addressed to the sender's source coordinate as last seen in a header, on its own interface, `m_tile_credit_*` (valid/ready, the 128-bit header): a return never waits behind data. `TILE_CREDIT_STATUS.RET_COUNT` counts them.

**Resync (TILE_SPEC 11.10).** A write of `SRC.TILE_RESYNC.GO` requests one; it is taken only while the top-level `tile_quiescent` input is high and waits for it otherwise (`TILE_CREDIT_STATUS.RESYNC_BUSY`). It then sends one `set` per legal sender in the saved mask: `TILE_LAND_FLITS` (class 0) to each cell and `TILE_DESC_PER_CORE` (class 1) to each control core, replacing that slot's pending `add`. Sets go before adds. RAPIDS sends one `set` per sender rather than the multicast TILE_SPEC 11.10 describes: it has no multicast header to build. The answering advertisements refill RAPIDS's own counters as at boot.

**Quiescent gating.** With `TILE_CREDITS`, a channel reset of either half (`CHANNEL_RESET`) acts only while `tile_quiescent` is high; a reset bit written while it is low has no effect until it rises.

**Stalled-data checks (`rapids_tile_stall_mon`).** Data that stops moving is a monitor error (TILE_SPEC 11.9, hazard rules). Each half carries one check, on the spare client 2 of its MonBus arbiter:

*Table 3.14.5: Stalled-data checks*

| Half | Pending | Moves on | Agent |
|---|---|---|---|
| Sink | The channel's sink SRAM holds data not yet written out (an allocated, unfilled segment does not count) | A W beat of the channel | `0x0B` |
| Source | The channel has read bursts outstanding (AR accepted, RLAST not seen; ARID low bits = channel) | An R beat for the channel | `0x0C` |

A channel pending for SRC.TILE_STALL_CYCLES cycles (default 1000, 10 us at 100 MHz; 0 turns both off) with no movement raises one error packet (PktTypeError, PROTOCOL_CORE, RAPIDS_EVENT_STALL; event data bit 32 = 0 sink / 1 source, bits [31:0] the threshold) and sets its bit in that half's TILE_STALLED, and raises nothing more until the data moves or drains.

## 29.8 Ordering Across the Fabric (open: delta DELTA-013)

Delta is moving to multi-path route tables, with failover as a table write (delta HAS ch07, Resilient Routing: Open Hazards). A request and its reply can then take different paths with different latencies, so delta guarantees the order of flits within one packet and nothing between packets. RAPIDS must not rely on fabric order. Under correct configuration, four places would; O4 has an interim rule (2026-10-19), the rest are open:

*Table 3.14.6: RAPIDS ordering hazards under multi-path routing*

| # | Where | Mechanism | Candidate direction |
|---|---|---|---|
| O1 | Read replies | The tag is the sink channel only. Two reads outstanding on one sink channel, to different cells or to one cell over two paths, can reply in either order, and the sink lands the bytes in arrival order | A transaction ID per read, echoed in the reply, and the sink places by ID (AXI-ID style), or one read outstanding per sink channel |
| O2 | Credit `set` vs `add` | A resync `set` (TILE_SPEC 11.10) can overtake, or be overtaken by, an `add` sent before it, and the counter ends wrong | A sequence or epoch on credit packets; an `add` older than the last `set` is discarded |
| O3 | Write, then a dependent packet | A cell's data write and a later packet that depends on it (a `desc` whose source is that data, a flag) can arrive in either order | Completion is signalled (the credit return says the data was consumed), never implied by send order |

| # | Where | Mechanism | Candidate direction |
|---|---|---|---|
| O4 | Writes from one cell to one sink channel | The sink lands a channel's data in arrival order, so a cell's later write that overtakes an earlier one lands in the wrong place | Interim rule: delta may interleave and reorder packets (delta HAS, Interface Assumptions), so the cell sends its next packet only once RAPIDS has returned every flit it has sent (its strap is whole). RAPIDS never returns a flit it has not received, so a whole strap proves every earlier packet landed and drained. No RAPIDS change, at the cost of one round trip per packet. Pencilled in (rapids TASK-033): tagged, indexed intake, below |

The flits of one packet stay in order. The header-then-payload framing of the source and sink halves also needs each CXS plane port to deliver a packet's flits together; whether delta keeps that at its plane ports once it interleaves is open.

## 29.9 Interleaved and Out-of-Order Intake (pencilled in, rapids TASK-033)

Not built. Validated in a cycle model only (`dv/models/tile_intake_model.py`, `dv/tests/models/test_tile_intake_model.py`). It lets delta interleave and reorder packets into the sink port with no send rule beyond the straps.

**The tag.** Every flit, data flits included, carries a sideband tag beside its data: `{sender, pos, head}`. `sender` is the source's 5-bit external slot (`rapids_pkg::tile_ext_slot`: cells 0-15, control cores 16-19). `head` marks a header flit. `pos` is the flit's place in the space RAPIDS holds for that sender and class. About 10 bits; delta carries it on AXIS `tuser` and CXS `user` (delta HAS, Interface Assumptions).

*Table 3.14.7: Tagged intake by class (pencilled in)*

| Class | Sender | Packet | pos | RAPIDS storage | Leaves in |
|---|---|---|---|---|---|
| 0, data | Cells | Header + up to `TILE_LAND_FLITS` - 1 beats | Flits sent, header included, mod `TILE_LAND_FLITS` | The landing space as a ring, written at `pos`, a valid bit per slot | Ring order: the packet at the read pointer, once its header and every flit to its last are present |
| 1, desc | Control cores | Header + one descriptor beat | Packets sent mod `TILE_DESC_PER_CORE` | One intake entry per `pos` | Entry order |

Debug never rides the fabric. Each cell emits one MonBus on AXI4-Lite, each tetrad merges its cells and its node into one AXI4-Lite MonBus, and the tile merges the tetrads and the root nodes into one MonBus (owner 2026-10-19, TILE_SPEC 5). So nodes never send into RAPIDS, and a MonBus class on the fabric (pencilled in earlier the same day) is retired. A header's class-0 flit returns when its packet drains, not on arrival, because its slot holds the packet's channel and length. The strap keeps a sender's flits in flight below the space, so a `pos` is never reused while its slot is full. Resync clears the rings and entries and both sides' `pos`.

Model results (one flit per cycle into RAPIDS, random packet sizes, fabric latency 2-120 cycles): both classes delivered exactly once and in send order at `TILE_LAND_FLITS` 16 and 8, every strap whole at the end. Controls: today's arrival-order landing corrupts as soon as a cell has two packets in flight (strap-only and the retired one-flit rule both fail); the wait-for- every-credit rule is correct

on it. Mutations caught: draining any complete packet instead of the one at the read pointer; `pos` counting payload only. Payload throughput, tagged intake vs wait-for-every-credit:

*Table 3.14.8: Model payload flits per cycle (link peak 1; headers take the rest)*

| Cells | Fabric latency 2-10 | Fabric latency 8-40 |
|---|---|---|
| 1 | 0.36 vs 0.28 | 0.21 vs 0.18 |
| 2 | 0.65 vs 0.48 | 0.42 vs 0.32 |
| 4 | 0.85 vs 0.77 | 0.70 vs 0.55 |
| 16 | 0.89 vs 0.89 | 0.88 vs 0.88 |

A cell’s own ceiling is its strap over the round trip, so the gain is largest with few active cells; with many, the link saturates either way.

## 29.10 Status Registers

SNK.TILE_STATUS (UNSUPPORTED, TAG_MISMATCH), SRC.TILE_DESC_DROPPED (DROPn) and SNK.TILE_COUNTS, plus, for TILE_CREDITS, SRC.TILE_ADV_WINDOW (RW), SRC.TILE_LEGAL_DEST_LO / _HI, SRC.TILE_CREDIT_STATUS (ADV_DONE, RESYNC_BUSY, LAND_OVERRUN, INTAKE_OVERRUN, RET_COUNT), SRC.TILE_RESYNC (GO, singlepulse), SRC.TILE_STALL_CYCLES (RW), SRC / SNK.TILE_STALLED and TILE_STRAPS (see the Register Map). TILE_STRAPS is the last config word of each half, 0x7FC below the monitor block (SRC 0x07FC, SNK 0x17FC, on the 8- and 16-channel maps alike), so the Hive finds it without the rest of the map: LAND_FLITS [15:0] = TILE_LAND_FLITS and DESC_PER_CORE [31:16] = TILE_DESC_PER_CORE, both 0 when TILE_CREDITS = 0. The W1C fields are live mirrors of stickies owned by the RTL. rapids_top_base decodes a write to either register from the register block's own request and sends a one-cycle clear (wr_data & wr_biten) to the owner, where a same-cycle event wins. The same values are on the top-level sts_tile_* ports.

## 29.11 Verification

`test_rapids_cxs_top.py` builds `rapids_cxs_top` (and `rapids_16ch_cxs_top`) with `TILE_HDR` = 1:

*Table 3.14.7: Tile Header Test Cells*

| Cell | Proves |
|---|---|
| `tile_source_write` | Header fields from the descriptor, ahead of the payload |
| `tile_source_read` | `op` = 1 is a header-only packet with no memory read |
| `tile_sink_write` | Header stripped, payload to memory |
| `tile_cell_descriptors` | Idle, busy and chaining injections on four channels (and channels 12-15 at 16) |
| `tile_desc_overflow` | A slot overflow drops, flags `DROPn` (and clears it by W1C); the running transfer and queued work finish |
| `tile_source_bad_ptype` | A source descriptor with `ptype` `credit` is refused |
| `tile_unsupported` | `credit` counted, a read request to RAPIDS flagged and cleared by W1C, the stream still carries data |
| `tile_credits[window]` | `TILE_CREDITS`: every node legal unadvertised, node credits ignored; the cells and core that advertise are legal; a zero and a late advertisement stay illegal; a descriptor kicked during the window waits for it |
| `tile_credits[no_oversend]` | A destination takes only what it advertised and returned; one flit short holds the packet; class-1 and unindexed credits change nothing |
| `tile_credits[shared_counter]` | Two channels to one cell share its counter (also on `rapids_16ch_cxs_top`) |
| `tile_credits[node]` | `cfg` writes to an unadvertised tetrad node go out uncharged; its control core stays illegal; a read request to the node is refused |

| Cell | Proves |
| --- | --- |
| `tile_credits[illegal]` | Descriptors to an unadvertised cell or core, a zero-advertised cell, an out-of-map or the tile-I/O destination are refused; one to a legal cell and one to that cell's node still go |
| `tile_straps[land]` | A cell streams past its strap into one channel: every packet lands, the cell keeps going only on the returns, every flit (header included) comes back, the first promptly |
| `tile_straps[isolation]` | A cell whose channel has no descriptor holds only its own landing space: another cell's packets land meanwhile; the waiting data drains once the descriptors arrive (also on `rapids_16ch_cxs_top`) |
| `tile_straps[discard]` | A disabled channel's data, a read request into RAPIDS and a reserved `ptype` are consumed and return every flit; a core's data packet returns none; the channel works afterwards |
| `tile_straps[coalesce]` | Returns held by `m_tile_credit_ready` leave as the held one plus one coalesced `add` |
| `tile_straps[intake]` | Core descriptors under the strap all run, in order per channel, none dropped, one class-1 entry back each |
| `tile_straps[intake_overrun]` | A flood past the intake queue: `INTAKE_OVERRUN`, every entry returned, the descriptor queue drops none, the survivors run in send order |
| `tile_straps[resync]` | A resync waits for `tile_quiescent`, then sets exactly the legal cells (`TILE_LAND_FLITS`) and core (`TILE_DESC_PER_CORE`) |
| `tile_straps[quiescent_reset]` | A channel reset written while not quiescent does nothing; written while quiescent it clears the channel's error |
| `tile_strap_regs[off]` / `[default]` / `[alt]` | `TILE_STRAPS` sits at 0x7FC in both halves and reads 0 / 16, 2 / 8, 1 (also on |

| Cell | Proves |
| --- | --- |
| | rapids_16ch_cxs_top) |
| tile_straps_alt[land] / [isolation] / [intake] | The straps hold in a TILE_LAND_FLITS = 8, TILE_DESC_PER_CORE = 1 build, with the senders modelling the same straps |
| tile_straps[stall_sink] / [stall_source] | Held WREADY / late R data raise one MonBus stall record (agent, channel, kind, threshold) and the TILE_STALLED bit, which clears when the data moves; the data then lands / goes out correctly |

test_descriptor_engine_shared.py inject_owed_beat covers the owed-beat guard. Every cell was mutation-checked.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

# 30 AXI4 Interface Specification

**Module:** `axi_read_engine.sv` (source AXI read master), `axi_write_engine.sv` (sink AXI write master) **Location:** `projects/components/dma-ip/rapids/rtl/fub/` **Status:** Implemented
**Last Updated:** 2026-09-30

---

## 30.1 Overview

RAPIDS drives two AXI4 masters: a read master on the source half and a write master on the sink half. The channel signals, the outstanding-transaction limits and the response handling are those of RAPIDS Beats and are specified in AXI4 Interface Specification (shared with RAPIDS Beats). This chapter records only what the byte-granular design adds.

The AXI side stays beat-oriented. Every burst starts at a beat-aligned address and moves whole beats. A descriptor that starts or ends inside a beat is handled by the byte lanes of the data path, not by narrow AXI transfers. Both engines are specified in AXI Read Engine and AXI Write Engine.

---

## 30.2 Byte Address to Beat Mapping

The scheduler splits a byte address into a beat address and a byte offset. `OFF_W` is `$clog2(DATA_WIDTH/8)`, which is 5 for the 256-bit board build.

*Table 4.1.1: Address Split*

| Quantity | Value |
|---|---|
| Offset | `addr[OFF_W-1:0]` |
| First beat address | `addr` with the low `OFF_W` bits cleared |
| Beats for a descriptor | `(offset + length + lanes - 1) >> OFF_W`, zero for zero length |

The engines receive the working byte address and the remaining beat count. `ARADDR` and `AWADDR` are driven with the low `OFF_W` bits at zero, so every burst is beat-aligned.

---

## 30.3 Burst Rules

*Table 4.1.2: Burst Rules*

| Rule | Read master | Write master |
|---|---|---|
| Burst start | beat-aligned | beat-aligned |
| Burst size | full bus width | full bus width |
| 4 KB boundary | no burst crosses one | no burst crosses one |
| Burst cap | smaller of the configured burst and the beats left to the 4 KB boundary | same |

The number of beats left to the boundary is computed from the beat-aligned address, so a byte offset never lets a burst run past it.

## 30.4 Write Strobes

The write master carries a byte enable with every beat. The data path stores the strobe next to the data in the sink SRAM, and the write engine drives it onto `WSTRB`. Interior beats of a packet are fully strobed. The first beat and the last beat of a descriptor that start or end inside a beat carry a partial strobe, so only the bytes of the descriptor are written and neighbouring bytes in memory are untouched.

The read master has no byte enables. It always reads whole beats and the source data path discards the bytes outside the descriptor.

*Table 4.1.3: Byte-Granular Write Signal*

| Signal | Width | Meaning |
|---|---|---|
| `m_axi_wstrb` | `DATA_WIDTH/8` | Byte lanes written by the beat, from the stored strobe. |

## 30.5 Limitations

- TYPE=EXT descriptors remain beat-aligned in address and length. This is a permanent limitation.
- A descriptor never issues a narrow AXI transfer. Sub-beat behavior is entirely in the strobes.

---

**Last Updated:** 2026-09-30

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 31 AXI-Stream Interface Specification

**Module:** `src_data_path_axis.sv` (source master), `snk_data_path_axis.sv` (sink slave)
**Location:** `projects/components/dma-ip/rapids/rtl/macro/` **Status:** Implemented **Last Updated:** 2026-09-30

---

## 31.1 Overview

RAPIDS has one AXI-Stream master on the source half and one AXI-Stream slave on the sink half. Both carry packed bytes. Unlike RAPIDS Beats, the stream is no longer a train of full beats: each descriptor is one packet, the last beat of a packet may be partial, and TSTRB says which byte lanes are valid. The handshake, the width parameters and the monitor tap are those of RAPIDS Beats, see AXI-Stream Interface Specification (shared with RAPIDS Beats). The two data paths are specified in Source Data Path AXIS and Sink Data Path AXIS.

---

## 31.2 Packet Contract

*Table 4.2.1: Packet Contract*

| Property | Rule |
|---|---|
| Packet | One descriptor is one packet, ending with TLAST. |
| Packing | Bytes are packed from lane 0 upward. The descriptor start offset in memory does not appear on the stream. |
| TSTRB | Contiguous from lane 0. All ones on every beat but the last. |
| Partial beat | Only the TLAST beat may be partial, carrying the remaining bytes. |
| Packet length | Equals the descriptor length in bytes. |
| Zero length | A zero-length descriptor produces no packet. |

## 31.3 Source Master Signals

*Table 4.2.2: Source Master Signals*

| Signal | Value |
|---|---|
| `m_axis_tdata` | Re-packed bytes from the drain shifter. |
| `m_axis_tstrb` | Low `min(bytes_left, DATA_WIDTH/8)` lanes set. |
| `m_axis_tlast` | Set when the remaining byte count equals the bytes in this beat. |
| `m_axis_tid` | Channel number. |
| `m_axis_tdest` | Channel number. |
| `m_axis_tuser` | Zero. |

The source path pops memory beats and re-packs them, so a stream beat is not tied one to one to a memory beat. The first pop of an offset descriptor only primes the hold register and produces no output unless the packet fits in that single beat. When the last memory beat leaves bytes in the hold register, one extra flush beat carries them.

---

## 31.4 Sink Slave Signals

*Table 4.2.3: Sink Slave Signals*

| Signal | Use |
|---|---|
| `s_axis_tdata` | Payload, shifted into the per-channel hold by the descriptor offset. |
| `s_axis_tstrb` | Valid byte lanes. Only the `TLAST` beat may be partial. |
| `s_axis_tid` | Channel select, low `CIW` bits. |
| `s_axis_tdest`, `s_axis_tuser` | Unused. |
| `s_axis_tlast` | Ends the packet. |
| `s_axis_tready` | Deasserted while the output register is busy, a flush beat is pending, or the selected channel has no packet record. |

The sink stalls a packet until the scheduler has issued the record of its descriptor. A packet for a channel whose record queue is empty is not accepted, so no byte enters the SRAM before the destination offset is known. The queue holds four records per channel.

The spill of a shifted beat is held per channel. After `TLAST`, if the hold register still holds bytes, `TREADY` drops for one flush beat that writes them to the SRAM.

---

## 31.5 Length Check

At TLAST the sink compares the bytes received with the length in the head record of the channel. A mismatch sets the sticky per-channel `r_pkt_len_error` bit, which is ORed with the write engine error into `sched_wr_error`. Nothing is padded or dropped silently. The bytes received are counted from the lanes set in TSTRB on each beat.

---

**Last Updated:** 2026-09-30

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 32 MonBus Interface Specification

**Last Updated:** 2025-01-10

## 32.1 Overview

The MonBus (Monitor Bus) is a 64-bit packet-based interface for reporting internal events, status, and performance data. All RAPIDS subsystems generate MonBus packets that are aggregated and output through a unified interface.

---

## 32.2 MonBus Signal Interface

### 32.2.1 Signal Table

*Table 4.3.1: MonBus Interface Signals*

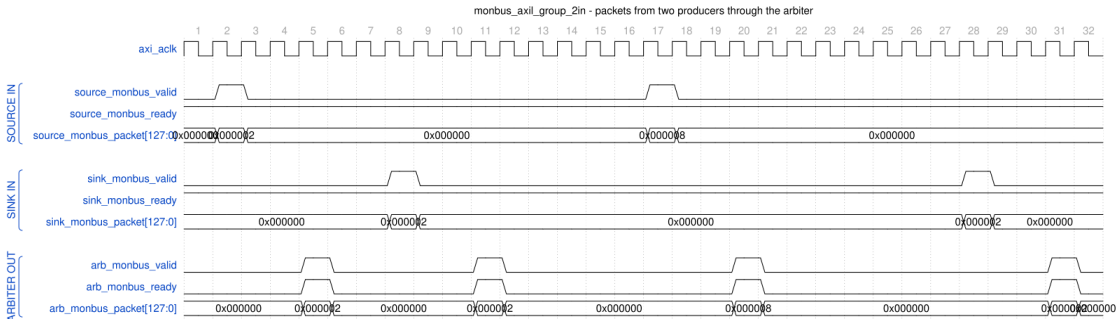
| Signal | Direction | Width | Description |
|---|---|---|---|
| `monbus_pkt_valid` | output | 1 | Packet valid |
| `monbus_pkt_ready` | input | 1 | Consumer ready |
| `monbus_pkt_data` | output | 64 | Packet data |

### 32.2.2 Handshaking Protocol

Standard valid/ready handshaking: - Data transfers when `valid & ready` both high - Producer holds `valid` until `ready` seen - Consumer asserts `ready` when able to accept

### 32.2.3 Timing Diagram

### 32.2.4 Figure 4.3.1: MonBus Packet Transfer Timing



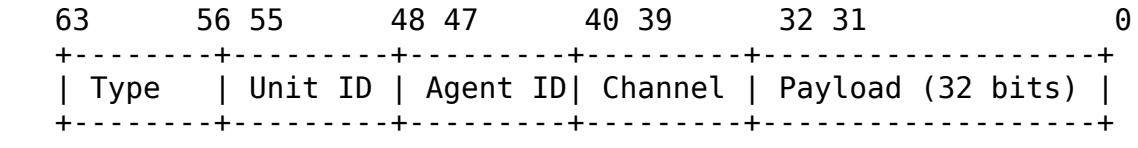

*monbus_axil_group_2in - packets from two producers through the arbiter*

**Source:** monbus_group_packet_transfer.json, captured from `dv/tests/macro/test_monbus_axil_group.py` (`basic_flow`, error FIFO 64 x 32, `TEST_LEVEL=full`) with `WAVES=1`. The 128-bit packets are shown as their top 24 bits (`[127:104]`: packet type nibble, then the protocol field at `[108:105]`, so `0x000002` is AXIS and `0x000008` is CORE).

Reading it: a MonBus link is a single-beat valid/ready handshake with the packet presented alongside `valid`. The group holds `ready` high on both inputs, so the source and sink packets are accepted the cycle they are offered, and each one reappears on the arbiter output three cycles later as another one-cycle `valid`/`ready` pair. Between packets the links idle low; nothing is held across transfers.

## 32.3 MonBus Packet Format

### 32.3.1 64-bit Packet Structure




### 32.3.2 Figure 4.3.2: MonBus Packet Format

*Table 4.3.2: MonBus Packet Fields*

| Field | Bits | Description |
|---|---|---|
| Type | [63:56] | Packet type code |
| Unit ID | [55:48] | Reporting unit identifier |
| Agent ID | [47:40] | Source agent within unit |
| Channel | [39:32] | Channel number (0-7) |
| Payload | [31:0] | Type-specific payload |

## 32.4 Packet Types

### 32.4.1 Type Code Definitions

*Table 4.3.3: MonBus Packet Types*

| Type Code | Name | Description |
|---|---|---|
| 0x01 | DESC_FETCH | Descriptor fetch started |
| 0x02 | DESC_COMPLETE | Descriptor fetch completed |
| 0x03 | SCHED_START | Scheduler started transfer |
| 0x04 | SCHED_DONE | Scheduler transfer complete |
| 0x10 | AXI_RD_START | AXI read burst started |
| 0x11 | AXI_RD_DONE | AXI read burst completed |
| 0x12 | AXI_WR_START | AXI write burst started |
| 0x13 | AXI_WR_DONE | AXI write burst completed |
| 0x20 | SRAM_FILL | SRAM fill event |
| 0x21 | SRAM_DRAIN | SRAM drain event |
| 0x30 | ERROR | Error event |
| 0x31 | TIMEOUT | Timeout event |
| 0xFF | HEARTBEAT | Periodic heartbeat |

### 32.4.2 Payload Formats by Type

#### *32.4.2.1 DESC_FETCH / DESC_COMPLETE (0x01, 0x02)*

*Table 4.3.4: Descriptor Packet Payload*

| Bits | Description |
|---|---|
| [31:16] | Descriptor address [47:32] |
| [15:0] | Descriptor address [31:16] |

#### 32.4.2.2 SCHED_START / SCHED_DONE (0x03, 0x04)

*Table 4.3.5: Scheduler Packet Payload*

| Bits | Description |
|---|---|
| [31:16] | Transfer ID |
| [15:0] | Beat count |

#### 32.4.2.3 AXI_RD/WR_START/DONE (0x10-0x13)

*Table 4.3.6: AXI Event Packet Payload*

| Bits | Description |
|---|---|
| [31:24] | AXI ID |
| [23:16] | Burst length |
| [15:0] | Reserved |

#### 32.4.2.4 ERROR (0x30)

*Table 4.3.7: Error Packet Payload*

| Bits | Description |
|---|---|
| [31:24] | Error code |
| [23:16] | Error source |
| [15:0] | Error details |

## 32.5 MonBus Source Aggregation

### 32.5.1 RAPIDS Core MonBus Sources

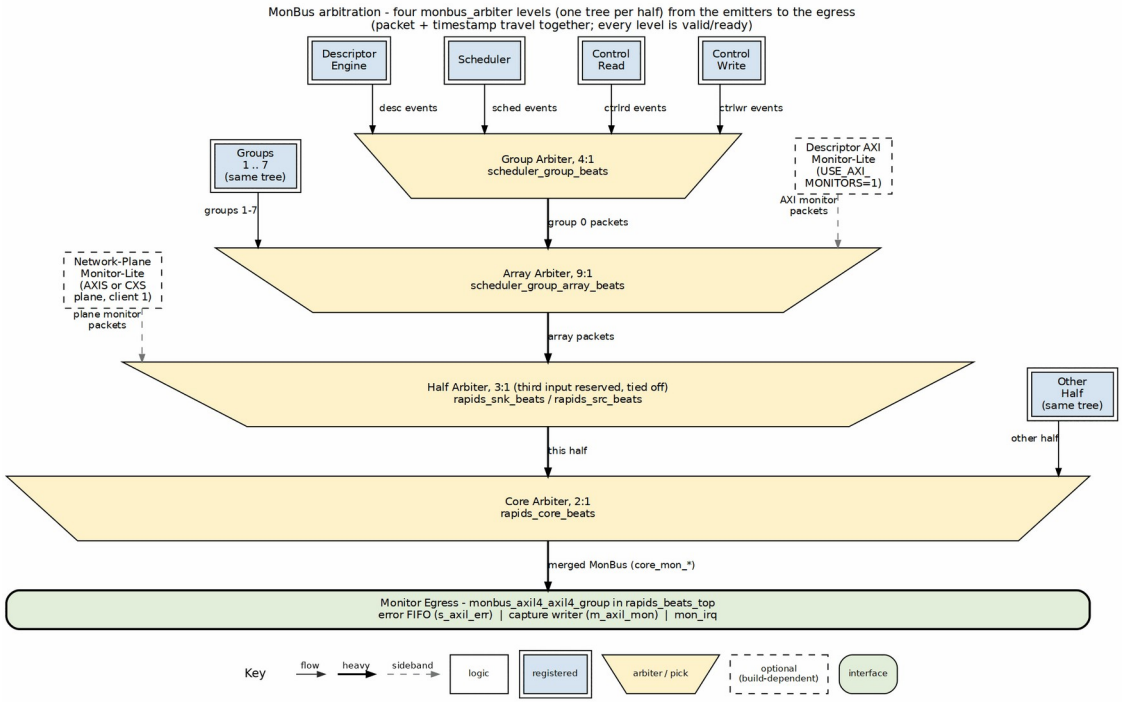
*Table 4.3.8: MonBus Source Assignment*

| Source ID | Agent | Description |
|---|---|---|
| 0-7 | Scheduler[0:7] | Per-channel scheduler events |
| 8-15 | DescEngine[0:7] | Per-channel descriptor events |
| 16-23 | SnkSRAM[0:7] | Sink SRAM controller events |
| 24-31 | SrcSRAM[0:7] | Source SRAM controller events |
| 32 | AXI Write Engine | Write transaction events |
| 33 | AXI Read Engine | Read transaction events |
| 34 | Error Handler | Error events |

Scheduler and DescEngine COMPLETION packets are subject to the half's `SCHED_CONFIG.COMPL_EN` (reset 1): the scheduler group drops them ahead of its arbiter when the bit is clear and acknowledges the emitter, so the data path is unaffected. Their ERROR packets are always forwarded (rapids ISSUE-005).

### 32.5.2 Aggregation Hierarchy

### 32.5.3 Figure 4.3.3: MonBus Aggregation Tree




*MonBus aggregation tree*

**Source:** 21_monbus_aggregation.dot

---

## 32.6 Integration Guidelines

### 32.6.1 Consumer Requirements

MonBus consumers must: 1. Provide sufficient ready cycles to prevent packet loss 2. Handle bursty traffic (multiple sources may have packets simultaneously) 3. Process or buffer packets at wire rate

### 32.6.2 Recommended Consumer Patterns

```
// Pattern 1: Direct FIFO buffering
gaxi_fifo_sync #(
    .DATA_WIDTH(64),
    .DEPTH(256)
) u_gaxi_fifo_sync (
    .axi_aclk               (axi_aclk),
    .axi_aresetn            (axi_aresetn),
    .wr_valid               (wr_valid),
    .wr_ready               (wr_ready),
    .wr_data                (wr_data),
    .rd_ready               (rd_ready),
    .count                  (count),
    .rd_valid               (rd_valid),
    .rd_data                (rd_data)
);
```

---

## 32.7 Timing Requirements

*Table 4.3.9: MonBus Timing Parameters*

| Parameter | Value | Description |
|---|---|---|
| Min Ready | 1 cycle | Must assert ready within N cycles |
| Max Latency | Configurable | Before packet dropped |
| Packet Rate | Variable | Depends on activity |

**Last Updated:** 2025-01-10

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

# 33 Packet Record Interface Specification

**Module:** `scheduler.sv` (producer), `src_data_path_axis.sv` and `snk_data_path_axis.sv` (consumers) **Location:** `projects/components/dma-ip/rapids/rtl/fub/` and `rtl/macro/`
**Status:** Implemented **Last Updated:** 2026-09-30

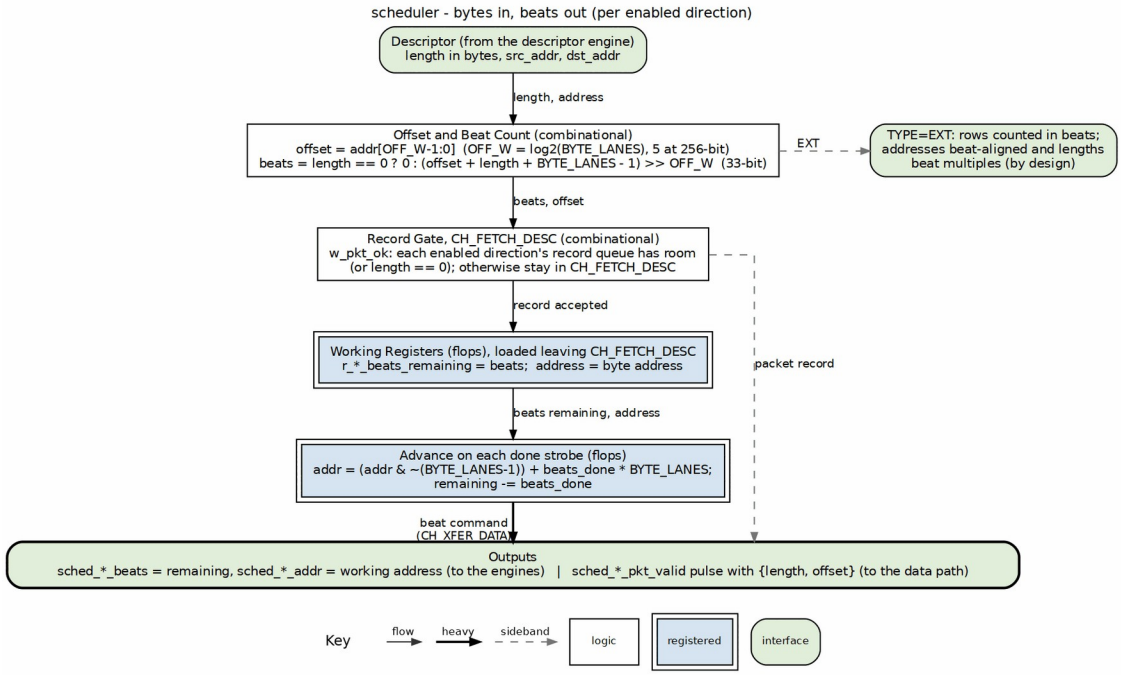
---

## 33.1 Overview

The packet record interface is new in the byte-granular design. It tells a data path how many bytes a descriptor covers and where in its first beat the data starts. RAPIDS Beats has no equivalent, because every transfer there is a whole number of aligned beats.

The scheduler is the producer. It emits one record per data descriptor. The source or sink data path is the consumer and holds the records in a small queue per channel. The record interface is internal to each half: it does not appear on the `rapids_core` or `rapids_top` ports.

### 33.1.1 Figure 4.4.1: Scheduler Byte Math




*Scheduler byte math*

**Source:** assets/graphviz/03_scheduler_byte_math.dot

---

## 33.2 Signals

There is one set per direction. The source half uses `sched_rd_pkt_*` and the sink half uses `sched_wr_pkt_*`. Each signal is a vector with one entry per channel.

*Table 4.4.1: Packet Record Signals*

| Signal | Direction (scheduler view) | Width per channel | Description |
|---|---|---|---|
| `sched_*_pkt_valid` | output | 1 | One-cycle pulse: a record is presented. |
| `sched_*_pkt_ready` | input | 1 | The consumer queue has room. |
| `sched_*_pkt_bytes` | output | 32 | Descriptor length in bytes. |
| `sched_*_pkt_offset` | output | `OFF_W` | Address offset inside the first beat. |

`OFF_W` is `$clog2(DATA_WIDTH/8)`, 5 for a 256-bit build.

## 33.3 Protocol

The scheduler raises `valid` for one cycle, in the cycle it leaves the descriptor fetch state. It does not leave that state until every enabled direction can take the record, so `ready` is a pre-condition of the pulse, not a wait state after it. Both directions of a channel see exactly one pulse per data descriptor.

A consumer accepts a record when `valid` and its queue is not full. It drives `ready` low when the queue is full, which is what holds the scheduler in its fetch state.

*Table 4.4.2: Record Rules*

| Rule | Detail |
|---|---|
| Records per descriptor | One, for data descriptors of nonzero length. |
| Zero-length descriptors | No record. |
| Non-data descriptors | No record. |
| Order | Records are consumed in the order pushed, per channel. |
| Queue depth | Four records per channel in each data path. |
| Pop | The record pops on the last beat of its packet. |

The offset is taken from the source address in the read direction and from the destination address in the write direction. The length is the same for both. When both directions are enabled, each data path receives its own offset.

## 33.4 Unused Direction

A scheduler group array instantiated for only one direction ties the other direction off inside the half. The unused `valid`, `bytes` and `offset` outputs are left open and the unused `ready` inputs are held at all ones, so the scheduler never waits on a queue that does not exist.

---

## 33.5 Consumer Behavior

- The source data path uses the record to compute the number of memory beats, to decide when a pop emits a stream beat, to set `TSTRB` and `TLAST`, and to retire the record.
- The sink data path uses the record to decide how far to shift each incoming beat, to check the received length at `TLAST`, and it holds off `TREADY` for a channel until the record exists.

---

**Last Updated:** 2026-09-30

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 34 CXS Interface Specification

**Applies to:** `rapids_beats_cxs_top`, `rapids_cxs_top` **Protocol:** Arm IHI 0079 (CXS) Issue D, one fixed profile **Last Updated:** 2026-10-16

---

## 34.1 Overview

The CXS tops replace the two AXIS network ports with one CXS link per direction: the **sink port** `s_cxs_*` (a CXS receiver) into the sink half and the **source port** `m_cxs_*` (a CXS transmitter) out of the source half. One link carries every channel; the channel rides in `CXSSRCID`. The planes that implement the links are described in the Network Planes chapter (Section 3.13).

## 34.2 Link Profile

*Table 4.4.1: CXS Link Profile*

| Property | Value |
|---|---|
| Issue | D (the issue with SRCID, TGTID and USER) |
| `CXSMAXPKTPERFLIT` | 2 |
| Packet start alignment | 16 bytes; RAPIDS always starts a packet at byte 0 |
| Packet mapping | One AXIS packet (`tlast`-delimited) is one CXS packet |
| `CXSSRCID` | The channel (AXIS `tid`) |
| `CXSTGTID` | AXIS `tdest` |
| `CXSUSER` | `{tuser, tstrb}`; `tuser` dropped when `DATA_WIDTH/8 + AXIS_USER_WIDTH > 128` |
| Stream framing | Per stream on the channel bits of `CXSSRCID` |
| Link control | Explicit credit return, activation, deactivation |
| Parity | Off |

## 34.3 Signal Table

*Table 4.4.2: CXS Interface Signals*

| Signal | Sink port | Source port | Width | Description |
|---|---|---|---|---|
| {s,m}_cxs_valid | in | out | 1 | Flit valid |
| {s,m}_cxs_data | in | out | DATA_WIDTH | Flit data |
| {s,m}_cxs_cntl | in | out | CXS_CNTL_PW | START / END / pointers |
| {s,m}_cxs_srcid | in | out | AXIS_ID_WIDTH | Channel |
| {s,m}_cxs_tgtid | in | out | AXIS_DEST_WIDTH | tdest |
| {s,m}_cxs_user | in | out | CXS_UW | {tuser, tstrb} |
| {s,m}_cxs_crdgnt | out | in | 1 | Credit grant (receiver to transmitter) |
| {s,m}_cxs_crdrtn | in | out | 1 | Credit return (transmitter to receiver) |
| {s,m}_cxs_activereq | in | out | 1 | Transmitter requests the link |
| {s,m}_cxs_activeack | out | in | 1 | Receiver acknowledges |
| {s,m}_cxs_deacthint | out | in | 1 | Receiver asks the transmitter to deactivate |
| snk_cxs_link_state, src_cxs_link_state | out | out | 2 | {ACTIVEREQ, ACTIVEACK} |
| snk_cxs_protocol_e | out | out | 1 | Sticky protocol violation |

| Signal | Sink port | Source port | Width | Description |
|---|---|---|---|---|
| rror, src_cxs_protocol_error | | | | |
| snk_cxs_unsupported | out | - | 1 | Sticky: a flit AXIS framing cannot carry |

## 34.4 Credit Flow

One credit permits one flit (IHI 0079D 2.1). The receiver grants up to `CXS_MAX_CREDIT` (default 32) credits, one per free entry in its receive FIFO, so every flit sent against a credit has space. The transmitter spends one credit per flit and returns the credits it still holds on deactivation. Credits are a single pool for the link; a grant carries no ID and cannot be reserved for a channel.

The grant -> flit -> free round trip between the RAPIDS endpoints is about 3 cycles, so 32 credits sustains one flit per cycle with margin for pipeline stages or a clock crossing on the link. More than 63 credits is beyond the spec and requires `CXS_CREDIT_BEYOND_SPEC` on both ends.

## 34.5 Link Activation

*Table 4.4.3: Link Activation Sequence*

| Step | Transmitter (`m_cxs_*`) | Receiver (`s_cxs_*`) |
|---|---|---|
| 1 | Raises `ACTIVEREQ` when it has a flit to send (and only once the previous activation's `ACTIVEACK` has fallen: 4-phase) | - |
| 2 | - | Raises `ACTIVEACK`; starts granting credits |
| 3 | Sends flits, one credit each | Frees an entry and regrants as the core drains |
| 4 | After `CXS_IDLE_CYCLES` idle cycles in RUN (or on `DEACTHINT`) drops `ACTIVEREQ`, then returns every held credit on `CRDRTN`, one per cycle | - |
| 5 | - | Drops `ACTIVEACK` once its credits are back |

## 34.6 Backpressure

On the sink port, a full receive FIFO means no further grants: the transmitter stalls for lack of credit. This is the CXS form of the AXIS sink's `s_axis_tready = 0`, including the byte-granular sink's hold until a channel's descriptor has been fetched. On the source port, the plane stalls the core's egress (`fub_axis_tready = 0`) while it has no credit.

## 34.7 Proposed Changes

A proposal (2026-10-16, not implemented) puts a tile packet header at the front of every packet: destination and source coordinates, payload type (`cfg` / `prog` / `mem`), length and target address. It is described once in the compute-engine tile's `TILE_SPEC.md` Section 11; CXS carries it as the first flit. The link signals, credits and per-stream framing on this page do not change, because `CXSSRCID` keeps the channel.

# 35 rapids-g Overview

**Status:** Implemented (rapids TASK-034) **Last Updated:** 2026-10-22

## 35.1 What rapids-g Is

rapids-g is RAPIDS with its scheduler state machines replaced by firmware. Each half of the core (source and sink) has one gatling, the four-context barrel RV32I core (`projects/components/riscv-ip/gatling-rv32i`), running one firmware image for all of that half's 4, 8 or 16 channels. The firmware makes every decision RAPIDS's state machine makes and does the EXT address arithmetic RAPIDS's `scheduler` does in hardware:

- taking a descriptor, validating it, starting it, finishing it, and following a chain;
- the EXT run bases, `base + ((i0 * stride_0) & wrap_0) + ((i1 * stride_1) & wrap_1)`, computed by repeated addition (gatling has no multiplier) and pushed ahead into a small queue per direction.

Everything that happens at the engines' rate stays hardware: the counters the done strobes decrement, the address advance within a run, the timeouts, the sticky errors, the packet records and the MonBus packets. That hardware is `scheduler_core`, the same module RAPIDS's `scheduler` uses (Section 2.1), so the two designs share one datapath.

rapids-g is register-compatible with RAPIDS: the same register map, the same descriptor format, the same host software, the same network planes.

Two capabilities RAPIDS does not have: the firmware pre-validates the next descriptor while the current one runs, so the hardware starts it the moment the current one completes (Section 5.2); and each half's gatling can read and write its own half's registers – including the eight per-channel `CH_SCRATCH` bits it shares with the host – through the host's register command queue (Section 5.3).

## 35.2 Structure

*Table 5.1.1: rapids and rapids-g, level by level*

| Level | rapids | rapids-g | How the -g module is made |
|---|---|---|---|
| fub | `scheduler` = state machine + ceilings + 2 run-base generators + `scheduler_core` | `scheduler_g` = command port + ceilings + base FIFOs + `scheduler_core` | hand-written (`rtl/fub_g/`) |
| macro | `scheduler_group_array` | `scheduler_group_array_g`: the same, with `scheduler_g` and one `rapids_seq_g` | generated |
| macro | `rapids_src`, `rapids_snk`, `rapids_core` | `rapids_src_g`, `rapids_snk_g`, `rapids_core_g` | generated |
| top | `rapids_top_base`, `rapids_top`, `rapids_cxs_top` | `rapids_g_top_base`, `rapids_g_top`, `rapids_g_cxs_top` | generated |
| sequencer | – | `rapids_seq_g`: gatling + instruction ROM + scratch + window decoder | hand-written (`rtl/macro_g/`) |
| firmware | – | `rtl/fw_g/sched_g.S` | assembled into `rtl/macro_g/generated/fw/sched_g_fw.svh` |

`bin/gen_rapids_g.py` derives every generated file from the RAPIDS file of the same name (Section 5.4); a change to RAPIDS reaches rapids-g by rerunning it.

## 35.3 Channels per gatling

By default each hart serves one channel at 4 and 8 channels (`NUM_HARTS` = `NUM_CHANNELS`): 4 or 8 harts, one slot each, every hart issuing one instruction every `NUM_HARTS` clocks. At 16 channels the default is 8 harts with two slots each (channels h and h + 8), measured best overall (Section 5.5; rapids TASK-036); 16 harts x 1 and 4 harts x 4 are also built and tested. `NUM_HARTS` = 4 at 8 channels gives each hart two slots, channels h and h + 4. The generated array refuses any other channel count, and `SHARED_ENGINES` = 0, at elaboration. An 8-channel core therefore has two gatlings, one per half, each sequencing eight schedulers; a 16-channel core has two, each sequencing sixteen.

## 35.4 What stays hardware, and why

Two transitions are interlocks, not policy, and stay hardware so they take effect in the cycle RAPIDS's `scheduler` would take them:

- a channel reset forces CH_IDLE;
- a fatal fault – a descriptor, engine or control error, an escalated timeout, or the serving hart halting – forces CH_ERROR, which drops the engine valids at once.

Every other transition is a firmware command, checked against an edge table (Section 5.2).

# 36 scheduler_g

**Module:** `scheduler_g.sv`, `scheduler_g_base_fifo.sv` **Location:** `projects/components/dma-ip/rapids/rtl/fub_g/` **Status:** Implemented (rapids TASK-034) **Last Updated:** 2026-10-22

---

## 36.1 Overview

`scheduler_g` has the whole port list of RAPIDS's `scheduler` plus a sequencer window. Inside it is `scheduler_core` (Section 2.1), the same two byte-to-beat ceilings as `scheduler`, and small pieces in place of its state machine and run-base generators:

- a state register written by command, with an edge table and the two interlocks;
- pre-validation: the firmware judges the next descriptor while this one runs, and the hardware takes the approved edges itself;
- the beat totals, computed in hardware with `scheduler`'s own ceiling and loaded from the skid head on the cycle it is taken, and the bases-done flags, written by the firmware;
- two four-entry run-base FIFOs (`scheduler_g_base_fifo`), one per direction.

## 36.2 Sequencer Port

*Table 5.2.1: Sequencer port*

| Signal | Dir | Description |
|---|---|---|
| `seq_wr_en`, `seq_wr_addr[4:0]`, `seq_wr_data[31:0]` | in | a word write into the window (registered in `rapids_seq_g`) |
| `seq_rd_addr[4:0]`, `seq_rd_data[31:0]` | in / out | a word read; combinational here, registered in `rapids_seq_g` |
| `seq_attn` | out | the firmware has something to do on this channel |
| `seq_fault` | in | the hart serving this channel has halted: a fatal fault |
| `seq_alg_max` | in | the firmware image's highest next-address selector (`FW_ALG_MAX`, rapids TASK-035), a constant from `rapids_seq_g` |

## 36.3 Register Window

*Table 5.2.2: The window, by word*

| Word | Read | Write |
|---|---|---|
| 0 | STATUS | CMD: `[6:0]` next state (0 = none), `[8]` read bases done, `[9]` write bases done, `[10]` STAGE, `[11]` ARM, `[12]` ARM to NEXT_DESC, `[13]` GEN_OK, `[14]` DECLINE, `[15]` STAGE_ARM, `[16]` STAGE_ARM to NEXT_DESC |
| 1-2 | SRC_LO, SRC_HI | 1: FW_ERROR (sends an error packet, data `{word, PKT_LO}`); 2: reserved |
| 3-4 | DST_LO, DST_HI | 3: RD_BASE_LO, 4: RD_BASE_HI (the HI write pushes `{HI, LO}`) |
| 5 | LENGTH (bytes) | WR_BASE_LO |
| 6 | NEXT_PTR | WR_BASE_HI (pushes) |
| 7-12 | chunk 1 words 0-5 (`descriptor_ext_t`) | 7: PKT_LO (the next packet's low data word); 8: FW_DEBUG (sends a debug packet, data `{word, PKT_LO}`); 9-12: reserved |
| 13 | FIFO: `[2:0]` read bases queued, `[6:4]` write bases queued, `[15:8]` CMD_ERR, `[16]` firmware packet pending | – |
| 14 | CFG: `[15:0]` BYTE_LANES, `[19:16]` | – |

| Word | Read | Write |
|---|---|---|
| | OFF_W, [20] EN_READ, [21] EN_WRITE, [22] EXT built, [23] TILE_HDR built | |
| 15-18 | the skid head, before it is taken: src [31:0], dst [31:0], length, descriptor [223:192] (valid, opcode, type, tile op) | – |
| 19-20 | the beat totals in use (hardware) | – |
| 21 | HEAD_SUM: the skid head's [0] valid (and its next-address selector one the image runs), [1] last, [2] chain (next_ptr != 0), [3] open (not yet staged or declined, in a state where STAGE and DECLINE are legal) | – |
| 22 | ALG: the latched descriptor's next-address selector, chunk 0 [224:217] | – |
| 23 | ALG_MAX: seq_alg_max | – |

*Table 5.2.3: STATUS*

| STATUS bits | Field |
|---|---|
| [6:0] | state (channel_state_t) |
| [7] | seq_attn |
| [8], [9], [10] | descriptor at the skid head, channel enable, channel reset active |
| [11], [13:12] | descriptor valid, opcode |

| STATUS bits | Field |
|---|---|
| [14], [15], [16] | last, gen_irq, chain (`next_ptr != 0`) |
| [17], [18], [19] | tile_bad, pkt_ok, exec_complete |
| [20], [21] | start pending, cmd_err (sticky) |
| [22], [23], [24], [25] | EXT, header only, read per-beat, write per-beat |
| [26], [27] | read bases done, write bases done |
| [28], [29], [30], [31] | armed, head staged, EXT set-up needed, head declined |

## 36.4 Commands and the Edge Table

A CMD write names the next state. The table below is the only way state changes outside the interlocks. A command it does not list is ignored, sets the sticky `cmd_err` and counts in CMD_ERR (saturating at 255; a channel reset clears both).

*Table 5.2.4: Legal edges*

| From | Command | Condition | Effect |
|---|---|---|---|
| IDLE | FETCH_DESC | a descriptor at the head, channel enabled | the head is taken on the same edge |
| NEXT_DESC | FETCH_DESC | a descriptor at the head | taken |
| FETCH_DESC | XFER_DATA | descriptor valid, not tile_bad | start pending; taken the cycle `pkt_ok` holds, which is the cycle the packet records fire |
| XFER_DATA | COMPLETE | exec_complete | |
| COMPLETE | IDLE or NEXT_DESC | – | |
| FETCH_DESC, XFER_DATA, COMPLETE, NEXT_DESC | ERROR | – | |

CH_ERROR is left only by a channel reset, as in RAPIDS's `scheduler`. The pending start is what keeps the packet records a single handshake: `scheduler_core` raises them only with `start_req`, so a FETCH_DESC held by the firmware never offers a record early.

A command that arrives after a channel reset it was not written for – the firmware was part-way through a handler when the reset came – is ignored by the same table and counted. The equivalence tests expect CMD_ERR to stay 0 on every channel that is never reset.

## 36.5 Pre-validation and the Automatic Edges

Owner direction, 2026-10-21: “Pre validation is a must.” While descriptor N transfers, the firmware:

- ARMs N: complete it on `exec_complete`, then go to NEXT_DESC (`[12]` = 1) or IDLE. Legal once per descriptor, in XFER_DATA.
- judges the descriptor at the skid head: valid -> STAGE; not valid -> DECLINE. Legal while a descriptor is at the head and not yet judged, from XFER_DATA through NEXT_DESC.
- with STAGE, may set STAGE_ARM (`[15]`) and its NEXT_DESC choice (`[16]`): the head’s own chain outcome, decided from HEAD_SUM. The hardware then takes that head already armed. STAGE_ARM without STAGE is illegal and counted.

STAGE_ARM is how the firmware keeps up with short descriptors. Without it, every descriptor needs an ARM written inside its own XFER_DATA, and the firmware’s path to that write is about 25 instructions – 200 clocks at 8 harts, where a 64-beat descriptor transfers in under 90. Each such descriptor finished before it was armed and went the hand path: 672 clocks per descriptor on a 64-beat sink chain (Section 5.5). With STAGE_ARM, a descriptor needs one firmware write, made any time before its predecessor completes.

With N armed, the hardware then takes, in the cycles RAPIDS’s `scheduler` would: XFER_DATA -> COMPLETE (on `exec_complete`) -> IDLE or NEXT_DESC -> FETCH_DESC (the staged head, its totals, the start already requested) -> XFER_DATA (when `pkt_ok`). The latched descriptor is still checked (valid, tile_bad) as on the hand path. Anything not approved stops the sequence and raises attention: an unarmed descriptor, an unstaged or declined head, a disabled channel.

Three ordering rules make it safe, each the fix of a race the equivalence bench found:

| Rule | Why |
|---|---|
| An EXT descriptor completes automatically only once both directions’ bases-done bits are set | the firmware writes them after its last push, so no push or done bit of this descriptor can land after the next is taken |
| GEN_OK (set-up done) is written before the set-up | a descriptor cannot complete before its done bits, which the set-up writes after GEN_OK, so a GEN_OK can never land on the next descriptor |
| The firmware judges both directions’ refills on one STATUS snapshot | a fresh STATUS read after the first direction’s last done bit may already describe the next descriptor |

A head the hardware takes is flagged (`[30]`) when it is EXT, and the firmware sets up its run bases in XFER_DATA; run 0 uses the descriptor's own address, so only short first runs wait for them.

## 36.6 Firmware Packets

The firmware can put its own packets on the channel's MonBus port, from the channel's agent:

*Table 5.2.5: Firmware packets*

| Write | Packet type | Event code | Event data |
|---|---|---|---|
| FW_DEBUG (word 8) | `PktTypeDebug` | `RAPIDS_EVENT_FW_DEBUG` (0x09) | `{written word, PKT_LO}` |
| FW_ERROR (word 1) | `PktTypeError` | `RAPIDS_EVENT_FW_ERROR` (0x0A) | `{written word, PKT_LO}` |

`scheduler_core`'s own packets are one-cycle pulses and always win the cycle; a firmware packet leaves in the first cycle with none and `mon_ready` high. One packet waits at a time (FIFO `[16]`); a send while one waits is dropped and counted in CMD_ERR, like a push to a full base FIFO. A firmware error packet only reports: it does not move the channel to CH_ERROR, which stays a CMD or an interlock. Neither packet type is a completion, so `cfg_sched_compl_enable` does not filter them.

## 36.7 Attention

`seq_attn` is high when the firmware has work on the channel, and is what the sequencer's SUMMARY register reports:

- IDLE with the channel enabled and an unstaged descriptor at the head; NEXT_DESC with an unstaged descriptor at the head;
- FETCH_DESC with no start pending, or turned tile_bad;
- XFER_DATA not yet armed, EXT set-up needed, an EXT base FIFO with room and bases still to come, or an unjudged head;
- COMPLETE not armed (the hand path).

Nothing raises attention in CH_ERROR, during a channel reset, or on a fault.

## 36.8 Run-Base FIFOs

`scheduler_g_base_fifo` holds four 64-bit bases (the depth of `stream_run_addr_gen`'s own queue). A LO write is held; the HI write pushes `{HI, LO}`. The FIFO is flushed whenever the channel is outside FETCH_DESC and XFER_DATA, and on a channel reset, so a base can never outlive its descriptor. A push to a full FIFO is dropped and counted in CMD_ERR. `scheduler_core` takes a base only between runs, exactly as from the generator.



---

# 37 rapids_seq_g and the Scheduler Firmware

**Module:** `rapids_seq_g.sv`; firmware `rtl/fw_g/sched_g.S` **Location:** `projects/components/dma-ip/rapids/rtl/macro_g/` **Status:** Implemented (rapids TASK-034, TASK-037, TASK-035, TASK-036, TASK-038) **Last Updated:** 2026-10-23

---

## 37.1 rapids_seq_g

One `gatling_core`, used unchanged through its own filelist, with the three memories it needs, each answering in one stage as gatling requires:

*Table 5.3.1: rapids_seq_g*

| Block | What | Size |
|---|---|---|
| Instruction memory | the firmware image, from `sched_g_fw.svh`; one read port serves every hart (the barrel fetches for one context per cycle). A ROM, or with `FW_WRITABLE` = 1 the same block RAM with a second port the host loads through (below) | `IMEM_WORDS` = 1024 |
| Scratch | per-hart memory, indexed `{context, word}`; the run-base generator state of each slot | `SCRATCH_WORDS` = 128 per hart (256 at four slots) |
| Window decoder | routes a data access to the slot's `scheduler_g` by context and slot | – |

Parameters: `NUM_CHANNELS` (4, 8 or 16), `NUM_HARTS` (4, 8 or 16, dividing `NUM_CHANNELS` with at most four slots per hart; default `NUM_CHANNELS`, one hart and one slot per channel; hart h serves channels h + `NUM_HARTS` x s), `IMEM_WORDS`, `SCRATCH_WORDS` (a power of 2, at least 64 per slot; a smaller scratch would alias slot 1 onto slot 0, and elaboration refuses it), `FW_WRITABLE` (0 = ROM, 1 = loadable; Loading Firmware Live).

### 37.1.1 Data Memory Map

*Table 5.3.2: Data memory map*

| Address | Region |
|---|---|
| 0x0000_0000 | instruction memory (fetch only; a data read returns 0) |
| 0x1000_0000 | slot 0 window (Section 5.2), |

| Address | Region |
|---|---|
| | word-indexed |
| 0x1000_0100 + 0x100 s | slot s window (s < SLOTS) |
| 0x1000_1000 | SUMMARY: `[SLOTS-1:0]` `seq_attn` of this hart's slots |
| 0x1000_1004 | SLOTS: slots per hart (1, 2 or 4) |
| 0x1000_3000 | register mailbox (below), aliased by context |
| 0x2000_0000 | scratch, aliased by context |

Contexts stay implicit (gatling PRD 7): every hart runs the same image, and "slot 0" is a different channel for each, chosen by the context the access comes from. Reads are registered (address in X, data in W); window writes are registered into `scheduler_g` one cycle after X, so a hart's next instruction, `NUM_HARTS` (at least four) cycles later, always sees the effect of its own write. Each hart issues one instruction every `NUM_HARTS` clocks: at the default 8 harts an instruction costs a hart 8 clocks, which is the unit of every firmware cost in Section 5.5.

### 37.1.2 Register Mailbox

Each hart can read and write its own half's PeakRDL registers – the same `rapids_regs` the host programs, config and status alike – through a mailbox in its data memory map at 0x1000_3000, aliased by context:

*Table 5.3.3: Register mailbox, per hart*

| Offset | Word | What |
|---|---|---|
| +0x00 | REG_ADDR | byte offset within the half, `[11:0]` |
| +0x04 | REG_WDATA | write data |
| +0x08 | REG_CTRL | `[0]` go, `[1]` write; a store with go set posts the access |
| +0x0C | REG_STATUS | `[0]` busy, `[1]` done, `[2]` error (the response's PSLVERR) |
| +0x10 | REG_RDATA | read data, valid once done |

The firmware posts an access and polls REG_STATUS: a single-stage barrel cannot stall on a bus. A store to REG_* while the hart's own access is busy is ignored, so a hart polls for done before it

posts the next one. The scheduler firmware does not use the mailbox today; the path is there for firmware that reads or adjusts its half's configuration, and `test_rapids_g_regpath` proves it (below).

#### *37.1.2.1 A Serialized Path*

**The register path is serialized, end to end.** The register block has one command/response path (`peakrdl_to_cmdrsp` in front of `rapids_regs`), and exactly one access is on it at any time. One agent – the host's APB, or one hart of one gatling – is granted the path, sends its one command, and **owns the path until its response is complete**: the grant is released only by the response handshake. No other agent's command enters the path in between, there is no pipelining and no second outstanding access, and a response is never returned to anyone but the agent that owns the path. This is the same one-outstanding discipline `apb4_slave` keeps on the host path alone, and the MON-window guard behind the arbiter depends on it: it registers its select when a command is accepted and steers the response on that registered select.

Two levels of round-robin arbitration feed that one path, and both hold their grant for the whole access:

**Level 1, the harts of one gatling** (`rapids_seq_g`). Each hart has its own mailbox state, indexed by context: `r_mb_addr[h][11:0]`, `r_mb_wdata[h]`, `r_mb_wr[h]`, and `r_mb_busy[h]` – set by the REG_CTRL store, and the hart's request – with `r_mb_done[h]`, `r_mb_err[h]` and `r_mb_rdata[h]` for the completion. An `arbiter_round_robin_simple_ack` takes `request = r_mb_busy` and grants one hart (`w_mb_gnt_valid`, `w_mb_gnt_id`). That hart's access goes out once on the gatling's one master port – `reg_cmd_valid = w_mb_gnt_valid && !r_mb_sent && r_mb_busy[gnt]`, with `reg_cmd_pwrite`, `reg_cmd_paddr[11:0]` and `reg_cmd_pwdata` from the granted hart – and `r_mb_sent` records that it has gone. `reg_rsp_ready = w_mb_gnt_valid && r_mb_sent`. The response handshake clears the hart's busy, latches done, `err <- reg_rsp_pslverr` and `rdata <- reg_rsp_prdata`, and is the arbiter's ack: only then does the grant move to the next requesting hart.

**Up the tree.** The port is `seq_reg_*` on `scheduler_group_array_g`, `rapids_src_g` / `rapids_snk_g` and `rapids_core_g` (`src_seq_reg_*`, `snk_seq_reg_*`), carried up unchanged by `gen_rapids_g.py`, and arrives in the top base as `w_src_seq_reg_*` and `w_snk_seq_reg_*`. Each carries `cmd_valid`, `cmd_ready`, `cmd_pwrite`, `cmd_paddr[11:0]`, `cmd_pwdata`, `rsp_valid`, `rsp_ready`, `rsp_prdata`, `rsp_pslverr`.

**Level 2, the host against the two gatlings** (`rapids_g_reg_arb`, included by `rapids_g_top_reg_arb.svh`). The generator renames `apb4_slave`'s command/response outputs to `host_cmd_*` / `host_rsp_*`, and three masters meet:

*Table 5.3.4: Register-path masters*

| Master | Index | Address into the 13-bit map |
|---|---|---|
| host APB, `host_cmd_*` / `host_rsp_*` | 0 | `host_cmd_paddr`, the whole map |
| SRC gatling, | 1 | `{1'b0,` |

| Master | Index | Address into the 13-bit map |
|---|---|---|
| w_src_seq_reg_* | | w_src_seq_reg_cmd_paddr}: pinned to SRC, 0x0000-0x0FFF |
| SNK gatling, w_snk_seq_reg_* | 2 | {1'b1, w_snk_seq_reg_cmd_paddr}: pinned to SNK, 0x1000-0x1FFF |

A second arbiter_round_robin_simple_ack takes request = m_cmd_valid[2:0] and grants one master (w_gnt_valid, w_gnt, w_gnt_id). The granted master's command passes through once – s_cmd_valid = w_gnt_valid && !r_cmd_sent && m_cmd_valid[gnt], m_cmd_ready[gnt] = s_cmd_ready && !r_cmd_sent – and r_cmd_sent holds the grant open for the response. The response goes back only to the owner: m_rsp_valid[gnt] = s_rsp_valid && r_cmd_sent and s_rsp_ready = r_cmd_sent && m_rsp_ready[gnt] (m_rsp_prdata and m_rsp_pslverr are shared wires, meaningful only with that master's valid). w_ack = (s_rsp_valid && s_rsp_ready) ? w_gnt : 0 releases the grant and clears r_cmd_sent. The arbiter's slave side is apb_cmd_* / apb_rsp_*, which runs on through the MON-window guard (an error response for paddr[11] when USE_MON_REGS = 0) to peakrdl_to_cmdrsp and rapids_regs.

*Table 5.3.5: One gatling access, end to end*

| Step | Owner of the path | Signals |
|---|---|---|
| a hart posts | – | REG_CTRL store sets r_mb_busy[h] |
| level 1 grants that hart | the hart, within its gatling | w_mb_gnt_id = h; reg_cmd_valid |
| level 2 grants that gatling | the hart, end to end | w_gnt_id; s_cmd_valid -> apb_cmd_valid; r_cmd_sent, r_mb_sent set |
| the register block answers | still the hart | apb_rsp_valid -> m_rsp_valid[gnt] -> reg_rsp_valid |
| the response is accepted | released at both levels | w_ack (level 2), reg_rsp_valid && reg_rsp_ready (level 1); r_mb_done[h] |

What the serialization costs and guarantees:

- **Ordering.** Accesses from all agents are totally ordered by grant; an access completes before the next one starts, so a read after a write – from any agent – sees the write.
- **Fairness.** Under full contention the host gets at least every third access and each gatling the same; inside a gatling each of its harts gets at least one of every `NUM_HARTS` of that gatling's turns (1 in 24 of the path at 8 harts).
- **Latency.** An access holds the path from command to response, including `peakrdl_to_cmdrsp`'s two-cycle request, so a busy gatling delays a host access by an access time, and a host access delays a waiting hart the same way. That is invisible at UART speeds and small at full APB rate; the mailbox is not a per-descriptor path.
- **No atomicity across accesses.** Each access is atomic; a read-modify-write is two accesses another agent can come between. The per-channel `CH_SCRATCH[n].SCRATCH` bits (0x300 in each half) are for the host and the gatling to share flags or semaphores: give each bit one writer.

`test_rapids_g_regpath` (dv/tests/top_g) replaces the scheduler image with a test program in which every hart reads SCHED_TIMEOUT_CYCLES of its own half through the mailbox and writes SCHED_TIMEOUT_LIMIT, while the host programs both halves and reads them back over APB through the same path. It checks that each gatling sees its own half's value, that its writes land in its own half, that no mailbox access reported an error, and that every host read during the contention returned the value the host expects.

### 37.1.3 Faults

`seq_fault[c]` is the halted bit of the hart serving channel c, so a firmware trap is a fatal fault for that hart's channels: they enter CH_ERROR and send the scheduler's error packet. A halted hart restarts from address 0 once every channel it serves is in channel reset.

## 37.2 Firmware

`sched_g.S` is RV32I with no stack and no data section; state that outlives one step lives in the slot's scratch. It is an **executive** over a **policy table** (rapids TASK-037). The executive polls and dispatches and is the same for every policy; the table says what each (state, event) pair does. Both firmware defects found on 2026-10-22 came from mixing the two (Section 5.5): the poll and the handlers disagreed about who owned an open head, and a handler that re-dispatched itself kept a hart for several descriptors. Neither can recur in this shape.

### 37.2.1 The Executive

Run to completion, no preemption: the barrel's harts are the threads. Each pass of the poll gives a slot at most one handler, and the handler returns to the poll. The image reads SLOTS at start-up and takes one of two polls.

**One slot per hart** (the default, `NUM_HARTS` = `NUM_CHANNELS`). The poll reads HEAD_SUM first. HEAD_SUM = 13 (open, valid, `next_ptr` != 0, not last) is a chained head: the poll writes STAGE | STAGE_ARM | STAGE_ARM_NEXT, held in a register, so a running chain costs four instructions from the read to the CMD write. This chain hook is the one policy entry that lives in the poll itself. Any other open head is judged inline (`judge_head`: stage it armed with its own chain outcome, or decline it). With no open head the poll reads SUMMARY; the idle pass is seven instructions. When SUMMARY shows attention, the poll reads HEAD_SUM again, so a head that arrived between the two reads is judged inline too, then calls `step`.

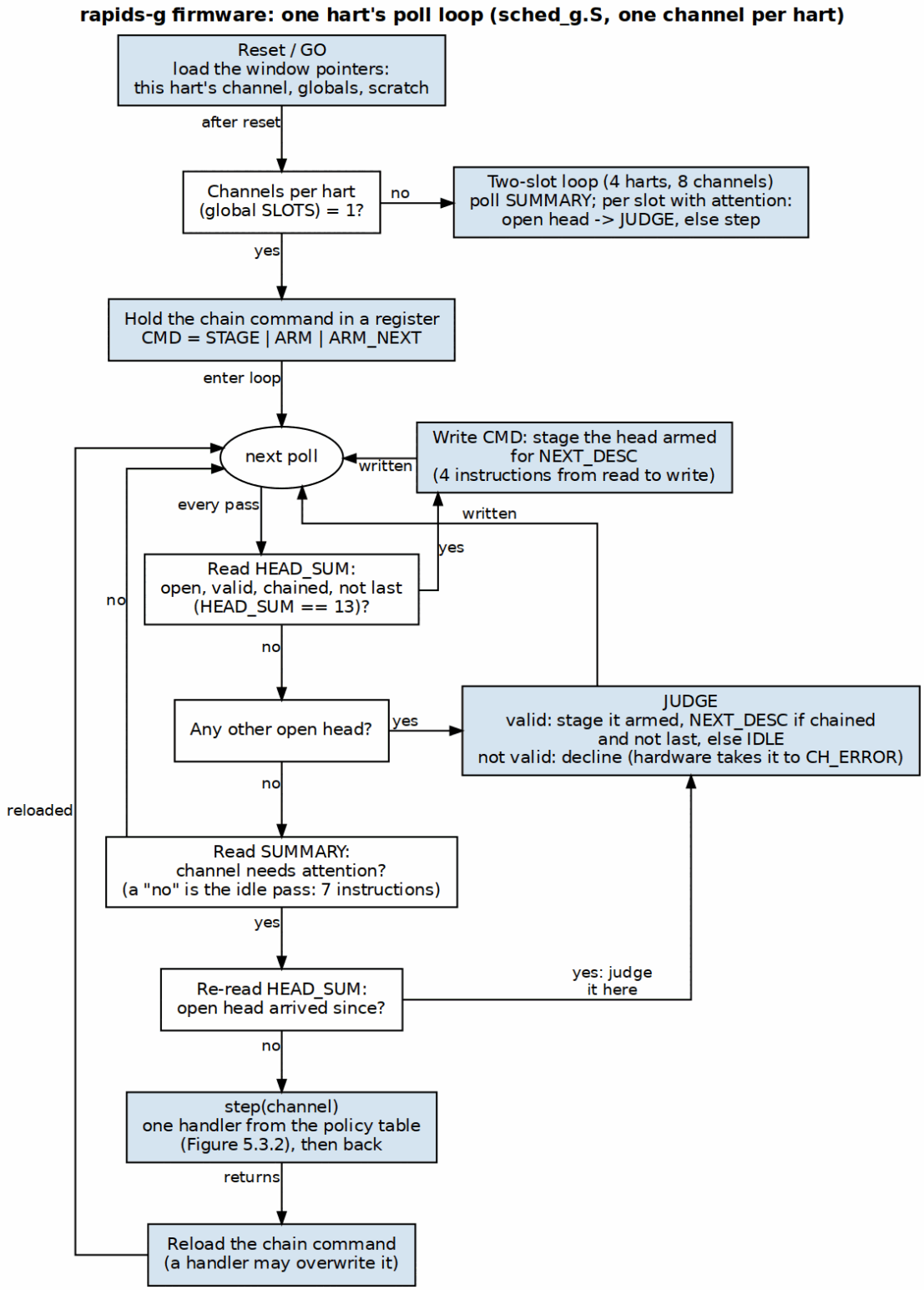
**Two slots per hart** (`NUM_HARTS` = 4 at 8 channels, or 8 at 16, the 16-channel default). The poll reads SUMMARY first, so the idle pass is two instructions. For each slot with attention it reads HEAD_SUM and, before anything else, takes the same chain hook: HEAD_SUM = 13 is staged armed for NEXT_DESC straight from the read, with the stage word held in `tp` (reloaded after every `step`, which may clobber it). Any other open head is judged inline; otherwise the poll calls `step`. Unrolled for the two slots.

A chain is serial: the hardware takes about 20 clocks per one-beat descriptor (FETCH_DESC, XFER_DATA, COMPLETE, NEXT_DESC), then waits in NEXT_DESC with the next head open until the firmware's stage command lands, so every instruction between the head opening and the CMD write is paid per descriptor. A cycle trace of the earlier two-slot polls (Section 5.5) showed it: the image before 2026-10-23 judged every head through SUMMARY and JUDGE, about 22 instructions (about 180 clocks a descriptor); a first fix that put the two HEAD_SUM checks ahead of SUMMARY alternated between a 36-clock wait (the check caught the head) and a 156-clock one (the head opened just after its check and the pass fell to JUDGE), about 120 on average. Checking SUMMARY first and the chain hook inside each slot's branch never misses: about 99 clocks a descriptor on the board, against about 44 with one slot per hart, which tests HEAD_SUM in a four-instruction loop.

**Four slots per hart** (`NUM_HARTS` = 4 at 16 channels, rapids TASK-036). The same poll as a loop over SUMMARY's bits, the slot's offset in `sp` (window and scratch share the 0x100 slot stride).

`step` reads STATUS and jumps through `ptab`, indexed by the 6-bit one-hot state: four instructions from the STATUS read to the state's row. `ptab` has 64 entries, so no value of the field can index past it; the five states with work have a jump to their row, and every other entry (CH_ERROR, which only a channel reset leaves, and the values that are not states) returns. The image is linked at 0 and `ptab` sits below 2 KB, so the jump is `jalr` off the index register alone and needs no base register (`gen_rapids_g_fw.py` refuses an image where it does not fit).

### 37.2.2 Figure 5.3.1: One Hart's Poll



*rapids-g firmware poll loop*

**Source:** `assets/graphviz/12_rapids_g_fw_poll_loop.dot`

### 37.2.3 The Policy Table

A state's row picks one handler by event, highest priority first. A handler does one step – one or two CMD writes, or one run-base set-up or top-up – and returns. No handler loops, re-reads STATUS to carry on, or calls `step`: the slot's next step is the next pass of the poll.

*Table 5.3.6: The policy table*

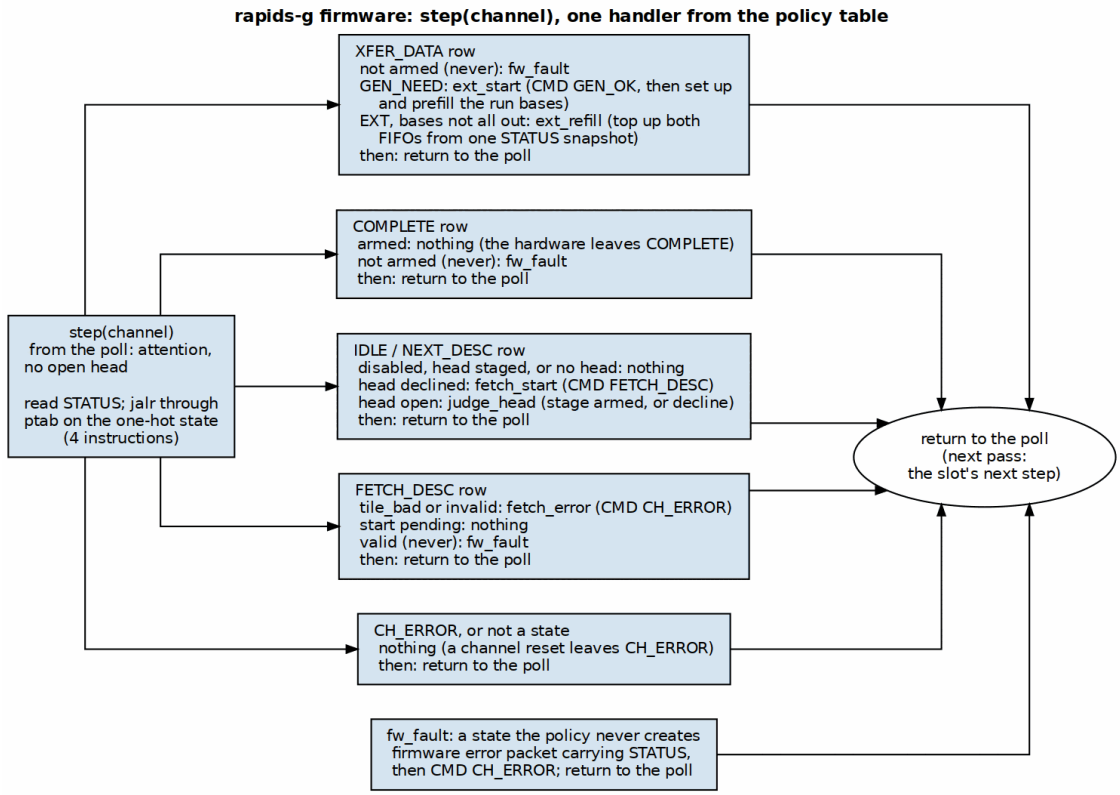
| State | Event, in priority order | Handler | What it does |
|---|---|---|---|
| any | an open head (in the poll) | `judge_head` | valid: STAGE armed with its chain outcome; not valid: DECLINE |
| IDLE (enabled), NEXT_DESC | head staged, or no head | – | nothing: the hardware takes a staged head |
| | head declined | `fetch_start` | CMD FETCH_DESC; FETCH_DESC then takes it to CH_ERROR |
| | head open | `judge_head` | as above |
| FETCH_DESC | tile_bad, or descriptor invalid | `fetch_error` | CMD CH_ERROR |
| | start already pending | – | nothing |
| | valid (never: see below) | `fw_fault` | firmware error packet carrying STATUS; CMD CH_ERROR |
| XFER_DATA | not armed (never) | `fw_fault` | as above |
| | GEN_NEED (an EXT descriptor the hardware took) | `ext_start` | CMD GEN_OK, then set up and prefill the run bases (`ext_init`) |
| | EXT, bases not all out | `ext_refill` | top up both directions' base FIFOs from one |

| State | Event, in priority order | Handler | What it does |
|---|---|---|---|
| | | | STATUS snapshot |
| COMPLETE | armed | – | nothing: the hardware leaves COMPLETE next cycle |
| | not armed (never) | `fw_fault` | as above |

**The rows marked “never”.** Every valid head is staged armed, so the hardware takes it, completes it and leaves COMPLETE on its own; the firmware takes a head itself (FETCH) only when it declined it, and a declined head is invalid by the same test the latch applies. A descriptor can therefore never be in XFER_DATA or COMPLETE unarmed, nor valid in FETCH_DESC by the firmware’s hand. TASK-034’s firmware still carried a hand path for those states (arm in XFER_DATA, route out of COMPLETE, start from FETCH_DESC); the mutation set showed no test could reach it (MAS 5.5), because nothing can. It is replaced by `fw_fault`: a state the policy never creates stops the channel visibly instead of being guessed at, and a channel reset recovers it.

The run-base routines (`ext_init`, `refill`) are a layer below the table: they are where the next-address algorithm lives, and the descriptor’s selector picks one (rapids TASK-035, below).

### 37.2.4 Figure 5.3.2: step(), the Policy Table




*rapids-g firmware step and policy table*

**Source:** `assets/graphviz/13_rapids_g_fw_service.dot`

### 37.2.5 EXT Run Bases

The positions, the step and the 16-bit index wrap are exactly `stream_run_addr_gen`'s: run-contiguous when `stride_0` equals the beat size (one base per run of `inner_count` beats, rows 1, 2, ...), otherwise per beat (every beat after the first, inner index fastest). A base is `base + ((i0 * s0) & m0) + ((i1 * s1) & m1)` in 64 bits. Without wrap masks (the common case) the firmware keeps the address itself and a base is one 64-bit add (`cur += s0` along a row, `row += s1` at a row's end); with masks it keeps the two products as running sums and masks them for each base. Whenever an EXT descriptor's FIFO has room, attention brings the firmware back to top it up; the last base sets the direction's bases-done bit.

### 37.2.6 Next-Address Selector

The descriptor's selector (chunk 0 `[224:217]`, HAS Section 5.1) chooses the algorithm. The image declares the highest one it implements with `.equ ALG_MAX`; `gen_rapids_g_fw.py` writes it into `sched_g_fw.svh` as `FW_ALG_MAX`, and `rapids_seq_g` drives it to every `scheduler_g` as `seq_alg_max`. The check is hardware, the same in both places it is made: `scheduler_core` latches a descriptor whose selector is above it (or non-zero without TYPE=EXT) as invalid, and

scheduler_g folds the same test into HEAD_SUM's valid bit, so the firmware declines such a head and the chain hook (HEAD_SUM = 13) is untouched. A selector is refused from reset, before the firmware runs an instruction.

*Table 5.3.7: Next-address algorithms in the image*

| Selector | Set-up | Top-up |
|---|---|---|
| 0 | gen_init: positions, step and masks as stream_run_addr_gen | refill (masked) or refill_fast (no wrap) |
| 1, Z-order | gen_init_z: unit = a beat or a run of inner_count beats; S0 = stride_0 or inner_count << OFF_W; position k = 1 | refill_z |

refill_z keeps x x S0 and y x stride_1 as running sums, like the masked path, and steps the position in Morton order without a multiplier: k -> k + 1 clears k's trailing ones and sets the bit above them, and bit 2i is x's bit i, bit 2i + 1 is y's, so each cleared bit subtracts the stride shifted by i and the set bit adds it – about two bits a step on average. The generator block's mode word tells refill which path to take (1 fast, 0 masked, -1 Z-order), so selector 0 costs one instruction more per masked top-up and nothing on the fast path.

### 37.2.7 Building the Image

bin/gen_rapids_g_fw.py runs riscv-none-elf-as / ld (text at 0) / objcopy -O binary and writes FW_WORDS and FW_IMAGE[] into rtl/macro_g/generated/fw/sched_g_fw.svh, which rapids_seq_g includes, so neither simulation nor synthesis needs a file path. --check fails when the include is stale. Never edit the include.

### 37.2.8 Loading Firmware Live

With FW_WRITABLE = 1 (rapids TASK-038) the instruction memory is the same 1024 x 32 block RAM with its second port used for a load: write and read back. It powers up holding the generated image, so a writable build that is never loaded runs exactly as the ROM build. The host reaches it through four registers in each half, at the same offsets in the 8- and 16-channel maps:

*Table 5.3.8: Firmware load registers, per half*

| Offset | Register | Fields |
|---|---|---|
| 0x3A0 | FW_CTRL | HOLD[0] RW: hold every hart of the half in reset, and allow FW_DATA writes; WRITABLE[1] RO: |

| Offset | Register | Fields |
|---|---|---|
| | | 1 on a FW_WRITABLE build; ALG_MAX[15:8] RW: the loaded image's highest next-address selector |
| 0x3A4 | FW_ADDR | ADDR[15:0] RW: the word index; advances by one after each FW_DATA write and each FW_RDATA read |
| 0x3A8 | FW_DATA | DATA[31:0]: a write stores the word at FW_ADDR, only while HOLD = 1 |
| 0x3AC | FW_RDATA | DATA[31:0] RO: the word at FW_ADDR (0 on a ROM build) |

A load is: FW_CTRL HOLD = 1 with the image's ALG_MAX; FW_ADDR = 0; one FW_DATA write per word; FW_ADDR = 0 and one FW_RDATA read per word to compare; FW_CTRL HOLD = 0. Releasing the hold restarts the harts from address 0 on the new image, like `seq_restart`, so hold only while every channel of the half is idle. While HOLD = 1 the next-address cap every `scheduler_g` applies (`seq_alg_max`) follows FW_CTRL.ALG_MAX and keeps that value after the release, so a loaded image carries its own selector set. On a ROM build and on RAPIDS the registers exist, WRITABLE reads 0 and the rest has no effect.

The top base's glue (`rapids_g_top_fw.svh`) turns each register access into one store or one advance. Behind `peakrdl_to_cmdrsp`, `swmod` and `swacc` are a two-cycle level, not a pulse (the handbook's generated-RTL note); the glue takes each access on its rising edge, one flop late so the field storage has the written value. The first version counted every cycle: it stored each word twice and advanced FW_ADDR by two, which a readback walking the same counter could not see below word 512 – the self-test's direct address probe now catches it.

Host side (`flows-rapids/host/run_characterization.py`): `--fw <sched.S>` assembles the source with the same generator that writes the ROM include, loads both halves, reads every word back and releases, and records the image's SHA-256 in the results (on a writable build the bitstream no longer names the firmware). `--fw-selftest` first proves the path: a FW_DATA write without HOLD is refused; a one-word image of 0x00000000, an illegal instruction, halts every hart, so every channel goes to CH_ERROR (the harts run what was loaded); direct probes across the memory read the right word and advance by one; loading the real image and a channel reset bring every channel back to CH_IDLE. On a ROM build it checks the opposite, that the same

writes change nothing. A full load is about 2,600 UART operations, too many for a 100 ms simulation beside a measurement, so a simulation can instead build the candidate in (`TEST_FW_IMAGE`, rendered by `gen_rapids_g_fw.py --src/--out`) and check it with `--fw-expect`, which reads back the first and last eight words; the memory holds the same image at GO either way.

The flow is: iterate a candidate live on a writable bitstream, check it in simulation built in, compare simulation and board, and once it is final commit it as `sched_g.S`, so the ROM build carries it. The two-slot poll above was developed this way, two candidates loaded on one bitstream; the committed image is byte-identical (SHA-256 `00b7003b...`) to the one measured on the board.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 38 The Generated rapids-g Tree

**Generator:** `bin/gen_rapids_g.py` **Status:** Implemented (rapids TASK-034) **Last Updated:** 2026-10-21

---

Above `scheduler_g` the rapids-g RTL is RAPIDS's, so it is derived rather than copied:

*Table 5.4.1: Generated files*

| Generated | From | Edits |
|---|---|---|
| `rtl/macro_g/generated/scheduler_group_array_g.sv` | `rtl/macro/scheduler_group_array.sv` | rename; `scheduler` becomes `scheduler_g`; the `seq_*` ports wired; `scheduler_group_array_g_seq.svh` (one `rapids_seq_g`) included in front of the scheduler loop; `$fatal` unless `SHARED_ENGINES` = 1 and `NUM_CHANNELS` is 4, 8 or 16 |
| `rtl/macro_g/generated/rapids_src_g.sv`, `rapids_snk_g.sv` | `rtl/macro/rapids_src.sv`, `rapids_snk.sv` | rename; the array becomes `scheduler_group_array_g` |
| `rtl/macro_g/generated/rapids_core_g.sv` | `rtl/macro/rapids_core.sv` | rename; the halves become `_g` |
| `rtl/top_g/generated/rapids_g_top_base.sv` | `rtl/top/rapids_top_base.sv` | rename; the core becomes `rapids_core_g`; `apb4_slave`'s cmd/rsp outputs renamed `host_*`; `rapids_g_top_reg_arb.svh` (host + both gatlings into the register command queue) included after it |
| `rtl/top_g/generated/rapids_g_top.sv`, | the -g base | the thin-top derivation of `bin/gen_plane_tops.py` |

| Generated | From | Edits |
|---|---|---|
| `rapids_g_cxs_top.sv` | | |
| `rtl/top_g/generated/rapids_g_16ch_top.sv`, `rapids_g_16ch_cxs_top.sv` | the -g base | the same, `NUM_CHANNELS` = 16; compiled with `rtl/filelists/top_g/rapids_g_16ch{,_cxs}_top.f`, which select the 16-channel register map (rapids TASK-036) |

The register-mailbox master (`seq_reg_*`) is added to every level's port list after `rst_n` and connected at every instance (array -> half -> core as `src_` / `snk_` -> top base), so the RAPIDS files need no knowledge of it.

Every edit matches its anchor exactly once or the generator fails, so a RAPIDS change that moves an anchor is a failed run, not a wrong file. `python3 bin/gen_rapids_g.py --check` exits 1 when any generated file is stale. The filelists are `rtl/filelists/fub_g/`, `macro_g/` and `top_g/`; they pull gatling through `$GATLING_ROOT/rtl/filelists/top/gatling_core.f`, and the 8-channel register map as `top/rapids_top_base.f` does.



---

# 39 rapids-g Verification and Measurements

**Status:** Implemented (rapids TASK-034) **Last Updated:** 2026-10-22

## 39.1 Verification

*Table 5.5.1: rapids-g suites*

| Suite | What it shows |
|---|---|
| `dv/tests/fub_g/test_rapids_g_equiv.py` | RAPIDS's `scheduler` and `scheduler_g` + firmware side by side (`rapids_g_equiv_tb_top`), 4, 8 and 16 channels, 256 and 512 bits, one, two and four channels per hart (`NUM_HARTS`: 8 on 4, 16 on 16, 8 and 4), same descriptors, engine models with different latencies per side. Every engine command, packet record, MonBus packet and state transition must match in order. The address sequences are also checked against an independent golden model (`RapidsGEquivTB.golden_cmds`: the byte-to-beat ceiling and `dma_address_gen`'s formula). Cells: linear, ext, reset_ext, backpressure, error_reset, reset_any (a channel reset in every state), hart_fault (every hart halted by an illegal word patched over its SUMMARY poll, found by encoding, faults its channels and restarts), alg (next-address selectors: unrunnable ones invalid on both sides, selector 1 on gatling against the golden model's Z-order and refused by RAPIDS), perf. |
| `dv/tests/fub_g/test_scheduler_g_edges.py` | `scheduler_g` alone with the testbench as its firmware: every command and flag in every state, 57 checks; legal edges taken, the rest ignored and counted, CH_ERROR left only by a reset; the automatic COMPLETE -> IDLE -> FETCH -> XFER of an armed |

| Suite | What it shows |
|---|---|
| | descriptor with a staged head, and the stop at a declined head. |
| `dv/tests/top_g/ test_rapids_g_regpath.py` | Both gatlings run a test program that reads and writes their own half's registers (`SCHED_TIMEOUT_CYCLES`, `SCHED_TIMEOUT_LIMIT`, `CH_SCRATCH`) through the mailbox while the host hammers the same command queue: every hart reads its own half's value, each gatling's writes land in its own half only, no host read is wrong. |
| `dv/tests/macro_g/` | Every RAPIDS `scheduler_group_array` test on `scheduler_group_array_g`, at 4, 8 and 16 channels. |
| `dv/tests/top_g/` | Every RAPIDS `rapids_top` and `rapids_cxs_top` test, unchanged, on `rapids_g_top` and `rapids_g_cxs_top`; the 16-channel cells on `rapids_g_16ch_top` and `rapids_g_16ch_cxs_top`. |

The test files are RAPIDS's own: the -g runners load the RAPIDS cocotb tests by module name (`macro_g`) or re-host the RAPIDS wrappers (`top_g/rapids_g_rehost.py`), so rapids-g is held to exactly RAPIDS's checks. They caught a firmware race on the way: a head that arrived between the tight loop's HEAD_SUM and SUMMARY reads went down `service`'s slower path, and `tile_source_bad_ptype` saw its channel error after the test's wait on three of eight seed bases; the loop now re-reads HEAD_SUM before `service`.

Mutation (re-run on the executive image, rapids TASK-037, and the selector, TASK-035; each from clean against the equivalence bench, or the edge test where it owns the logic):

*Table 5.5.2: Mutation, executive image*

| Mutant | Where | Result |
|---|---|---|
| JUDGE never chains | firmware | caught at two slots per hart (8 on 4, 16 on 8); with one slot a chained head always takes the chain hook, so JUDGE |

| Mutant | Where | Result |
|---|---|---|
| | | never sees one |
| route out of an unarmed COMPLETE | firmware | survived: unreachable (below); the code is now `fw_fault` |
| stride sign extension | firmware | caught |
| wrap mask dropped | firmware | caught |
| XFER_DATA's `ptab` entry returns | firmware | caught |
| GEN_OK not written | firmware | caught |
| Z-order: y subtracts | firmware | caught (`alg`) |
| selector ignored (always selector 0) | firmware | caught (`alg`) |
| edge-table hole: COMPLETE always legal | `scheduler_g` | caught by the edge test; the bench cannot reach it, because the firmware never commands COMPLETE |
| fault interlock removed | `scheduler_g` | caught |
| HEAD_SUM ignores the selector | `scheduler_g` | caught (`alg`) |
| the latch ignores the selector | `scheduler_core` | caught (`alg`) |

The first run left three survivors, and each said something different. JUDGE's chain decision is live only when a hart serves two slots, a configuration the bench did not build until it gained its `NUM_HARTS` axis. The edge-table hole belongs to the edge test. The unarmed-COMPLETE route, with the rest of the old hand path, cannot run at all: every valid head is staged armed, and the firmware takes a head itself only when it declined it, which the latch also calls invalid (Section 5.3). That code became the `fw_fault` row rather than a handler no test can reach.

Two RAPIDS bugs were found on the way and fixed in RAPIDS: rapids BUG-016 (a held CH_FETCH_DESC sent DESC_START every cycle) and rapids BUG-017 (stale EXT run bases after a channel reset; also in RAPIDS Beats and STREAM).

## 39.2 Performance on the Board

rapids-8ch and rapids-g-8ch ran the same 45-point `gcompare` profile on the Genesys 2 harness (`projects/fpga-systems/Genesys2/dma-ip/rapids/flows-rapids`): 8 channels, DATA_WIDTH 256, 100 MHz, peak **3,200 MB/s per direction**. A figure is the bytes moved over the hardware timer's cycles from the first request to the last write, descriptor fetch and launch included. Every point is golden-checked and passes on both designs. The point names read `<group>_ch<N>_b<B>[_d<D>][_il]`: `beat` is one descriptor per channel and `chain` a linked chain, `ch<N>` the active channels, `b<B>` beats (32 bytes) per descriptor, `d<D>` descriptors per channel (absent: one), and `il` an interleaved sink. Without `il` the sink stream feeds the channels one after another, so only one channel holds data at a time; the source reads memory for every channel at once in every point.

Records: `reports/board/flow_results/byte_perf_gcompare_fsm_il_rc.json` (rapids-8ch) and `byte_perf_gcompare_gfin_rc.json` (rapids-g-8ch).

The simulation of the board build (`make sim-gcompare`, run in seven chunks) matches the board cycle for cycle: 85 of 85 windows on each design.

The executive image with the next-address selector (rapids TASK-037 and TASK-035; `byte_perf_gcompare_gexe_rc.json`, 2026-10-23) passes all 45 points on the board, matches its simulation on 85 of 85 windows, and is identical to the 2026-10-22 image (gfin) on all 85: every gcompare point is a linear descriptor whose head the poll stages itself, the path the rewrite did not lengthen, so the board numbers in Table 5.5.3 stand unchanged. That build takes 111,738 LUTs, 65,246 registers and 58 block RAM tiles and closes 100 MHz at +0.061 ns. rapids-8ch's simulation, on the shared datapath with the selector check, still matches its own board run on 85 of 85. Matching needed one change to the host program. The shared descriptor engine's round-robin pointer (`descriptor_engine_shared`, `r_rr_ptr`) is reset only by `aresetn`, so it carries from one point to the next. A chunk that starts cold in simulation served the channels in a different order from the board, where the previous point had run, and took 16 clocks longer (8 on rapids-8ch). Each chunk after the first now runs the previous chunk's last point first, unrecorded (`byte_perf.chunk_warmup`). The RTL is unchanged.

*Table 5.5.3: Genesys 2 gcompare, MB/s (peak 3,200; – = not measured: the source side has no interleaved variant)*

| Point | rapids-8ch sink | rapids-g-8ch sink | rapids-8ch source | rapids-g-8ch source |
|---|---:|---:|---:|---:|
| beat_ch1_b1 | 54 | 14 | 50 | 14 |
| beat_ch1_b4 | 197 | 56 | 191 | 56 |
| beat_ch1_b16 | 624 | 210 | 648 | 212 |
| beat_ch1_b64 | 1,575 | 701 | 1,613 | 709 |
| beat_ch1_b256 | 2,544 | 1,693 | 2,568 | 1,703 |
| beat_ch1_b1024 | 3,006 | 2,617 | 3,015 | 2,624 |

| Point | rapids-8ch sink | rapids-g-8ch sink | rapids-8ch source | rapids-g-8ch source |
|---|---|---|---|---|
| beat_ch1_b4096 | 3,149 | 3,031 | 3,152 | 3,033 |
| beat_ch1_b32768 | 3,194 | 3,178 | 3,194 | 3,178 |
| beat_ch2_b1 | 77 | 29 | 74 | 28 |
| beat_ch2_b4 | 281 | 111 | 281 | 110 |
| beat_ch2_b16 | 853 | 394 | 890 | 398 |
| beat_ch2_b64 | 1,896 | 1,151 | 1,941 | 1,160 |
| beat_ch2_b256 | 2,731 | 2,214 | 2,754 | 2,223 |
| beat_ch2_b1024 | 3,068 | 2,879 | 3,075 | 2,883 |
| beat_ch2_b4096 | 3,166 | 3,113 | 3,168 | 3,114 |
| beat_ch2_b32768 | 3,196 | 3,189 | 3,196 | 3,189 |
| beat_ch4_b1 | 99 | 45 | 100 | 45 |
| beat_ch4_b4 | 363 | 174 | 368 | 180 |
| beat_ch4_b16 | 1,056 | 589 | 1,095 | 621 |
| beat_ch4_b64 | 2,122 | 1,517 | 2,161 | 1,569 |
| beat_ch4_b256 | 2,840 | 2,505 | 2,857 | 2,540 |
| beat_ch4_b1024 | 3,102 | 2,993 | 3,107 | 3,005 |
| beat_ch4_b4096 | 3,175 | 3,145 | 3,176 | 3,149 |
| beat_ch4_b32768 | 3,197 | 3,193 | 3,197 | 3,194 |
| beat_ch8_b1 | 116 | 66 | 121 | 66 |
| beat_ch8_b4 | 425 | 252 | 436 | 250 |
| beat_ch8_b16 | 1,198 | 806 | 1,237 | 811 |
| beat_ch8_b64 | 2,257 | 1,837 | 2,291 | 1,843 |
| beat_ch8_b256 | 2,897 | 2,699 | 2,911 | 2,703 |
| beat_ch8_b1024 | 3,119 | 3,058 | 3,123 | 3,059 |
| beat_ch8_b4096 | 3,179 | 3,163 | 3,180 | 3,164 |
| beat_ch8_b32768 | 3,197 | 3,195 | 3,198 | 3,195 |
| chain_ch1_b1_d16 | 149 | 73 | 278 | 73 |
| chain_ch1_b64_d16 | 2,214 | 1,875 | 3,015 | 1,788 |
| chain_ch1_b1024_d16 | 3,113 | 3,103 | 3,188 | 3,177 |

| Point | rapids-8ch sink | rapids-g-8ch sink | rapids-8ch source | rapids-g-8ch source |
|---|---|---|---|---|
| chain_ch1_b8192_d4 | 3,186 | 3,180 | 3,194 | 3,188 |
| chain_ch8_b1_d16 | 164 | 74 | 1,177 | 419 |
| chain_ch8_b64_d16 | 2,276 | 1,975 | 3,123 | 3,059 |
| chain_ch8_b1024_d16 | 3,121 | 3,117 | 3,195 | 3,191 |
| chain_ch8_b8192_d4 | 3,190 | 3,188 | 3,198 | 3,195 |
| beat_ch8_b16_il | 1,048 | 726 | – | – |
| beat_ch8_b64_il | 2,098 | 1,717 | – | – |
| chain_ch8_b1_d16_il | 822 | 418 | – | – |
| chain_ch8_b64_d16_il | 2,709 | 2,662 | – | – |
| chain_ch8_b1024_d16_il | 3,166 | 3,162 | – | – |

The firmware's cost per descriptor sets the gap. Its history on the hardest point, a one-channel chain of 64-beat sink descriptors:

*Table 5.5.4: Firmware cost per chained descriptor (the 16-channel rows' clocks are the point's measured cycles / 16)*

| Firmware | chain_ch1_b64_d16 sink | Clocks per descriptor |
|---|---|---|
| first 8-hart image (one hart per channel) | 1,118 MB/s | about 288 |
| one-slot tight chain loop, HEAD_SUM re-read before `service` | 1,833 MB/s (board) | about 90 |
| seven-instruction idle pass (this image) | 1,875 MB/s | about 90 |
| rapids-8ch, for reference | 2,214 MB/s | – |
| two slots per hart (16 channels), SUMMARY then JUDGE | 1,122 MB/s (board) | 184 |
| two slots, HEAD_SUM checks ahead of SUMMARY | 1,546 MB/s (board) | 134 |
| two slots, SUMMARY then the chain hook per slot (this image) | 2,008 MB/s (board) | 103 |
| rapids-16ch, for reference | 2,232 MB/s | 93 |

Before STAGE_ARM (Section 5.2) the same point cost 672 clocks a descriptor. The tight loop (Section 5.3) follows a chained head in four instructions; the shorter idle pass recovered a 1-2% dip on the eight-channel single-descriptor sink points (`beat_ch8_b4` to `b256`).

## 39.3 Scheduler Limit

Measured with `cocotb_test_perf`: engines that take every burst in one cycle and never stall, so the scheduler is the only limit; 8 channels and 8 harts, DATA_WIDTH 256, cycles at 100 MHz. The bench runs both directions on every channel, twice the load of one RAPIDS half (each half moves one direction).

*Table 5.5.5: Cycles to finish, schedulers alone*

| Scenario | rapids-8ch | rapids-g-8ch, 10-22 | rapids-g-8ch, executive | executive / rapids-8ch |
|---|---|---|---|---|
| 16 one-beat descriptors, one channel | 131 | 2,371 | 2,371 | 18x |
| 16 one-beat descriptors on each channel | 131 | 2,316 | 2,316 | 18x |
| 16 64-beat descriptors on each channel | 1,255 | 2,389 | 2,389 | 1.9x |
| 4 64 KB descriptors on each channel | 9,248 | 9,529 | 9,412 | 1.02x |
| 2 1 MB descriptors on each channel | 84,277 | 84,420 | 84,443 (*) | 1.002x |
| EXT, runs of 4, 64 beats, 4 per channel | 407 | 40,433 | 39,201 | 96x |
| EXT, per beat, 64 beats, 4 per channel | 1,043 | 135,848 | 130,086 | 125x |
| EXT, runs of 256, 1 MB, 1 per channel | 29,574 | 72,329 | 70,641 | 2.4x |

The executive image (rapids TASK-037) is 2-4% faster on every EXT row and unchanged on the linear ones: an XFER_DATA step costs four instructions of dispatch and returns, where `service` re-read STATUS and re-judged the head before returning. (*) The scenarios run in sequence in one simulation, so when the 64 KB row finishes sooner the 1 MB row starts at another phase of the barrel; run alone from the same cycle, both images take 84,420 (and 9,454 for 64 KB).

With one hart per channel the channels no longer share a hart, so one busy channel and eight busy channels cost the same (2,371 and 2,316). On 4 harts the same rows were 7,215 and 14,139. Pre-validation hides the firmware behind the descriptor in flight whenever the transfer outlasts it: two 1 MB descriptors per channel finish 0.17% behind rapids-8ch, and what remains is each channel's first descriptor. Descriptors shorter than the firmware's work still pay it in full. EXT descriptors are limited by base generation: a base costs the firmware about 15-35 instructions,

each NUM_HARTS clocks, against one cycle in `stream_run_addr_gen`, so with both directions on every channel even 256-beat runs cannot be fed at the engines' rate. Custom gatling instructions for DMA (`gatling_dma`) are the lever for that case.

### 39.3.1 Sixteen Channels

rapids-g-16ch (rapids TASK-036): the same bench at 16 channels, DATA_WIDTH 256, with each way of spreading the 16 channels over one gatling's harts, measured on the committed image (the two-slot poll with the chain hook, Section 5.3).

*Table 5.5.6: Cycles to finish at 16 channels*

| Scenario | rapids-16ch | 16 harts x 1 | 8 harts x 2 (default) | 4 harts x 4 |
|---|---|---|---|---|
| 16 one-beat descriptors, one channel | 131 | 4,731 | 3,131 | 1,619 |
| 16 one-beat descriptors on each channel | 131 | 4,620 | 5,383 | 5,627 |
| 16 64-beat descriptors on each channel | 1,255 | 4,700 | 5,469 | 5,704 |
| 4 64 KB descriptors on each channel | 9,248 | 9,609 | 9,585 | 9,579 |
| 2 1 MB descriptors on each channel | 84,277 | 84,610 | 84,612 | 84,612 |
| EXT, runs of 4, 64 beats, 4 per channel | 407 | 78,362 | 77,998 | 77,714 |
| EXT, per beat, 64 beats, 4 per channel | 1,043 | 260,192 | 257,831 | 256,406 |
| EXT, runs of 256, 1 MB, 1 per channel | 29,574 | 140,577 | 139,688 | 139,184 |

Sixteen harts make every instruction cost a hart 16 clocks, which a lone busy channel pays in full (4,731); four harts are best for one channel and on EXT, but lose 22% to sixteen when all sixteen channels run one- or 64-beat descriptors, each hart walking four slots. Eight harts with two slots each are 16% behind sixteen there, within 0.6% of the best on EXT, 1.5x faster than sixteen harts for a single channel, and keep half the hart state; `rapids_seq_g` defaults to them at 16 channels. Against the image this table first carried (the two-slot poll before the chain hook), the eight-hart column is about 10% slower on the one- and 64-beat rows and 1.5% on EXT: this bench's descriptors are independent (`next_ptr` = 0), so none takes the hook, and the SUMMARY-first pass costs a few instructions per head. What the hook buys is on chains, which this bench does not run:

Tables 5.5.9 and 5.5.10. The 1 MB rows are within 0.4% of rapids-16ch, the 64 KB rows within 3.6%.

## 39.4 Sixteen Channels on the Genesys 2

rapids-g-16ch and rapids-16ch in the full Genesys 2 harness (rapids TASK-036), `make bitstream-perf CHANNELS=16 ROW_COL=1`, 2026-10-23. The first rapids-g-16ch build placed at 198,555 LUTs (97.4%, every slice) and did not route: the harness's two 2048-entry descriptor RAMs had fallen to distributed RAM (25.6k LUTs each), because `sdpram_core`'s byte-enabled write port will not map to block RAM. The harness writes whole descriptors, so it now builds them with `USE_WSTRB` = 0, as the STREAM harness does; the 8-channel bitstreams take the same saving, not yet re-measured.

*Table 5.5.8: 16-channel bitstreams*

| Bitstream | LUTs | WNS at 100 MHz | Board == simulation |
|---|---|---|---|
| rapids-16ch | 135,389 (66.4%) | +0.192 ns | 109/109 windows identical |
| rapids-g-16ch, ROM | 151,472 (74.3%) | +0.148 ns | 109/109 windows identical |
| rapids-g-16ch, `FW_WRITABLE` | 152,813 (75.0%) | +0.034 ns | 109/109 identical, the committed image loaded live |

The 16-channel gcompare profile is 57 points: the 8-channel profile's single descriptors and chains, plus 16-channel single descriptors, chains at 1, 8 and 16 channels in size-major order, and the interleaved sink at 16 (`byte_perf.build_points`); in simulation it runs as 18 chunks to keep each under the 100 ms cap. "ROM" is the image before the two-slot chain hook; "v2" is the committed image (Section 5.3), loaded live on the writable bitstream with `--fw`.

*Table 5.5.9: Genesys 2 gcompare at 16 channels, MB/s (peak 3,200; – = no source variant)*

| Point | rapids-16ch sink | rapids-g ROM sink | rapids-g v2 sink | rapids-16ch source | rapids-g ROM source | rapids-g v2 source |
|---|---|---|---|---|---|---|
| beat_ch1_b1 | 1,600 | 1,600 | 1,600 | 356 | 356 | 356 |
| beat_ch1_b256 | 2,643 | 1,736 | 1,625 | 3,103 | 3,103 | 3,103 |

| Point | rapids-16ch sink | rapids-g ROM sink | rapids-g v2 sink | rapids-16ch source | rapids-g ROM source | rapids-g v2 source |
|---|---|---|---|---|---|---|
| beat_ch8_b16 | 1,241 | 813 | 788 | 3,012 | 3,012 | 3,012 |
| beat_ch8_b256 | 2,913 | 2,704 | 2,686 | 3,188 | 3,188 | 3,188 |
| beat_ch16_b1 | 133 | 70 | 67 | 2,048 | 247 | 230 |
| beat_ch16_b16 | 1,309 | 974 | 938 | 3,103 | 2,626 | 2,498 |
| beat_ch16_b256 | 2,935 | 2,820 | 2,810 | 3,194 | 3,194 | 3,194 |
| beat_ch16_b32768 | 3,198 | 3,197 | 3,197 | 3,200 | 3,200 | 3,200 |
| chain_ch1_b1_d16 | 178 | 19 | 36 | 397 | 19 | 36 |
| chain_ch1_b64_d16 | 2,232 | 1,122 | 2,008 | 3,175 | 1,208 | 2,190 |
| chain_ch8_b1_d16 | 169 | 21 | 38 | 2,677 | 154 | 284 |
| chain_ch8_b64_d16 | 2,278 | 1,236 | 2,152 | 3,197 | 3,197 | 3,197 |
| chain_ch16_b1_d16 | 168 | 20 | 36 | 3,091 | 172 | 346 |
| chain_ch16_b | 2,280 | 1,232 | 2,084 | 3,198 | 3,198 | 3,198 |

| Point | rapids-16ch sink | rapids-g ROM sink | rapids-g v2 sink | rapids-16ch source | rapids-g ROM source | rapids-g v2 source |
|---|---|---|---|---|---|---|
| 64_d16 | | | | | | |
| chain_ch16_b1024_d16 | 3,121 | 3,119 | 3,120 | 3,200 | 3,200 | 3,200 |
| chain_ch16_b8192_d4 | 3,190 | 3,189 | 3,190 | 3,200 | 3,200 | 3,200 |
| beat_ch16_b16_il | 1,309 | 839 | 813 | – | – | – |
| beat_ch16_b64_il | 2,351 | 1,879 | 1,845 | – | – | – |
| chain_ch16_b1_d16_il | 948 | 155 | 292 | – | – | – |
| chain_ch16_b64_d16_il | 2,772 | 2,702 | 2,738 | – | – | – |
| chain_ch16_b1024_d16_il | 3,172 | 3,166 | 3,169 | – | – | – |

The two-slot chain hook roughly doubles the short chains (19 to 36 MB/s on one-beat descriptors, 1,122 to 2,008 on 64-beat ones) and costs single descriptors one to three instructions: 1-7% on the points where the firmware is the limit, nothing from 256 beats per channel up.

### 39.4.1 Where rapids-g Catches Up

The sweep that sizes the gap, in simulation (`make sim-gscale CHANNELS=8|16`; the same simulation matches the board on all 109 gcompare windows): single descriptors and 16-descriptor interleaved-sink chains on every channel of the build, in 2x size steps, rows in beats per descriptor (32 bytes each). Each cell is rapids cycles / rapids-g cycles (1.00 = parity). Columns: `chain` is the 16-descriptor interleaved-sink chain (8 channels run one slot per hart; at 16, ROM is the image before the chain hook and v2 the committed one); `sink`, `source` and `il sink` are a single descriptor per channel, plain or with the sink streams interleaved.

*Table 5.5.10: rapids / rapids-g cycles, every channel busy*

| Beats | chain 8ch | chain 16ch ROM | chain 16ch v2 | sink 8ch | sink 16ch v2 | source 8ch | source 16ch v2 | il sink 8ch | il sink 16ch v2 |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 0.52 | 0.17 | 0.32 | 0.57 | 0.53 | 0.55 | 0.49 | 0.57 | 0.53 |
| 2 | 0.49 | 0.22 | 0.41 | 0.57 | 0.53 | 0.56 | 0.51 | 0.58 | 0.53 |
| 4 | 0.60 | 0.41 | 0.68 | 0.59 | 0.56 | 0.57 | 0.54 | 0.61 | 0.56 |
| 8 | 0.90 | 0.76 | 0.93 | 0.63 | 0.62 | 0.61 | 0.60 | 0.66 | 0.62 |
| 16 | 0.95 | 0.93 | 0.97 | 0.67 | 0.72 | 0.66 | 0.70 | 0.71 | 0.67 |
| 32 | 0.97 | 0.96 | 0.98 | 0.74 | 0.82 | 0.73 | 0.82 | 0.76 | 0.73 |
| 64 | 0.98 | 0.97 | 0.99 | 0.81 | 0.88 | 0.80 | 0.88 | 0.83 | 0.80 |
| 128 | 0.99 | 0.99 | 0.99 | 0.88 | 0.92 | 0.88 | 0.92 | 0.89 | 0.87 |
| 256 | 0.99 | 0.99 | 1.00 | 0.93 | 0.96 | 0.93 | 0.96 | 0.93 | 0.92 |
| 512 | 1.00 | 1.00 | 1.00 | 0.96 | 0.98 | 0.96 | 0.98 | 0.96 | 0.96 |
| 1024 | 1.00 | 1.00 | 1.00 | 0.98 | 0.99 | 0.98 | 0.99 | 0.98 | 0.98 |
| 2048 | – | – | – | 0.99 | 0.99 | 0.99 | 0.99 | 0.99 | 0.99 |
| 4096 | – | – | – | 0.99 | 1.00 | 0.99 | 1.00 | 1.00 | 0.99 |

How much a channel must move per descriptor before rapids-g is within 10%, 5% and 1% of rapids, from the sweeps (DATA_WIDTH 256, so 32 bytes a beat; the smallest size from which every larger size in the sweep stays inside the bound), with every channel busy and with one channel busy on each build. A chain row is bytes per descriptor in a 16-descriptor chain, so the channel moves sixteen times that in the run:

*Table 5.5.11: Bytes per descriptor for rapids-g to be within 10%, 5% and 1% of rapids*

| Shape | Busy channels | Within 10% | Within 5% | Within 1% |
|---|---|---|---|---|
| single descriptor, sink | 1 of 8 | 64 KB | 128 KB | 512 KB |
| single descriptor, sink | 8 of 8 | 8 KB | 16 KB | 64 KB |
| single descriptor, sink | 1 of 16 | 64 KB | 128 KB | 1 MB |
| single descriptor, sink | 16 of 16 | 4 KB | 8 KB | 64 KB |
| single descriptor, source | 1 of 8 | 64 KB | 128 KB | 512 KB |
| single descriptor, source | 8 of 8 | 8 KB | 16 KB | 128 KB |
| single descriptor, source | 1 of 16 | 64 KB | 128 KB | 1 MB |
| single descriptor, source | 16 of 16 | 4 KB | 8 KB | 64 KB |
| single descriptor, interleaved sink | 8 of 8 | 8 KB | 16 KB | 64 KB |
| single descriptor, interleaved sink | 16 of 16 | 8 KB | 16 KB | 128 KB |
| 16-descriptor chain, sink | 1 of 8 | 4 KB | 4 KB | 16 KB |
| 16-descriptor chain, sink | 1 of 16 | 2 KB | 4 KB | 32 KB |
| 16-descriptor chain, source | 1 of 8 | 4 KB | 8 KB | 16 KB |
| 16-descriptor chain, source | 1 of 16 | 4 KB | 8 KB | 32 KB |
| 16-descriptor chain, interleaved sink | 8 of 8 | 512 B | 512 B | 4 KB |
| 16-descriptor chain, interleaved sink | 16 of 16 | 256 B | 512 B | 4 KB |

The one-channel rows come from the same sweep run with one channel busy (`make sim-gscale1 CHANNELS=8|16`, both directions, chains plain rather than interleaved), on each build:

*Table 5.5.12: rapids / rapids-g cycles, one channel busy (- - = not in the sweep)*

| Beats | chain sink (8ch build) | chain sink (16ch build) | chain source (8ch build) | chain source (16ch build) | sink (8ch build) | sink (16ch build) | source (8ch build) | source (16ch build) |
|---|---|---|---|---|---|---|---|---|
| 1 | 0.49 | 0.22 | 0.26 | 0.12 | 0.27 | 0.26 | 0.28 | 0.27 |
| 2 | 0.53 | 0.24 | 0.26 | 0.12 | 0.27 | 0.26 | 0.29 | 0.27 |

| Beats | chain sink (8ch build) | chain sink (16ch build) | chain source (8ch build) | chain source (16ch build) | sink (8ch build) | sink (16ch build) | source (8ch build) | source (16ch build) |
|---|---|---|---|---|---|---|---|---|
| 4 | 0.62 | 0.28 | 0.26 | 0.12 | 0.29 | 0.27 | 0.29 | 0.28 |
| 8 | 0.56 | 0.36 | 0.27 | 0.12 | 0.31 | 0.29 | 0.30 | 0.29 |
| 16 | 0.48 | 0.45 | 0.44 | 0.20 | 0.34 | 0.32 | 0.33 | 0.31 |
| 32 | 0.54 | 0.60 | 0.78 | 0.36 | 0.38 | 0.35 | 0.37 | 0.35 |
| 64 | 0.85 | 0.90 | 0.59 | 0.66 | 0.45 | 0.42 | 0.44 | 0.41 |
| 128 | 0.96 | 0.96 | 0.92 | 0.95 | 0.54 | 0.51 | 0.54 | 0.51 |
| 256 | 0.99 | 0.98 | 0.99 | 0.98 | 0.67 | 0.63 | 0.66 | 0.63 |
| 512 | 0.99 | 0.99 | 0.99 | 0.99 | 0.78 | 0.75 | 0.78 | 0.75 |
| 1024 | 1.00 | 0.99 | 1.00 | 0.99 | 0.87 | 0.85 | 0.87 | 0.85 |
| 2048 | – | – | – | – | 0.93 | 0.92 | 0.93 | 0.92 |
| 4096 | – | – | – | – | 0.96 | 0.96 | 0.96 | 0.96 |
| 8192 | – | – | – | – | 0.98 | 0.98 | 0.98 | 0.98 |
| 16384 | – | – | – | – | 0.99 | 0.99 | 0.99 | 0.99 |
| 32768 | – | – | – | – | 1.00 | 0.99 | 1.00 | 0.99 |

With one channel busy a lone descriptor has nothing to hide the firmware behind: against the FSM it costs a fixed 162 clocks per single descriptor on the 8-channel build and 194 on the 16-channel one, whose hart walks a second, idle slot each pass (the cycle gap is the same from 8 KB to 1 MB), so the ratio is 1 - gap / beats: within 10% from 64 KB, 5% from 128 KB, and 1% from 512 KB (8 channels) or 1 MB (16). A chain on one channel overlaps the next head with the running descriptor and is within 5% from 4-8 KB and 1% from 16-32 KB; the 16-channel build is later on chains below 2 KB because its hart walks a second, idle slot every pass. With every channel busy the other channels' data hides the cost, which is why those rows close eight times earlier.

Chains close the gap at 512 bytes (the 8-channel chain reads 0.90 at 256 B in Table 5.5.10 but is 0.899) and are within 1% at 4 KB on both builds; a single descriptor per channel needs 8-16 KB to be within 5% and 64-128 KB to be within 1%, because its whole firmware cost (fetch, judge, stage and complete, with no next head to overlap) is paid once per descriptor against a data phase that only grows with the size. With one gatling per half serving 16 channels the expectation was that chains need twice the size to catch the FSM, since each hart serves two channels: on the ROM image they did (0.95 at 16 beats on 8 channels, at 32 on 16). With the chain hook in the two-slot poll they catch up at 16 on both, and are ahead of the 8-channel build from 4 beats; one- and two-beat chains remain behind it (0.32-0.41 against 0.49-0.52), the cost of walking two slots per pass. Single descriptors are at most 0.04 behind the 8-channel build at any size (and ahead of it on the

plain sink from 16 beats, where rapids-16ch slows too): the firmware’s work is per descriptor, so more channels do not move it.

## 39.5 Area and Timing

`rapids_core` and `rapids_core_g`, out of context on the Genesys 2 part (xc7k325t-2), DATA_WIDTH 256, `SHARED_ENGINES` 1, `USE_AXI_MONITORS` 0, `USE_ROW_COL_MAJOR_ADDRESSING` 1, default harts (8 at 8 channels, 8 at 16), placed and routed at 100 MHz, measured 2026-10-23 on the executive image with the next-address selector:

*Table 5.5.7: Out-of-context results*

| | rapids-8ch | rapids-g-8ch | rapids-16ch | rapids-g-16ch |
|---|---|---|---|---|
| LUTs | 69,184 | 77,975 (+12.7%) | 126,820 | 141,067 (+11.2%) |
| Registers | 56,105 | 58,837 | 101,808 | 104,738 |
| Block RAM tiles | 64 | 70 | 128 | 134 |
| DSPs | 64 | 0 | 128 | 0 |
| WNS at 100 MHz | +0.684 ns | +0.533 ns | +0.079 ns | +0.050 ns |

The selector check costs RAPIDS 411 LUTs at 8 channels (68,773 on 2026-10-22). At 16 channels each `scheduler_g` is about 1,630 LUTs against 1,430 for `scheduler`: its window read mux and base FIFOs in fabric outweigh the run-base generators, whose multipliers are DSPs on the RAPIDS side; over 32 schedulers that is most of the difference. Both 16-channel builds close 100 MHz, with the thinner margin (+0.050 ns) the cost of the larger part fill. The register arbiter (`rapids_g_reg_arb`) is in the top, outside `rapids_core_g`, and not in these numbers. The 2026-10-22 8-channel figures below are the first image's. Per half, `rapids_seq_g` is about 1.95k LUTs (gatling about 1.39k of it); the sink half is 38,693 LUTs against 34,943 for `rapids_snk`, and the source half 37,180 against 32,746 for `rapids_src`. The 4-hart build measured 76,350 LUTs, 56,645 registers, 69 block RAM tiles and +0.249 ns: eight harts cost about 600 LUTs and 2k registers, and gatling's own path is no longer the worst. The worst path of `rapids_core_g` is now in the source half, from a scheduler's source address (`gen_schedulers[3].u_scheduler/u_core/r_src_addr_reg`) to the AXI read engine's outstanding counter enable, 20 levels. The 64 DSPs of `rapids_core` are the run-base generators' index-by-stride multipliers, which the firmware replaces with additions.

In the full Genesys 2 harness of Table 5.5.3, rapids-g-8ch takes 111,738 LUTs (54.8%), 65,246 registers, 58 block RAM tiles and no DSPs and closes 100 MHz at +0.061 ns on the executive image (110,662 LUTs at +0.072 ns on 2026-10-22); rapids-8ch takes 100,007 LUTs at +0.357 ns (2026-10-22 build).

# 40 RAPIDS Beats Architecture MAS

**Version:** 0.14 **Date:** 2026-10-20 **Purpose:** Module Architecture Specification for RAPIDS Phase 1 “Beats” Architecture

---

## 40.1 Document Organization

**Note:** All chapters linked below for automated document generation.

### 40.1.1 Front Matter

- Document Information

### 40.1.2 Chapter 1: Overview

- Architecture Overview
- Top-Level Port List
- Clocks and Reset

### 40.1.3 Chapter 2: FUB (Functional Unit Blocks)

**Control Path:** - Scheduler - Descriptor Engine

**AXI Engines:** - AXI Read Engine - AXI Write Engine

**Flow Control:** - Beats Alloc Control - Beats Drain Control - Beats Latency Bridge

**Control Engines:** - Control-Read Engine - Control-Write Engine

### 40.1.4 Chapter 3: Macro (Integration Blocks)

**Scheduler Integration:** - Beats Scheduler Group - Beats Scheduler Group Array

**Sink Data Path (Network to Memory):** - Sink Data Path - Sink Data Path AXIS - Sink SRAM Controller

**Source Data Path (Memory to Network):** - Source Data Path - Source Data Path AXIS - Source SRAM Controller

**Top-Level Integration:** - RAPIDS Core Beats - RAPIDS Registers - RAPIDS Config Block - RAPIDS Beats Top - Network Planes

### 40.1.5 Chapter 4: Interfaces

- Interface Overview
- AXI4 Interface Specification
- AXIS Interface Specification
- MonBus Interface Specification
- CXS Interface Specification

---

## 40.2 Quick Reference

### 40.2.1 FUB Modules (fub_beats/)

*FUB Module Summary*

| Module | File | Purpose | Status |
|---|---|---|---|
| scheduler_beats | `scheduler_beats.sv` | Transfer coordinator, descriptor processing | Implemented |
| descriptor_engine_beats | `descriptor_engine_beats.sv` | Descriptor fetch/parse (256-bit) | Implemented |
| axi_read_engine_beats | `axi_read_engine_beats.sv` | AXI read master, streaming pipeline | Implemented |
| axi_write_engine_beats | `axi_write_engine_beats.sv` | AXI write master, streaming pipeline | Implemented |
| alloc_ctrl_beats | `alloc_ctrl_beats.sv` | Space allocation tracking (virtual FIFO); kept and tested, no longer in the SRAM path since `bdf4e0dff` | Implemented |
| drain_ctrl_beats | `drain_ctrl_beats.sv` | Data availability tracking (virtual FIFO); kept and tested, no longer in the SRAM path | Implemented |
| latency_bridge_beats | `latency_bridge_beats.sv` | Registered-FIFO to valid/ready skid bridge; kept and tested, no longer in the SRAM path | Implemented |
| ctrlrd_engine | `ctrlrd_engine.sv` | Control-read consumer gate (poll-until-match, retry budget) | Implemented |
| ctrlwr_engine | `ctrlwr_engine.sv` | Control-write producer doorbell | Implemented |

| Module | File | Purpose | Status |
|---|---|---|---|
| | | (single-beat write) | |

### 40.2.2 Macro Modules (macro_beats/)

*Macro Module Summary*

| Module | File | Purpose | Status |
|---|---|---|---|
| scheduler_group_beats | `scheduler_group_beats.sv` | Scheduler + Descriptor Engine wrapper | Implemented |
| scheduler_group_array_beats | `scheduler_group_array_beats.sv` | 8-channel scheduler array with arbitration | Implemented |
| snk_data_path_beats | `snk_data_path_beats.sv` | Network-to-memory path integration | Implemented |
| snk_data_path_axis_beats | `snk_data_path_axis_beats.sv` | AXIS variant of sink path | Implemented |
| src_data_path_beats | `src_data_path_beats.sv` | Memory-to-network path integration | Implemented |
| src_data_path_axis_beats | `src_data_path_axis_beats.sv` | AXIS variant of source path | Implemented |
| snk_sram_controller_beats | `snk_sram_controller_beats.sv` | Sink SRAM: naming wrapper around STREAM's `sram_controller` (one instance, per-channel FIFOs inside it) | Implemented |
| src_sram_controller_beats | `src_sram_controller_beats.sv` | Source SRAM: naming wrapper around STREAM's `sram_controller` | Implemented |
| rapids_src_beats / rapids_snk_beats | `rapids_src_beats.sv`, `rapids_snk_beats.sv` | One half each: scheduler array + data path + monbus merge | Implemented |

| Module | File | Purpose | Status |
|---|---|---|---|
| rapids_core_beats | `rapids_core_beats.sv` | Both halves + the core monbus arbiter | Implemented |
| rapids_regs | `regs/generated/rtl/rapids_regs.sv` (generated from `rtl/macro_beats/rapids_regs.rdl`) | PeakRDL register block (base + MON regfile @ 0x1000) | Implemented |
| rapids_config_block | `rapids_config_block.sv` | Maps register `hwif_out` to core/monitor `cfg_*` | Implemented |
| rapids_beats_top | `rapids_beats_top.sv` | Top-level: APB slave, kickoff, monitors, MonBus AXI-Lite group | Implemented |

## 40.3 Architecture Comparison: RAPIDS vs STREAM

The “beats” architecture is a Phase 1 implementation of RAPIDS that shares concepts with STREAM but targets network-to-memory (and vice versa) transfers rather than memory-to-memory.

*RAPIDS Beats vs STREAM Comparison*

| Feature | STREAM | RAPIDS Beats |
|---|---|---|
| **Primary Use** | Memory-to-memory DMA | Network-to-memory accelerator |
| **Data Paths** | Single bidirectional | Separate sink/source paths |
| **Network Interface** | None | AXIS master/slave |
| **SRAM Buffering** | Shared buffer | Separate sink/source buffers |
| **Descriptor Format** | 256-bit | 256-bit (compatible) |
| **Channel Count** | 8 | 8 |
| **Flow Control** | alloc_ctrl/ drain_ctrl | STREAM `stream_alloc_ctrl`/`stream_drain_ctrl` inside the shared `sram_controller` (`alloc_ctrl_beats`/`drain_ctrl_beats` kept, out of the path) |

## 40.4 Clock and Reset Summary

### 40.4.1 Clock Domains

*Clock Domains*

| Clock | Frequency | Usage |
|---|---|---|
| `aclk` | 100-500 MHz | Primary - all RAPIDS logic, AXI/AXIS interfaces |

### 40.4.2 Reset Signals

*Reset Signals*

| Reset | Polarity | Type | Usage |
|---|---|---|---|
| `aresetn` | Active-low | Async assert, sync deassert | Primary - all RAPIDS logic |

**See:** Clocks and Reset for complete timing specifications

---

## 40.5 Interface Summary

### 40.5.1 External Interfaces

*External Interfaces*

| Interface | Type | Width | Purpose |
|---|---|---|---|
| AXI4 (Descriptor) | Master | 256-bit | Descriptor fetch |
| AXI4 (Sink Write) | Master | 512-bit (param) | Sink data write to memory |
| AXI4 (Source Read) | Master | 512-bit (param) | Source data read from memory |
| AXIS (Sink) | Slave | 512-bit (param) | Network data ingress |
| AXIS (Source) | Master | 512-bit (param) | Network data egress |
| MonBus | Master | 64-bit | Monitor packet output |

### 40.5.2 Internal Buses

*Internal Buses*

| Interface | Width | Purpose |
|---|---|---|
| MonBus | 64-bit | Internal monitoring bus |
| Descriptor Bus | 256-bit | Descriptor distribution |
| SRAM Data Bus | 512-bit | SRAM read/write data |

## 40.6 Related Documentation

- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide
- **RAPIDS Beats HAS** - Architecture specification (the original pre-beats rapids_spec/ tree was retired)

## 40.7 Specification Conventions

### 40.7.1 Signal Naming

- **Clock:** `aclk`
- **Reset:** `aresetn` (active-low)
- **Valid/Ready:** Standard AXI/AXIS handshake
- **Registers:** `r_` prefix (e.g., `r_state`, `r_counter`)
- **Wires:** `w_` prefix (e.g., `w_next_state`, `w_grant`)

### 40.7.2 Parameter Naming

- **Uppercase:** `NUM_CHANNELS`, `DATA_WIDTH`, `ADDR_WIDTH`
- **Derived:** `CHAN_WIDTH = $clog2(NUM_CHANNELS)`

### 40.7.3 Figure and Table Numbering

- **Figures:** `Figure X.Y.Z: Title` where X=chapter, Y=section, Z=figure number
- **Tables:** Caption line `: Table Title` after table markdown

---

**Last Updated:** 2026-07-13 **Maintained By:** RAPIDS Architecture Team